

Scaling Cooperative Intelligence via Inverse Planning and Probabilistic Programming

by

Tan Zhi-Xuan

B.S., Yale University (2018)

S.M., Massachusetts Institute of Technology (2022)

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2025

© 2025 Tan Zhi-Xuan. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Tan Zhi-Xuan
Department of Electrical Engineering and Computer Science
May 16, 2025

Certified by: Vikash K. Mansinghka
Principal Research Scientist, Brain and Cognitive Sciences
Thesis Supervisor

Certified by: Joshua B. Tenenbaum
Professor of Brain and Cognitive Science
Thesis Supervisor

Accepted by: Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

Scaling Cooperative Intelligence via Inverse Planning and Probabilistic Programming

by

Tan Zhi-Xuan

Submitted to the Department of Electrical Engineering and Computer Science
on May 16, 2025 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

How can we build cooperative machines that model and understand human minds — machines that assist us with our goals, coordinate on plans, infer the intentions behind our words, and even learn our norms and values? This thesis presents a scalable model-based approach to building such systems via inverse planning and probabilistic programming. First, we introduce a probabilistic programming architecture that implements a Bayesian *theory of mind*. This architecture, Sequential Inverse Plan Search (SIPS), performs online inference of human goals and plans by inverting a Bayesian model of incremental human planning. By combining high-performance symbolic planners with sequential Monte Carlo (SMC) inference, SIPS achieves faster-than-real-time speed, while scaling to hundreds of possible goals, and remaining robust to human mistakes due to boundedly-rational planning. Second, we present Cooperative Language-guided Inverse Plan Search (CLIPS), a system that integrates SIPS with large language models (LLMs) to model *communicative cooperation*. By using LLMs as likelihood functions within probabilistic programs, CLIPS can infer human goals from ambiguous instructions, then provide uncertainty-aware assistance with much higher levels of reliability than LLMs can on their own. In addition, CLIPS can be used to infer the shared intentions of communicating agents from their actions and words. Third, we show how inverse planning can model the acquisition of *social normativity*, formalizing norm-guided societal behavior as a norm-augmented stochastic game (NSG). In NSGs, agents assume that society follows a shared set of social norms, and infer these norms from the actions of other agents. By doing so, agents can rapidly learn cooperative social norms using orders of magnitude less data than model-free approaches. Finally, we present advances in *probabilistic programming infrastructure* that have enabled architectures such as SIPS and CLIPS. Through interfaces for programmable SMC and probabilistic programming with LLMs, developers can readily compose modeling and inference subroutines when designing probabilistically coherent intelligent systems. Together, these innovations demonstrate the feasibility and scalability of rational AI engineering for cooperatively intelligent machines, while illuminating the computational and algorithmic foundations of human cooperative intelligence.

Thesis supervisor: Vikash K. Mansinghka

Title: Principal Research Scientist, Brain and Cognitive Sciences

Thesis supervisor: Joshua B. Tenenbaum

Title: Professor of Brain and Cognitive Science

Acknowledgments

An unfortunate aspect of writing one’s thesis up to the last minute is that it is hard to recall all the people one should thank. Nonetheless, I shall endeavor to do right by at least some of you.

The beginnings of the work presented in this thesis go back to at least my undergraduate years, when I first began thinking seriously about computational moral epistemology — or, as I sometimes put it — when I was “nerd-sniped into thinking about AI alignment”. For that, I have to thank Mojmír Stehlík for recommending the right book at the right time, though I think we would both agree that, counterfactually, it could have easily been someone else. I should also thank quite a few college professors, particularly the ones who taught me Confucian and Buddhist philosophy — Jay Garfield, Eric Greene, Michael Hunter, Pauline Lin — since it was studying Buddhism that led me to studying Confucianism, and it was studying Confucianism that got me interested in norm learning, moral epistemology, and the importance of 禮. Thanks should go as well to Daniel Greco, who read what is probably my earliest essay on moral learning and its relevance to AI. And thank you to Scaz and Jake Brawer for supporting my inane idea of teaching robots ownership norms, allowing me to eventually turn it into a paper for AAAI 2019.

While I had hoped to continue focusing on norms and norm learning as I entered the PhD — speaking of which, thanks to Desmond Ong, for his mentorship, and for putting me in touch with the CoCoSci-verse — I learned very quickly that if I wanted to build AI systems that could learn norms from *groups* of agents, I would first have to solve the sub-problem of inferring the goals of *individual* agents. As a result, most of this thesis is about that sub-problem.

For allowing me the freedom to learn that fact the hard way — and for the many years of invaluable mentorship thereafter — I have to thank my thesis supervisors, Vikash Mansinghka, and Josh Tenenbaum. To Vikash, thank you for teaching me concepts like “Rao-Blackwellization” and “model-inference co-design” and how to try and save the world with probabilistic programming. To Josh, thank you for all the advice on how to design a good human experiment, and for showing me that it is possible to make a career doing research that is as conceptually ambitious as it is technically rigorous. To the remaining members of my thesis committee — Dylan Hadfield-Menell and Leslie Kaelbling — thank you playing defense while I played offense. But more importantly, I am honored to be carrying torch of both assistance games and POMDPs forward — even if language-augmented assistance games are just a special case, not an extension, and even though all them are just Dec-POMDPs.

This is where I say, “This thesis would not have been possible...”, because it truly wouldn’t have been possible without many amazing collaborators over the years. **Part I** is based on work done with Jordyn Mann, Tom Silver, Arwa Alanqary, Gloria Lin, Joie Le, and Gloria Kang, with Nathalie Fernandez and Stefan Lunis helping out with a project in between on LLMs as goal priors. Gloria Lin’s portraits of me will be some of my favorite mementos from graduate school. Stefan, it

was a blast being your mentor, and I hope I one day know at least half as much about cocktails as you. Nathalie, thank you for toughing it out. **Part II** is based on work done with Lance Ying, who has been my closest collaborator at MIT. Between Lance's hustle and my perfectionism, a lot of good work ends up getting done. Lance is also an amazing chef, and I hope whoever ends up working with him in the future also gets to try his cooking. **Part III**, unsurprisingly, has a special place in my heart, and exists due to the tremendous efforts of Ninell Oldenburg. Thank you, Ninell, for being an amazing mentee to work with, and for keeping my dream of norm learning alive. Without you, I would have spent my entire PhD solving a subproblem. Finally, **Part IV** is really a testament to all the time I spent in ProbComp absorbing everything there is to know about probabilistic programming — but special thanks must go to Alex Lew, who has taught me everything I know about proper weighting and meta-inference. We are not the right initials for my K -kernel to be your L -kernel, but I am always here to be the Z .

Other collaborators I have had the privilege of working with over the years include Sydney Levine, Joe Kwon, Diego Trujillo Jiménez, Mindy Zhang, Jovana Kondic, Stewy Slocum, Micah Carroll, Matija Franklin, Hal Ashton, Kai Sandbrink, Mel Andrews, Zachary Peck, and Agustín Martínez Suñé. Thank you for all the papers together, and thank you Jovana, for the book. I got to work with Mel, Zach, Ninell and Agustín because of the PIBBSS Summer Fellowship, which would not exist if not for TJ and Nora Ammann.

I learned an incredible amount during my early years in the PhD from various members of CoCoSci and ProbComp, many of whom I have already thanked in my S.M. thesis, but will take the opportunity to thank again: Alex Lew, Lio Wong, Sam Tenka, Tom Silver, Sholei Croom, Nathalie Fernandez, Junyi Chu, Zhutian Yang, Kelsey Allen and João Loula were all members of the GPS crew shortly after I joined MIT. I can think of few other intellectual environments with that kind of interdisciplinarity. Most of what I know of (P)PL design and theory are due to interactions with Marco Cusumano-Towner, Feras Saad, Alex Lew, McCoy Becker, George Matheos, and Mathieu Huot. A special shoutout goes to Nishad Gothoskar for beating me to doctorhood, and Arijit Dasgupta for bringing a bit more of Singapore to ProbComp. And of course, thank you to Amanda Brower and Rachel Paiste, for keeping ProbComp afloat.

I have too many friends to thank, but I am excited to see those of you in Singapore more often. I will, on the other hand, miss those of you staying here. Tessa, I will miss you especially, but I am so grateful we found each other again. Hopefully this time, the continents won't keep us apart.

Finally, thank you to my parents, and to my family, for loving me through all the transitions in my life. I look forward to being home with you soon.

Contents

<i>List of Figures</i>	11
<i>List of Tables</i>	13
Introduction	17
Publications	19
 I Theory of Mind	 21
1 Online Bayesian Goal Inference via Sequential Inverse Plan Search	23
1.1 Modeling Agents as Bounded Incremental Planners	25
1.1.1 Modeling Goals, States and Observations	26
1.1.2 Modeling Plans and Actions	27
1.2 Sequential Inverse Plan Search	28
1.2.1 Online Updating of Hypothesized Partial Plans	29
1.2.2 Managing Hypothesis Diversity via Resampling and Rejuvenation	30
1.3 Experiments	30
1.3.1 Domains	30
1.3.2 Baselines	31
1.3.3 Human-Like Goal Inference from Sub-optimal and Failed Plans	31
1.3.4 Accuracy, Speed and Robustness of Inference	32
1.4 Related Work	34
1.5 Discussion and Future Work	35
 2 A Theory of Boundedly-Rational Minds	 37
2.1 Computational Model	38
2.1.1 Mistaken Goals via Temporary Goal Confusion	38
2.1.2 Mistaken Plans via Resource-Bounded Planning	38
2.1.3 Mistaken Actions via Execution Errors	40
2.1.4 Bayesian Goal Inference	40
2.2 Experiments	40
2.2.1 Experiment 1: Doors, Keys, & Gems	41
2.2.2 Experiment 2: Block Words	42
2.3 Discussion	45
2.4 Appendix	46

2.4.1	Experiment interface	46
2.4.2	Model fitting	46
2.4.3	Additional qualitative results	47
3	Infinite Ends from Finite Samples	49
3.1	Computational Model	50
3.1.1	Modeling Boundedly Rational Plans and Actions	51
3.1.2	Bottom-Up Sampling of Plausible Goals	51
3.1.3	Bayesian Filtering of Bottom-Up Samples	52
3.2	Experiments	53
3.2.1	Human Similarity	57
3.2.2	Accuracy and Runtime	57
3.3	Discussion	59
3.4	Appendix	60
3.4.1	Experiment Interface	60
3.4.2	Model Fitting and Sensitivity Analysis	60
3.4.3	Bottom-Up Proposals	62
3.4.4	Additional Similarity Metrics	64
3.4.5	Additional Step-by-Step Comparisons	64
II	Communicative Cooperation	67
4	Cooperative Language-Guided Inverse Plan Search	69
4.1	Language-Augmented Goal Assistance Games	70
4.2	Modeling Cooperative Action and Communication	71
4.3	Inverse Planning from Actions and Instructions	73
4.4	Pragmatic Instruction Following as Goal Assistance	74
4.5	Experiments	75
4.5.1	Model Configuration	76
4.5.2	Baselines	76
4.5.3	Human Experiments	77
4.5.4	Performance Metrics	78
4.5.5	Results	78
4.6	Related Work	80
4.7	Discussion and Future Work	81
4.8	Appendix	83
4.8.1	Model Configuration	83
4.8.2	Goal Assistance Problems	88
4.8.3	Human Experiments	89
4.8.4	GPT-4V Responses	90
4.8.5	Alternate Assistance Methods	92

5	Inferring Shared Goals from Communicating Agents	95
5.1	Modeling Communicating Teams as Group Agents	96
5.2	Team-Level Goal Inference from Actions and Instructions	97
5.3	Experiments	97
5.3.1	Dataset Generation	97
5.3.2	Human Experiments	98
5.3.3	Model Configuration	99
5.3.4	Results	99
5.4	Discussion and Future Work	101
III	Social Normativity	103
6	Modeling Social Normativity via Norm-Augmented Stochastic Games	105
6.1	Preliminaries	106
6.2	Norm-Augmented Stochastic Games	108
6.3	Normative Equilibria and Norm Convergence	112
6.4	Discussion and Related Work	115
7	Learning Social Norms via Bayesian Inverse Planning	117
7.1	Representing Social Norms	119
7.2	Norm-Compliant Planning	120
7.3	Bayesian Norm Learning	121
7.4	Experiments	122
7.4.1	Passive Norm Learning	123
7.4.2	Norm-Enabled Social Outcomes	124
7.4.3	Intergenerational Norm Transmission	125
7.4.4	Norm Emergence and Convergence	126
7.5	Discussion	127
7.6	Appendix	130
IV	Probabilistic Programming Infrastructure	133
8	Programmable Sequential Monte Carlo	135
8.1	Background	136
8.2	Particle Initialization	137
8.3	Particle Updating	139
8.4	Particle Rejuvenation	142
8.5	Particle Resampling	143
8.6	Particle Filter Resizing	145
8.7	Discussion and Future Work	147

9	Probabilistic Programming with Large Language Models	149
9.1	Language Models as Generative Functions	150
9.1.1	Batched LLM Generative Functions for Efficiency	151
9.2	Roles for Language Models in Probabilistic Programs	152
9.2.1	Likelihood Functions	152
9.2.2	Informative Priors	153
9.2.3	Proposal Distributions	154
9.3	Discussion and Related Work	156
	Conclusion	159
	<i>References</i>	161

List of Figures

1.1	An example of online Bayesian goal inference via Sequential Inverse Plan Search.	24
1.2	Overview of the SIPS modeling and inference architecture.	25
1.3	Overview of the SIPS class of agent models.	26
1.4	A partial policy computed by real-time heuristic search.	28
1.5	Human vs. SIPS inferences for a sub-optimal trajectory.	31
2.1	Overview of our boundedly rational agent model.	39
2.2	Goal inference results for the Doors, Keys & Gems domain.	41
2.3	Goal inference results for the Block Words domain.	43
2.4	Sensitivity plots for human-model correlations.	44
2.5	Experiment interface for Block Words.	46
3.1	Goal inference in Block Words via open-ended SIPS.	51
3.2	Step-by-step inference results on two illustrative Block Words scenarios.	55
3.3	Human similarity and accuracy of open-ended goal inference models.	56
3.4	Algorithmic-level properties of open-ended SIPS vs. humans and baseline algorithms.	56
3.5	Accuracy vs. runtime for exact inference and open-ended SIPS.	58
3.6	Interface for our open-ended goal inference task.	60
3.7	Human-model IoU across generative model parameters.	61
3.8	Human-model IoU across inference parameters.	61
3.9	Additional measures of human-model similarity in open-ended goal inference.	64
3.10	Additional step-by-step results for open-ended goal inference.	65
4.1	Overview of cooperative language-guided inverse plan search (CLIPS).	70
4.2	CLIPS model architecture.	72
4.3	Example goal assistance problem in VirtualHome.	76
4.4	Example goal assistance problems and inferences in Doors, Keys & Gems.	79
4.5	GPT-4V zero-shot prompt for a goal assistance problem.	87
4.6	Web interface for human experiments.	89
4.7	Human vs. model correlation plots for CLIPS in Doors, Keys & Gems.	90
4.8	Example responses by GPT-4V to CLIPS goal assistance problems.	91
4.9	Examples comparing (offline) Q_{MDP} and $\bar{\pi}_{\text{MDP}}$ assistance in Doors, Keys & Gems.	92
5.1	A CLIPS-based model of communicating teams.	96
5.2	Goal inferences over time from the actions and instructions of a human-robot team.	98
5.3	Human vs. model correlation plots for team-level goal inferences.	100

7.1	Overview of our architecture for Bayesian norm learning.	118
7.2	Passive norm learning curves for a norm-learning agent.	123
7.3	Social outcomes in the presence of a norm learning agent.	124
7.4	Inter-generational transmission or decay of norms.	125
7.5	Norm emergence and convergence over time.	127
7.6	Emergent norms sorted by frequency of emergence.	127
7.7	A multi-agent environment for norm learning.	130
8.1	Documentation for <code>pf_initialize</code>	138
8.2	Documentation for <code>UpdatingTraceTranslator</code>	140
8.3	Documentation for the most general form of <code>pf_update!</code>	141
8.4	Documentation for <code>pf_rejuvenate!</code> and its variants.	142
8.5	Documentation for <code>pf_resample!</code> and its variants.	144
8.6	Documentation for <code>pf_resize!</code> and other resizing operations.	146
9.1	Overview of the <i>GenGPT3.jl</i> library for probabilistic programming with LLMs. . .	150
9.2	Probabilistic programming with batched LLM generative functions.	151
9.3	LLM utterance likelihood $P(u \pi)$ in a model $P(\pi, u)$ of intentional utterances. . .	152
9.4	LLM goal prior in a simple agent model.	153
9.5	LLM goal proposal given observed subgoals.	154

List of Tables

1.1	Accuracy, runtime, and robustness of inference.	33
2.1	Best-fitting model parameters across domains and model types.	47
3.1	Accuracy and runtime across open-ended goal inference methods.	58
3.2	Effect of bottom-up proposal on human similarity and accuracy.	63
4.1	Performance of CLIPS vs. baseline methods.	78
4.2	Breakdown of goal assistance problems in Doors, Keys & Gems.	88
4.3	Breakdown of goal assistance problems in VirtualHome.	88
4.4	Comparison of CLIPS assistance methods.	93
5.1	Team-level goal inference accuracy for humans and inverse planning models. . .	100
7.1	Precision and recall for passive norm learning	123
7.2	List of potential norms $\mathcal{N}$ considered by norm-learning agents.	131

Introduction

Introduction

*Then they would pose to themselves a different practical question. Each would ask, not ‘What should I do?’, but rather, ‘What should **we** do?’.*

Elizabeth Anderson

Humans routinely cooperate with each other to accomplish tasks that none of us could achieve as effectively on our own. Whether we are assembling furniture with the help of friends, assisting an older relative with an unfamiliar mobile application, or coordinating with oncoming drivers on a single-track road, we regularly exercise our *cooperative intelligence*: our ability to model and understand other agents, groups, and society as a whole so that we may work towards shared goals. As AI systems are deployed to fulfill similar social functions — perhaps as collaborative robots, virtual assistants, or autonomous vehicles — they will require similar cooperative capabilities [1, 2]. How then can we build AI systems that cooperate well with humans and other agents, and ensure that they do so reliably, safely, and efficiently?

This thesis introduces a scalable model-based approach to engineering cooperatively intelligent machines, grounded in two core technologies: *inverse planning* and *probabilistic programming*. By modeling humans and other agents as approximately rational planners that act and communicate in accordance with their desires and intentions, we can invert this planning process, recovering goals, plans, and other mental states from via Bayesian inference from agents’ actions and words. And by using probabilistic programming [3, 4] to specify such agent models and accelerate inference over them, we can scale inverse planning to complex environments that have proved intractable for earlier approaches, while sharply reducing the engineering effort involved in Bayesian modeling and inference.

The thesis is organized into several parts, with each of the first three parts centered around a core capacity required for human-like cooperative intelligence. **Part I** presents a probabilistic programming architecture that implements a Bayesian *theory of mind*. This architecture, Sequential Inverse Plan Search (SIPS), performs online inference of human goals and plans by inverting a Bayesian model of incremental human planning [5]. Unlike earlier Bayesian theory-of-mind models [6] and inverse reinforcement learning methods that require exhaustive dynamic programming [7], plan recognition algorithms that rely on full-horizon planning [8], and data-intensive neural approaches to online goal inference [9], SIPS integrates high-performance online planners with sequential Monte Carlo inference, producing accurate and well-calibrated goal inferences with faster-than-real-time speed [5], while scaling to environments with hundreds of possible goals [10]. In addition, SIPS captures mistakes that arise due to boundedly-rational planning, enabling human-like goal inference from suboptimal or failed plans [11].

In **Part II**, we cover Cooperative Language-guided Inverse Plan Search (CLIPS), a system that integrates SIPS with large language models (LLMs) to model *communicative cooperation* [12]. As a formal setting for such cooperation, we introduce language-augmented goal assistance games, an extension of the assistance game formalism [13] where the human principal may communicate in natural language. To solve these assistance games, CLIPS models humans as cooperative planners who communicate aspects of their plans in addition to taking actions, using LLMs as likelihood functions over human utterances given an underlying plan. As a result, CLIPS is capable of inverse planning from natural language, using observed actions or inferred goals to disambiguate ambiguous or incomplete instructions. This enables pragmatic instruction following and uncertainty-aware goal assistance with much higher reliability than state-of-the-art LLMs. In addition, CLIPS can be used for team-level goal inference, inferring the intentions of communicating agents from their actions and words [14].

In **Part III**, we turn to cooperation at the larger, societal scale, demonstrating how Bayesian inverse planning can model the acquisition of *social normativity*. Building on earlier work in game theory [15], philosophy [16], and multi-agent systems [17], we formalize norm-guided societal behavior as a norm-augmented stochastic game (NSG) [18]. In NSGs, agents assume that society follows a shared but unknown set of social norms — rules that classify behavior as either norm-compliant or norm-violating — and infer these norms from the actions of other agents via inverse planning. Agents are also motivated to comply with these norms via utility functions that depend on their beliefs about which norms are practiced. By designing agents with these features, they learn rapidly to comply with cooperative social norms, achieving mutually beneficial outcomes with orders of magnitude less data than model-free reinforcement learning, while avoiding the tragedy of the commons.

Finally, **Part IV** presents advances in *probabilistic programming infrastructure* that have enabled systems like SIPS and CLIPS. Extending the programmable inference paradigm introduced in Gen [4, 19], we introduce the *GenParticleFilters.jl* library for *programmable sequential Monte Carlo* (SMC), providing composable building blocks for writing SMC algorithms like SIPS, while automating the low-level math that ensures their soundness [20]. We also introduce infrastructure for *probabilistic programming with LLMs* in the form of *GenGPT3.jl*, enabling the use of LLMs as likelihood functions, priors, and proposals inside of Bayesian architectures like CLIPS. Together, these libraries provide new compositional tools for the rational engineering of intelligent systems.

We conclude by discussing how these contributions demonstrate the scalability of model-based AI engineering for cooperatively intelligent machines, how they shed light on the computational foundations of human cooperative intelligence, and how they lay fertile ground for a new generation of cooperative AI applications that are both rationally designed and reliably human-aligned.

Publications

Most chapters in this thesis are based on prior published work or software packages, with new material added on specific topics and extensions. These publications are as follows:

I Theory of Mind

1. Online Bayesian Goal Inference via Sequence Inverse Plan Search
 - Based on: T. Zhi-Xuan, J. Mann, T. Silver, J. Tenenbaum, and V. Mansinghka. “Online Bayesian Goal Inference for Boundedly Rational Planning Agents”. In: *Advances in Neural Information Processing Systems* 33 (2020)
2. A Theory of Boundedly Rational Minds
 - Based on: A. Alanqary, G. Z. Lin, J. Le, T. Zhi-Xuan, V. K. Mansinghka, and J. B. Tenenbaum. “Modeling the Mistakes of Boundedly Rational Agents Within a Bayesian Theory of Mind”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 43. 43. 2021
3. Infinite Ends from Finite Samples
 - Based on: T. Zhi-Xuan, G. Kang, V. Mansinghka, and J. Tenenbaum. “Infinite Ends from Finite Samples: Open-Ended Goal Inference as Top-Down Bayesian Filtering of Bottom-Up Proposals”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 46. 2024

II Communicative Cooperation

4. Cooperative Language-Guided Inverse Plan Search
 - Based on: T. Zhi-Xuan, L. Ying, V. Mansinghka, and J. B. Tenenbaum. “Pragmatic Instruction Following and Goal Assistance via Cooperative Language-Guided Inverse Planning”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024
5. Inferring Shared Goals from Communicating Agents
 - Based on: L. Ying, T. Zhi-Xuan, V. Mansinghka, and J. B. Tenenbaum. “Inferring the goals of communicating agents from actions and instructions”. In: *Proceedings of the AAIL Symposium Series*. Vol. 2. 1. 2023, pp. 26–33

III Social Normativity

6. Modeling Social Normativity via Norm-Augmented Markov Games
 - Develops upon the theory in: N. Oldenburg and T. Zhi-Xuan. “Learning and Sustaining Shared Normative Systems via Bayesian Rule Induction in Markov Games”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024
7. Learning Social Norms via Inverse Planning
 - Based on: N. Oldenburg and T. Zhi-Xuan. “Learning and Sustaining Shared Normative Systems via Bayesian Rule Induction in Markov Games”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024

IV Probabilistic Programming Infrastructure

8. Programmable Sequential Monte Carlo
 - Based on the following software package: T. Zhi-Xuan. *GenParticleFilters.jl*. 2020
9. Probabilistic Programming with Large Language Models
 - Based on the following software package: T. Zhi-Xuan. *GenGPT3.jl: GPT-3 as a generative function in Gen.jl*. Nov. 2022

Part I

Theory of Mind

Chapter 1

Online Bayesian Goal Inference via Sequential Inverse Plan Search

[F]irst you decide to treat the object whose behavior is to be predicted as a rational agent... Then you figure out what desires it ought to have... and finally you predict that this rational agent will act to further its goals in the light of its beliefs.

Daniel Dennett

The capacity to model and understand the minds of others is a core ability that enables humans to function as a cooperative species. This capacity — a theory of mind — is present in humans from a very young age [22, 23], and supports a wide range of social and cooperative behaviors, including helping [24], communication [25], group coordination [26, 27], norm learning [28, 29] and moral reasoning [30, 31]. In efforts to computationalize a theory of mind, both AI researchers [8, 32] and cognitive scientists [6, 33] have often followed — implicitly or explicitly — Daniel Dennett’s account of folk psychology as taking the *intentional stance* [34]. Taking the intentional stance involves viewing and predicting another entity as a *rational agent*. In doing so, we attribute intentions, beliefs, and other mental states as rational explanations for that entity’s behavior.

Over the past decade or so, one of the most successful approaches for modeling the intentional stance has been a paradigm known as *Bayesian theory-of-mind* [33, 35, 36] (BToM). In this paradigm, the attribution of mental states is framed in Bayesian terms: Given a generative model of how an agent’s mental states lead to approximately rational behavior, an observer can infer those hidden states via Bayesian inference. Since this generative model is usually a model of approximately rational planning, inference over such models is often known as *Bayesian inverse planning* [6].

Despite the successes of this paradigm at capturing human-like inferences of goals [6, 37], beliefs [33, 38], rewards [36, 39], intentions [31], and even emotions [40], an algorithmic challenge that has long been unaddressed by BToM models is the intractability of planning in most domains [41, 42], and hence the apparent intractability of inverse planning. If this intractability holds, it is hard to see how any cognitive system could infer mental states reliably or speedily via inverse planning [43, 44]. How then, are humans solving this problem efficiently — and how can we engineer machines to do the same?

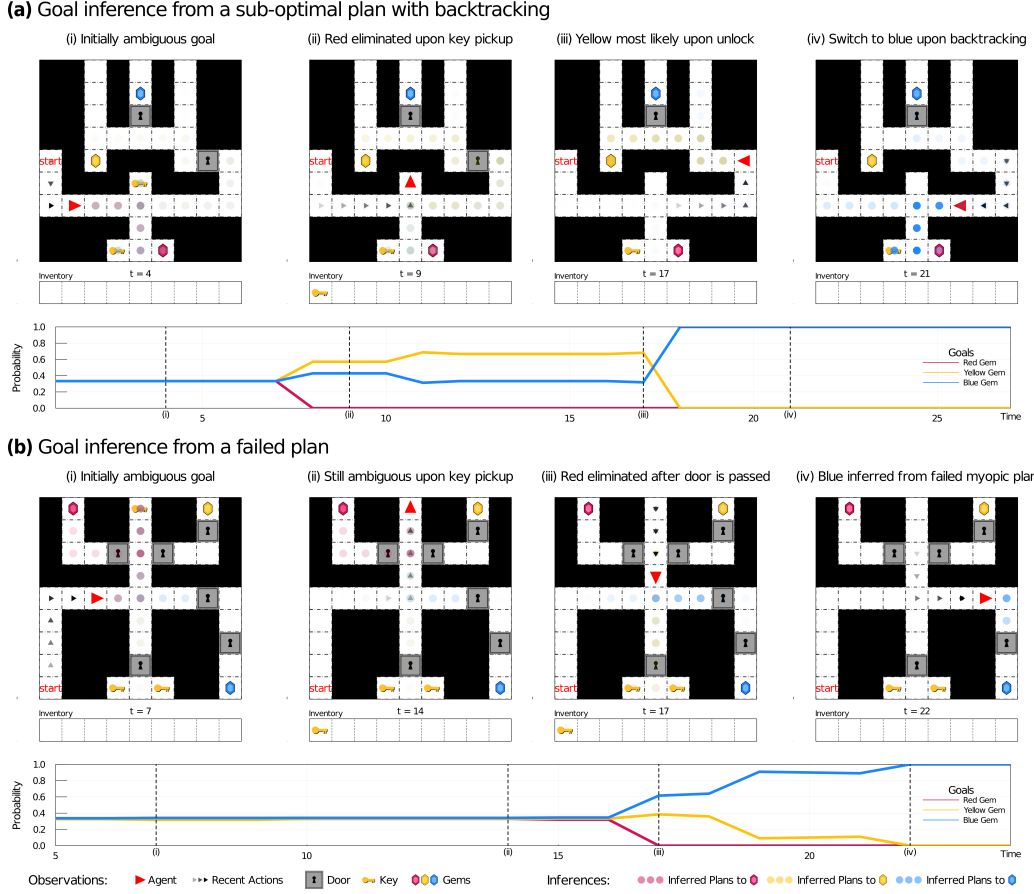


Figure 1.1: Our architecture performing online Bayesian goal inference via Sequential Inverse Plan Search. In (a), an agent exhibits a *sub-optimal plan* to acquire the blue gem, backtracking to pick up the key required for the second door. In (b), an agent exhibits a *failed plan* to acquire the blue gem, myopically using up its first key to get closer to the gem instead of realizing that it needs to collect the bottom two keys. In both cases, our method not only manages to infer the correct goal by the end, but also captures sharp human-like shifts in its inferences at key points, such as (a.ii) when the agent picks up a key unnecessary for the red gem, (a.ii) when the agent starts to backtrack, (b.iii) when the agent ignores the door to the red gem, or (b.iv) when the agent unlocks the first door to the blue gem.

In this and the next two chapters, we present a unified modeling and inference architecture (Figure 1.2) that addresses this challenge, while also delivering new insights about the nature of human theory-of-mind. This architecture, **Sequential Inverse Plan Search (SIPS)**, generalizes prior generative models of approximately rational decision-making to incorporate both *incremental planning* and *bounded rationality*, while providing a sequential Monte Carlo (SMC) inference algorithm capable of highly efficient online inference of an agent’s goals and plans. This chapter introduces the SIPS architecture, presenting experiments from its original publication [5]. In Chapter 2, we focus on the ability of SIPS to capture human-like inferences about the goals of boundedly rational agents who make mistakes when planning (e.g. Figure 1.1). Finally, in Chapter 3, we show how SIPS can be scaled to perform open-ended goal inference with many hundreds of possible goals, while also demonstrating that humans solve such problems via a similar sampling-based strategy.

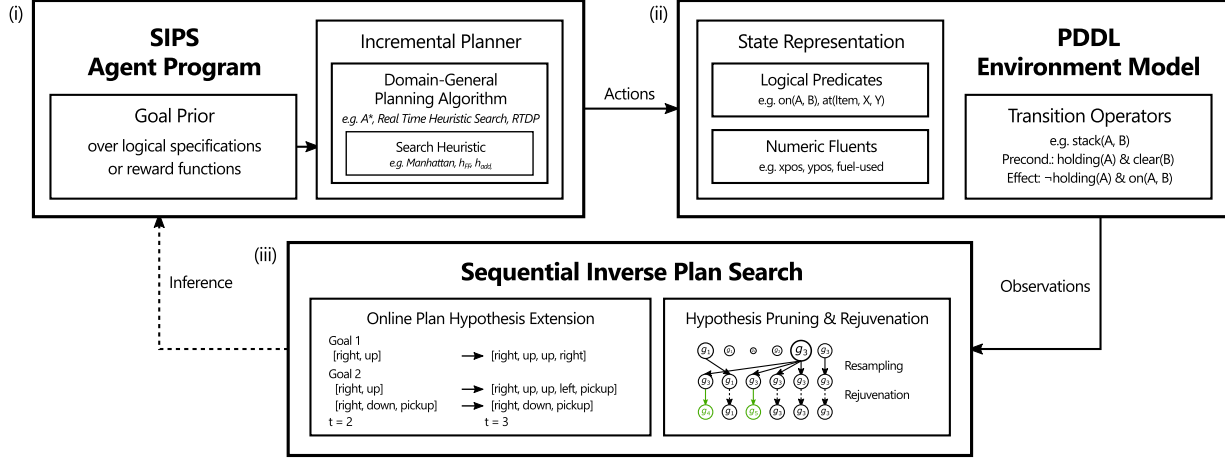
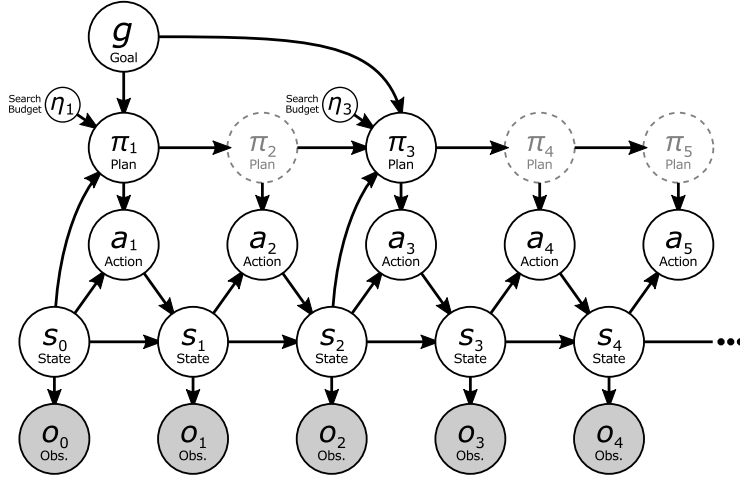


Figure 1.2: Our modeling and inference architecture is comprised of: **(i)** A programmatic model of an incremental planning agent in the SIPS model class, implemented in the Gen probabilistic programming system; **(ii)** An environment model specified in the Planning Domain Definition Language (PDDL), facilitating support for a wide variety of planning domains and state-of-the-art symbolic planners; **(iii)** Sequential Inverse Plan Search (SIPS), a SMC algorithm that exploits incremental planning in our agent model to reduce computation, extending hypothesized plans or policies only as new observations arrive.

In our architecture, we model agents as probabilistic programs (Figure 1.3), comprised of goal priors and domain-general planning algorithms (Figure 1.2i) interacting with a symbolic environment model (Figure 1.2ii). Inference is then performed via by the namesake of the architecture, Sequential Inverse Plan Search (SIPS), a sequential Monte Carlo (SMC) algorithm that exploits the incremental nature our agent models, incrementally inferring partial plans or policies while limiting computational cost (Figure 1.2iii). SIPS delivers both accuracy and speed by being built in Gen, a general-purpose probabilistic programming system that supports customized inference using data-driven proposals and involutive rejuvenation kernels [4, 19, 45], alongside an embedding of the Planning Domain Definition Language [46], enabling the use of compiler optimized general-purpose planners provided by the PDDL.jl library [47] as customizable modeling components. We evaluate our approach against a Bayesian inverse reinforcement learning baseline [48] on a wide variety of planning domains that exhibit compositional task structure and sparse rewards (e.g. Figure 1.1), achieving high accuracy on many domains, often with orders of magnitude less computation.

1.1 Modeling Agents as Bounded Incremental Planners

In contrast to prior work that models agents as *actors* that are *noisily rational* or *Boltzmann rational* [7, 48, 49], a key distinction of the SIPS class of agent models is an explicit representation of how agents’ compute plans or policies on-the-fly in order to achieve their final goal. Due to resource-limited planning, these plans might be incomplete or overly optimistic at any point in time (potentially leading to sub-optimality), and may be filled in or refined only at later time steps. To capture these dynamics, we model agents and their environments as generative processes of the following form.



(a) One realization of our agent and environment model.

```

model PLAN-UPDATE( $t, s_{t-1}, \pi_{t-1}, g$ )
  parameters: PLANNER,  $h, \theta_d$ 
   $d \sim \text{CATEGORICAL}([\text{replan}, \text{refine}, \text{keep}], \theta_d)$ 
   $\eta \sim \text{BUDGET-PRIOR}()$ 
  if  $d = \text{replan}$  then
     $\pi_t \sim \text{PLANNER}(s_t, g, h, \eta)$ 
  else if  $d = \text{refine}$  then
     $\pi_t \sim \text{REFINE\_PLANNER}(\pi_{t-1}, s_t, g, h, \eta)$ 
  else
     $\pi_t \leftarrow \pi_{t-1}$ 
  end if
  return  $\pi_t$ 
end model

```

(i) Plan Update $P(\pi_t | s_{t-1}, \pi_{t-1}, g)$

```

model ACTION-SELECTION( $t, s_t, \pi_t$ )
  parameters:  $\epsilon$ 
  return  $a_t \sim \text{EPSILON-GREEDY}(\pi_t, s_t, \epsilon)$ 
end model

```

(ii) Action Selection $P(a_t | s_t, \pi_t)$

(b) Replanning agent programs.

Figure 1.3: We model agents as incremental planners that interleave search and execution of partial plans as they interact with the environment. In (a) we depict one possible realization of this model, where the agent initially samples a search budget η_1 and searches for a plan π_1 that is two actions long. At $t = 2$, no additional planning needs to be done, so π_2 is copied from π_1 , as denoted by the dashed lines. The agent then replans at $t = 3$ from state s_3 , sampling a new search budget η_3 and a new partial plan π_3 with three actions. We formally specify this agent model using probabilistic programs, with pseudo-code shown in (b). PLAN-UPDATE samples updated plans or policies π_t given previous ones π_{t-1} , while ACTION-SELECTION selects an action a_t according the current plan π_t and state s_t .

$$\text{Goal Prior: } g \sim P(g) \quad (1.1)$$

$$\text{Plan Update: } \pi_t \sim P(\pi_t | s_{t-1}, \pi_{t-1}, g) \quad (1.2)$$

$$\text{Action Selection: } a_t \sim P(a_t | s_t, \pi_t) \quad (1.3)$$

$$\text{State Transition: } s_t \sim P(s_t | s_{t-1}, a_t) \quad (1.4)$$

$$\text{Observation Noise: } o_t \sim P(o_t | s_t) \quad (1.5)$$

Here, g , π_t , a_t , s_t are the agent's goals, the state of the agent's plan or policy, the agent's action, and the environment's state at time t respectively. Optionally, observers may have imperfect observability of the environment due to observation noise, resulting in an observed state o_t . Under perfect observability, they instead observe the action a_t and state s_t . We describe each component of this process in greater detail below.

1.1.1 Modeling Goals, States and Observations

To represent states, observations, goals, and distributions over goals in a general and flexible manner, our architecture embeds the Planning Domain Definition Language (PDDL) [46, 50], representing states s_t and goals g in terms of predicate-based facts, relations, and numeric expressions (Figure 1.2ii). State transitions $P(s_t | s_{t-1}, a_{t-1})$ are modeled by transition operators that specify the preconditions and effects of actions. While we focus on deterministic transitions in this paper, we also support stochastic transitions, as in Probabilistic PDDL [51]. Given this representation, an observer's prior over goals $P(g)$ can be specified as a probabilistic program over

PDDL goal specifications (including numeric reward functions) as well as sets of goal predicates (e.g. `has(gem)`), which define the set of terminal states in the induced (stochastic) shortest path problem [52]. Observation noise $P(o_{t+1}|s_{t+1})$ can also be modeled by corrupting each Boolean predicate with some probability, and adding continuous (e.g. Gaussian) noise to numeric fluents.

1.1.2 Modeling Plans and Actions

To model how agents plan in SIPS, the basic insight we follow is that agents like ourselves are *incremental planners*: Rather than form a complete plan or policy to the goal from the beginning, we may only compute a partial plan or policy due to limited cognitive resources. Since full-horizon planning is often costly or intractable, it may make more sense to form partial plans towards promising states, start acting, and replan along the way.

We capture the overall process of incremental planning by the probabilistic program shown in Figure 1.3b(i). At each step t , we assume that the agent:

- Randomly decides (d) whether to **replan** from scratch, **refine** π_{t-1} , or **keep** π_{t-1} as is.¹
- Samples a search budget $\eta \sim \text{BUDGET-PRIOR}()$ for future planning.
- If $d = \text{replan}$, computes a new plan/policy π_t starting at s_t until the budget η is used up.
- If $d = \text{refine}$, refines π_{t-1} into a new policy π_t until the budget η is used up.

This randomized plan update captures multiple sources of variability in how an agent might plan. The replanning decision d accounts for whether an agent (re)plans at all, and whether they reuse past information. If they do (re)plan, η captures how much effort they put into additional planning.

A flexibility of the SIPS model class is that each random decision above has tunable parameters. For example, $\text{BUDGET-PRIOR}()$ could be a negative binomial distribution (as used in the original version of SIPS [5], with parameters r and q), capturing the intuition that both very high and very low search budgets η are unlikely. Alternatively, a uniform prior over a representative set of search budgets might be enough to robustly capture a range of qualitatively distinct behaviors,

Furthermore, a variety of different planning algorithms can be used to model how an agent plans. In the original version of SIPS [5], for instance, we used a probabilistic variant of A^* search, which expands nodes on the frontier with probability $P_{\text{expand}}(s) \propto \exp(-f(s, g)/\gamma)$, where γ is a noise parameter controlling the randomness of search, and $f(s, g) = c(s) + h(s, g)$ is the estimated total plan cost under a search heuristic h .

A more robust approach is to use anytime planners such as Real-Time Dynamic Programming (RTDP) [53, 54] or Real-Time Heuristic Search (RTHS) [55–58], which compute not just partial plans, but a partial *policies*. In contrast to partial plans, which only contain information about what action to take over the next few steps, partial policies map states to recommended actions in a *neighborhood* of the current state s_t (Figure 1.4). In addition, within this neighborhood, both RTDP and RTHS compute estimates $\hat{V}_{\pi_t}(s)$ of the *optimal value function* $V^*(s)$ also known as the *cost-to-goal* function:

$$V^*(s) = \min_a Q^*(s, a) \quad Q^*(s, a) = C(s, a) + \sum_{s'} P(s'|s, a) V^*(s) \quad (1.6)$$

where $Q^*(s, a)$ is known as the optimal Q -value function, and $C(s, a)$ is the cost of taking action a at state s . We note that once we have $\hat{V}_{\pi_t}(s)$ estimates, we can also estimate $Q_{\pi_t}^*(s, a)$ as $\hat{Q}_{\pi_t}(s, a) = C(s, a) + \sum_{s'} P(s'|s, a) \hat{V}_{\pi_t}(s)$.

¹Note that **replan** may be forced sometimes, e.g. when there are no actions left in the previous plan π_{t-1} .

Computing these value estimates is useful for two reasons: First, the value estimates $\hat{V}_{\pi_t}(s)$ stored in a policy π_t provide search guidance for future calls to RTDP and RTHS that *refine* the policy π_t . Indeed, in the context of RTHS variants like Real-Time Adaptive A* [56], $\hat{V}_{\pi_t}(s)$ plays the role of the search heuristic h that guides the embedded A* search algorithm. As such, planning is accelerated whenever a policy π_t is refined.

More importantly, the $\hat{Q}$ -values computed by policy-based planning can be used to compute a more *graded* distribution $P(a_t, s_t, \pi_t)$ over what action the agent will take given their policy π_t . In particular, we can parameterize an *approximate* Boltzmann-rational distribution over actions:

$$P(a_t, s_t, \pi_t) \propto \exp\left(-\beta \hat{Q}_{\pi_t}(s_{t-1}, a_t)\right) \quad (1.7)$$

In contrast, when using an algorithm like probabilistic A*, which produces only partial plans, the most graded action distribution we can compute is an ϵ -greedy action selection (shown in Figure 1.3b). While we do not present experiments in this chapter demonstrating the policy-based version of SIPS, Chapter 3 relies on the policy-based version to scale to much larger goal spaces.

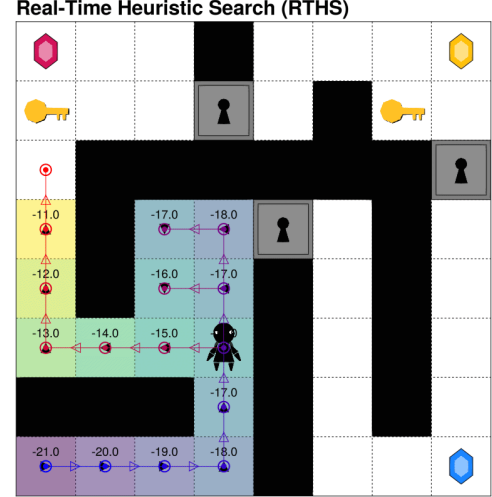


Figure 1.4: A partial policy π computed by RTHS when searching for the yellow gem. The policy stores cost-to-goal estimates $\hat{V}(s)$ and recommended actions $\pi(s)$ in a neighborhood around the current state.

1.2 Sequential Inverse Plan Search

Having specified our model, we can now state the problem of Bayesian goal inference. We assume that an observer receives a sequence of potentially noisy state observations $o_{1:t} = (o_1, \dots, o_t)$. Given the observations up to timestep t and a set of possible goals $\mathcal{G}$, the observer's aim is to infer the agent's goal $g \in \mathcal{G}$ by computing the posterior:

$$P(g|o_{1:t}) \propto P(g) \sum_{\substack{s_{1:t} \\ a_{1:t} \\ \pi_{1:t}}} \prod_{\tau=0}^{t-1} P(o_{\tau+1}|s_{\tau+1})P(s_{\tau+1}|s_{\tau}, a_{\tau})P(a_{\tau}|s_{\tau}, \pi_{\tau})P(\pi_{\tau}|s_{\tau}, \pi_{\tau-1}, g) \quad (1.8)$$

Computing this posterior exactly is intractable, as it requires marginalizing over all the random latent variables s_{τ} , a_{τ} , and π_{τ} . Instead, we develop a sequential Monte Carlo procedure, shown in Algorithm 1, to perform approximate inference in an online manner, using samples from the posterior $P(g|o_{1:t-1})$ at time $t-1$ to inform sampling from the posterior $P(g|o_{1:t})$ at time t . We call this algorithm Sequential Inverse Plan Search (SIPS), because it sequentially inverts a search-based planning algorithm, inferring sequences of partial plans that are likely given the observations, and consequently the likely goals.

As in standard particle filtering schemes, we first sample a set of particles or hypotheses $i \in [1, k]$, with corresponding weights w_i (lines 3-5). Each particle corresponds to a particular plan p_{τ}^i and goal g^i . As each new observation o_{τ} arrives, we extend the particles (lines 12-14) and reweight them by their likelihood of producing that observation (line 15). The collection of weighted particles thus approximates the full posterior over the unobserved variables in our model, including the agent's plans and goals. We describe key features of this algorithm below.

Algorithm 1 Sequential Inverse Plan Search (SIPS)

```

1: procedure SIPS( $s_0, o_{1:t},$ )
2:   parameters:  $k$ , number of particles;  $c$ , resampling threshold
3:    $w^i \leftarrow 1$  for  $i \in [1, k]$  ▷ Initialize particle weights
4:    $s_0^i, \pi_0^i, a_0^i \leftarrow s_0, [], \text{no-op}$  for  $i \in [1, k]$  ▷ Initialize states, plans and actions
5:    $g^i \sim \text{GOAL-PRIOR}()$  for  $i \in [1, k]$  ▷ Sample  $k$  particles from goal prior
6:   for  $\tau \in [1, t]$  do
7:     if  $\text{EFFECTIVE-SAMPLE-SIZE}(w^1, \dots, w^k)/k < c$  then ▷ Resample and rejuvenate
8:        $g^i, s_{1:\tau}^i, \pi_{1:\tau}^i, a_{1:\tau}^i \sim \text{RESAMPLE}([g^i, s_{1:\tau}^i, \pi_{1:\tau}^i, a_{1:\tau}^i]^{1:k})$  for  $i \in [1, k]$ 
9:        $g^i, s_{1:\tau}^i, \pi_{1:\tau}^i, a_{1:\tau}^i \sim \text{REJUVENATE}(g^i, o_{1:\tau}, s_{1:\tau}^i, \pi_{1:\tau}^i, a_{1:\tau}^i)$  for  $i \in [1, k]$ 
10:    end if
11:    for  $i \in [1, k]$  do ▷ Extend each particle to timestep  $\tau$ 
12:       $s_\tau^i \sim P(s_\tau | s_{\tau-1}^i, a_{\tau-1}^i)$  ▷ Sample state transition
13:       $\pi_\tau^i \sim \text{PLAN-UPDATE}(\pi_\tau | s_\tau^i, \pi_{\tau-1}^i, g^i)$  ▷ Update plan if necessary
14:       $a_\tau^i \sim \text{ACTION-SELECTION}(a_\tau | s_\tau^i, \pi_\tau^i)$  ▷ Select action
15:       $w^i \leftarrow w^i \cdot P(o_\tau | s_\tau^i)$  ▷ Update particle weight
16:    end for
17:  end for
18:   $\tilde{w}^i \leftarrow w^i / \sum_{j=1}^k w^j$  for  $i \in [1, k]$  ▷ Normalize particle weights
19:  return  $[(g^1, w^1), \dots, (g^k, w^k)]$  ▷ Return weighted goal particles
20: end procedure
21:
22: procedure REJUVENATE( $g, o_{1:\tau}, s_{1:\tau}, \pi_{1:\tau}, a_{1:\tau}$ ) ▷ Metropolis-Hasting rejuvenation move
23:   parameters:  $\pi_g$ , goal rejuvenation probability
24:   if  $\text{BERNOULLI}(\pi_g)$  then ▷ Heuristic-driven goal proposal
25:      $g' \sim Q(g) := \text{SOFTMAX}([h(o_\tau, g) \text{ for } g \in \mathcal{G}])$  ▷ Propose  $g'_0$  based on est. distance to  $o_\tau$ 
26:      $s'_{1:\tau}, \pi'_{1:\tau}, a'_{1:\tau} \sim P(s_{1:\tau}, \pi_{1:\tau}, a_{1:\tau} | g)$  ▷ Sample trajectory under new goal  $g$ 
27:      $\alpha \leftarrow Q(g)/Q(g')$  ▷ Compute proposal ratio
28:   else ▷ Error-driven replanning proposal
29:      $t_* \sim Q(t_* | s_{1:\tau}, o_{1:\tau})$  ▷ Sample a time close to when  $s_{1:\tau}$  diverges from  $o_{1:\tau}$ 
30:      $s'_{t_*:\tau}, \pi'_{t_*:\tau}, a'_{t_*:\tau} \sim Q(s_{t_*:\tau}, \pi_{t_*:\tau}, a_{t_*:\tau} | o_{t_*:\tau})$  ▷ Propose new plan sequence  $\pi'_{t_*:\tau}$ 
31:      $\alpha \leftarrow Q(s_{t_*:\tau}, \pi_{t_*:\tau}, a_{t_*:\tau} | o_{t_*:\tau}) / Q(s'_{t_*:\tau}, \pi'_{t_*:\tau}, a'_{t_*:\tau} | o_{t_*:\tau})$  ▷ Compute proposal ratio
32:      $\alpha \leftarrow \alpha \cdot Q(t_* | s'_{1:\tau}, o_{1:\tau}) / Q(t_* | s_{1:\tau}, o_{1:\tau})$  ▷ Reweight by auxiliary proposal ratio
33:   end if
34:    $\alpha \leftarrow \alpha \cdot P(o_{1:\tau} | s'_{1:\tau}) / P(o_{1:\tau} | s_{1:\tau})$  ▷ Compute acceptance ratio
35:   return  $g'_0, s'_{1:\tau}, \pi'_{1:\tau}, a'_{1:\tau}$  if  $\text{BERNOULLI}(\min(\alpha, 1))$  else  $g_0, s_{1:\tau}, \pi_{1:\tau}, a_{1:\tau}$  ▷ Accept or reject proposals
36: end procedure

```

1.2.1 Online Updating of Hypothesized Partial Plans

A key aspect that makes SIPS a genuinely *online* algorithm is the modeling assumption that agents also plan *online*. This obviates the need for the observer to precompute a complete plan or policy for each of the agent’s possible goals in advance, and instead defers such computation to the point where the agent reaches a time t that the observer’s hypothesized plans do not yet reach. In particular, for each particle i , the corresponding plan hypothesis π_{t-1}^i is updated (Algorithm 1, line 13) by running the PLAN-UPDATE procedure in Figure 1.3b.i, which only performs additional computation if replanning is forced (i.e. π_{t-1}^i does not already contain a planned action for time t and state s_t), or when $d \neq \text{keep}$. This means that at any given time t , only a small number of plans require extension, limiting the number of expensive planning calls.

1.2.2 Managing Hypothesis Diversity via Resampling and Rejuvenation

We also introduce resampling and rejuvenation steps into SIPS in order to ensure particle diversity. Whenever the effective sample size falls below a threshold c (line 7), we resample the particles (line 8), thereby pruning low-weight hypotheses. We then rejuvenate by applying a mixture of two data-driven Metropolis-Hastings kernels to each particle. The first kernel uses a heuristic-driven goal proposal (lines 25-27), proposing goals $\tilde{g} \in \mathcal{G}$ which are close in heuristic distance $h(o_\tau, \tilde{g})$ to the last observed state o_τ . This allows SIPS to reintroduce goals that were pruned, but later become more likely. The second kernel uses an error-driven replanning proposal (lines 29-32), which samples a time close to the divergence point between the hypothesized and observed trajectories, and then proposes to replan from that time, thereby constructing a new sequence of hypothesized partial plans that are less likely to diverge from the observations. Despite the complexity of these proposals, acceptance ratios are automatically calculated via Gen’s support for involutive kernels [19]. Collectively, these steps help to ensure that hypotheses are both diverse and likely.

1.3 Experiments

We conducted several sets of experiments that demonstrate the human-likeness, accuracy, speed, and robustness of our approach. We first present experiments demonstrating the novel capacity of SIPS to infer goals from sub-optimal trajectories involving backtracking and failure (Figure 1.1). Comparing these inferences against human goal inferences shows that SIPS is more human-like than baseline approaches (Figure 1.5). We also evaluate the accuracy and speed of SIPS on a variety of planning domains (Table 1.1a), showing that it outperforms Bayesian IRL baselines. Finally, we present robustness experiments showing that SIPS can infer goals even when the data-generating model differs from the model assumed by the algorithm (Table 1.1b).

1.3.1 Domains

We validate our approach on domains with varying degrees of complexity, both in terms of the size of the state space $|\mathcal{S}|$ and the number of possible goals $|\mathcal{G}|$. All domains are characterized by compositional structure and sparse rewards, posing a challenge for standard approaches.

- **Taxi.** ($|\mathcal{G}| = 3, |\mathcal{S}| = 125$): A benchmark domain used in hierarchical reinforcement learning [59], where a taxi has to transport a passenger from one location to another in a gridworld.
- **Doors, Keys, & Gems.** ($|\mathcal{G}| = 3, |\mathcal{S}| \sim 10^5$): A domain in which an agent must navigate a maze with doors, keys, and gems (Figure 1.1). Each key can be used once to unlock a door, allowing the agent to acquire items behind that door. Goals correspond to acquiring one out of three colored gems.
- **Block Words.** ($|\mathcal{G}| = 5, |\mathcal{S}| \sim 10^5$): A Blocks World variant adapted from [8] where blocks are labeled with letters. Goals are block towers that spell one of a set of five words.
- **Intrusion Detection.** ($|\mathcal{G}| = 20, |\mathcal{S}| \sim 10^{30}$): A cybersecurity-inspired domain drawn from [8], where an agent might perform a variety of attacks on a set of servers. There are 20 possible goals, each corresponding to a set of attacks on up to 10 servers.

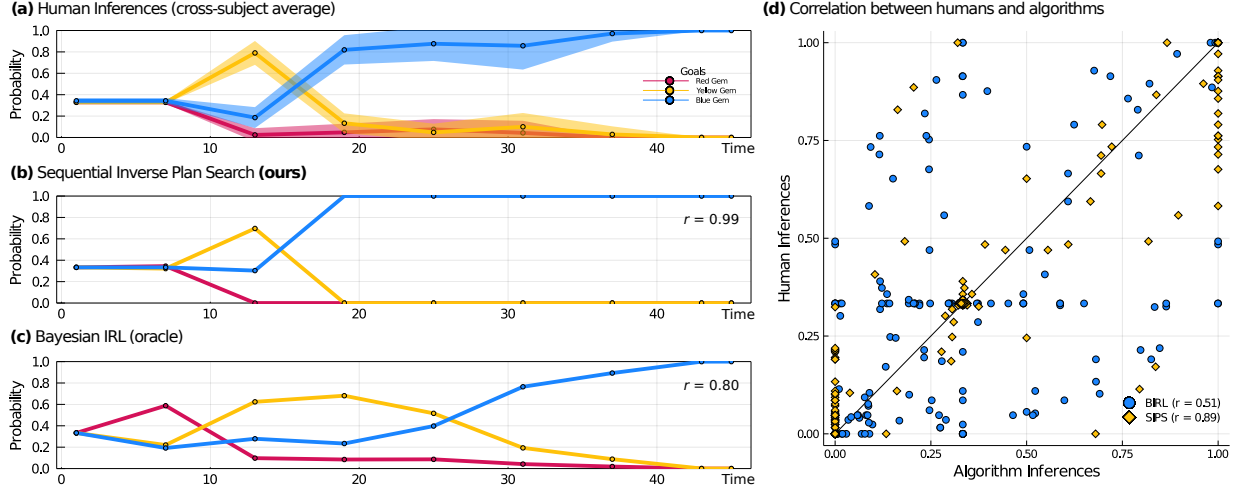


Figure 1.5: (a) Average human goal inferences over time (± 1 std.) for the sub-optimal trajectory in Figure 1.1a, compared to (b) inferences made by SIPS and (c) oracle BIRL. We omit unbiased BIRL because unbiased VI fails to converge for this domain, producing a flat posterior. In (d) we show a scatterplot of mean human inferences against algorithm inferences across all trajectories.

1.3.2 Baselines

We implemented Bayesian IRL (BIRL) baselines by running value iteration to compute a Boltzmann-rational policy $\pi(a_t|s_t, g)$ for each possible goal $g \in \mathcal{G}$. Following the setting of early Bayesian theory-of-mind approaches [6], we treated goals as indicator reward functions, and assumed a uniform prior $P(g)$ over goals. Inference was then performed by exact computation of the posterior over reward functions, using the policy as the likelihood for observed actions. Unless otherwise stated, we used a discount factor of 0.9, and Boltzmann noise parameter $\alpha=1$.

Due to the exponentially large state space of many of our domains, standard value iteration (VI) often failed to converge even after 10^6 iterations. As such, we implemented two variants of BIRL that use asynchronous VI, sampling states instead of fully enumerating them. The first, *unbiased BIRL*, uses uniform random sampling of the state space up to 250,000 iterations, sufficient for convergence in the Block Words domain. The second, *oracle BIRL*, assumes oracular access to the full set of observed trajectories in advance, and performing biased sampling of states that appear in those trajectories. Although inapplicable in practice for online use, this ensures that the computed policy is able to reach the goal in all cases, making it a useful benchmark for comparison.

1.3.3 Human-Like Goal Inference from Sub-optimal and Failed Plans

To investigate the novel human-like capabilities of our approach, we performed a set of qualitative experiments on a set of trajectories designed to exhibit notable sub-optimality or failure. The experiments were performed on the Doors, Keys & Gems domain because it allows for irreversible failures. Two illustrative examples are shown in Figure 1.1, and more are provided in the supplement of the paper this chapter is based on. In Figure 1.1a, SIPS accurately infers goals from a sub-optimal plan with backtracking, initially placing more posterior mass on the yellow gem when the agent acquires the first key (panel ii), but then switching to the blue gem once the agent

backtracks to the second key (panel iv). In Figure 1.1b, SIPS remains uncertain about all three goals when the first key is acquired (panel ii), but discards the red gem as a possibility when the agent walks past the door (panel iii), and finally converges upon the blue gem when the agent myopically unlocks the first door required to access that gem (panel iv).

In addition to posterior convergence, the inferences made by SIPS display human-like changes at key timepoints. We quantified this human-likeness by collecting human goal inferences on ten trajectories (six sub-optimal or failed) in a pilot study with $N=8$ subjects. Human inferences were collected every six timesteps, and a comparison against SIPS and the oracle BIRL baseline is shown in Figure 1.5. For the trajectory in in Figure 1.1a, human inferences (Figure 1.5a) display extremely similar qualitative trends as SIPS (Figure 1.5b, $r=0.99$). Oracle BIRL correlates less well (Figure 1.5c, $r=0.80$), assigning high probability to the yellow gem even after the agent backtracks at $t \geq 18$. This is because Boltzmann action noise assigns significant likelihood to the undoing of past actions. As Figure 1.5d shows, SIPS also correlates more strongly with mean human inferences across the dataset. Inferences made SIPS (yellow) hew closely to the diagonal, achieving a correlation of $r=0.89$, indicating that the agent model assumed by SIPS is similar to humans’ theory-of-mind. In contrast, inferences made by BIRL (blue) are more diffuse, achieving a correlation of only $r=0.51$.

1.3.4 Accuracy, Speed and Robustness of Inference

To evaluate accuracy and speed, we ran each inference method on a dataset of optimal and non-optimal agent trajectories for each domain, assuming a uniform prior over goals. The optimal trajectories were generated using A* search with an admissible heuristic for each possible goal in the domain. Non-optimal trajectories were generated using the replanning agent model in Figure 1.3b, with parameters $r=2$, $q=0.95$, $\gamma=0.1$, and $\theta_d = [0, 0, 1.0]$ (i.e. replanning only when forced). We found that with matched model parameters, SIPS achieved good performance with 10 particles per goal without the use of rejuvenation moves, so we report those results here.

We summarize the results of these experiments in Table 1.1a. Our method greatly outperforms the unbiased BIRL baseline in both accuracy and speed in three out of four domains, with an average runtime (AC) often several orders of magnitude smaller. This is largely because unbiased VI fails to converge except for the highly restricted Taxi domain. In contrast, SIPS requires far less initial computation, albeit with higher marginal cost due its online generation of partial plans. In fact, it achieves comparable accuracy and speed to the oracle BIRL baseline, sometimes with less computation (e.g. in Doors, Keys & Gems). SIPS also produces higher estimates of the goal posterior $P(g_{\text{true}}|o)$. This is a reflection of the underlying agent model, which assumes randomness at the level of planning, not acting. As a result, even a few observations can provide sharp evidence that a particular plan and goal was chosen.

Given the specific assumptions made by our agent model, a reasonable question is whether inference is robust to plans generated by other agent models or actual humans. To address this, we also performed a series of robustness experiments for two domains (Table 1.1b) on data generated by mismatched model parameters r , q , γ , mismatched planning heuristics h , Boltzmann-rational RL agents, optimal agents, and 5 pilot human subjects (30 trajectories per subject).

As Table 1.1b shows, SIPS is relatively robust to data generated by these other models and parameters. Although performance can degrade with mismatch, this is partly due to the difficulty of inference from highly random behavior (e.g. $q=0.8$, $h=GC$). On the other hand, when mismatched parameters are *more* optimal, performance can *improve* (e.g. $h=Mz$). Importantly, SIPS also does

Domain	Method	Accuracy						Runtime			
		$P(g_{\text{true}} o)$			Top-1			C_0 (s)	MC (s)	AC (s)	N
		Q1	Q2	Q3	Q1	Q2	Q3				
Taxi (3 Goals)	SIPS (ours)	0.44	0.50	0.62	0.53	0.56	0.67	13.0	1.80	2.55	1429
	BIRL (unbiased)	0.34	0.35	0.79	0.33	0.42	0.92	2.22	0.00	0.16	10000
	BIRL (oracle)	0.37	0.47	0.81	0.42	0.44	0.86	1.63	0.00	0.12	2500
Doors, Keys & Gems (3 Goals)	SIPS (ours)	0.37	0.51	0.61	0.74	0.74	0.74	3.30	0.70	0.86	2099
	BIRL (unbiased)	0.33	0.33	0.33	0.33	0.33	0.33	3326	0.12	154	250000
	BIRL (oracle)	0.37	0.36	0.42	0.44	0.60	0.80	150	0.12	7.01	10000
Block Words (5 Goals)	SIPS (ours)	0.47	0.83	0.90	0.78	0.84	0.91	20.8	2.46	4.15	2506
	BIRL (unbiased)	0.20	0.20	0.21	0.42	0.49	0.56	687	0.27	63.6	250000
	BIRL (oracle)	0.20	0.29	0.45	0.73	0.80	0.96	22.2	0.05	2.12	10000
Intrusion Detection (20 Goals)	SIPS (ours)	0.56	0.87	0.87	0.65	0.87	0.87	375	6.60	28.0	13321
	BIRL (unbiased)	0.05	0.05	0.05	0.05	0.05	0.05	18038	0.75	1069	250000
	BIRL (oracle)	0.09	0.24	0.53	0.94	1.00	1.00	98	0.02	6.00	10000

(a) **Accuracy and runtime of goal inference across domains and inference methods.** We quantify accuracy at the 1st, 2nd and 3rd quartiles (Q1–Q3) of each observed trajectory via the posterior probability of the true goal $P(g_{\text{true}}|o)$, and the fraction of problems where g_{true} is top-ranked (Top-1). We measure runtime in terms of the start-up cost (C_0), marginal cost per timestep (MC), and average cost per timestep (AC) in seconds. We also report the total number of states visited (N) during either search or value iteration as a platform-independent measure. Excluding the oracle baseline, the best metrics are bolded.

Domain	Persistence (r)			Persistence (q)			RL	Optimal
	1	2*	4	0.8	0.9	0.95*	$\alpha=50$	
Doors, Keys, Gems	0.60	0.73	0.73	0.53	0.60	0.73	0.58	0.80
Block Words	0.90	0.87	0.90	0.70	0.83	0.87	0.82	0.80
Domain	Search Noise. (γ)			Heuristic (h)			Humans	
	0.5	0.1*	0.02	Mh.*	Mz.	GC.	$hadd^*$	$n=5$
Doors, Keys, Gems	0.67	0.73	0.77	0.73	0.90	–	–	0.79
Block Words	0.83	0.87	0.87	–	–	0.43	0.87	0.73

(b) **Robustness to model mismatch.** Top-1 accuracy of SIPS at the third time quartile (Q3), evaluated on data generated by mismatched parameters, Boltzmann-rational RL agents, optimal agents, and humans. We ran SIPS assuming $r=2$, $q=0.95$, $T=10$. For Doors, Keys, Gems, we assumed a Manhattan (Mh.) heuristic against a maze distance (Mz.) heuristic. For Block Words, we assumed $hadd$ against the naive goal count (GC.) heuristic. Matched parameters are starred (*).

Table 1.1: Accuracy, runtime, and robustness of inference.

well on human data, showing robustness even when the planner is unknown. While our boundedly rational agent model cannot possibly capture all aspects of human planning, these experiments suggest that it serves as a reasonable approximation, similar to our intuitive theories of other people’s minds.

1.4 Related Work

Inverse Reinforcement Learning (IRL). A long line of work has shown how to learn reward functions as explanations of goal-directed agent behavior via inverse reinforcement learning [13, 48, 60, 61]. However, most such approaches are too costly for online settings of complex domains, as they require solving the underlying Markov Decision Process (MDP) for every posited goal or reward function, and for all possible initial states [62, 63]. Our approach instead assumes that agents are online model-based planners. This greatly reduces computation time, while also better reflecting humans’ intuitive understanding of other agents.

Bayesian Theory-of-Mind (BToM). Computational models of humans’ intuitive theory-of-mind posit that we understand other’s actions by Bayesian inference of their likely goals and beliefs. These models, largely built upon the same MDP formalism used in IRL, have been shown to make predictions that correspond closely with human inferences [6, 33, 35, 36, 64–66]. Some recent work also models agents using probabilistic programs [67, 68]. Our research extends this line of work by explicitly modeling an agent’s partial plans, or *intentions* [69]. This allows our architecture to infer final goals from instrumental subgoals produced as part of a plan, and to account for sub-optimality in those plans, thereby enriching the range of mental inferences that BToM models can explain.

Plan Recognition as Planning (PRP). Our work is related to the literature on plan recognition as planning, which performs goal and plan inference by using classical satisficing planners to model plan likelihoods given a goal [8, 70–74]. However, because these approaches use a heuristic likelihood model that assumes goals are always achievable, they are unable to infer likely goals when irreversible failures occur. In contrast, we model agents as online planners who may occasionally execute partial plans that lead to dead ends.

Online Goal Inference. Several recent papers have extended IRL to an online setting, but these have either focused on maximum-likelihood estimation in 1D state spaces [75, 76], or utilize an expensive value iteration subroutine that is unlikely to scale [77]. In contrast, we develop a sequential Monte Carlo algorithm that exploits the online nature of the agent models in order to perform incremental plan inference with limited computation cost.

Inferences from Sub-Optimal Behavior. We build upon a growing body of research on inferring goals and preferences while accounting for human sub-optimality [7, 68, 78–81], introducing a model of boundedly-rational planning as resource-limited search. This reflects a natural principle of *resource rationality* under which agents are less likely to engage in costly computations [82, 83]. Unlike prior models of myopic agents which assign zero reward to future states beyond some time horizon [78, 80], our approach accounts for myopic planning in domains with instrumental subgoals and sparse rewards.

1.5 Discussion and Future Work

In this paper, we demonstrated an architecture capable of online inference of goals and plans, even when those plans might fail. However, important limitations remain. First, we considered only finite sets of goals, but the space of goals that humans pursue is easily infinite. We take up how to address this challenge in Chapter 3. Second, unlike the domains considered here, the environments we operate in often involve stochastic dynamics and infinite action spaces [84, 85]. A natural extension would be to integrate Monte Carlo Tree Search or sample-based motion planners into our architecture as modeling components [67], potentially parameterized by learned heuristics [86]. With hope, our architecture might then approach the full complexity of problems that we face everyday, whether one is stacking blocks as a kid, finding the right keys for the right doors, or writing a PhD thesis.

Chapter 2

A Theory of Boundedly-Rational Minds

What are the cognitive limitations of humans, and how do they manifest themselves in the kind of imperfect decisions that we make? All of this is really important... if we want to build machines that can accurately interpret human behavior.

Stuart Russell

A key aspect of human intuitive psychology is our understanding that other agents are fallible: they may possess false beliefs [87], lack knowledge [88], fail to plan ahead [89], or act unintentionally [90]. This capacity is crucial to social life, allowing us to teach others [91], or forgive harms that we take as unintended [92, 93]. Remarkably, even 18-month old infants seem to account for such errors when inferring the goals of others, enabling them to offer assistance [24].

What are these errors, and how do we understand them in a way that allows us to infer the goals and intentions of others? As we have discussed in the previous chapter, inferring goals and intentions can be framed as *inverse planning* with respect to a Bayesian theory of mind [6, 33]. However, while prior BToM models successfully explain how humans can infer others' goals, desires, and intentions [66, 94], and have even explained how we might infer mistaken beliefs [33, 79], little attention has been paid to mistaken goals, plans, or actions. With a few exceptions [78, 95], most BToM models only account for mistakes through Boltzmann-distributed action noise [7, 48]. This fails to capture higher-level mistakes, and has been challenged elsewhere as a model of sequential decision making [96, 97].

In this chapter, we build upon the pilot human experiments presented in Chapter 1 on goal inference from sub-optimal plans, rigorously evaluating SIPS as a model of human-like goal inference in the presence of mistakes. Unlike earlier BToM agents which plan via exhaustive computation of expected value over the entire state space [6, 48], SIPS agent models do not always plan optimally, but, like ourselves, only plan several steps ahead before executing that partial plan and replanning. Though this approach to planning may be more resource rational [89, 98], it can also lead to planning mistakes. We extend this model with goal mistakes, due to confusion of goals with semantically similar specifications, and action mistakes, due to occasional execution of unplanned actions. The resulting model thus accounts for sub-optimality at three distinct levels of human decision-making.

We hypothesize that human goal inferences given sub-optimal action sequences are better explained by Bayesian inference in this boundedly rational agent model, in contrast to previously introduced BToM approaches. We test this hypothesis by eliciting human inferences in the Doors, Keys & Gems, and Block Words domains introduced in Chapter 1. In these experiments, participants observe an agent as it progresses towards its goal, but makes mistakes along the way. At selected points, they are asked to judge which goals are the most likely. Observed sequences are designed to exhibit mistakes at the levels of goals, plans, and actions. By comparing human judgements against inferences for each computational model, we evaluate the fidelity of these models to our intuitive theory of boundedly rational minds.

2.1 Computational Model

To account for mistakes at multiple levels of decision making, we extend the original SIPS model class introduced in Zhi-Xuan et al. [5], considering generative processes of the following form:

$$\text{Goal prior: } g_0 \sim P(g_0) \quad (2.1)$$

$$\text{Goal transition: } g_t \sim P(g_t | g_{t-1}, g_0) \quad (2.2)$$

$$\text{Plan update: } p_t \sim P(p_t | s_{t-1}, p_{t-1}, g_{t-1}) \quad (2.3)$$

$$\text{Action selection: } a_t \sim P(a_t | s_t, p_t) \quad (2.4)$$

$$\text{State transition: } s_t \sim P(s_t | s_{t-1}, a_t) \quad (2.5)$$

$$\text{Observation noise: } o_t \sim P(o_t | s_t) \quad (2.6)$$

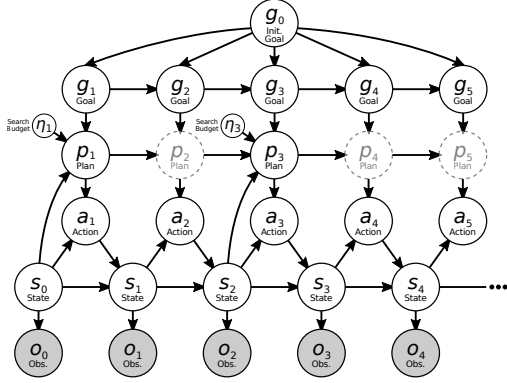
where g_0 is the agent’s original intended goal, and g_t , p_t , a_t , s_t are the agent’s current (potentially corrupted) goal, the internal state of the agent’s plan, the agent’s action, and the environment’s state at time t respectively. These generative processes are specified as probabilistic programs (Figure 2.1), and their corresponding mistakes are described below.

2.1.1 Mistaken Goals via Temporary Goal Confusion

Consistent with previous BToM literature [6], we restrict our scope to inference over a fixed set of context-relevant goals with associated prior probabilities (Eq. 2.1), leaving aside how people come up with these contextual hypotheses. However, within a fixed set of goals, complex goals can still get confused. For example, when stacking blocks to spell a word, one might have in mind a misspelling: Is it “firey” or “fiery”? To account for these mistakes, we introduce goal transition noise (Eq. 2.2), specified by the procedure in Figure 2.1b. At each step t with probability ϵ_g , the original goal g_0 may be corrupted to produce a similar *temporary* goal g_t , or the corrupted goal corrected to the original one (Line 4). Goal noise can be domain-specific, e.g. in Blocks World, CORRUPT might correspond to a random permutation.

2.1.2 Mistaken Plans via Resource-Bounded Planning

As described in the introduction, we model agents that interleave resource-bounded planning with plan execution. This can cause mistakes due to failure to plan ahead. At each step t , agents already have a previous partial plan p_{t-1} , and construct an updated plan p_t (Eq. 2.3).



(a) Single realization of our model in graphical form.

```

1: model GOAL-TRANSITION( $t, g_{t-1}, g_0$ )
2:   parameters:  $\epsilon_g$  (goal noise)
3:   if BERNOULLI( $\epsilon_g$ ) = TRUE then
4:     return CORRUPT-OR-REPAIR( $g_0, g_{t-1}$ )
5:   else
6:     return  $g_{t-1}$ 
7:   end if
8: end model

```

(b) Samples goals from $P(g_t | g_{t-1}, g_0)$.

```

1: model PLAN-UPDATE( $t, s_t, p_{t-1}, g$ )
2:   parameters: PLANNER,  $r, q, \gamma, h$ 
3:   if  $t > \text{LENGTH}(p_{t-1})$  or  $s_t \notin p_{t-1}[t]$  then
4:      $\eta \sim \text{NEGATIVE-BINOMIAL}(r, q)$ 
5:      $\tilde{p}_t \sim \text{PLANNER}(s_t, g, h, \gamma, \eta)$ 
6:      $p_t \leftarrow \text{APPEND}(p_{t-1}, \tilde{p}_t)$ 
7:   else
8:      $p_t \leftarrow p_{t-1}$ 
9:   end if
10:  return  $p_t$ 
11: end model

```

(c) Samples plans from $P(p_t | s_t, p_{t-1}, g)$.

```

1: model ACTION-SELECTION( $t, s_t, p_t$ )
2:   parameters:  $\epsilon_a$  (action noise)
3:   if BERNOULLI( $\epsilon_a$ ) = TRUE then
4:     return UNIFORM(ACTIONS( $s_t$ ) \  $p_t[t][s_t]$ )
5:   else
6:     return  $p_t[t][s_t]$ 
7:   end if
8: end model

```

(d) Samples actions from $P(a_t | s_t, p_t)$.

Figure 2.1: Boundedly rational agent model. (a) A single realization of the model. At $t=1$, the agent samples a search budget η_1 and searches for a plan p_1 that is two actions long. At $t=2$, no additional planning needs to be done, so p_2 is copied from p_1 , as denoted by the dashed lines. The agent then replans at $t=3$, sampling a new search budget η_3 and an extended plan p_3 with three more actions. (b-d) Generative subroutines specifying (b) goal transition noise, (c) plan updates, and (d) action selection.

This plan update procedure is shown in Figure 2.1c¹, and it captures boundedly rational planning in the following ways: (i) The agent does not update its previous plan (i.e., a sequence of intended actions) if it already extends to the current step t (Line 8). (ii) If the plan *does* need to be extended, the agent only spends a limited budget η to construct a new partial plan $\tilde{p}_t$. For example, if the agent plans via forward search, η is the number of steps the agent thinks ahead. We sample η from a negative binomial distribution (Line 4), encoding the assumption that neither very short nor very long plans are likely. (iii) We assume that the planning procedure itself, PLANNER, is noisy (Line 5). While in principle, any planning algorithm could be used, we adopt a probabilistic version of A* search, capturing the intuition that humans often plan by thinking a few steps ahead, guided by a heuristic $h(s, g)$ that evaluates how promising a state s is relative to the goal g . In regular A* search, only the most promising state s is expanded at each iteration. However, since humans may not rank states perfectly, we instead sample s from the Boltzmann distribution $P_{\text{expand}}(s) \propto \exp(-f(s, g)/\gamma)$, where higher γ increases randomness, $c(s)$ is the cost of reaching s from the initial state, and $f(s, g) = c(s) + h(s, g)$ is the estimated total cost of reaching g by passing through s . The search algorithm terminates when either the goal state g is reached or when the plan budget is exhausted, at which point a partial plan $\tilde{p}_t$ to last-expanded state s is returned.

¹The main distinction from the PLAN-UPDATE procedure in Chapter 1 is that here we do not model plan refinement.

2.1.3 Mistaken Actions via Execution Errors

Humans may not always execute plans as intended. Instead, due to carelessness or lack of motor control, we may sometimes commit execution errors, e.g., dropping a block by accident, or walking a step more than intended. Hence, we model action selection (Eq. 2.4, Figure 2.1d) as a process where the agent usually executes their intended action given the current plan p_t and state s_t (Line 6), but with probability ϵ_a randomly executes another possible actions in state s_t (Line 4).

2.1.4 Bayesian Goal Inference

We model observers as performing Bayesian inference over an agent’s intended goal g_0 given observations $o_{1:t}$. We assume that the agent and observer have a shared model of the environment, specified in PDDL [46]. Observations are provided as symbolic predicates, and to model observation noise, Boolean predicates are corrupted with probability ϵ_o , while numeric predicates have Gaussian noise added with variance σ_o^2 . As in Chapter 1, we use SIPS to approximate the intractable posterior $P(g_0|o_{1:t})$.

As baselines for comparison, we compute goal inferences using lesioned agent models: **G-Lesioned**, where goal mistakes are absent, **P-Lesioned**, where resource bounded planning is absent, or **A-Lesioned**, where action mistakes are absent. We also compare with the **Boltzmann agent model** used in earlier BToM approaches [6] and Bayesian Inverse Reinforcement Learning [48]. In this model, agents precompute the expected future reward $V(s)$ of every state s , and follow a Boltzmann policy, noisily selecting actions that tend to maximize reward:

$$\pi(a|s) \propto \exp(\alpha[R(s, a, s') + V(s')]) \quad (2.7)$$

Here, s' is the successor state, $R(s, a, s')$ is the reward from taking action a from s to s' , and higher α leads to lower noise. We set $R(s, a, s') = -1$ for all actions and treat goal states as terminal, leading agents to prefer shorter routes to goals. Because this model exhaustively plans (i.e. computes the expected reward $V(s)$) over the entire state space, it only accounts for low-level action mistakes. As such, we hypothesize that it will not explain mistaken goals or plans as well as our boundedly-rational model.

2.2 Experiments

To demonstrate the generality of our model, we conducted experiments in two domains: (i) a gridworld puzzle called Doors, Keys & Gems, and (ii) a Blocks World variant called Block Words, where an agent spells words out of lettered blocks. These domains exhibit the compositional structure that humans encounter in daily life, making them tractable to plan in, but also complex enough for mistakes to arise.

In each domain, we elicited goal inferences from human observers as they watched a variety of optimal and suboptimal agent trajectories unfold. Given the complexity of our model, sub-optimal trajectories might admit multiple interpretations: “Was that a mistaken action, or a mistaken plan?” As such, we designed trajectories to make some mistakes more likely than others. For example, if someone walked a step out of the house before turning to get their keys, an observer might take that as a mistaken action, but if they walked all the way to the bus stop before turning around, a

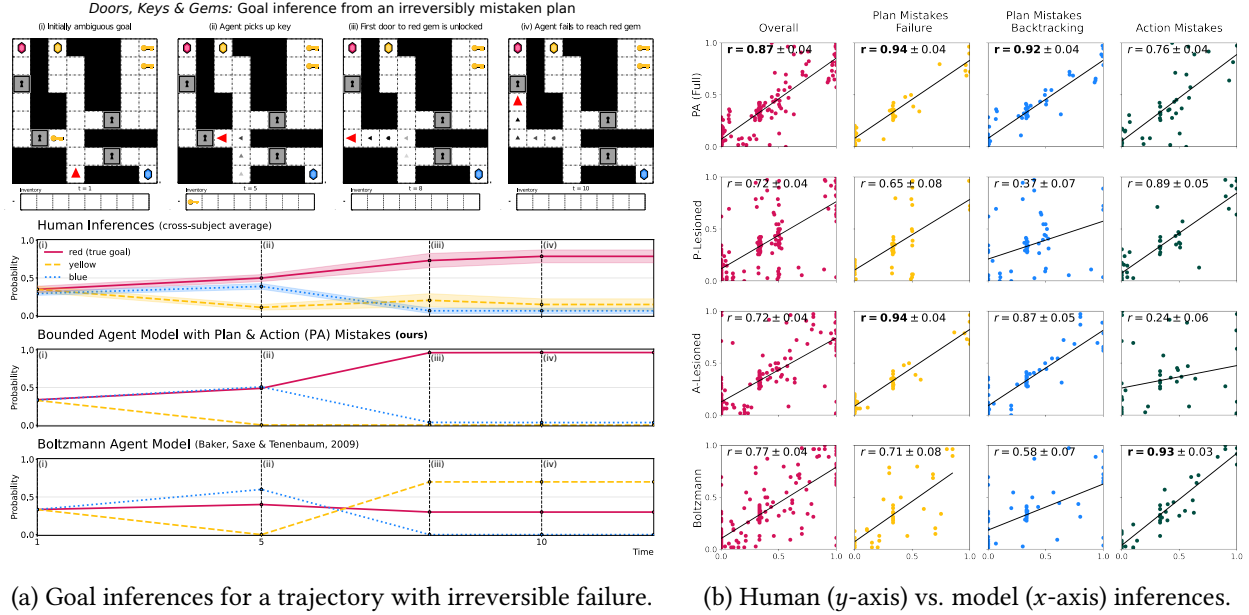


Figure 2.2: Results for the Doors, Keys & Gems domain. In (a), we show goal inferences of humans (avg., $n=18$, w. standard error), our model (PA) and the Boltzmann agent model for an illustrative trajectory. In (b), we compare human vs. model inferences across models (full model, lesioned models for plan/action mistakes, Boltzmann model) and stimulus types.

mistaken plan would seem more likely. In the Block Words domain specifically — which has a richer goal space — we also designed trajectories with mistaken goals: agents would sometimes stack a misspelled word, before correcting themselves.

2.2.1 Experiment 1: Doors, Keys, & Gems

In this domain, an agent must navigate a maze in order to collect one of three colored gems, which may be locked behind doors (Figure 2.2(a)). Keys are required to unlock doors, and can only be used once, leading to the possibility of irreversible failure if the agent does not plan ahead. We assume that planning is guided by a maze distance heuristic that ignores the presence of doors, leading occasionally to myopic plans which neglect the need for keys. Because the goals in this domain are simple, we do not model mistaken goals.

Stimuli. We designed 16 agent trajectories as stimuli for human participants, organized into four subsets: (1) A control set of optimal trajectories. (2) Trajectories with irreversibly mistaken plans, where the agent myopically uses up all obtainable keys, such that the goal gem is locked out of reach (Figure 2.2(a)). (3) Trajectories with mistaken actions, where the agent takes a few false steps, then corrects their behavior. (4) Trajectories with short-sighted plans, leading the agent to backtrack to obtain keys. Participants accessed a web interface which presented all 16 stimuli in random order. Stimuli were presented as animated videos which paused at selected judgement points. At each point, participants provided goal inferences by selecting the gem(s) they believed to be the agent’s most likely goal. Participants could select multiple gems if more than one seemed equally likely, and these responses were converted to probability distributions.

Participants. We recruited 20 US participants (mean age 36.4, SD 10.4; 8 women, 12 men, 0 non-binary/other) via Amazon Mechanical Turk (AMT), restricting to those with a HIT approval rate of 99% and above. Participants went through a tutorial and answered four comprehension questions before viewing the stimuli. Participants also earned points proportional to the probability they gave the true goal (mean score 28, SD 10), and were paid \$1 per 10 points, incentivizing accurate guesses. Two participants were excluded, either for failing two or more comprehension checks, or for guessing indiscriminately and failing to reach a threshold of 10 points.

Analysis. We fit the parameters of each model via grid search, maximizing the overall correlation (Pearson’s r) between average human judgements and model inferences. Model inferences were computed by averaging 10 runs of the SIPS particle filter [5] with 300 particles per run. We then computed 95% bootstrap confidence intervals for the r values using 500 resamples of the human data. We also conducted a sensitivity analysis to validate whether any the models performed better just because of overfitting. Details and model parameters can be found in the Appendix.

Results. Figure 2.2 shows the results of our analysis. An illustrative example from the Doors, Keys, & Gems domain is shown in Figure 2.2(a), where an agent locks their desired gem out of reach due to myopic planning. The panels show how the agent mistakenly uses up a key to unlock the first door to the red gem, instead of collecting the other two keys. They then approach the second door, and are stuck. Below these panels, we show average human goal inferences over time (with standard error ribbons), alongside the inferences for our boundedly-rational agent model and the baseline Boltzmann agent model. Humans are able to recognize the true goal as soon as the first door is unlocked ($t=8$) despite knowing that the agent won’t be able to reach it. Our model, which accounts for plan and action (PA) mistakes, exhibits highly similar behavior to humans, placing high confidence in the red gem once the door is unlocked ($t=8$). For the Boltzmann agent model, however, the probability of the red gem decreases at $t=8$, and more weight goes to the yellow gem, since it becomes the only gem reachable without any keys.

We present correlation plots between average human and model inferences in Figure 2.2(b). The left column shows correlations across all 16 stimuli, with our full model (PA) best explaining human inferences ($r=0.87 \pm 0.04$). The next two columns show correlations for stimuli with mistaken plans, leading to failure and backtracking respectively. In both cases, our full model fit the human data best. The final column shows correlations only for stimuli with action mistakes, and here our model performed slightly worse than models that account only for action noise (P-Lesioned, Boltzmann). Notably, however, these models performed much more poorly on mistaken plans, mirroring how the A-Lesioned model performed much worse with mistaken actions. Sensitivity analysis (Figure 2.4(a)) indicated that the high correlation for our full model was not just due to overfitting: Apart from a few parameter settings, our full model achieved consistently higher correlations than the Boltzmann or lesioned models.

2.2.2 Experiment 2: Block Words

In this domain adapted from Ramírez and Geffner [8], blocks are labeled with letters, and an agent may pick up a block, put it down, or stack it on top of another. The agent’s goal is to stack a block tower that spells (top-down) one out of five English words, which are provided to the observer in advance. We assume that planning is guided by a domain-general relaxed distance heuristic [99], leading agents to occasionally neglect how stacking one block on another will prevent the block

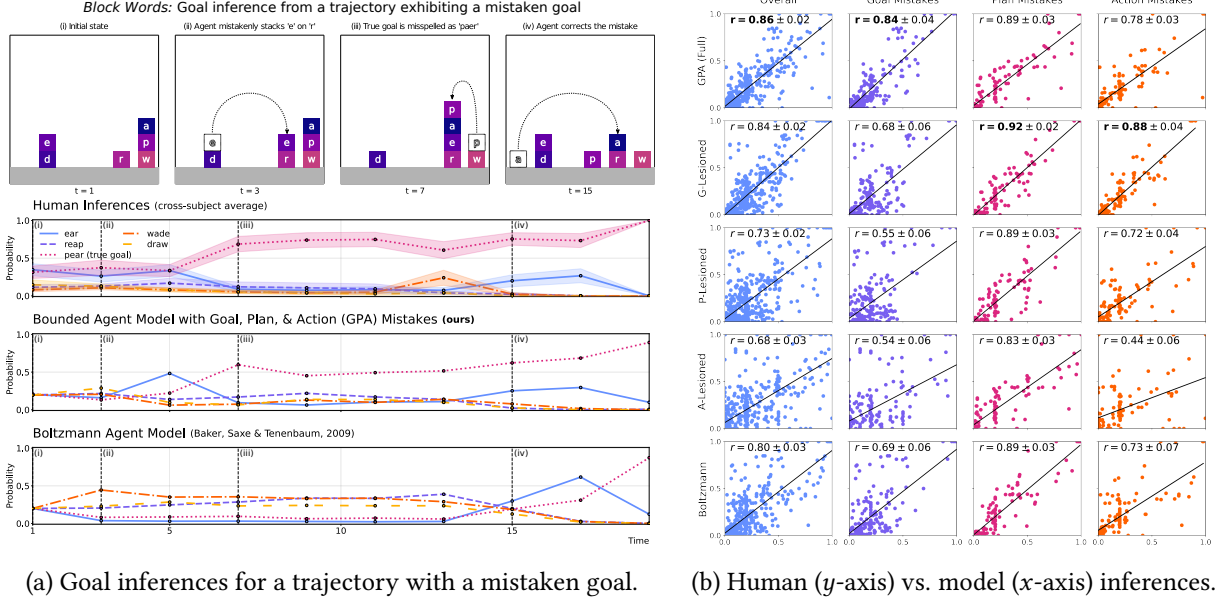


Figure 2.3: Results for the Block Words domain. In (a), we show goal inferences of humans (avg., $n=27$, w. standard error), our model (GPA) and the Boltzmann agent model on an illustrative trajectory. In (b), we compare human vs. model inferences across models (full model, lesioned models for goal/plan/action mistakes, Boltzmann model) and stimulus types.

underneath from being reached. Due to the complexity of goals in this domain, we account for temporary goal confusion, where the agent tries to spell a random permutation of the original word, for example, “paer” instead of “pear” (Figure 2.3(a)).

Stimuli. As in Experiment 1, we designed 16 stimuli, organized into four subsets: (1) A control set of optimal trajectories. (2) Trajectories with mistaken goals (misspellings of the intended block tower). (3) Trajectories with mistaken plans, where the agent stacks a block on top of other block(s) it will later need. (4) Trajectories with mistaken actions, where the agent drops a block in an unintended location, or picks up a block adjacent to the one intended. Participants accessed a web interface which presented 10 stimuli in random order, with at least two stimuli from each subset. Stimuli were presented as animations which paused every two actions (picking and placing a block). At these pauses, subjects selected which word(s) they believed to be the most likely goal. Participants could select multiple words if they seemed equally likely, and these responses were converted to probability distributions.

Participants. We recruited 32 US participants (mean age 40.8, SD 12.5; 13 women, 19 men, 0 non-binary/other) via AMT, restricting to those with a HIT approval rate of 99% and above. Participants went through a tutorial and answered five comprehension questions before proceeding to the stimuli. Following Experiment 1, we awarded points proportional to the probability they assigned to the true goal (mean score 37, SD 7). Five participants were excluded from our analysis, either for failing two or more comprehension checks, or for failing to reach a threshold of 20 points.

Analysis. Similar to Experiment 1, we fit the parameters of each model via grid search, computed 95% bootstrap confidence intervals, and conducted a sensitivity analysis by examining how r values changed across parameter sets. Model inferences were computed by averaging 10 runs of

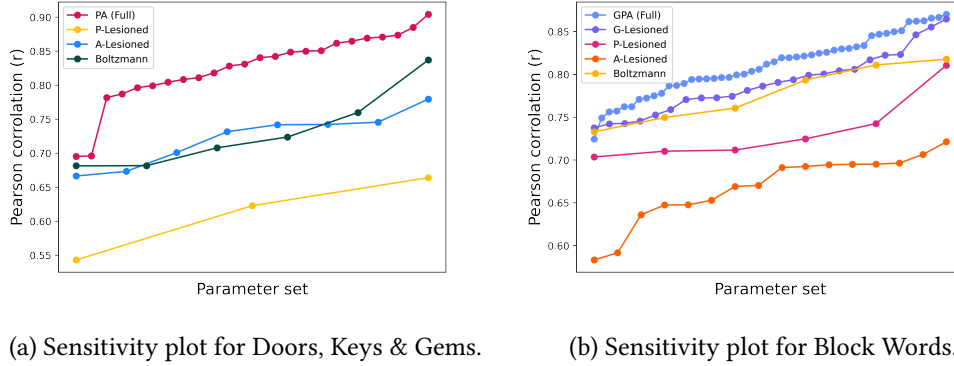


Figure 2.4: Sensitivity plots for human-model correlations (Pearson’s r) with respect to model parameters. For each model, we plot r values for each parameter set that we performed grid search over, ordering them from low to high in terms of correlation.

the SIPS particle filter with 500 particles per run (100 particles per potential goal, as in Experiment 1). Details and parameters can be found in the Appendix.

Results. The results of our analysis are presented in Figure 2.3. We show an illustrative stimulus with goal confusion in Figure 2.3(a). The panels show the agent intending to spell the word “pear” but misspells it as “paer” instead. Below these panels, we show average human inferences over time (with standard error ribbons), alongside the results for our bounded agent model and the baseline Boltzmann agent model. Humans are able to identify the true goal as soon as all the letters in “pear” are stacked, despite the wrong order ($t=7$). When the agent corrects their mistake ($t=15$), humans remain confident in this inference. Our model, which encompasses goal, plan, and action (GPA) mistakes, exhibits similar behaviour, assigning higher credence to “pear” from $t=7$ onwards. In contrast, the Boltzmann agent model fails to account for “paer” as a misspelling, assigning higher probability to a goal like “reap” because it takes less steps to reach from “paer” compared to “pear”.

We compare average human vs. model inferences in Figure 2.3(b). The left column shows the overall correlation across all 16 stimuli, indicating that our full model (GPA) best explains human inferences ($r = 0.86 \pm 0.02$). The next few columns show correlations for each category of mistake. We see that our full model fit human data best when goal mistakes are present, compared to models which do not account for goal errors (G-lesioned, Boltzmann, etc.). The full model also did well with plan and action mistakes, though slightly out-performed by other models. Surprisingly, the Boltzmann model correlated highly even when trajectories exhibited short-sighted planning mistakes. We hypothesize that this is because the plan mistakes in our dataset are sufficiently short-lived (i.e. require backtracking only 1–2 actions) that they are readily attributed to action noise. In contrast, the model without action mistakes (A-lesioned) performed worst across the board, demonstrating the importance of modeling execution errors in this domain.

Sensitivity analysis indicated that the performance of our full model was robust across parameters. As Figure 2.4(b) shows, the full model outperforms the P-Lesioned and A-Lesioned models across all parameter sets. For all but one parameter set, our full model also dominates the G-Lesioned and Boltzmann models in an ordinal sense: For each k , the top k th parameter set outperforms the corresponding parameter sets for the other two models.

2.3 Discussion

Our experimental results suggest that humans are robust to observed mistakes when inferring the goals of others. By comparing collected human inferences against inferences produced by our models, we find considerable support for our hypothesis that human inferences are better explained by a Bayesian Theory of Mind that accounts for mistaken goals, plans, and actions. In the Doors, Keys & Gems domain, we find that models which only account for mistaken actions are not robust to mistaken plans, and vice versa, while in the Block Words domain, we find that models which do not account for goal confusion result in poor inferences when mistaken goals are observed. Together, these findings indicate that it is important to model *distinct* errors at *multiple* levels of cognition and action in order to understand others' goals, and that people intuitively distinguish these possibilities when observing sub-optimal behavior.

Nonetheless, many open questions remain. For one, what kinds of mistakes do humans understand beyond the three explored here? Prior research has also modeled biases like time inconsistency [79], capability limitations [100], and errors in environment models [101] when inferring the goals or preferences of others. Other cognitive limitations abound. Which of these do humans intuitively apprehend, and which do we not? For another, given the complexity of these models, how do human observers rapidly and tractably infer the goals and mistakes of others? If they draw such inferences via sampling, then could a Monte Carlo inference algorithm such as SIPS explain the inferences made by individual humans? By answering these questions, we hope to build an even richer (theory) theory of our boundedly rational minds.

2.4 Appendix

2.4.1 Experiment interface

Figure 2.5 shows the web interface used by participants. The interface displays animated stimuli in random order, each divided into segments. After observing each segment, participants provide goal inferences by selecting the goal(s) they believe to be the agent's most likely goal, then proceed to next segment. Responses are converted to uniform probability distributions over the model parameters (e.g., if two goals were selected, 50% probability is allocated to each goal). If no single goal seems more likely than the others, participants can choose the *I don't know* option, which is treated as a uniform distribution over all of the potential goals.

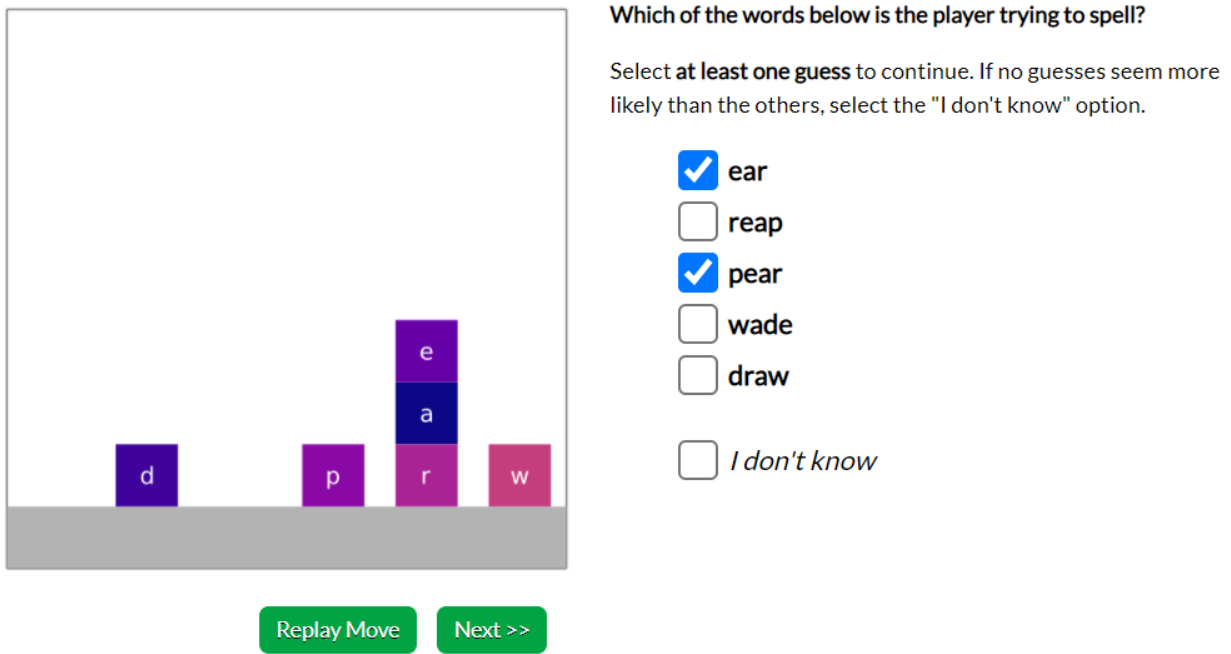


Figure 2.5: Experiment interface for Block Words.

2.4.2 Model fitting

We fit our full model and the lesioned models to maximize human-model correlation by performing grid search over the following sets of parameters:

- Goal noise: $\epsilon_g \in [0.1, 0.2]$
- Parameters for the negative binomial distribution over the planning budget η : $r \in [2, 4]$ and $q \in [0.9, 0.95]$.
- A* search noise: $\gamma \in [0.02, 0.5]$
- Action noise: $\epsilon_a \in [0.05, 0.1, 0.2]$

	ϵ_g	r	q	γ	ϵ_a	α
Full Model	–	2.0	0.9	0.5	0.05	–
G-Lesioned	–	–	–	–	–	–
P-Lesioned	–	–	–	–	0.05	–
A-Lesioned	–	2.0	0.9	0.5	–	–
Boltzmann	–	–	–	–	–	0.125

(a) Model parameters for Doors, Keys & Gems.

	ϵ_g	r	q	γ	ϵ_a	α
Full Model	0.2	2.0	0.9	0.02	0.05	–
G-Lesioned	–	2.0	0.9	0.50	0.20	–
P-Lesioned	0.2	–	–	–	0.20	–
A-Lesioned	0.2	4.0	0.9	0.02	–	–
Boltzmann	–	–	–	–	–	2.0

(b) Model parameters for Block Words.

Table 2.1: Best-fitting model parameters across domains and model types: goal noise ϵ_g , planning budget parameters r and q , search noise γ , action noise ϵ_a , and inverse temperature for the Boltzmann model α .

Across these models, we fixed the observation noise parameters to $\epsilon_o = 0.05$ and $\sigma_o = 0.25$ for Doors, Keys & Gems, and $\epsilon_o = 0.1$ for Block Words. For the A* search heuristic in Doors, Keys & Gems, we used an optimistic maze distance metric which ignored the presence of locked doors, and the Fast Forward relaxed distance heuristic in Block Words [102]. For the Boltzmann agent model, we fitted the inverse temperature parameter α over the range $[0.125, 0.25, 0.5, 1, 2, 4]$. Table 2.1 shows the fitted parameters.

To compute 95% confidence intervals for the fitted r values, we used the bootstrap method, resampling the human data without replacement 500 times and computing r between the average human inferences and the model inferences. This produces an approximately normal histogram over r values, for which we can compute a confidence interval which spans 95% of the samples.

2.4.3 Additional qualitative results

In addition to collecting human goal inferences, we also asked participants to identify whether they believed a mistake occurred after viewing each of the last two stimuli they were presented with. If they stated that a mistake occurred, they were asked to provide a qualitative description of the mistake. For the Doors, Keys & Gems domain, we found that 100% of participants responded correctly when mistakes were absent, and that 81% of participants detected a mistake when present. For Block Words domain, 100% of participants responded correctly when mistakes were absent, and 70% detected a mistake when present, but sometimes could not remember details well enough to describe the mistake. In addition, participants accurately identified action mistakes only 20% of the time. These results validate the intuitive hypothesis that humans can recognize mistakes when they occur, but also suggest that conscious recognition of mistakes is not necessary for accurate goal inference.

Chapter 3

Infinite Ends from Finite Samples

[I]t would be impossible for any mind to comprehend the infinite variety of different needs of different people which compete for the available resources, and to attach a definite weight to each.

Friedrich Hayek

Whether one is watching a play, reading a novel, or spending time with a friend at their house, inferences about others' goals and motivations often arise spontaneously and unbidden [103]: Is the person crouching behind a tree trying to hide from, or spy on someone? Does the strange warrior who has just entered the fray of battle intend to kill the protagonist, or save them? When your friend gets up from the couch and walks to the kitchen, are they getting a snack, or making some tea? Despite the seemingly infinite space of possible goals, we have little trouble in coming up with plausible hypotheses, and then — as the story unfolds — filtering out those that fail to explain our observations. If the warrior defends our protagonist from a stray arrow, they are likely an ally. If your friend opens the fridge, they are probably having a snack. What computational mechanisms underlie this ability to both hypothesize and evaluate the goals that explain others' behavior, even when the set of possibilities is vast and open-ended?

While psychologists have long studied how people both *generate* [104–107] and *evaluate* hypotheses about the goals that other agents have [94, 108, 109], computational models of human goal inference have focused on the latter, assuming a small and *fixed* set of possible goals, then modeling how people infer their relative likelihoods [6, 36, 110–112]. This leaves open how people come up with plausible goals in the first place, especially in large hypothesis spaces where enumeration over all possibilities makes inference intractable [113, 114]. How then are people solving this seemingly intractable problem (if they do so at all)? Even though recent advances such as SIPS (Chapter 1) have shown how modeling the plans of other agents [5, 11] can be made orders of magnitude more efficient, they do not address the key challenge posed by open-ended settings: Efficiently generating plausible goal hypotheses.

In this chapter, we develop an algorithmic account of open-ended goal inference, extending the SIPS algorithm for online goal inference by combining top-down Bayesian inverse planning with bottom-up sampling from fast proposals. Instead of exhaustively enumerating the space of goals, open-ended SIPS assumes that humans are familiar with the statistics of their environments [115],

and can rapidly generate relevant hypotheses based on contextual, data-driven cues [88, 116]. In particular, we assume familiarity with the statistics of co-occurring subgoals, such that complete goals can rapidly be generated once some subgoals have been achieved. Open-ended SIPS then filters these goals according to the principle of rational action [6, 108], keeping those that best explain the agent’s actions. We evaluate open-ended SIPS in Block Words, a game where observers have to guess the word that someone is stacking out of lettered blocks [8, 11]. Subgoals correspond to partial words, so observers can generate plausible goals by “auto-completion”. However, this bottom-up strategy is insufficient in general — some goals may be irrational given the actions observed so far, necessitating inverse planning.

To test the predictions of our model, we conduct an experiment where human participants play a series of rounds in Block Words. Each round is carefully designed so as to elicit various patterns of inference — some where bottom-up guessing is sufficient, some where inverse planning is required to filter out irrational goals, and some intended to produce garden-path inferences that exact Bayesian reasoning should avoid. As alternatives to open-ended SIPS, we test pure bottom-up sampling, as well as an exact Bayesian baseline that performs enumerative inference over all English words that can be spelled in each round. We compare these models by measuring their similarity to human responses in terms of their mean, variance, sample efficiency, and computational cost, allowing us to determine their fidelity to human goal inference in both behavioral and algorithmic terms.

3.1 Computational Model

Building upon the framework introduced in Chapter 1, we assume that observers perform approximately Bayesian inference over a generative model of how other agents plan and act to achieve their goals:

$$\text{Goal Prior: } g \sim P(g) \quad (3.1)$$

$$\text{Online Planning: } \pi_t \sim P(\pi_t | s_{t-1}, \pi_{t-1}, g) \quad (3.2)$$

$$\text{Action Selection: } a_t \sim P(a_t | s_{t-1}, \pi_t) \quad (3.3)$$

$$\text{State Transition: } s_t \sim P(s_t | s_{t-1}, a_t) \quad (3.4)$$

Here g is the agent’s goal, and at each step t , π_t is the agent’s current plan or policy, a_t is the agent’s action, and s_t is the state of the environment. Given a sequence of states $s_{0:T}$ and actions $a_{1:T}$, the observer’s task is to infer the goal g by approximating the posterior $P(g | s_{0:T}, a_{1:T})$. Note that unlike the experiments presented in the previous two chapters, we assume that states and actions are directly observed.

Approximating this posterior presents numerous computational challenges. Among these, our focus is on the challenge posed by *open-ended* settings, where the set of possible goals $g \in \mathcal{G}$ is large or potentially infinite. In this section, we briefly review how — under the assumption of deterministic but bounded planning — SIPS renders goal inference over *fixed* spaces algorithmically tractable, allowing us to perform exact Bayesian goal inference by tracking just one hypothesis per goal. We then explain how can extend these ideas to open-ended spaces.

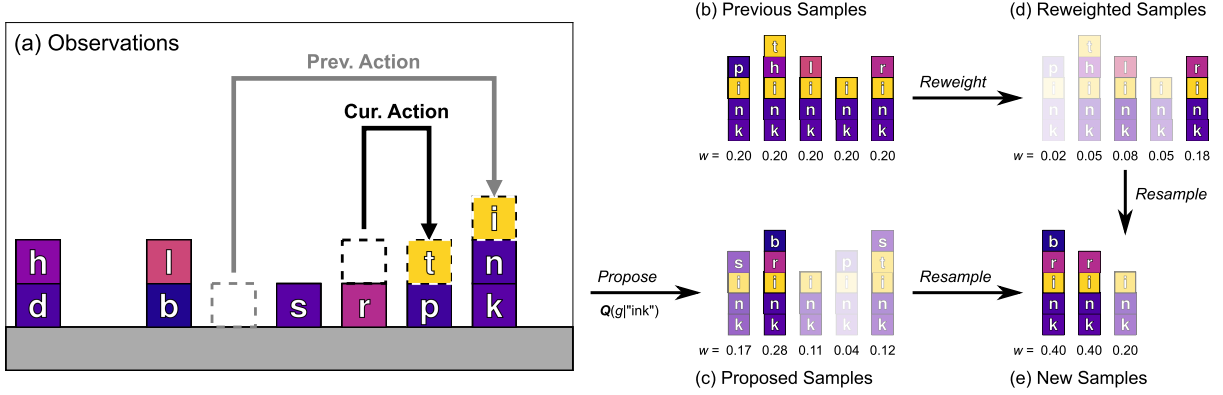


Figure 3.1: Goal inference in Block Words via open-ended SIPS. Initially, **i** is stacked on **n**, leading **pink** to be proposed as a goal. In the next step, however, **t** is stacked on **p**. This makes **pink** much less likely after reweighting, and hence removed after resampling.

3.1.1 Modeling Boundedly Rational Plans and Actions

Since computing the posterior requires simulating the plans π_t that an agent might follow to each goal g , this process is also known as *Bayesian inverse planning*. In general this is a difficult problem, because planning itself is a complicated and often intractable task. However, as Chapter 1 shows, this difficulty can be alleviated by treating agents as *bounded incremental planners*, who spend only limited computation at each step t on planning. We consider a special case of the incremental plan update discussed in Section 1.1.2, modeling agents that *deterministically* update a *policy* π_t (i.e. a conditional plan) that defines a Boltzmann distribution over actions a_t that can be taken at state s_{t-1} :

$$P(a_t | s_{t-1}, \pi_t) \propto \exp \left(-\beta \hat{Q}_{\pi_t}(s_{t-1}, a_t) \right) \quad (3.5)$$

Here $\hat{Q}_{\pi_t}(s_{t-1}, a_t)$ denotes the estimated cost of the shortest path to goal g from s_{t-1} that starts with action a_t . As such, π_t assigns higher probability to actions along more optimal paths to the goal, with β controlling the degree of optimality. To compute $\hat{Q}_{\pi_t}(s_{t-1}, a_t)$ efficiently, we use real-time heuristic search (RTHS), which updates the $\hat{Q}_{\pi_{t-1}}$ values computed for π_{t-1} via a search process guided by the Q -values themselves [53, 55, 56]. This process is limited to a computational budget of up to size B (e.g. the number of search iterations). As such, given $G = |\mathcal{G}|$ possible goals, the computational cost of simulating the plans of an agent for T steps (and hence exactly inferring their goals) is $O(GBT)$.

3.1.2 Bottom-Up Sampling of Plausible Goals

While the analysis above suggests that Bayesian inverse planning is not only tractable, but *linear* in computational complexity, it neglects the fact that the number of goals G can grow very large. As suggested by Blokpoel et al. [114], this might be because G itself grows exponentially with some other natural parameter — in Block Words, for example, just 9-11 lettered blocks can be used to spell anywhere from 150 to 800 English words. But even without this exponential dependence, a large value of G can quickly render (exact) goal inference too costly to be algorithmically plausible.

How might people manage the complexity of their inferences in these open-ended settings? We posit that in sufficiently familiar contexts, people are familiar with the statistics of co-occurring

subgoals, such that given some subgoal y_i , they can rapidly sample a complementary subgoal $y_j \sim P(y_j|y_i)$. This means that once an agent achieves some subgoal y_i — say, boiling a kettle of water, or stacking the letter $\boxed{n}$ on top of $\boxed{g}$ — an observer can rapidly generate a complete goal $g = y_i \wedge y_j$ — perhaps adding tea to the boiled water, or spelling the word $\boxed{s} \boxed{o} \boxed{n} \boxed{g}$. These conditional distributions can be efficiently learned using either neural networks or classical sequence models such as n -grams. Since the goals in our study are English words, we use a character-level n -gram model. Regardless of what sequence model is used, a key property is that retrieval and sampling can occur in essentially *constant* time [117], providing a plausible mechanism for relevance-guided hypothesis generation [88, 116].

3.1.3 Bayesian Filtering of Bottom-Up Samples

Given the ability to rapidly generate plausible goals, it is tempting to forgo inverse planning altogether. As Figure 3.1 illustrates, however, this strategy can go awry. Suppose you see someone stack the block $\boxed{i}$ on top of $\boxed{n} \boxed{k}$, and the word $\boxed{p} \boxed{i} \boxed{n} \boxed{k}$ comes to mind. But then you see $\boxed{t}$ stacked on top of $\boxed{p}$. Is $\boxed{p} \boxed{i} \boxed{n} \boxed{k}$ still a plausible goal? From a bottom-up perspective, $\boxed{p}$ is still a likely completion of $\boxed{i} \boxed{n} \boxed{k}$. But if we understand agents as *rational* planners, this no longer seems likely. If $\boxed{p} \boxed{i} \boxed{n} \boxed{k}$ had been the agent’s goal, stacking $\boxed{t}$ on $\boxed{p}$ would be quite suboptimal.

If humans actually engage in the reasoning above, then modeling their inferences requires uniting top-down Bayesian inverse planning with bottom-up cues. Following other sampling-based accounts of sequential human inferences [75, 118, 119], we model this integration with a sequential Monte Carlo (SMC) algorithm (Algorithm 2), extending the SIPS algorithm presented in Chapter 1. SMC algorithms are also known as particle filters, which approximate Bayesian posteriors by maintaining a weighted set of hypotheses or particles, then updating the weights of those particles as observations arrive [120]. At each step, they may also *resample* particles according to their weights, or *rejuvenate* the particles, making perturbations to the sample collection to increase hypothesis diversity [121, 122].

A variant of this rejuvenation phase is where our bottom-up samplers come in: As Algorithm 2 shows, after observing each state s_t and action a_t at step t , we use these samplers as *proposal distributions* over goals $Q(g|s_t, a_t)$, generating N new goal hypotheses $\tilde{g}^i$ (L10) based on bottom-up cues. These new hypotheses assigned an importance weight $P(\tilde{g}^i, \tilde{\pi}_{1:t}^i, s_{0:t}, a_{1:t})/Q(\tilde{g}^i|s_t, a_t)$, where the numerator $P(\tilde{g}^i, \tilde{\pi}_{1:t}^i, s_{0:t}, a_{1:t})$ accounts for how well the plans $\tilde{\pi}_{1:t}^i$ that lead to $\tilde{g}^i$ explain the actions $a_{1:t}$, and the denominator $Q(\tilde{g}^i|s_t, a_t)$ compensates for $\tilde{g}^i$ having been sampled from the proposal (L11–12). Open-ended SIPS also *reweights* previous samples based on how well they

Algorithm 2 Open-Ended SIPS

```

1: procedure OPEN-ENDED-SIPS( $s_{0:T}, a_{1:T}, N$ )
2:   using:  $Q(g|s_t, a_t)$ 
3:    $\hat{P} \leftarrow \{\}$ 
4:   for  $t$  in  $[1, T]$  do
5:     for  $i \in [1, |\hat{P}|]$  do ▷ Update  $|\hat{P}|$  old particles.
6:        $\pi_t^i \leftarrow \text{UPDATE-POLICY}(\pi_{t-1}^i, g^i)$ 
7:        $w^i \leftarrow w^i \cdot P(a_t|s_{t-1}, \pi_t^i)$ 
8:     end for
9:     for  $i \in [1, N]$  do ▷ Propose  $N$  new particles.
10:       $\tilde{g}^i \sim Q(g|s_t, a_t)$ .
11:       $\tilde{\pi}_{1:t}^i \leftarrow \text{SIMULATE-POLICY}(\tilde{g}^i, t)$ 
12:       $\tilde{w}^i \leftarrow \frac{P(\tilde{g}^i, \tilde{\pi}_{1:t}^i, s_{0:t}, a_{1:t})}{Q(\tilde{g}^i|s_t, a_t)}$ 
13:    end for
14:     $\hat{P} \leftarrow \text{RESAMPLE}(\hat{P} \cup \{(\tilde{g}^i, \tilde{\pi}_{1:t}^i, \tilde{w}^i)\}_{i=1}^N, N)$ 
15:     $\hat{P} \leftarrow \text{COALESCE-IDENTICAL}(\hat{P})$ 
16:  end for
17:  return  $\hat{P}$ 
18: end procedure

```

explain the current action a_t (L6–7). Finally, we resample the particle collection back down to N samples (L14), coalescing identical samples by summing their weights (L15). Our algorithm thus implements the high-level logic described earlier: Incrementally generate plausible hypotheses, evaluate them, then filter out those that do not make sense. This can be viewed as an epistemic analogue to recent accounts of open-ended decision making [88, 123].¹

3.2 Experiments

We evaluated open-ended SIPS as a model of human goal inference on a set of 16 scenarios in Block Words, a variant of the classic Blocksworld domain where the goal is to *infer* the word that an agent is spelling by stacking a tower of lettered blocks. In contrast to previous Block Words tasks [8, 11, 70, 125], we did not specify a fixed set of 5 to 20 goal words. Instead, we told participants that the goal might be any English word between 3 to 8 letters long, with the implied restriction that the word had to be spelled out of the available blocks.

Structure & Design Participants were first shown the initial layout of the blocks. They could then advance the scenario, watching several actions play out as an animated video. The video would then pause at a judgment point, giving participants time to guess the word being spelled via text box entry. Participants could *add* as many guesses as they liked, and also *remove* any previous guesses that they no longer considered likely. They could then advance to the next judgment point, continuing in this way until the end of the scenario. Each participant was presented 8 out of the 16 scenarios, after first completing a tutorial and a comprehension quiz. To incentivize high quality responses, we paid participants a reward based on the accuracy of their guesses ($\$0.1/n$ for every correct answer out of n guesses), and presented the bonus point breakdown after they completed each scenario.

Experimental Conditions. To tease apart the predictions of our model from those that would be made by either exact Bayesian inference or pure bottom-up proposals, we designed our 16 scenarios to fall into one of four conditions:

Bottom-Up Friendly. Words are stacked more-or-less linearly, such that it is sufficient to guess words that complete either the most recently stacked tower, or any partial word.

Irrational Alternatives. Blocks are stacked so that some bottom-up guesses are made irrational, like our

p	i	n	k
---	---	---	---

 example from Figure 3.1.

Garden Paths. Cases where bottom-up guessing suggests a plausible but misleading interpretation of the first few actions, which turn out to be merely instrumental for the true goal.

Uncommon Words. The true goal is either a longer or more uncommon word (aft, chump, wizard, banish), which people and bottom-up proposals might find difficult to initially guess. Otherwise similar to the first condition.

¹While this algorithm has $O(T^2)$ runtime due to rejuvenation, in practice runtime is closer to $O(T)$ due to reuse of likelihood computation. A slight variant can guarantee $O(T)$ runtime by forgetting previous observations [124].

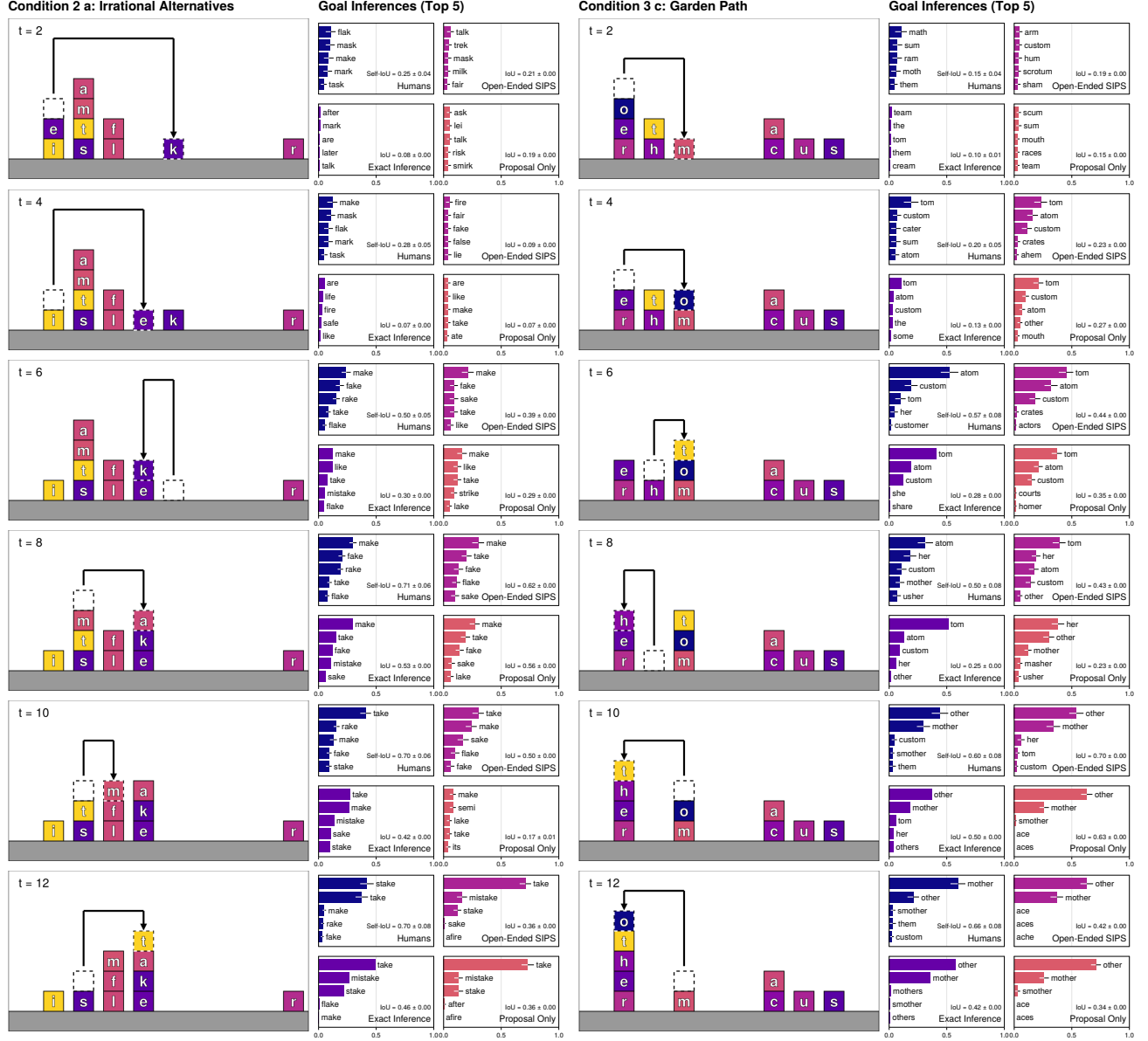
Participants. We recruited 100 US participants fluent in English via Prolific (mean age 39.4; 44 women, 54 men, 2 non-binary), such that every scenario was completed by 50 individuals. Participants were paid US\$15/hr along with the bonus described earlier. Familiarity with word games varied, with 17 reporting that they played word games daily, 22 weekly, 30 every 1-2 months, 11 yearly, and 20 almost never.

Despite comprehension checks, a subset of participants did not follow instructions correctly, either because they never updated their guesses (36 out of 800 scenario responses), or only added guesses without removing them (139 out of 800). We excluded such responses from our analysis.

Model Configuration. We implemented open-ended SIPS using the particle filtering extension of the Gen.jl probabilistic programming framework [4, 20], and the Blocksworld domain in the Planning Domain Definition Language [46, 47]. We fit parameters via grid search to improve model similarity with humans as measured by the intersection over union (IoU) between distributions (see Appendix), which gave an inverse temperature of $\beta = 1.0$, planning budget of $B = 100$, and prior $P(g)$ fitted to tempered word frequencies from the wordfreq library [126], using the 3of6game word list as our dictionary [127]. We ran open-ended SIPS with $N \in \{2, 5, 10, 20, 50\}$ particles, taking the mean and variance over $M = \max(10, 200/N)$ trials. We describe the proposals $Q(g|s_t, a_t)$ in the next section.

Alternative Models. As alternatives to our hybrid SMC model, we tested both (i) exact inference via fully enumerative Bayesian inverse planning over all valid English goal words (145–807 words, depending on the scenario) and (ii) pure bottom-up sampling using a subgoal-conditioned proposal. Exact inference was implemented using the same model parameters as open-ended SIPS, except that all goals were considered as hypotheses from the outset.

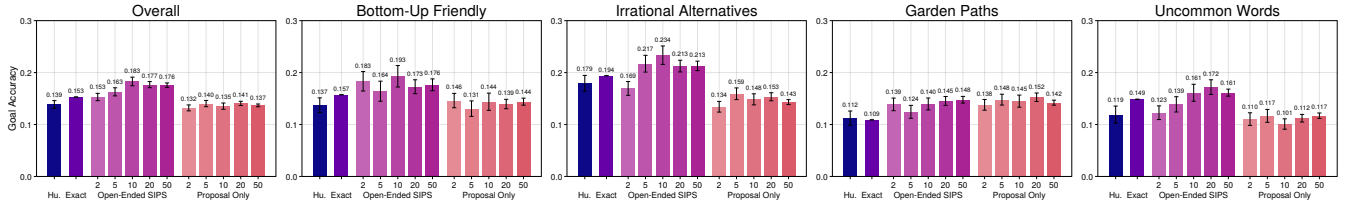
For the bottom-up proposal, we sampled complete words g by conditioning an n -gram model on some partial word that can be stacked from blocks in the current state s_t . We used $n = 5$, fitting the n -gram on the same tempered word frequencies used for the prior $P(g)$. To decide which partial word to complete, the proposal first considers the tower stacked by the last action a_t , sampling a completion if it is sufficiently word-like (as determined by the n -gram model). If not, it considers if the last block was moved because the agent intended to reach some block *underneath* it, and tries to form a word with one of those blocks. If no way of using those blocks is sufficiently word-like, the proposal samples a random tower in state s_t (weighted by how word-like it is), then samples a completion. Other proposals are discussed in the Appendix.



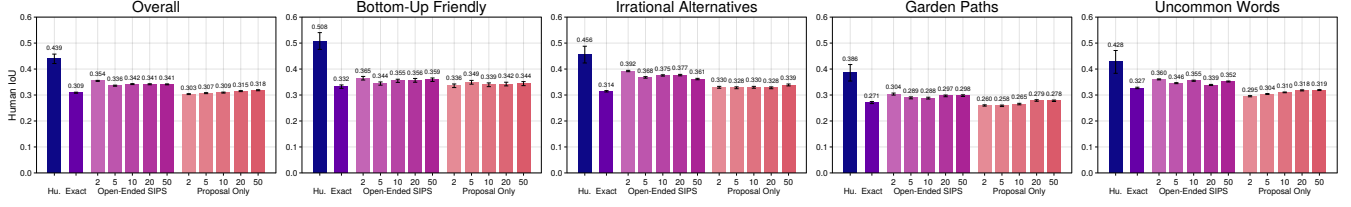
(a) Irrational Alternatives scenario: **stake**

(b) Garden Path scenario: **mother**

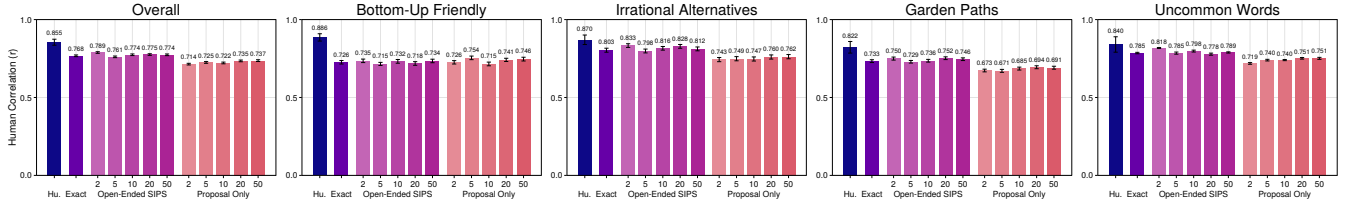
Figure 3.2: Step-by-step inference results on two illustrative Block Words scenarios. On the left, we show the sequence of actions, and on the right, the 5 most probable goals at each step for humans and our models ($N = 2$ for the sampling-based methods), averaged across humans and algorithm runs (error bars reflect the standard error). In (a), only the bottom-up proposal fails to infer that **m** being stacked on **f** at $t = 10$ implies that **make** and **fake** are less likely than **take**. In (b), both open-ended SIPS and humans exhibit sticky inferences at $t = 8$, assigning high weight to **atom** and **custom** as guesses as a result of the garden path trajectory. In contrast, the bottom-up proposal displays a recency bias since it does not store previous guesses.



(a) Average goal accuracy (posterior probability of true goal) across humans, models, and conditions.

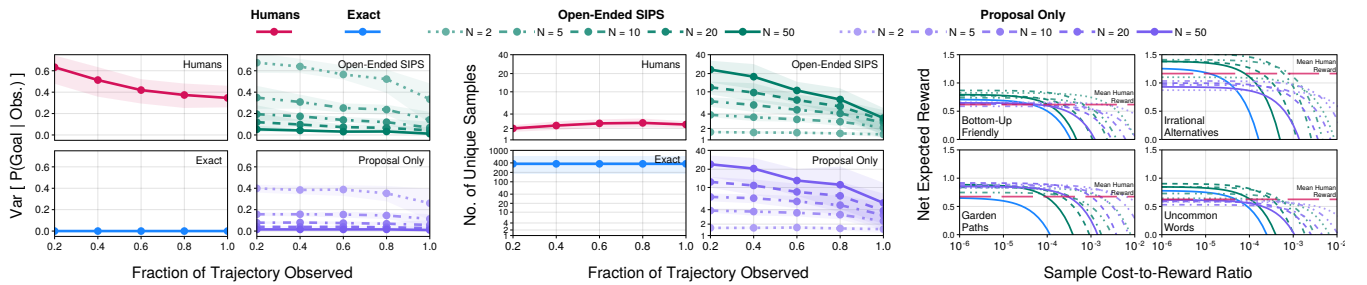


(b) Intersection over Union (IoU) between model inferences and the mean human distribution of guesses.



(c) Pearson's correlation r between human inferences and model inferences across conditions.

Figure 3.3: Human similarity and accuracy of open-ended goal inference models, measured in terms of (a) average goal accuracy, (b) IoU with mean human inferences, and (c) Pearson's r with mean human inferences. Each bar corresponds to a model (and sample size N), while each column is a condition. We computed human-human IoU through repeated sampling of 50-50 splits. Error bars denote 95% confidence intervals.



(a) Total variance of goal inferences.

(b) Sample efficiency.

(c) Reward minus cognitive cost.

Figure 3.4: Algorithmic-level properties of open-ended SIPS vs. humans and baseline algorithms. Ribbons show 10th-90th quantiles. (a) Total variance of the goal probability estimates produced by each method vs. humans, averaged over scenarios. (b) Sample efficiency of inference methods over time, measured by the number of *unique* tracked hypotheses (averaged over scenarios). (c) Expected reward minus cognitive cost, as a function of the ratio of sample cost to the reward of accurate goal inference.

3.2.1 Human Similarity

We analyzed human responses and model outputs by comparing them in terms of distribution similarity (Fig. 3.3b), average accuracy (Fig. 3.3a), step-by-step inferences (Fig. 3.2), response variance (Fig. 3.4a), sample efficiency (Fig. 3.4b), and resource rationality (Fig. 3.4c). Additional results (e.g. accuracy vs. runtime) and sensitivity analyses are in the Appendix.

Open-ended SIPS is most similar to human inferences across all conditions. As we predicted, human inferences showed the highest similarity with open-ended SIPS (IoU = 0.33–0.36 for all N) compared to exact inference (IoU = 0.31) or bottom-up guessing (IoU = 0.30–0.32), with $N = 2$ samples being the most similar. Notably, open-ended SIPS was more similar to humans in the *Irrational Alternatives* condition, with both achieving considerably higher accuracy than the bottom-up only heuristic, indicating that humans indeed engage in inverse planning. Our model was also more similar to humans than exact inference, especially in the *Bottom-Up Friendly* and *Garden Path* conditions, consistent with the hypothesis that humans engage in sampling.

Step-by-step human inferences are best matched by open-ended SIPS. The step-by-step comparisons in Figure 3.2 help to elucidate these aggregate findings. On one hand, open-ended SIPS and the proposal-only model make initial guesses that are biased towards words that complete the first few stacked letters, whereas the exact posterior is much more uncertain. On the other hand, humans account for the rationality of the observed actions when drawing inferences (e.g. Figure 3.2(a), $t = 10$), just like our exact and approximate Bayesian inverse planning algorithms.

Our model’s algorithmic properties best explain human variance and guess counts. In Figure 3.4, we compare the *algorithmic* properties of the models. Human variance was best matched by open-ended SIPS with $N = 2$. Bottom-up proposals had lower variance, and did not prune samples as effectively as sample-matched counterparts. Exact inference is zero-variance, but at the cost of tracking drastically more hypotheses. As such, it was dominated by open-ended SIPS in terms of net reward when accounting for cognitive costs (Fig. 3.4c). The comparison with pure bottom-up proposals was more nuanced. If reweighting a sample via inverse planning is costly enough, pure bottom-up guessing can be more resource-rational [128]. However, there is a large range of cost-ratios where it pays to do inverse planning. Since humans attained more reward than all proposal-only baselines in the *Irrational Alternatives* condition, this suggests that they indeed find inverse planning worthwhile.

3.2.2 Accuracy and Runtime

While our analysis in this chapter focused on open-ended SIPS as a rational process model of human goal inference, our algorithm can also be used as a practical tool for building AI systems that better infer people’s goals in order to assist them [129]. To that end, we compare the accuracy-runtime tradeoffs of both open-ended SIPS and the baseline methods. All experiments were conducted on a laptop with an i7-1370P 1.90 GHz CPU and 64 GB of RAM.

Effect of planner configuration. In Figure 3.5, we show how the accuracy of goal inference changes as a function of the planner configuration used in the generative model. Accuracy is plotted against algorithm runtime per observed action. We find that accuracy generally increases with planning budget, indicating the importance of spending enough computation on calculating good $\hat{Q}_{\pi_t}$ estimates, which improves the quality of the action likelihood $P(a_t|g)$. The effect of the search strategy is more subtle. Using A^* search as the RTHS search strategy can *improve*

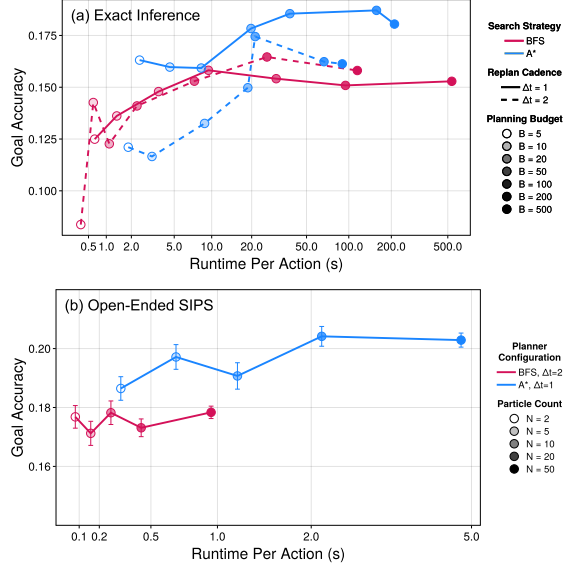


Figure 3.5: Accuracy vs. runtime for (a) exact inference and (b) open-ended SIPS across planner configurations.

Method	Search Strat.	Sample Count	Runtime / Act. (s)	Accur. $P(g_{\text{true}})$	Accur. Std. Dev.
Exact	BFS	—	7.31	0.153	<i>0.000</i>
Inference	A*	—	37.95	0.186	<i>0.000</i>
OE-SIPS	BFS	2	0.08	0.177	0.218
		5	0.16	0.171	0.140
		10	0.26	0.178	0.095
		20	0.44	0.173	0.069
		50	0.95	0.178	0.049
OE-SIPS	A*	2	0.32	0.186	0.225
		5	0.67	0.197	0.146
		10	1.18	0.191	0.108
		20	2.14	<i>0.204</i>	0.080
		50	4.73	0.203	0.055
Proposal	—	2	<i>0.0004</i>	0.145	0.171
Only		5	0.0006	0.147	0.104
		10	0.0008	0.139	0.073
		20	0.001	0.147	0.052
		50	0.003	0.146	0.035

Table 3.1: Accuracy and runtime across inference methods, search strategies, and sample counts. **Bold:** Best in each method. *Italicized:* Best across methods.

accuracy when $\Delta t = 1$ (i.e. when the policy is updated at every timestep). However, this leads to an increased runtime compared to BFS for any given planning budget B , with A^* being 2-5 times slower than BFS for low planning budgets. Still, using A^* with $\Delta t = 1$ achieves higher accuracy than BFS ever does, indicating its value when accuracy is paramount. Unlike the planning budget, increasing the particle count N does not appear to substantially improve the average accuracy of open-ended SIPS, with changes in planner configuration dominating any accuracy improvement from additional particles.

Overall comparison. In Table 3.1, we show accuracy and runtime measures across all inference methods and particle counts. For the inverse planning methods, we fixed the planning budget to $B = 100$, but compare both the accuracy-maximizing planner configuration ($\sigma = A^*$, $\Delta t = 1$) and the runtime-minimizing configuration ($\sigma = \text{BFS}$, $\Delta t = 2$). While exact inference has zero variance, the cost of tracking all goal hypotheses leads to an unacceptably high runtime for many applications. The bottom-up proposal is on the opposite end of the spectrum, achieving millisecond or less runtimes but with a subpar accuracy that does not increase with particle count. In contrast, open-ended SIPS combines the best of both worlds, achieving the highest goal accuracies while maintaining real-time speed. Changing the particle count N trades off variance in inference results vs. runtime. By using $N = 20$, the variance in the probability estimate of the true goal can also be limited to less than 10% while still taking less than half a second process each new observation. These results demonstrate the promise of open-ended SIPS as a practical algorithm for real-time open-ended goal inference.

3.3 Discussion

In comparison to alternative models, our sampling-based account of open-ended goal inference is best supported on both empirical and theoretical grounds, providing an algorithmically plausible explanation for the speed and flexibility of human goal inference. Still, our experiments find that humans remain more similar to themselves (IoU = 0.44) than our best-fitting model (IoU = 0.35). Part of this might be explained by the discrepancy between the statistics of how humans guess in word games versus the text corpus frequencies that inform our model. This could be addressed by deriving a prior and proposal from human guesses. Humans also appear to exhibit *stickier* inferences in garden path cases, whereas open-ended SIPS tends to avoid them when run with larger values of N by proposing new goals at every step. This suggests that humans may be *adaptive* in deciding when to rejuvenate their hypotheses [130, 131]. Finally, unlike our model, humans might forget older observations, becoming more inaccurate, but also more efficient at inference. SMC algorithms that selectively forget past observations could mimic this [124].

Another open question is how bottom-up sampling can be made more general. In future work, we plan to explore how the statistics of co-occurring subgoals can be distilled from web-scale language models [132] into domain-specific models for rapid hypothesis generation. These statistics might be augmented by static analysis of environment models, automatically determining which subgoals are instrumental for other goals [133]. Such mechanisms for flexible domain adaptation could provide an even richer picture of how we contend with the infinitude of ends that others pursue, even in the face of our very finite means.

3.4 Appendix

3.4.1 Experiment Interface

The web interface used by participants is shown in Figure 3.6. At each judgment point, participants typed their guesses into the text box, which validated whether the guess was between 3 and 8 characters and used only the letters that were available. Participants could also remove their guesses by clicking the $\otimes$ symbol next to each guess. The list of guesses was converted into a probability distribution by assigning a probability of $1/n$ to each word among the n guesses. Participants could rewatch the most recent segment of the animation by pressing the *Replay* button, or rewatch the whole animation up to the judgment point by pressing the *Replay All* button. This interface is accessible at <https://block-words.web.app/?local=true>.

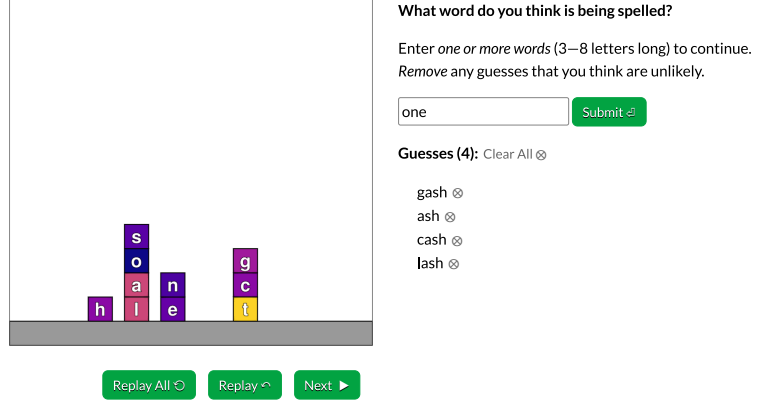


Figure 3.6: Interface for our open-ended goal inference task.

3.4.2 Model Fitting and Sensitivity Analysis

Our model of open-ended goal inference is characterized by two sets of parameters: The parameters of the *generative model* $P(g, \pi_{1:t}, s_{0:t}, a_{1:t})$, and the parameters of the *inference algorithm* which approximates $P(g|s_{0:t}, a_{1:t})$. We fit the generative model across the following parameter ranges:

- Goal prior word temperature $T_w \in \{1, 2, 4, 8, 16\}$
- Inverse temperature $\beta \in \{\frac{1}{4}, \frac{1}{2}, 1, 2, 4\}$
- Planning budget $B \in \{5, 10, 20, 50, 100, 200, 500\}$
- Replanning cadence $\Delta t \in \{1, 2\}$
- RTHS search strategy $\sigma = A^*$ or BFS

T_w controls tempering of the wordfreq-derived word frequencies used for the goal prior $P(g)$, and β controls the optimality of action selection. B is the planning budget for real-time heuristic search (RTHS) algorithm, Δt is the number of timesteps between each call to RTHS that updates the policy π_t , and σ controls how nodes are expanded by RTHS: via A^* search around each neighbor of the current state s_t (guided by the FF heuristic as the default $\hat{Q}_{\pi_t}$ value) as in LSS-LRTA* [57], or via breadth-first search (BFS) around the current state s_t , as in LRTA*-LS [134].

For the inference algorithm, we fit these parameters:

- n -gram word temperature $T_w \in \{1, 2, 4, 8, 16\}$
- n -gram termination bias $\epsilon \in \{0, 0.05, 0.1, 0.15, 0.2, 0.25\}$
- Bottom-up proposal strategy $Q \in \{\text{LAST-AND-NEXT}, \dots\}$
- Number of samples $N \in \{2, 5, 10, 20, 50\}$

T_w tempers the word frequencies used to fit the n -gram model for the bottom-up proposal Q , and is matched to be the same value used for the goal prior $P(g)$. To capture the difficulty of guessing

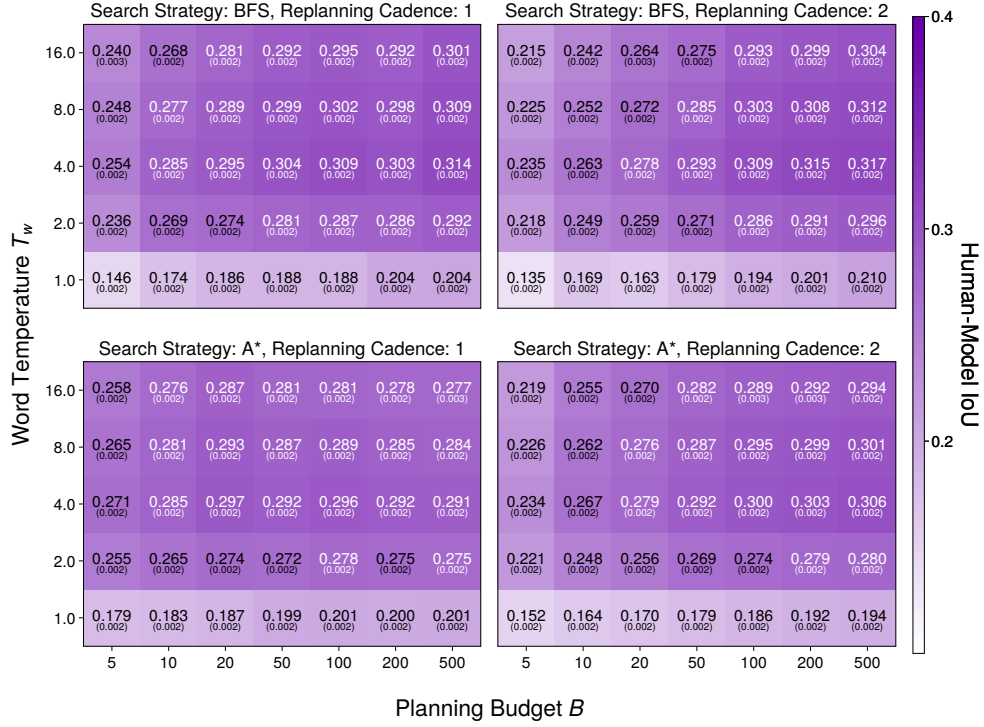


Figure 3.7: Human-model IoU across generative model parameters, using exact inference.

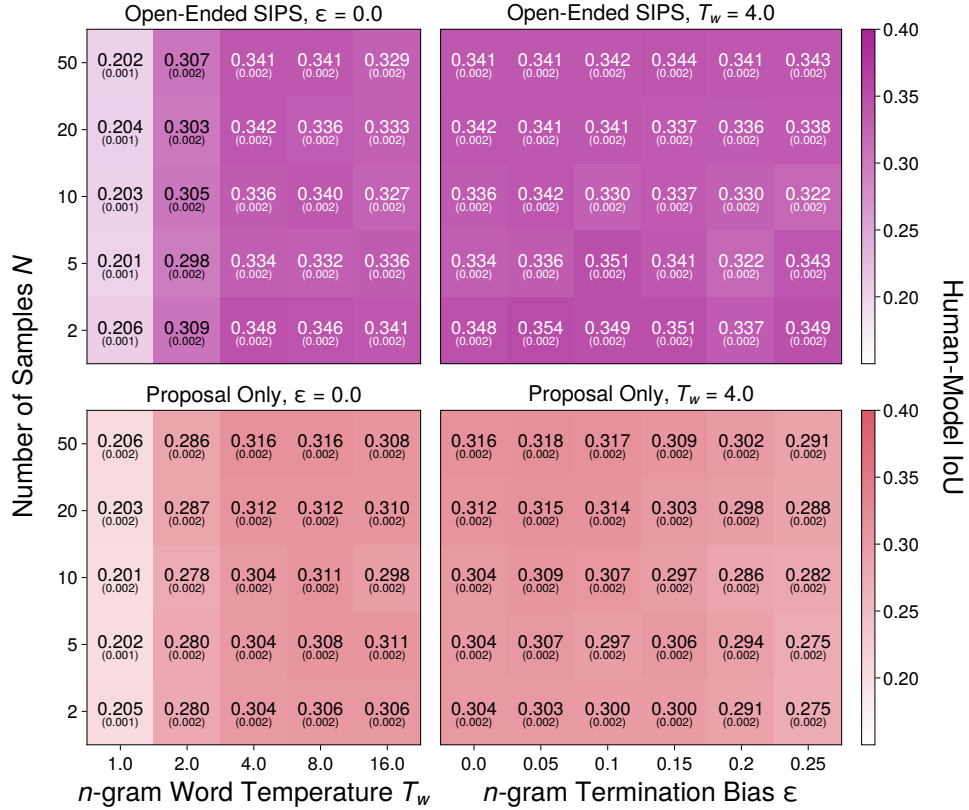


Figure 3.8: Human-model IoU across inference parameters for open-ended SIPS / bottom-up proposals.

longer words, we modified the n -gram to have an additional ϵ probability of terminating after each character. For simplicity, we fixed the context length of the n -gram model to $n = 5$. Various ways of implementing the bottom-up proposal Q are discussed in the next section. We also vary the number of particles N used by open-ended SIPS.

Fitting procedure. Model fitting proceeded in two stages. We first fit the generative model parameters to improve similarity with humans, using *exact* inference to factor out stochasticity or performance issues in the inference algorithm from the quality of the generative model itself. Instead of Pearson’s correlation coefficient (commonly used in other BToM studies), we used the intersection-over-union between human and model distributions (i.e. the Jaccard index) as our similarity metric, since it does not consider two probability vectors similar just because they both contain many zeros. Having determined values of $B = 100$, $\Delta t = 2$ and $\sigma = \text{BFS}$ that led to the most similarity with humans under the constraint of a reasonable runtime, we then fit the parameters of the open-ended SIPS algorithm. The best fitting inference parameters were $T_w = 4$ (which was matched with the goal prior’s T_w), $\epsilon = 0.05$, $Q = \text{LAST-AND-NEXT}$, and $N = 2$.

Generative model sensitivity analysis. Figure 3.7 shows how similarity with humans varies across generative model parameters when using exact inference. A higher planning budget B leads to a stronger fit, showing the importance of computing a good estimate of the agent’s policy via planning. Interestingly however, the more informed search strategy, A^* , led to slightly worse fits, suggesting that humans may not be explicitly modeling other’s detailed search processes when performing inverse planning over large numbers of goals. Human similarity also improved when using a tempered word prior with $T_w = 4.0$, whereas using the raw corpus frequencies as the prior ($T_w = 1.0$) led to substantially poorer fits. In other words, people’s intuitions for what words are likely in a word game are substantially broader than everyday word usage statistics.

Inference algorithm sensitivity analysis. Figure 3.8 shows how human-model IoU varies with different inference parameters for both open-ended SIPS and the proposal-only baseline. Again, untempered word frequencies ($T_w = 1$) lead to a poor fit, which cannot be overcome even with a high sample count N . Intermediate tempering ($T_w = 4$) leads to the best fit, capturing the broader distribution of human word guesses. The termination bias ϵ has a less pronounced effect, with the best value ($\epsilon = 0.05$) capturing the additional difficulty of proposing long words. Open-ended SIPS dominates the bottom-up proposal for almost all parameter settings.

3.4.3 Bottom-Up Proposals

For bottom-up sampling, we experimented with proposals $Q(g|s_t, a_t)$ of varying degrees of sophistication. In all of these proposals, we sample complete words g by conditioning an n -gram language model on some partial word that can be stacked from the blocks in the current state s_t . However, this still leaves undetermined *which* partial words, or subgoals, to consider. We implemented the following strategies for selecting partial words to complete:

ANY-TOWER samples a random block tower τ in state s_t , then tries to complete it into a full word. The probability of sampling a tower is proportional to how word-like the tower is — i.e., how probable it is according to the n -gram model.

LAST-TOWER tries to sample a word that completes the tower *most recently* stacked by action a_t . However, if this tower τ is not sufficiently word-like (or if a_t is not a stacking action), the proposal falls back to ANY-TOWER instead. This is implemented by comparing the probability *plast*

Proposal Q	Open-Ended SIPS					Bottom-Up				
	$N = 2$	$N = 5$	$N = 10$	$N = 20$	$N = 50$	$N = 2$	$N = 5$	$N = 10$	$N = 20$	$N = 50$
Human-Model Similarity (IoU)										
ANY-TOWER	0.294	0.310	0.297	0.314	0.319	0.211	0.218	0.217	0.224	0.228
LAST-TOWER	0.348	0.340	0.317	0.339	0.344	0.300	0.301	0.301	0.306	0.310
NEXT-TOWER	0.302	0.304	0.306	0.309	0.329	0.226	0.223	0.221	0.231	0.237
LAST-AND-NEXT	0.354	0.336	0.342	0.341	0.341	0.303	0.307	0.309	0.315	0.318
Goal Accuracy										
ANY-TOWER	0.115	0.162	0.156	0.159	0.160	0.067	0.071	0.072	0.069	0.070
LAST-TOWER	0.163	0.171	0.184	0.174	0.176	0.141	0.134	0.139	0.138	0.134
NEXT-TOWER	0.118	0.157	0.160	0.170	0.165	0.073	0.074	0.068	0.072	0.074
LAST-AND-NEXT	0.153	0.163	0.183	0.177	0.176	0.132	0.140	0.135	0.141	0.137

Table 3.2: Effect of bottom-up proposal strategy Q on human-model similarity (IoU) and goal accuracy.

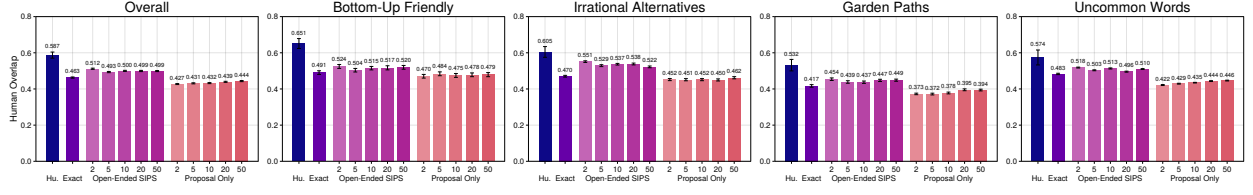
of the most recently stacked tower under the n -gram against the probability p_{rand} of an equally tall tower of random blocks, then deciding to complete the last tower with probability $\frac{p_{\text{last}}}{p_{\text{last}} + p_{\text{rand}}}$.

NEXT-TOWER focuses on cases where the agent is unstacking a tower of blocks in order to reach a block in that tower. The proposal considers all ways of using a block in the most recently unstacked tower to complete some other block tower. One of these candidate towers τ is selected with probability proportional to how word-like it is, and then a completion is sampled from to the n -gram model. If no candidate is word-like enough compared to the probability of selecting a random block, then the proposal defaults to ANY-TOWER.

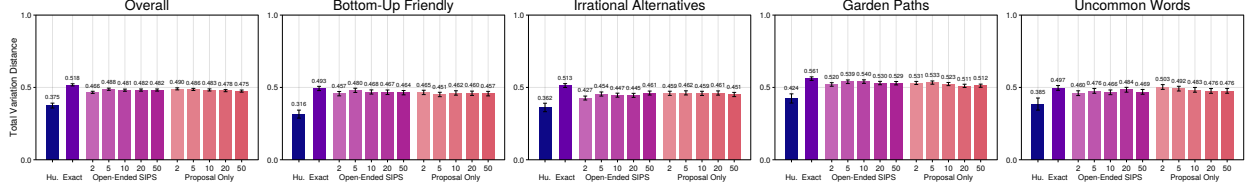
LAST-AND-NEXT is the most sophisticated of our proposals, which we describe and use in the main text. It is equivalent to LAST-TOWER, except it defaults to NEXT-TOWER if the last action is not a stacking action, or if the last stacked tower is not sufficiently word-like.

Table 3.2 shows how these different proposal strategies compare in terms of both human similarity and goal inference accuracy. As expected, the LAST-AND-NEXT proposal best matches human inferences whether it is incorporated into open-ended SIPS (Table 3.2(a)) or used on its own (Table 3.2(b)), while achieving (close to) the highest accuracy. The LAST-TOWER also performs well in this regard, albeit with slightly lower human similarity. In contrast, both ANY-TOWER and LAST-TOWER fare poorly. Open-ended SIPS is able to make up for their weakness to some degree, showing the value of inverse planning even with a weak proposal.

Handling auxiliary randomness. Note that all of our bottom-up proposals make use of *auxiliary randomness* [135] when sampling a tower τ to complete into a full word g . This means that even though we can *sample* from $g \sim Q(g|s_t, a_t)$, we cannot exactly *evaluate* the probability $Q(g|s_t, a_t)$ used in the importance weight. Instead, we can only evaluate $Q(g|s_t, a_t, \tau)$ using the n -gram model, which is conditional on the choice of tower $\tau \sim Q(\tau)$. In the context of an SMC algorithm like open-ended SIPS, however, we can use an *unbiased density sampler* of $Q(g|s_t, a_t)$ [135, 136], which returns both g and a weight w such that $\mathbb{E}_Q[w^{-1}|g] = Q(g|s_t, a_t)^{-1}$. This weight w can then be used as the denominator when computing importance weights in L6 of Algorithm 2. Using $w = Q(g|s_t, a_t, \tau)$ satisfies this property.



(a) Distribution overlap between human inferences and model inferences across conditions.



(b) Total variation distance between human inferences and model inferences across conditions (lower is better).

Figure 3.9: Similarity of average human and model goal inferences measured in terms of (a) distribution overlap ($\sum_g \min[P(g), Q(g)]$ for distributions P and Q), and (b) total variation distance ($\frac{1}{2} \sum_g |P(g) - Q(g)|$). As in Figure 3.3, error bars denote 95% CIs, calculated from 1000 bootstrap samples of the distribution of human responses.

3.4.4 Additional Similarity Metrics

In Figure 3.9, we report additional measures of similarity between human goal inferences and model outputs: (a) the overlap coefficient between distributions (a generalized version of recall), and (b) total variation distance (which captures the *maximum* difference in the probability of any event under two distributions). Regardless of the metric, open-ended SIPS is more similar to humans than exact inference or the bottom-up proposal. This is most pronounced in the *Irrational Alternatives* condition.

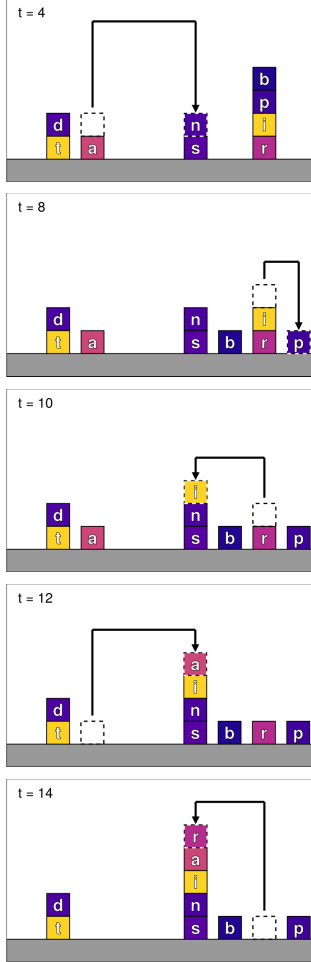
3.4.5 Additional Step-by-Step Comparisons

In Figure 3.10, we show step-by-step comparisons between human and model inferences for the two experimental conditions not covered in the main text.

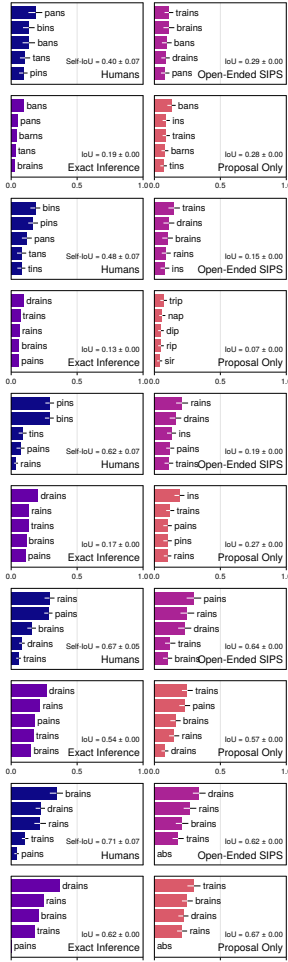
In the *Bottom-Up Friendly* scenario, limited inverse planning is necessary, and so the bottom-up proposal is as good an explanation for human goal inference as open-ended SIPS for all steps except $t = 8$. At this step, the bottom-up proposal generates words that end in **p** by following the LAST-TOWER proposal strategy. This fails to take into account the previous actions of stacking block **n** onto block **s**, highlighting the importance of inverse planning in even simple scenarios.

In the *Uncommon Words* scenario, we chose the uncommon word **chump** to be the goal, and designed actions to make more common distractors like **jump** and **hump** seem likely. As expected, most humans failed to think of **chump** as a possibility until the very last step ($t = 12$). Open-ended SIPS and the proposal only baseline with $N = 2$ particles reflected this tendency, demonstrating how inferring rare events is difficult with a small number of samples. In contrast, the exact inference baseline enumerates all possible words at every step, and hence assigns **chump** a significant probability at $t = 10$. This is the case even though **chump** is less likely under the goal prior $P(g)$: an event being rare under the proposal Q leads to qualitatively different behavior than exact inference over events with low prior probabilities.

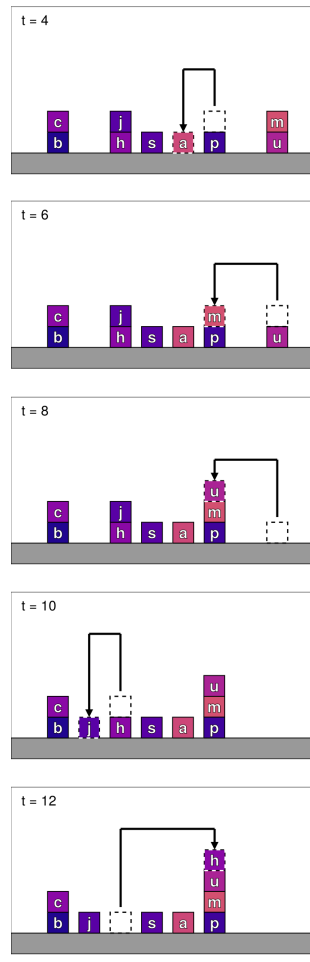
Condition 1 d: Bottom-Up Friendly



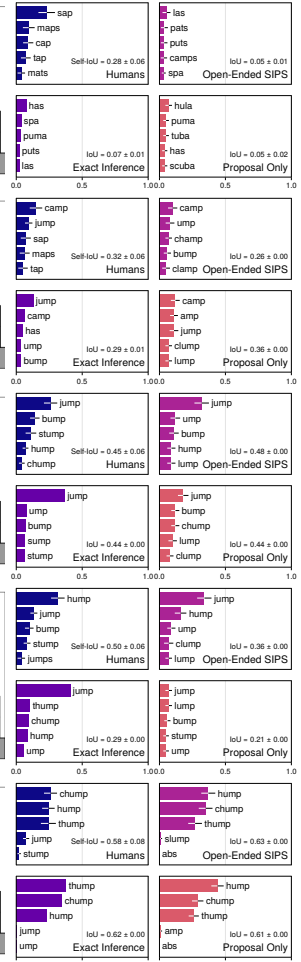
Goal Inferences (Top 5)



Condition 4 a: Uncommon Words



Goal Inferences (Top 5)



(a) Bottom-Up Friendly scenario: **drains**

(b) Uncommon Words scenario: **chump**

Figure 3.10: Step-by-step inference results on scenarios from the *Bottom-Up Friendly* and *Uncommon Words* conditions. In (a) there are few qualitative differences between the inference methods, apart from $t = 8$, where the proposal only baseline generates words that end in **p** instead of taking into account the fact that **n** was previously stacked on **s**. In (b), humans, open-ended SIPS, and the bottom-up proposal largely fail to guess the uncommon word **chump** until the very last timestep ($t = 12$), in contrast to the fully enumerative baseline, which assigns a non-trivial probability to **chump** by $t = 10$.

Part II

Communicative Cooperation

Chapter 4

Cooperative Language-Guided Inverse Plan Search

If we are to model natural human communication, our computer conversant must know how to be helpful. It should be able to debug someone’s plans and, by such means as making suggestions, put [them] back on the right path to [their] goal. Being helpful, however, is a business that requires tact.

Phillip R. Cohen

When people cooperate, we do not just coordinate our behavior by silently inferring the goals and plans of others. We also *communicate* with each other, signaling our intentions through a variety of mediums, and interpreting these signals in order to achieve shared goals. The primary medium that humans use for such communication is natural language, allowing us to express our intentions, and have others understand them. Yet the machinery introduced in [Part I](#) does not obviously apply to the comprehension of such language. Why is a theory of mind necessary for understanding linguistic communication, if it is already so detailed? And how could one model the cooperative production of natural language in a manner suitable for inverse planning?

We give an answer to these questions with **cooperative language-guided inverse plan search (CLIPS)**, an extension of the SIPS architecture ([Chapter 1](#)) for pragmatic instruction following and goal assistance ([Figure 4.1](#)). Recognizing that human instructions are often ambiguous without context (e.g. “Can you hold that door?”), CLIPS infers a speaker’s intentions from both their actions and words, under the assumption that humans are pragmatic speakers who exploit context to convey intentions [[137–140](#)]. To capture such speech, CLIPS models humans as cooperative planners who communicate *joint plans* [[141–143](#)], using a large language model (LLM) probabilistic program ([Chapter 9](#)) to score the probability of observed utterances given a latent plan. As such, CLIPS can perform inverse planning directly from natural language. With the resulting goal inferences, CLIPS computes a policy that assists the speaker under uncertainty about their goals, effectively solving the assistance game [[13, 32](#)] that characterizes cooperative instruction following.

We evaluate CLIPS on a suite of multi-step goal assistance problems in a doors-and-keys gridworld [[5](#)] and the VirtualHome domain [[144, 145](#)]. In these problems, the assistant must infer

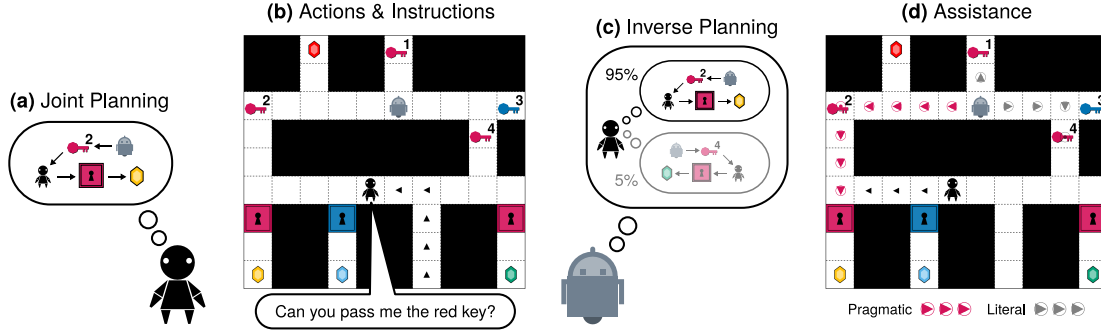


Figure 4.1: Overview of *cooperative language-guided inverse plan search* (CLIPS). We model a human principal as (a) cooperatively planning a joint policy for the human and the (robot) assistant. The human is (b) assumed to take actions from this joint policy while communicating planned actions as an instruction (“Can you pass me the red key?”). Observing this, (c) CLIPS infers the human’s goal and policy via Bayesian inverse planning. CLIPS then (d) acts by minimizing expected goal achievement cost, pragmatically interpreting the ambiguous instruction by picking up *Key 2*. In contrast, a literal instruction follower might pick up *Key 1* or *Key 4*, which are also red in color.

the human’s goal from their actions and possibly ambiguous instructions, then decide how best to help. Even when leveraging LLMs, standard instruction following methods struggle with this setting because they disregard pragmatic context [146–148], while action-only goal inference [5, 6, 145] ignores linguistic information. Multimodal LLMs have access to all information, but they fail to ground it in a coherent theory of mind [38, 149, 150]. In contrast, CLIPS is able to use observed actions and inferred goals to resolve *ambiguous language*, interpret *joint instructions*, and correct for *incomplete commands*, achieving much higher goal accuracy and cooperative efficiency than GPT-4V, LLM-based literal instruction following, and unimodal inverse planning, while correlating strongly with goal inference and assistance judgments provided by human raters.

4.1 Language-Augmented Goal Assistance Games

CLIPS is an architecture for solving cooperative problems involving language. But which problems? Here we focus on the fully assistive setting, where only the speaker (assumed to be a human) has a goal, and the listener tries to help by taking actions. We formulate this as a *language-augmented goal assistance game*, an extension of assistance games [13, 32] with linguistic utterances and uncertainty over goals (i.e. desired terminal states) [151].

Definition 4.1. A *language-augmented goal assistance game* (LGAG) is a stochastic game [152] with a principal/speaker h and an assistant/listener r , given by the tuple $(S, U, A^h, A^r, C, G, H, P_s, P_i)$:

- S is the set of environment states.
- U a space of utterances u that the principal may use to communicate at any step.
- A^h the set of principal/speaker actions a^h .
- A^r the set of assistant/listener actions a^r .
- C a set of cost functions $C : S \times U \times A^h \times A^r \rightarrow \mathbb{R}$ over state-action transitions.
- $G \subseteq \mathcal{P}(S)$ a set of possible goals g where each goal $g \subseteq S$ is a set of states.
- H a horizon (i.e. number of steps) after which the game terminates.

- $P_s(s'|s, a^r, a^h)$ the environment transition distribution.
- $P_i(s_i, C, g)$ a prior distribution over initial environments, cost functions, and goals.

At the outset of the game, the principal initially knows (i.e. observes) the *true* cost function C and goal g sampled from P_i , but the assistant only observes the initial environment s_i . Thus, the assistant has to *infer* the true goal g . The true goal g corresponds to a set of *terminal* states unknown to the assistant.

LGAGs extend the original assistance game formalism [13] in two ways. Most prominently, the principal now has a joint action space $A^h \times U$ that includes communicative utterances $u \in U$ that have no impact on the environment, but can convey information to the assistant. In CLIPS, we let U be the set of all text strings, thereby including (the text representation of) natural languages like English. Secondly, we assume the principal has an unknown *goal* rather than an unknown reward function, and seeks to minimize the cost of achieving their goal rather than maximize cumulative reward, as in goal-POMDPs [153] and goal-uncertain stochastic shortest path (GUSSP) problems [151].¹ We can further extend LGAGs by allowing *both* the principal and assistant to speak, but CLIPS is designed for the single-speaker variant.

Assistance games admit cooperative equilibria [13]. However, these equilibria are intractable to compute in general [32], and also assume more game-theoretic optimality from humans than may be warranted [154]. This is especially true for LGAGs, where optimality would imply the creation of an *de novo* linguistic convention for how to produce and interpret utterances $u \in U$ [155]. As such, our goal is to *approximately* solve the assistance game under reasonable modeling assumptions about how humans cooperate and communicate. By fixing a human model — including a model of how people speak — the game becomes a partially observable Markov decision process (POMDP) from the assistant’s perspective [156], which can be solved through approximate methods [157, 158]. We describe this human model, then show how an assistant can perform Bayesian inference over such a model, using the acquired information to better assist humans.

4.2 Modeling Cooperative Action and Communication

How should a cooperative agent model the actions and instructions of an interlocutor? When humans cooperate, we direct our actions towards a shared goal while expecting that others will do the same. This capacity for *joint intentionality* [26] allows us to act as a single “we”, avoiding nested reasoning about each other’s minds [141, 143]. In CLIPS, we capture joint intentionality by modeling the human principal as a *cooperative planner*, who computes a *joint policy* π for both agents to achieve the goal g . The human follows π by taking an action a_t^h at each step t , and assumes the assistant will take an action a_t^r accordingly. These actions lead to a change in the state s_t . The human may also speak at any time t (with the decision denoted by d_t), issuing a command c_t that summarizes their intended policy π , in line with a plan-based theory of speech acts [159–161]. This command is then rendered in natural language as an utterance u_t . The overall generative process is shown in Figures 4.2a–b, and is summarized below.

¹While a goal and cost function pair (g, C) can be converted to an equivalent reward function — namely, the history-dependent reward function $R(s_{1:T}, u, a^h, a^r)$ that delivers negative reward $-C(s_T, u, a^h, a^r)$ before the goal g is achieved, and zero reward once any $s_t \in s_{1:T}$ is in g — the goal-centric representation lends itself to rich goal specifications defined by conjunctions of predicates.

model CLIPS-MODEL(s_1, G, T)

$g \sim \text{GOAL-PRIOR}(G)$

$\pi \sim \text{POLICY-INIT}(g, s_1)$

for $t \in [1, T]$ **do**

$\pi \leftarrow \text{RTHS-POLICY-UPDATE}(\pi, g, s_t)$

$a_t^h, a_t^r \sim \text{BOLTZMANN-DIST}(s_t, \pi)$

$u_t, c_t, d_t \sim \text{UTTERANCE-MODEL}(s_t, \pi)$

$s_{t+1} \sim \text{TRANSITION}(s_t, a_t^h, a_t^r)$

end for

end model

(a) Full CLIPS model as a probabilistic program.

model UTTERANCE-MODEL(s_t, π)

$d_t \sim \text{BERNOULLI}(p_{\text{speak}})$

if $d_t = \text{TRUE}$ **then**

$a_{t:t+L} \leftarrow \text{ROLLOUT-POLICY}(s_t, \pi, L)$

$c_t \sim \text{RAND-SALIENT-SUBSET}(a_{t:t+H}, K)$

$u_t \sim \text{LANGUAGE-MODEL}(c_t, \mathcal{E})$

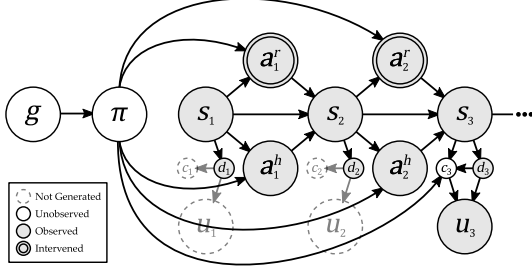
else

$c_t, u_t \leftarrow \emptyset, ""$

end if

end model

(c) Utterance model $P(u_t, c_t, d_t | \pi)$ as a probabilistic program.



(b) One graphical realization of our Bayesian model.

Command: (handover you me key1) where (iscolor key1 blue)
 Utterance: Hand me the blue key.
 Command: (unlock you key1 door1)
 Utterance: Can you unlock the door for me?
 Command: (pickup me key1) (unlock you key2 door1) where (iscolor key1 blue) (iscolor door1 green)
 Utterance: I'm getting the blue key, can you open the green door?
 Command: (handover you me key1) (handover you me key2) where (iscolor key1 green) (iscolor key2 red)
 Utterance: Can you pass me the green and red keys?

(d) Paired examples $\mathcal{E}$ of commands c_t and utterances u_t .

Figure 4.2: Model architecture. In CLIPS, we model the human as a cooperative planner who computes a joint policy π for a goal $g \in G$ (here we omit the dependence on action costs C). The policy π dictates the human's and assistant's actions a_t^h, a_t^r at each state s_t , as well as the command c_t and utterance u_t that the human may decide d_t to communicate at step t . We implement this process as probabilistic program, shown in (a). One realization of this process is depicted in (a), showing a case where an utterance u_3 is only made at $t = 3$. Utterance generation is modeled by another probabilistic program in (c), which summarizes salient actions from policy π as a command c_t , then samples an utterance u_t using a LLM prompted with c_t and (d) few-shot examples $\mathcal{E}$ demonstrating how commands are translated into natural language.

$$\text{Goal \& Cost Prior:} \quad g, C \sim P(g, C) \quad (4.1)$$

$$\text{Joint Planning:} \quad \pi \sim P(\pi | g, C) \quad (4.2)$$

$$\text{Action Selection:} \quad a_t^h, a_t^r \sim P(a_t^h, a_t^r | s_t, \pi) \quad (4.3)$$

$$\text{Utterance Generation:} \quad u_t, c_t, d_t \sim P(u_t, c_t, d_t | s_t, \pi) \quad (4.4)$$

$$\text{State Transition:} \quad s_{t+1} \sim P(s_{t+1} | s_t, a_t^h, a_t^r) \quad (4.5)$$

This generative process extends the SIPS model family introduced in Chapter 1, replacing single-agent planning with joint planning (Eq. 4.2), and incorporating a model of language production (Eq. 4.4, Fig. 4.2c). We model joint planning with the same heuristic search (RTHS) algorithm [53, 55, 56] used in SIPS (RTHS-POLICY-UPDATE in Fig. 4.2a), but configured to plan over the joint action space $A^h \times A^r$.² Given a goal g and cost C , RTHS computes a Boltzmann policy π over joint actions a_t^h, a_t^r :

$$\pi(a_t^h, a_t^r | s_t) \propto \exp \left(-\beta \hat{Q}_\pi(s_t, a_t^h, a_t^r) \right) \quad (4.6)$$

²In practice, we only use RTHS for the mDKG domain, finding that a simple Q -value estimator is sufficient for VirtualHome. We also assume that the principal and assistant take turns acting (so a_t^r is a no-op when a_t^h is not). This reduces the branching factor while planning, but preserves the optimal solution [162].

As previously introduced, β controls the optimality of the policy, and $\hat{Q}_\pi(s_t, a_t^h, a_t^r)$ is the cost-to-goal estimate for taking actions a_t^h, a_t^r at state s_t for goal g under cost function C .

Having computed the policy π , we model human communication as an approximately *pragmatic* process, focusing specifically on imperative speech. Since the human wants to achieve their goal, they are likely to convey information about the policy they have in mind. However, long utterances are costly, and so the human is likely to convey only a salient summary of the policy, reducing the cost of communication. We capture these aspects of cooperative communication through a structured generation procedure that leverages the linguistic competence of LLMs:

1. At each step t , the human decides whether to communicate (d_t) with probability p_{speak} . If d_t is true, we rollout the policy π for L steps, producing a series of planned actions $a_{t:t+L}$.
2. The planned actions $a_{t:t+L}$ are filtered to a domain-specific set of *salient* actions (see Appendix for details), and a random subset of up to K such actions are sampled to form a command c_t .
3. Finally, the command c_t is translated into a natural language utterance u_t using an LLM prompted with few-shot examples (Figure 4.2d). This defines a likelihood function $P(u_t|c_t)$.

This procedure is implemented as the probabilistic program shown in Figure 4.2c. While it does not capture all pragmatic preferences a speaker might have (or the possibility of non-imperative language), key features of pragmatic speech are modeled: Utterances are about plans, and hence practically useful, but are also restricted to small ($\leq K$) but salient subsets of those plans, ensuring they are informative without being costly. Furthermore, using an LLM as an utterance model $P(u_t|c_t)$ provides broad coverage over a rich space of linguistic expressions.

4.3 Inverse Planning from Actions and Instructions

Using this probabilistic model, our assistant can infer a posterior distribution over the human’s goal g , cost function C , and policy π given a sequence of observed actions $a_{1:T}^h$, states $s_{1:T}$ utterance decisions $d_{1:T}$, and utterances $u_{1:T}$ (where u_t is the empty string whenever $d_t = \text{FALSE}$). Some care must be taken, however. Since the assistant is also taking actions $a_{1:T}^r$ alongside the human principal, it is tempting to infer the human’s goal by conditioning on *both* agents’ actions:

$$\begin{aligned} P(g, C, \pi | s_{1:T}, d_{1:T}, u_{1:T}, a_{1:T}^h, a_{1:T}^r) &\propto P(g, C, \pi, s_{1:T}, d_{1:T}, u_{1:T}, a_{1:T}^h, a_{1:T}^r) \\ &= P(g, C, \pi) \prod_{t=1}^T P(s_t | s_{t-1}, a_{t-1}^{hr}) P(u_t, d_t | s_t, \pi) \underline{P(a_t^h, a_t^r | s_t, \pi)} \end{aligned} \quad (4.7)$$

For an *external* observer, this is the appropriate distribution to compute [14, 163] – indeed we shall take this approach when we examine team-level goal inference in Chapter 5. However, prior work on cooperative inference also computes Equation 4.7 when observers are *internal* to the environment [141, 143], which can lead to pathologies if the joint policy is not a good model of the assistant’s actions. For example, the assistant might condition on the fact that it has not moved so far ($a_{1:T}^r$ are no-ops), leading it to infer that the human is pursuing a goal g where the assistant’s help is unnecessary. To address this, it is crucial to recognize that the assistant is *intervening* upon the environment through its actions (Figure 4.2b, double-lined nodes), not observing them.³ As

³Alternatively, it is enough to note the assistant’s actions are selected according to *different* policy π' than the human’s inferred joint policy π . Thus, even if one is an evidential decision theorist [164, 165], it is *safer* for the assistant not to update its beliefs as if its actions come from π .

such, we should detach the evidential connection between the assistant’s actions $a_{1:T}^r$ and any causal ancestors, which we denote using Pearl’s **do**-operator [166]:

$$\begin{aligned} P(g, C, \pi | s_{1:T}, d_{1:T}, u_{1:T}, a_{1:T}^h, \mathbf{do}(a_{1:T}^r)) &\propto P(g, C, \pi, s_{1:T}, d_{1:T}, u_{1:T}, a_{1:T}^h, \mathbf{do}(a_{1:T}^r)) \\ &= P(g, C, \pi) \prod_{t=1}^T P(s_t | s_{t-1}, a_{t-1}^{hr}) P(u_t, d_t | s_t, \pi) \underline{P(a_t^h | s_t, \pi)} \end{aligned} \quad (4.8)$$

The difference between Equations 4.7 and 4.8 lies in the final underlined term: Whereas an external observer reweights its beliefs by incorporating the likelihood of both agents’ actions $P(a_t^h, a_t^r | s_t, \pi)$, an assistive agent should only incorporate the likelihood of the human’s action $P(a_t^h | s_t, \pi)$.

We compute the distribution in Equation 4.8 via the appropriate generalization of SIPS, shown in Algorithm 3. An initial belief b_0 is returned by BELIEF-INIT, which generates a set of N weighted samples by either sampling or enumerating over the goal and cost prior $P(g, C)$ and initializing the corresponding policy π . Then at each step t , the previous belief b_{t-1} is updated by adjusting the weight w^i associated with each goal-cost-policy sample (g^i, C^i, π^i) . To do this, BELIEF-UPDATE refines the Q -value estimates that define the policy π^i by running more iterations of RTHS (L10). The weights are then updated to reflect the new state s_t and decision to speak d_t (L11-12). If d_t is true (i.e. an utterance u_t is observed), we also marginalize over all commands c_t that could have generated u_t from π , and update w^i with the resulting mixture likelihood (L13-14). Finally, we condition on the human’s action a_t^h , and return the resulting sample collection (L16-18).

Algorithm 3 CLIPS belief initialization and update

```

1: procedure BELIEF-INIT( $N$ )
2:    $w^i \leftarrow 0$ ;
3:    $g^i, C^i \sim P(g, C)$ ;  $\pi^i \sim P(\pi | g^i, C^i)$  for  $i \in [1, N]$ 
4:    $b_0 \leftarrow \{(w^i, g^i, C^i, \pi^i)\}_{i=1}^N$ ; return  $b_0$ 
5: end procedure
6:
7: procedure BELIEF-UPDATE( $b_{t-1}, s_{t-1:t}, d_t, u_t, a_{t-1:t}^h, a_{t-1}^r$ )
8:    $\{(w^i, g^i, \pi^i)\}_{i=1}^N \leftarrow b_{t-1}$ 
9:   for  $i \in [1, N]$  do
10:     $\pi^i \leftarrow \text{RTHS-POLICY-UPDATE}(\pi^i, g^i, s_t)$ 
11:     $w^i \leftarrow w^i \cdot P(s_t | s_{t-1}, a_{t-1}^h, a_{t-1}^r)$ 
12:     $w^i \leftarrow w^i \cdot P(d_t | s_t, \pi)$ 
13:    if  $d_t = \text{TRUE}$  then
14:       $P(u_t | s_t, \pi) \leftarrow \sum_{c_t} P(u_t | c_t) P(c_t | s_t, \pi)$ 
15:       $w^i \leftarrow w^i \cdot P(u_t | s_t, \pi)$ 
16:    end if
17:     $w^i \leftarrow w^i \cdot P(a_t^h | s_t, \pi)$ 
18:  end for
19:   $w^i \leftarrow w^i / \sum_{j=1}^N w_j$ 
20:   $b_t \leftarrow \{(w^i, g^i, \pi^i)\}_{i=1}^N$ ; return  $b_t$ 
21: end procedure

```

If we enumerate over all goals and costs in BELIEF-INIT, and compute policies deterministically (e.g. via RTHS with a fixed search budget), this is an *exact* Bayesian inference algorithm, since we also enumerate over commands c_t . In environments where there are too many hypotheses for this to be efficient, Algorithm 3 can readily be extended to a sequential Monte Carlo algorithm [120, 122] in same manner as SIPS (Chapter 1). For our experiments, however, enumeration is feasible, and we opt for this approach to avoid variance from sampling.

4.4 Pragmatic Instruction Following as Goal Assistance

With Algorithm 3, our assistive agent is able to infer a distribution over the human’s goal g from their instructions and actions. How should this information guide action, especially if the goal g remains uncertain? In keeping with our Bayesian approach, we minimize *expected* cost, helping the human achieve their likely goal as quickly as possible [167]. Since each inferred policy π^i

corresponds to a $\hat{Q}_{\pi^i}$ -value function, the assistant can select actions by minimizing the expected $\hat{Q}_{\pi^i}$ value given its uncertainty over π^i :

$$a_t^{r*} = \arg \min_{a_t^r} \mathbb{E}_{\pi^i} [\hat{Q}_{\pi^i}(s_t, a_t^h, a_t^r)] \quad (4.9)$$

We implement this policy with Algorithm 4, which effectively solves the Q_{MDP} approximation of the assistive POMDP [158, 168, 169]. At each step t , we assume the assistant has updated its belief b_{t-1} from the previous step $t-1$, and also observes a_t^h , the human’s action⁴. We then iterate over all samples $(w^i, g^i, \pi^i) \in b_{t-1}$, update the policies π^i , and perform a weighted sum of $\hat{Q}_{\pi^i}$ -values for all assistant actions a_t^r according to their weights w^i . This produces a vector of *assistive* Q -values, which we minimize over to compute the best assistive action.

Algorithm 4 CLIPS Q_{MDP} assistance policy

```

1: procedure ASSISTANCE-POLICY( $b_{t-1}, s_t, a_t^h$ )
2:   for  $a_t^r \in \mathcal{A}^r(s_t)$  do
3:      $\hat{Q}_{\text{assist}}(s_t, a_t^h, a_t^r) \leftarrow 0$ 
4:     for  $(w^i, g^i, \pi^i) \in b_{t-1}$  do
5:        $\pi^i \leftarrow \text{RTHS-POLICY-UPDATE}(\pi^i, g^i, s_t)$ 
6:        $\hat{Q}_{\text{assist}}(s_t, a_t^h, a_t^r) \leftarrow w^i \cdot \hat{Q}_{\pi^i}(s_t, a_t^h, a_t^r)$ 
7:     end for
8:   end for
9:    $a_t^{r*} \leftarrow \arg \min_{a_t^r} \hat{Q}_{\text{assist}}(s_t, a_t^h, a_t^r)$ 
10:  return  $a_t^{r*}$ 
11: end procedure

```

4.5 Experiments

We evaluate our method against a variety of baselines in two domains: a cooperative gridworld puzzle called **multi-agent Doors, Keys & Gems** (mDKG) originally developed in [5, 14], and **VirtualHome** (VH), a virtual household simulator [144, 145]. In both domains, a human principal and a (robot) assistant cooperate on multi-step tasks to accomplish the human’s goals. In mDKG, the goals are four colored gems which are often secured behind doors. Doors can be unlocked by keys of the same color, and the assistant can help by collecting keys or unlocking doors (Figure 4.1). In VirtualHome, the goals are 6–12 household tasks, including setting up tables and preparing ingredients (Figure 4.3). Both domains admit encodings in the Planning Domain Definition Language (PDDL) [46] with some extensions [47], which we use as the environment representation for our planning and inverse planning algorithms.

To systematically test the ability of assistive agents to pragmatically follow a human’s instructions and assist with their goals, we developed a dataset of *goal assistance problems* for each domain (30 problems in mDKG, 25 problems in VH). In each problem, the goal is initially unknown to the assistant. While taking a sequence of actions $a_{1:T}^h$, the human communicates one or more instructions $u_{1:T}$ to the assistant. The assistant then has to make an inference about the human’s goal g , and chooses actions to best assist them. We designed our problems to cover a range of scenarios inspired by human communication we observed in exploratory studies, varying the information that can be deciphered from the human’s actions or instructions (see Figure 4.4 and Appendix). In mDKG, we restricted the human to only one instruction u_T , since more information would render goal inference trivial. In VH, however, the goal space was larger, allowing us to construct 10 problems with multiple utterances such as “Can you get the plates?” followed by “Bring the bowls too!”

⁴This assumption is natural in our turn-based setting. When actions are simultaneous the assistant can minimize expectation over both policies π^i and human actions a_t^h .

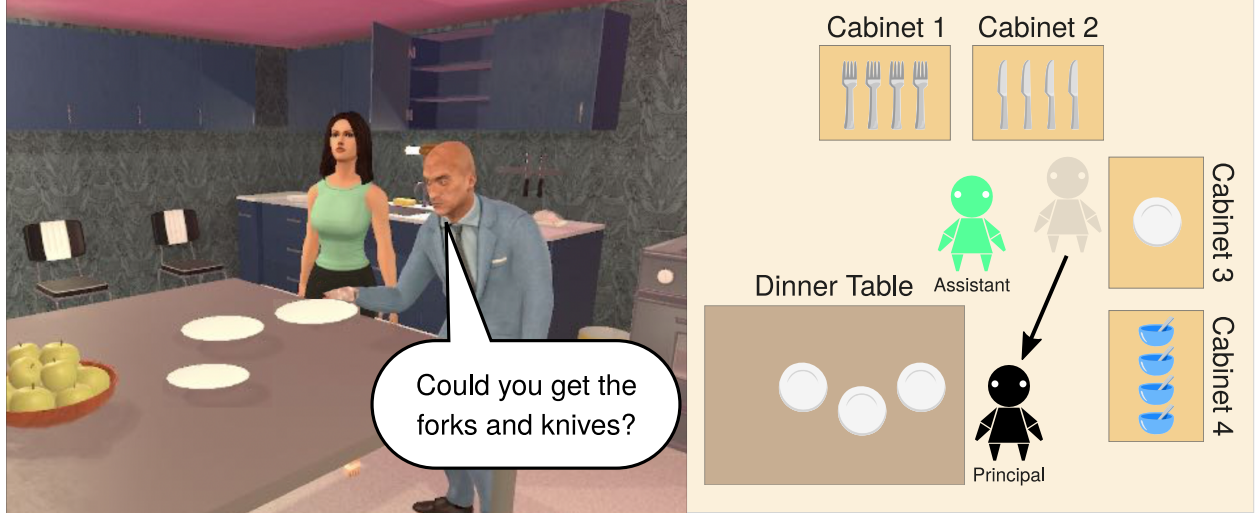


Figure 4.3: A goal assistance problem in VirtualHome, where the principal and assistant collaborate to set the table. The principal places three plates on the table, then says “*Could you get the forks and knives?*”. A pragmatic assistant has to infer the number of forks and knives from context (in this case, three each).

4.5.1 Model Configuration

In our experiments, we configured CLIPS to compute Q -values using the real-time adaptive A^* variant of RTHS [56], and a Gamma prior over the rationality parameter β . For the utterance model, we used either OpenAI’s 6.7B curie model [170] or the davinci-002 as our LLMs, due to the more diffuse probabilities they provided as base models. For goal assistance, we ran Algorithm 4 in offline mode, fixing the inferred posterior after observing $a_{1:T}^h$ and $u_{1:T}$, then selecting actions by minimizing expected Q -values with respect to the fixed posterior. This was evaluated against a simulated human agent which followed a joint policy to the true goal, unless it was apparent that the assistant was not doing the same. While Algorithm 4 can also be run in online mode, we fixed the posterior to better match the information that our human raters were provided.

4.5.2 Baselines

To evaluate the benefit of pragmatic instruction following over either a literal interpretation of instructions, or goal inference from only one modality, we included the following baselines:

Unimodal Inverse Planning

We implemented action-only and language-only inverse planning (IP) baselines as ablations of CLIPS, where the action-only baseline is equivalent to the exact Boltzmann-rational variant of SIPS (Chapter 1) for inference. In these baselines, the assistive agent uses the same belief update shown in Algorithm 3, except that it conditions on only actions $a_{1:t}^h$ or only the utterance u_t . In cases with no actions, CLIPS and the language-only baseline are equivalent.

LLM-Based Literal Listener

The literal listener baseline interprets the instruction u_t in state s_t without further information about the human’s actions or goals. To implement this baseline, we adapt the utterance model in Figure 4.2c to sample from the space of all assistant-directed commands c_t that are possible in state s_t . In particular, we enumerate over all assistant actions $\mathcal{A}^r$, and select a subset of up to K salient actions to form a command c_t . This defines a distribution over commands $P(c_t|s_t)$ which *does not* depend the principal’s policy π or goal g . Given the command c_t , we use an LLM (text-davinci-002 for mDKG, davinci-002 for VH) as an utterance likelihood $P(u_t|c_t)$. Assuming a uniform prior over commands, we can perform enumerative inference to compute the posterior $P(c_t|u_t, s_t)$ given an instruction u_t in state s_t .

Given a command c_t , the assistant still needs to generate an assistive plan. We did this in two ways: The *naive* literal assistant interprets a command c_t by randomly selecting one concrete grounding (e.g. “*Can you pick up the red key?*” could mean picking up the red key closest to the human or some other key), then planning to achieve that concrete goal. In contrast, the *efficient* literal assistant tries to directly satisfy the command c_t in the most efficient way⁵ (e.g. “*Could you unlock the blue door?*” is satisfied by unlocking the blue door closest to the assistant). For both variants, the assistant can either satisfy the most likely command c_t^* , or sample a command from the full distribution $P(c_t|u_t, s_t)$. We report results for the latter after averaging over 10 samples, using systematic sampling to reduce variance.

Multimodal LLM (GPT-4V)

For the mDKG domain, we used GPT-4 with Vision (GPT-4V) [171] as a purely neural baseline, prompting it with the same set of rules and instructions that we provided to our human raters, along with the final frame of each animated visual that we showed to humans (similar to Figure 4.1(b)). This allowed us to probe the degree to which multimodal LLMs are capable of intuitive pragmatic reasoning given spatially-grounded actions and verbal instructions [149]. Due to cost and rate limits, we report zero-shot performance with a temperature of zero.

More details about our models and datasets can be found in the Appendix (Section 4.8). Source code is available at <https://github.com/probcomp/CLIPS.jl>.

4.5.3 Human Experiments

As an additional standard for comparison, we conducted a study with 100 human participants from the US through Prolific (mean age = 39.8, 59 men, 38 women, 2 non-binary), presenting each of them with 15 goal assistance problems from the mDKG domain. In each problem, participants saw the actions taken by the human principal and observed the instruction at the end. Participants were then asked to select the set of gems that they thought were likely to be principal’s goal, and then indicate how the robot agent should best assist the principal. This gave 50 goal and assistance ratings for each problem (93% power for Cohen’s $d=0.5$ at $\alpha=0.05$).

For the assistance question, we presented each participant with a set of *assistance options*, corresponding to either picking up keys or unlocking doors. This allowed us to query participants’

⁵Note that the naive/efficient distinction was not meaningful in the VirtualHome problems, where utterance ambiguity was not due to grounding ambiguity.

Table 4.1: Performance of CLIPS vs. baseline methods, measured in terms accuracy (posterior probability of true goal, precision and recall for assistance options), helpfulness (plan length and human cost relative to CLIPS), and human similarity (correlation of goal inferences and assistance options with mean human ratings) Metrics are averaged across the dataset per domain, with standard errors reported in brackets.

Method	$P(g_{true})$	Accuracy		Helpfulness		Human Similarity	
		Assist. Prec.	Assist. Rec.	Rel. Length	Rel. Hum. Cost	Goal Cor.	Assist. Cor.
Doors, Keys & Gems							
Humans	0.67 (0.04)	0.83 (0.02)	0.85 (0.01)	–	–	–	–
CLIPS (Ours)	0.74 (0.05)	0.97 (0.03)	0.97 (0.03)	1.00 (0.00)	1.00 (0.00)	0.93 (0.01)	0.96 (0.01)
Lang. Only IP	0.55 (0.05)	0.90 (0.06)	0.83 (0.06)	1.26 (0.10)	1.18 (0.07)	0.74 (0.01)	0.83 (0.01)
Action Only IP	0.31 (0.04)	0.43 (0.09)	0.40 (0.09)	1.68 (0.15)	1.46 (0.08)	0.15 (0.01)	0.45 (0.01)
Literal Efficient	–	0.65 (0.08)	0.54 (0.07)	1.55 (0.11)	1.58 (0.10)	–	0.47 (0.01)
Literal Naive	–	0.58 (0.04)	0.47 (0.02)	1.54 (0.07)	1.52 (0.05)	–	0.54 (0.01)
GPT-4V	0.29 (0.08)	0.39 (0.08)	0.37 (0.08)	–	–	0.10 (0.01)	0.11 (0.01)
VirtualHome							
CLIPS (Ours)	0.63 (0.04)	0.87 (0.04)	1.00 (0.00)	1.00 (0.00)	1.00 (0.00)	–	–
Lang. Only IP	0.37 (0.05)	0.59 (0.05)	0.96 (0.03)	1.33 (0.07)	1.35 (0.08)	–	–
Action Only IP	0.25 (0.03)	0.61 (0.07)	0.84 (0.07)	1.30 (0.07)	1.33 (0.07)	–	–
Literal Listener	–	0.64 (0.06)	0.70 (0.07)	1.48 (0.08)	1.54 (0.09)	–	–

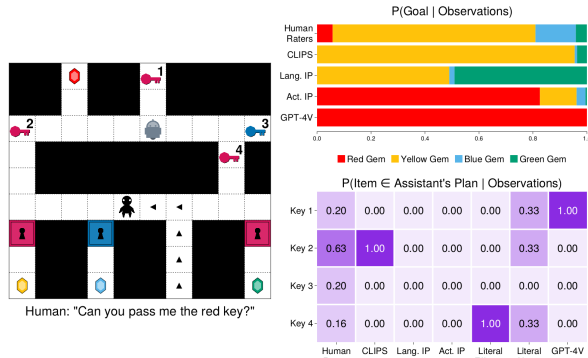
understanding of how to provide assistance without having them solve the entire problem. To compare these human-provided assistance options with CLIPS and our baselines, we extracted the corresponding actions from the assistive plans generated by each method, and (where applicable) estimated the marginal probability of a particular assistance option occurring in the assistive plan via sampling.

4.5.4 Performance Metrics

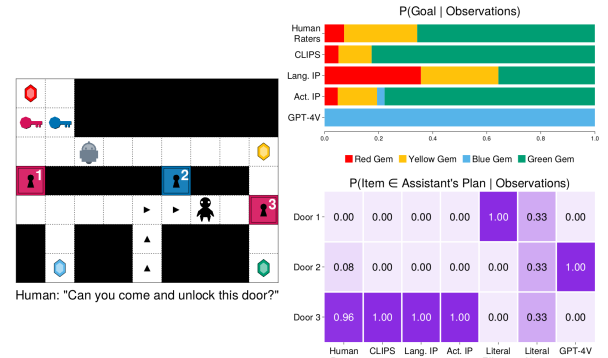
We evaluated CLIPS and the baseline methods in terms of their goal and assistance accuracy, the helpfulness and efficiency of the generated plans, and their similarity to human judgments. To evaluate accuracy, we calculated the probability $P(g_{true})$ assigned to the true goal for the non-literal methods, as well as precision and recall for selecting the optimal assistance options that were consistent with the instruction. To evaluate helpfulness and efficiency, we computed the length of the generated plan and the total action cost incurred by the human principal, relativized to CLIPS. Finally, to evaluate human similarity in mDKG, we calculated the correlation between each method’s outputs with the average ratings provided by our participants. We calculated Pearson’s r as well its variability from 1000 bootstrapped samples of the human dataset.

4.5.5 Results

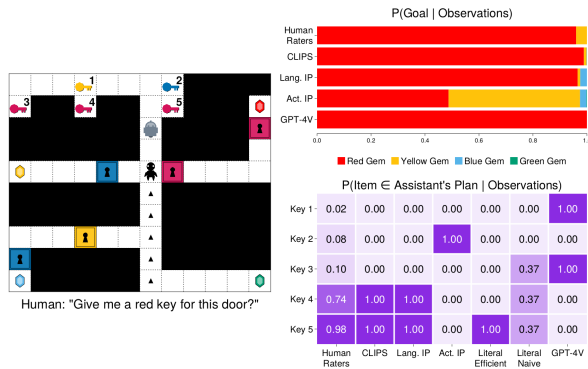
The results of our experiments are presented in Table 4.1. In terms of accuracy, CLIPS assigned higher probability to the true goal than the baselines (note that 100% is not possible since goals cannot always be distinguished, as in Fig. 4.4d), while achieving close to perfect precision and recall in selecting the optimal assistance options. Indeed, CLIPS outperformed the average human in this regard. In contrast, unimodal inverse planning was much less accurate at inferring the true goal, and all baselines selected the correct assistance options at significantly lower rates. This affected the efficiency of assistance: CLIPS was 1.26 to 1.68 times faster in achieving the human’s goal than the baselines, and produced plans that were 1.18 to 1.58 times less costly for the human.



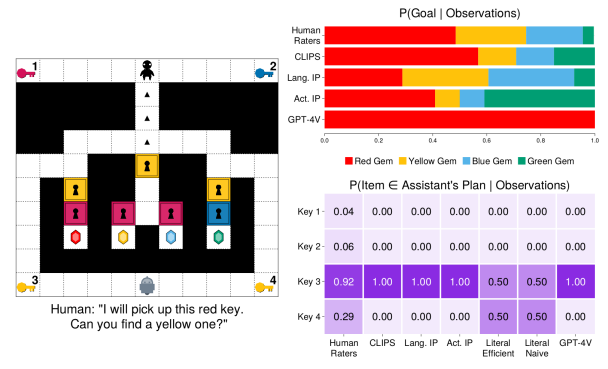
(a) **Ambiguous Predicates.** The human player asks for a red key when three are available. CLIPS resolves this ambiguity, inferring that the human wants *Key 2* to reach the yellow gem.



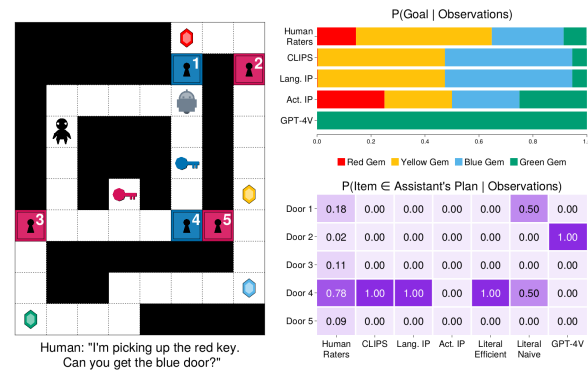
(b) **Ambiguous Indexicals.** The human player refers to a door using the indexical "this". CLIPS interprets this pragmatically, inferring that *Door 3* is intended even though *Door 2* is equally close to the human.



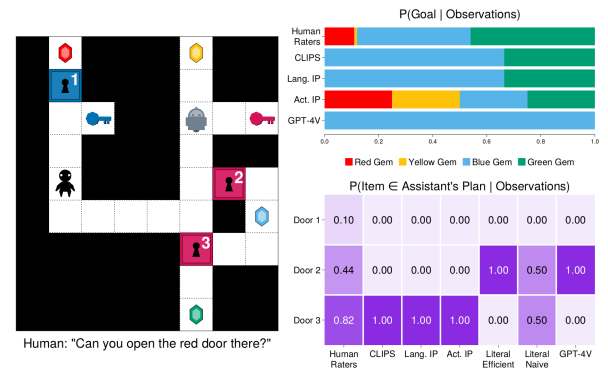
(c) **Partial Instructions.** The human player asks for a single red key, but requires two to reach the red gem. CLIPS infers this and picks up both keys (*Key 4* and *Key 5*), unlike the literal listener baselines.



(d) **Uncertain Goals.** The human player asks for a yellow key while going to pick up a red key. CLIPS pragmatically infers their goal to be one of the three gems on the left, and hence picks up the left key (*Key 3*).



(e) **Joint Instructions.** The human player asks for a blue door to be unlocked while indicating that they will pick up a red key. The CLIPS assistant infers that it should unlock *Door 4* so that the human can unlock *Door 5*.



(f) **Safe Assistance.** The human player asks the robot to unlock a red door when there are two such doors. The CLIPS assistant reasons that unlocking *Door 3* is safer as it leads to *both* blue and green gems, unlike *Door 2*.

Figure 4.4: Goal assistance problems in Doors, Keys & Gems. Each sub-figure contains a visual (*left*), instruction (*bottom left*), goal posteriors produced by each method (*top right*), and the probability of a key or door appearing in the assistive plans generated by each method (*bottom right*). Our pragmatic goal assistance method, CLIPS, best matches the goal inferences and assistance options produced by human raters (averaged across raters). In contrast, language and action-only inverse planning (Lang. IP & Act. IP) have higher goal uncertainty, the literal baselines fail to resolve instruction ambiguity, and GPT-4V often produces incoherent responses.

For human similarity in the mDKG domain, we found that both the goal inferences and the assistance options produced by CLIPS correlated highly with human ratings, achieving Pearson’s r of 0.93 (95% CI: 0.91–0.94) and 0.96 (95% CI: 0.95–0.96) respectively. In contrast, unimodal inverse planning produced goal inferences that were highly dissimilar from humans. Correlation with human-selected assistance options was also poor. These results demonstrate the importance of accounting for pragmatic context. Out of all methods, GPT-4V performed the worst, possibly because of the spatial reasoning and multi-step planning required for coherent goal inference in mDKG. Among participants themselves, we found that the median Pearson’s r of each participant’s ratings with the mean rating was 0.85 (IQR: 0.71–0.94) for goal inferences and 0.87 (IQR: 0.65–0.95) for assistance options, indicating that our dataset of human judgments is a reliable measure of average human performance.

In Figure 4.4, we compare the results of CLIPS against human judgements and baselines on six illustrative goal assistance problems, providing qualitative observations in the captions. The results show how CLIPS closely mirrors human judgements for both goal inference and assistance, resolving ambiguity in predicates (Fig. 4.4a) and indexicals (Fig. 4.4b) while successfully completing partial instructions (Fig. 4.4c) and interpreting joint instructions (Fig. 4.4e). In comparison, the unimodal and literal baselines make less confident inferences or fail to assist appropriately, while GPT-4V provides incoherent answers. We also find that both humans and CLIPS are able to assist appropriately when there is significant goal uncertainty (Fig. 4.4d), and even when the optimal assistive plans for each goal make diverging recommendations (Fig. 4.4f), illustrating the importance of uncertainty-aware assistance.

4.6 Related Work

Theory of Mind as Bayesian Inverse Planning. As introduced in Part I, Bayesian Theory of Mind (BToM) posits that humans reason about the actions and mental states of others through (approximately) Bayesian inference [33, 35, 36, 163]. To efficiently solve these inference problems, CLIPS builds upon SIPS (Chapter 1) to perform online inference of goals and plans [5, 11].

Pragmatic Language Understanding. Understanding the pragmatic use of language has long been of interest to philosophers [137, 172], AI researchers, and computational linguists, with early work developing planning-based theories of pragmatic conversation [160, 161]. While CLIPS contains many elements of these earlier theories, it mostly closely follows Rational Speech Act (RSA) theory. RSA frames communication as a recursive Bayesian process, where speakers make utterances that optimize relevance to listeners, and listeners interpret speaker utterances in light of the pragmatic goals that underlie those utterances [138–140, 173]. CLIPS instantiates the RSA framework in the context of cooperative planning, while vastly expanding the variety of utterances that can be modeled by using LLMs as utterance likelihoods. As such, CLIPS follows recent proposals [174] that LLMs can play the role of a probabilistic and context-sensitive “meaning function” that maps between natural language and a probabilistic language-of-thought [175].

Multimodal Goal Inference and Reward Learning. Goal inference can be framed as online inverse reinforcement learning (IRL), where the aim is to learn a reward or cost function explaining the agent’s behavior in a single episode [176]. CLIPS can thus be viewed as a language-informed IRL method, though IRL is primarily applied offline given a dataset of expert demonstrations [177–179].

Particularly relevant is *reward-rational implicit choice* [180], a framework for multimodal Bayesian reward learning from heterogeneous human feedback. CLIPS can be seen as an application of this framework for specific modalities, but extended to the cooperative setting.

Decentralized Cooperation and Joint Intentionality. When multiple agents cooperate on a task, they usually need to track each other’s mental states to make decisions. Common approaches to this involve recursive reasoning over each other’s goals and plans, which can quickly grow intractable [156, 181]. To address this challenge, computational cognitive scientists have developed models of *joint intentionality* [26], where cooperating agents conceive of themselves as a *group agent* with a common goal in mind [142, 182]. This approach allows groups of agents to converge on shared goals [141] and efficient task decompositions [143] in a decentralized manner with limited recursion. CLIPS uses this insight to model joint planning and pragmatic instruction.

Value Alignment and Assistance Games. CLIPS is a solution strategy for language-augmented goal assistance games, an extension of the assistance game formalism for human-AI value alignment [13, 183]. In contrast to prior solution strategies, our framework leverages joint intentionality when modeling human principals, thereby requiring less recursion than iterated best-response [13], while avoiding the intractability of equilibrium solutions [32, 154]. CLIPS also combines LLMs as a learned model of human language with a rational model of human planning, similar to recent assistant game solvers that integrate learned and model-based components [184].

Instruction Following with Language Models. CLIPS performs grounded instruction following from language [185–187], using LLMs to score utterances with respect to grounded commands. While many studies have used LLMs to translate language into task specifications [188–190] or actions [146, 148], LLM instruction following is often unreliable, due to weaknesses in planning [191] and theory of mind [150, 192]. We instead use LLMs for their key strength — modeling language — leaving reasoning to sound algorithms for planning and inference.

Probabilistic Programming with Language Models. To translate between formal commands and natural language, CLIPS uses LLM probabilistic programs (Chapter 9) — i.e. probabilistic programs that embed calls to a LLM for sampling or scoring. Early work in this area first used LLMs as priors rather than likelihoods [193]. Recent advances have exploited LLMs as both proposals and priors in probabilistic programs that express constrained decoding tasks (e.g. syntactic or semantic constraints) which can be solved via sequential Monte Carlo sampling [194–196].

4.7 Discussion and Future Work

In this chapter, we introduced cooperative language-guided inverse plan search (CLIPS), a Bayesian architecture for pragmatic instruction following and goal assistance. By using a rational cognitive model of how speakers produce both actions and utterances given their goals, CLIPS is able to integrate information from observed actions and ambiguous instruction, inferring a distribution over speaker’s goals and intentions. Using this distribution, a CLIPS assistant is then able to assist the speaker in achieving their goal through expected cost minimization. Through our experiments, we show that the proposed architecture produces human-like outputs on goal inference and assistance tasks. Compared with the baselines, CLIPS also makes more accurate goal inferences, and chooses actions that result in more efficient plans.

While CLIPS demonstrates compelling theoretical properties and strong empirical performance, a number of challenges remain before it can be applied to scenarios with a larger number of goals and commands. As noted in Section 4.3, the Bayesian inference strategy we adopt is fully enumerative, but this approach breaks down once the space of possible goals, plans, and commands grows sufficiently large. Recent advances in probabilistic programming could overcome these bottlenecks. For example, more sophisticated sequential Monte Carlo strategies could be used to flexibly propose and update hypotheses, focusing computation on only the most likely sets of latent variables [5, 122]. These strategies can be applied to perform *constrained decoding* from large language models [194, 195, 197], thereby allowing LLMs to be used as sound proposal distributions over a grammar for commands. This would benefit from LLMs’ performance at few-shot semantic parsing and translation [198], while preserving the Bayesian nature of our framework.

The assistance policy we introduced in Algorithm 4 could also be extended in a number of ways. In many assistive settings, uncertainty is sufficiently high that it makes sense for the assistant to take *information gathering actions*. Indeed, this is what people typically do when we are confused: we ask questions. To enable this ability, assistants should perform *belief-space planning* over the set of possible goal beliefs [158, 199], thinking ahead about which actions reveal more information about the principal’s goals. Such planning would alleviate issues that can arise due to the Q_{MDP} approximation [168, 169] used by Algorithm 4.

In the longer run, we envision extending CLIPS into an instructable AI assistant capable of *grounded task dialogue* in cooperative tasks. Such an assistive agent would not only be able to interpret human speech acts in a grounded environment, but also *plan* to produce speech acts such as questions, informative statements, and requests. If successful, this would constitute a considerable step towards the “ideal helpful computer conversant” outlined by early AI researchers, in contrast to the “meager techniques for getting machines to reason explicitly about their utterances” that existed at the time (Cohen [159]), and that arguably still do today.

4.8 Appendix

4.8.1 Model Configuration

CLIPS & Unimodal Inverse Planning

We describe the configuration for CLIPS and unimodal inverse planning together, since the only difference between the former and the latter is whether both action likelihoods $P(a_t^h|s_t, \pi)$ and utterance likelihoods $P(u_t|s_t, \pi)$ are incorporated into the weight w associated with each hypothesized goal g , cost function C and policy π in Algorithm 3.

Goal and Cost Priors. In both domains, we assumed a uniform goal prior $P(g)$ over the set of possible goals (4 in mDKG, 6–12 in VH). In mDKG, we considered four possible cost functions $C \in \mathcal{C}$, and placed a uniform prior $P(C)$ over such cost functions. This made CLIPS more robust to the possibility that the human would prefer to do less (or more) work, asking the robot to pick up keys or unlock doors that would be sub-optimal if action costs were equal between both agents. The costs for each action were as follows:

- Human pick-up: 5. Robot pick-up: 1. Wait: 0.6. All other action costs: 1.
- Human pick-up: 5. Robot pick-up: 1. Wait: 0.6. Other human costs: 2. Other robot costs: 1.
- Human pick-up: 5. Robot pick-up: 1. Robot unlock: 5. Wait: 0.6. All other action costs: 1.
- Human pick-up: 5. Robot pick-up: 1. Robot unlock: 5. Wait: 0.6. Other human costs: 2. Other robot costs: 1.

In VH, we avoided the need to perform online inference over multiple cost functions by using a slightly more flexible utterance model that allowed for the possibility that the principal might ask the assistant to carry out sub-optimal actions (e.g. asking the assistant to acquire *all* remaining items, even though the optimal joint plan would have both agents share the work more equally). This allowed us to use a *single* cost function that assigned unit cost to all actions for both agents. Future work should investigate how more flexible models for multi-agent task division (e.g. [143]) could be integrated with CLIPS, along with offline learning of cost function priors.

Policy Configuration. In Bayesian inverse planning, an algorithm is required to compute the joint policy π for each possible goal. While it is possible to do this using a domain-general planning algorithm and search heuristic, we configured our planners in a domain-specific manner to achieve greater speed and reliability.

Doors, Keys & Gems. In mDKG, we used the Real-Time Adaptive A* (RTAA*) variant [56] of real-time heuristic search [55] to incrementally compute the value function $V(s)$ for the policy π . Given $V(s)$, one can then compute $Q(s, a) = C(s, a) + \sum_{s'} P(s'|s, a)V(s')$. Each time `RTHS-POLICY-UPDATE` is called, we perform A* search up to a search budget of 2^{18} nodes from each state s_n that neighbors the current state s , using the current estimate $\hat{V}$ of the value function as the search heuristic. Once A* search terminates at state s_f (either by reaching the goal or using up the budget), we update the value function for *all* states s_i in the interior of the search tree as follows:

$$\hat{V}(s_i) \leftarrow \hat{V}(s_f) - C(s_n, s_f) + C(s_n, s_i)$$

where $C(s_n, s_i)$ is the cost of getting from s_n to s_i by traversing the search tree. As shown in [56], as long as $\hat{V}$ is initialized to a *consistent* (under-)estimate of the true value function, subsequent

$\hat{V}$ values will remain consistent. We ensured this by (lazily) initializing $\hat{V}$ with a minimum spanning tree heuristic [200] over the graph of keys and doors required to reach the goal, using a preprocessing step used in inventory-aware pathfinding [201] to extract necessary keys and doors. To further accelerate planning, we pruned actions that were irrelevant to the goal (e.g. picking up unnecessary keys), and implemented cost-minimal path reuse from the tree-adaptive variant of RTAA* [202], allowing us to terminate A* search whenever an existing path to the goal was found. Collectively, these algorithmic choices allowed us to estimate the optimal value function V^* in close to real-time for most timesteps, despite the difficulty of optimal multi-agent planning.

VirtualHome. In VH, we also experimented with using RTAA* to compute the policy π . However, this unpredictably increased the latency of inverse planning when the search budget was too high, and led to unstable Q -value estimates when the search budget was too low. Instead we found it sufficient to compute an analytic lower bound $\hat{V}$ to the optimal value function V^* by counting the number of remaining items and movements required. Using $\hat{V}$ was both faster (no search was required), and produced more consistent estimates of the relative Q -values $Q(s, a) - V(s)$ (also known as the *advantage function*) for each state s and goal g , resulting in more stable action likelihoods $P(a|s, \pi)$ and goal inferences.

Rationality Modeling As in hierarchical Bayesian approaches to inverse reinforcement learning [203] and theory-of-mind [37], we placed a Gamma prior over the rationality parameter β for the Boltzmann policy, discretized across 33 values from 2^{-3} to 2^5 by incrementing the exponent with a step size of 0.25. In mDKG, we used a $\text{Gamma}(\alpha = 0.5, \lambda = 1)$ prior, while in VH, we used a $\text{Gamma}(\alpha = 1, \lambda = 2)$ to match the less optimal actions taken in that domain. For efficient inference, we used Rao-Blackwellization [204], marginalizing over all values of β when computing the action likelihood $P(a^h|s_t, \pi)$, then analytically updating the local posterior $P(\beta|a_{1:t}^h, s_{1:t}, \pi, g)$.

Utterance Model. In the CLIPS utterance model, salient actions $\alpha_{1:m}$ are first extracted from the joint policy π , before being sub-sampled into a command c , which is then translated into an utterance u . In our implementation, we enumerated over commands to reduce variance. This section describes how we extracted salient actions, the structure of commands, and the command enumeration strategy for each domain.

Extracting Salient Actions. Following work in linguistics [205, 206] and RSA theory [138], we assumed that some types of actions were more likely to be communicated because of their *salience*. To model this, after rolling out the joint policy π for up to L steps, we filtered out all non-salient actions, leaving only the salient actions $\alpha_{1:m}$ to be communicated. In VH, to account for the possibility of the principal delegating *all* remaining items to the assistant (instead of equally sharing), we enforced that the principal could not collect new items when rolling out the policy.

As in [14], we manually defined action salience: In mDKG, pickup, unlock, and handover actions and iscolor predicates were salient, while in VH, grab actions and itemtype predicates were salient. In general, however, action salience could be learned, e.g. by offline estimation of which actions tend to be more *unique* and hence *informative* for a particular goal and plan.

Command Enumeration. Formally, we define a command $c := ([a, \dots], \{p, \dots\}, \{(v, \tau), \dots\})$ as a sequence of (possibly lifted) actions $[a, \dots]$, a set of associated predicates $\{p, \dots\}$ that qualify an action’s arguments, and a set $\{(v, \tau), \dots\}$ of free variables v with associated types τ which appear as an argument in at least one action in $[a, \dots]$. For example, the command (handover robot human ?key1) where (iscolor ?key1 blue) has one partially lifted action $a = (\text{handover robot human$

?key1), along with a predicate $p = (\text{iscolor } ?\text{key1 } \text{blue})$ that describes the variable $v = ?\text{key1}$ as blue, where the type of $?key1$ is $\tau = \text{key}$.

To generate commands from a set of salient actions $\alpha_{1:m}$ and associated predicates $p_{1:m}$, we enumerated over all subsets of salient action-predicate pairs of up to size $K = 3$ in mDKG and $K = 4$ in VH, before pruning away combinations of actions that were less likely. In mDKG, we did this by excluding commands with more than two distinct action types, commands where *all* actions were taken by the speaker/principal, and also commands which involved chains of dependent actions (e.g. picking up a key, followed by handing over the same key). In VH, we excluded all commands that involved *any* action by the speaker (since our VH dataset did not contain joint instructions). Following enumeration of these (ground) commands, we then lifted the commands, replacing concrete object names with corresponding typed variables. We also replaced all agent identifiers such as human and robot with corresponding indexicals (me and you). Duplicate lifted commands were then removed.

These choices were made partly to limit the commands enumerated. Future extensions could make use of constrained LLM decoding [194, 195, 198] to avoid the limitations of command enumeration, directly translating an utterance u into e.g. an infinite grammar over commands.

Language Models. In mDKG, we used OpenAI’s 6.7B curie model [170] as our LLM, while in VH, we used the more recent davinci-002 model. Both of these were base models, providing more diffuse probabilities that avoid issues with over-confidence and mode collapse that can arise in LLMs finetuned on instructions or human preferences [207, 208].

LLMs were instantiated as generative functions in the Gen probabilistic programming system [4] using the GenGPT3.jl library [21] (see Chapter 9), allowing us to easily score the probability $P(u|c, \mathcal{E})$ of an utterance u given a command c and few-shot examples $\mathcal{E}$ provided in the prompt. Each utterance was scored against all enumerated commands in parallel via a batched API request. In mDKG, we provided 18 examples of commands being translated to utterances, while in VH, we provided 12 examples. Examples can be found in the source code.

Assistance Policy. When running the Q_{MDP} assistance policy shown in Algorithm 4, we filtered out goal hypotheses with a weight w^i of less than 0.02, so as to avoid the cost of updating the policies π^i for unlikely goals. In mDKG, we updated the policies using RTAA* with a node budget 2^{16} and a time limit of 10s. In VH, we updated Q -values using the analytic lower bound $\hat{V}$ mentioned earlier.

We evaluated the assistance policy against a simulated human principal, which was designed to (i) follow a “ground truth” plan whenever possible, (ii) switch to following an approximately optimal joint policy when the following the original plan was not possible, (iii) fallback to a single-agent policy upon detection that the assistant was not following the joint policy to the true goal (i.e. not being helpful). To detect this, the principal compared how likely the assistant was to be following a joint policy to the true goal (Boltzmann, with $\beta = 1$) vs. a random baseline, switching to the fallback once it was 10 times more likely for the assistant to be acting at random.

LLM-Based Literal Listener

Our literal listener baselines consisted of two components: literal command inference, and literal assistance. We describe each below.

Literal Command Inference. Command inference was implemented similarly to inference over the CLIPS utterance model. We first enumerated over a set of possible commands in the current state s , then used a LLM to score the probability $P(u|c, s)$ of the observed utterance u for each command c . Normalizing these probabilities produced a posterior over commands $P(c|u, s)$.

Utterances. In mDKG, we made command inference easier by replacing joint instructions (e.g. “I’m getting the red key, can you get the blue key?”) with assistant-directed instructions (e.g. “Can you get the blue key?”). In VH, we handled multiple utterances u_{t_1}, u_{t_2} by merging the inferred commands $c = c_{t_1} \cup c_{t_2}$. This created a product distribution $P(c|u_{t_1}, u_{t_2}, s_{t_1}, s_{t_2}) = P(c_{t_1}|u_{t_1}, s_{t_1})P(c_{t_2}|u_{t_2}, s_{t_2})$ over merged commands.

Command Enumeration. To enumerate commands, we first enumerated all salient actions that were reachable from the current state s . We then enumerated all subsets the set of salient actions of up to size $K = 3$ in mDKG and $K = 4$, pruning away certain combinations as in the CLIPS utterance model. Commands were then lifted, and duplicate commands were removed.

Language Models. We used text-davinci-002 for mDKG. Given the larger space of commands for the literal listener, we found that using a RLHF finetuned model such as text-davinci-002 ensured that more posterior probability mass concentrated on the intuitively correct set of commands. To simulate this effect in our VH experiments after text-davinci-002 access was shutdown by OpenAI, we used davinci-002 with a temperature of 0.5. 12 few-shot examples were used for both mDKG and VH.

Literal Assistance. Literal assistance involved several steps: (i) Systematically sampling a command c from the inferred distribution $P(c|u, s)$; (ii) Converting the command c to an equivalent goal formula g_c ; (iii) Generating an assistive plan that first achieved g_c , then the true goal g .

Sampling Commands. As described in the main text, we estimated the *mean* performance of literal assistance by drawing $m = 10$ samples from the distribution $P(c|u, s)$ and averaging our metrics across these samples. To do this in a low-variance manner, we first sorted the commands by their inferred probabilities. We then used systematic sampling [209], selecting a command c_i as long the random variable x_i fell within its corresponding probability bin, where $x_i = (i - 1)/m + x$ for $i \in [1, m]$, and $x \sim U(0, 1/m)$.

Mapping Commands to Goals. Given a sampled command $c = ([a, \dots], \{p, \dots\}, \{(v, \tau), \dots\})$, we mapped each action a to a set of goal predicates $\{p_a, \dots\}$ that it achieves. By combining these predicates with the qualifier predicates $\{p, \dots\}$, then adding an existential quantifier for any variables, we created a PDDL goal formula g_c that could be used by a planning algorithm. For example, the command (handover robot human ?key1) where (iscolor ?key1 blue) would be converted to the goal formula (exists (?key1 - key) (and (pickedup-by robot ?key1) (has human ?key1) (iscolor ?key1 blue))).

Some commands had multiple possible groundings. In mDKG (but not in VH), these groundings corresponded to plans with different lengths. As such, we implemented both a *naive* and *efficient* form of literal assistance. In naive assistance, we generated plans for all possible groundings of the lifted command c , and averaged performance metrics across these plans. For example, if key2 was blue, then a grounding of (handover robot human ?key1) where (iscolor ?key1 blue) might be (handover robot human key2), which translates to the goal (and (pickedup-by robot key2) (has human key2)). In efficient assistance, we simply generated a plan for existential goal formula implied by the lifted command, automatically finding the most efficient way to satisfy that formula.

Plan Generation. We generated an assistive plan in two phases. In the first phase, we generated an optimal *joint* plan to the commanded goal g_c using A* search with a node budget of 2^{16} (for commands with handover actions, this might involve both agents walking towards a convenient meeting point). If finding a joint plan was unsuccessful, we fell back to computing a plan where only the assistant was allowed to move. In the second phase, we then enforced that only the principal was allowed to move (in mDKG) or pick up items (in VH), and computed the remaining plan to achieve the principal’s true goal g (again, using A* search with a node budget of 2^{16}).

Multimodal LLM (GPT-4V)

We used the public preview of GPT-4V (gpt-4-1106-vision-preview) as a baseline model in our mDKG experiments. We used zero-shot prompting, providing the model with effectively the same instructions as given to the human raters, along with the final frame of the GIF animation shown to humans (Figure 4.5). Responses were generated with a temperature of 0.

You’re watching a human and a robot collaborating on a treasure game shown below. The human player wants to collect exactly one of the four target gems. The two players cooperate to achieve the human’s goal.

During the game, the human may communicate with the robot to coordinate their actions. Any communication that happens is shown below the visuals.

The rules of the game are as follows:

- The human player’s goal is to collect one target gem.
- The robot plays an assistive role, and cannot pick up gems on its own.
- The players can move on the white squares.
- The players have a full view of the map at all times.
- Keys are used to unlock doors with the same color (e.g. red keys only unlock red doors).
- Each key can only be used once. Keys disappear after being used.
- One player can pass keys to the other if they’re next to each other.
- Players can occupy the same square and walk past each other.
- The keys and doors are labeled. The labels are shown on the top right corner of each cell.

stimulus.png

In the scenario above, the human player took several actions, then gave the instruction: "Can you hand me that key?"

Given this information, what gem is most likely to be the human agent’s goal? Which key(s) should the robot pick up to best assist the human?

Please respond in the following JSON format:

```
{ "goal": [red/yellow/blue/green], "assist": [key1, key2, etc.] }
```

Figure 4.5: GPT-4V zero-shot prompt for an example key assistance problem in Doors, Keys & Gems.

4.8.2 Goal Assistance Problems

Table 4.2 and Table 4.3 describe the scenarios and conditions for our goal assistance problems in multi-agent Doors, Keys & Gems (mDKG) and VirtualHome (VH) respectively. VH scenarios were encoded in PDDL [46] for use with our inverse planning system, but they can be translated to VirtualHome’s evolving graph simulator or Unity simulator [144].

Table 4.2: Goal assistance problems in Doors, Keys & Gems. 30 problems were divided into those that required assistance with either picking up keys or unlocking doors. Actions and utterances were varied according to the following factors.

Factor	Condition	Description
Instruction Uncertainty	Ambiguous	Ambiguous instruction that can be associated with many possible items. (e.g. “Can you get me the red key?” when multiple red keys are available.)
	Partial	The principal only requests a subset of the items they require to achieve their goal. (e.g. “Can you get me the red key?” when both red and blue keys to reach its goal.)
Agent Involvement	Assistant Actions	The instruction involves only actions that the assistant should take. (e.g. “Can you pick up that key?”, “Please unlock the red doors.”)
	Joint Actions	The instructions contain information about both agents’ actions. (e.g. “I’m getting the red key, can you get the blue key?”)
Goal Uncertainty	Low Uncertainty	The actions and instructions combined are largely sufficient to uniquely determine the goal. (e.g. In Figure 4.4b, only the red gem is likely.)
	High Uncertainty	The principal’s goal remains ambiguous even after observing both their actions and instructions. (e.g. In Figure 4.4d, the three gems on the left are plausible.)

Table 4.3: Goal assistance problems in VirtualHome. 25 problems were divided into (a) three scenarios, which correspond to different initial states of the environment, and had different sets of possible goals. Actions and utterances (b) were varied across several conditions.

Scenario	Goal Category (k)	Description
Food Preparation (6 goals)	Salads (3)	Place a chef’s knife & ingredients (onion, cucumber / potato & tomato / chicken) on the table.
	Stews (3)	Place an onion, carrot, a main (salmon / chicken / potato) & a bottle of wine on the table.
Table Setting (12 goals)	Plates (4)	Place n plates on the table, where $n \in [1, 4]$.
	Plates & Cutlery (4)	Place n plates, n forks & n knives on the table, where $n \in [1, 4]$.
	Plates & Bowls (4)	Place n plates, n forks, n knives & n bowls on the table, where $n \in [1, 4]$.
Drink Pairing (12 goals)	Wine (3)	Place wine and n wineglasses on the table, $n \in [1, 3]$.
	Wine Pairing (3)	Place wine, cheese, n wineglasses & n forks on the table, $n \in [1, 3]$.
	Juice (3)	Place juice & n waterglasses on the table, $n \in [1, 3]$.
	Juice Pairing (3)	Place juice, n waterglasses, n cupcakes & n forks on the table, $n \in [1, 3]$.

(a) Scenarios and goals.

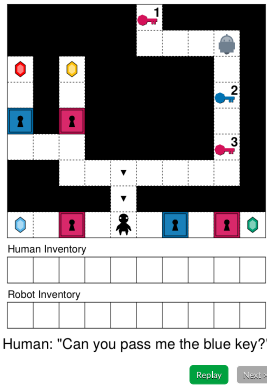
Factor	Condition	Description
Instruction Uncertainty	Ambiguous Number	Instruction which mentions an ambiguous number of items to be collected. (e.g. “Can you help with these plates?”)
	Ambiguous Type	Instruction which is ambiguous about the type of item to be collected. (e.g. “Can you get two glasses?”, “And I need the veggies for the stew.”)
Utterance Count	Single	The principal gives a single instruction to the assistant. (e.g. “Go get the forks and knives.”, “Can you get some wine?”)
	Multiple	The principal gives an instruction to the assistant, takes a few more actions, then gives another instruction. (e.g. “Could you find two glasses?” followed by “Get some forks as well.”)

(b) Factors and conditions.

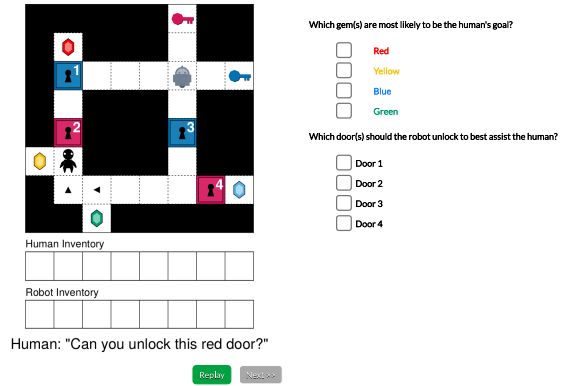
4.8.3 Human Experiments

Here we provide additional details about our human experiments for the mDKG domain. Participants first completed a tutorial that explained the task and experimental interface, then answered 5 comprehension questions before proceeding to the main experiment. In the main experiment, they were shown 15 stimuli in randomized order, which involved either keys or doors as assistance options (Figure 4.6).

To incentivize accurate but calibrated responses, participants were rewarded for accurately guessing the true goal or selecting the optimal assistance option. Specifically, they earned $1/N$ bonus points if they selected N goals out of which one was the true goal, but 0 points if none of their selected goals was the true goal. They earned one additional point if they selected exactly the right assistance options. Participants were paid US\$1 for every 30 bonus points they earned, on top of a base pay of US\$16/hr (US\$4 per experiment).



(a) Key assistance problem.



(b) Door assistance problem.

Figure 4.6: Web interface for human experiments. Participants had to select checkboxes for all gems they thought were likely to be human principal’s goal, and select all keys or doors they thought the robot assistant should pick-up or unlock.

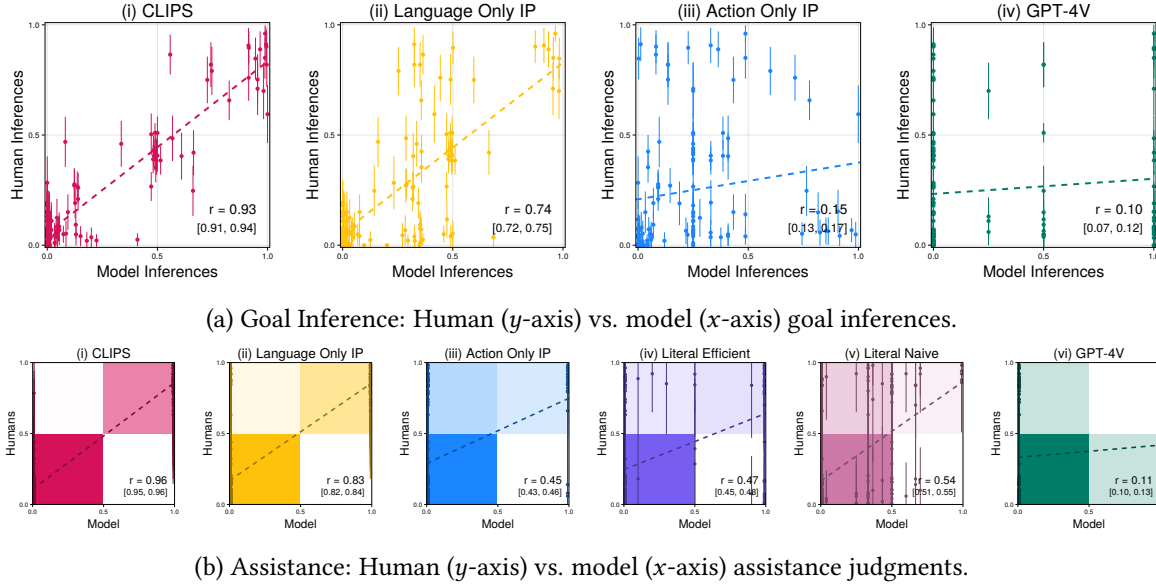


Figure 4.7: Human-model correlation plots for Doors, Keys & Gems. In (a), we show scatter plots comparing average human goal inferences vs. model goal inferences across all stimuli. In (b), we compare average human assistance judgments vs. model outputs by overlaying a scatter plot over a binarized confusion matrix. Error bars correspond to 95% confidence intervals for the mean human response. At the bottom-right of each plot we report Pearson’s r (95% bootstrap CI below).

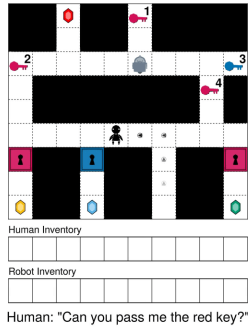
In Figure 4.7, we show correlation plots that provide a more comprehensive view of how well each model matches human responses. As Figure 4.7a shows, CLIPS best reproduces human goal inferences. In contrast, unimodal inverse planning tends to be under-confident relative to humans, since they only incorporate evidence. GPT-4V tends to be over-confident on stimuli where humans are not (and vice versa), resulting in a very low correlation overall.

To compare human assistance judgments with model outputs, Figure 4.7b shows scatter plots laid over binarized confusion matrices, generated by thresholding human and model responses at 0.5, then plotting bin counts as a heatmap. Since many of our models return either 1 or 0 for assistance, this provides a better way to visualize human-model agreement. CLIPS best matches human judgments, followed by language-only IP. The other models often fail to select an assistance option in cases where humans do (deeper colored top-left quadrants). GPT-4V selects options in many cases where humans do not (bottom-left quadrant), achieving the lowest correlation.

4.8.4 GPT-4V Responses

To better illustrate the capabilities and limitations of multimodal LLMs, we show several example responses by GPT-4V in Figure 4.8. As the responses illustrate, GPT-4V is highly inaccurate in its ability to perceive objects in the provided image and determine their spatial relationships, sometimes confabulating object labels that do not exist (Fig. 4.8a–b), failing to notice which gems are locked behind which doors (Fig. 4.8b–c), and failing to perceive which items are along a path to the goal (Fig. 4.8c). This weakness at spatial perception makes current multimodal LLMs inadequate for assistive tasks like ours, which require coherent spatial reasoning.

(a) Ambiguous predicates.



GPT4V Response:

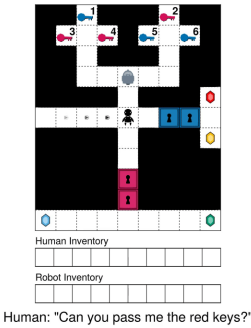
Based on the image and the rules provided, the human player is asking for the red key, which suggests that their goal is to collect the red gem. The red gem is behind the red door labeled "1," and the red key is required to unlock it. Since the human player can only collect one target gem, and they are asking for the red key, it is most likely that the red gem is their goal.

The robot should pick up the red key to assist the human player. The robot cannot pick up gems, so it only needs to focus on the keys that will help the human player achieve their goal.

Here is the response in the requested JSON format:

```
{ "goal": ["red"], "assist": ["key1"] }
```

(b) Ambiguous number.



GPT4V Response:

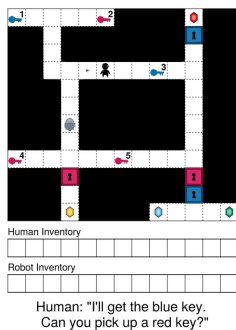
Based on the image and the information provided, the human player has asked the robot to pass the red keys. This implies that the human player is aiming for a gem that requires passing through a red door. In the image, the red gem is behind a red door, so it is the most likely target for the human player.

The robot should pick up the keys that will allow the human to reach the red gem. The red gem is behind door 3, which requires key 3. However, to reach key 3, the robot first needs to unlock door 1 with key 1. Therefore, the robot should pick up key 1 and then key 3 to assist the human player.

Here is the response in the requested JSON format:

```
{ "goal": ["red"], "assist": ["key1", "key3"] }
```

(c) Joint instructions.



GPT4V Response:

Based on the information provided and the layout of the game board, the human player has expressed an intention to pick up the blue key. This key can unlock the blue door, which is blocking the path to the green gem. Since the human player's goal is to collect one target gem and they are going for the blue key, it is reasonable to assume that the green gem is their intended target.

The robot, in an assistive role, has been asked to pick up a red key. There are two red keys (key4 and key5) and two red doors on the map. However, since the human is going for the green gem, the robot should pick up the red key that will assist in this goal. Key4 is the red key that unlocks the red door on the path to the green gem. Therefore, the robot should pick up key4.

Here is the JSON response with the likely goal and the key the robot should pick up:

```
{ "goal": ["green"], "assist": ["key4"] }
```

Figure 4.8: Example responses by GPT-4V. GPT-4V produces two paragraphs of reasoning text, then gives its final response in JSON. A full set of GPT-4V's responses can be found at <https://osf.io/v8ru7/>.

However, the reasoning text produced by GPT-4V appears to demonstrate internal means-ends consistency (keys are required to unlock doors of the same color, which in turn allow the human to reach the gems behind them). As prior work has shown [210], this suggests that if an abstract symbolic representation is provided to a state-of-the-art LLM with the appropriate high-level relations (spatial connectivity etc.), they may be able to perform significantly better. Nonetheless, deriving an abstract symbolic representation from the underlying gridworld is itself a form of complex spatial reasoning [201], which likely presents a challenge to contemporary LLMs.

4.8.5 Alternate Assistance Methods

In the main text, we reported results for an *offline* version of the Q_{MDP} assistance policy in Algorithm 4, which keeps the goal posterior fixed after observing $a_{1:T}^h$ and $u_{1:T}$, rather than continuing to update the posterior as the human principal takes more actions. Here we report results for the *online* version of Q_{MDP} assistance. We also report results for an alternative assistance method, $\bar{\pi}_{\text{MDP}}$, which is constructed by averaging over *policies* π for each inferred goal g : Rather than minimizing the expected Q -value as in Q_{MDP} assistance [168], $\bar{\pi}_{\text{MDP}}$ first samples a goal g from the posterior, then follows the policy π for that goal. This can be viewed as Thompson sampling [211] over policies.



Figure 4.9: Examples comparing (offline) Q_{MDP} and $\bar{\pi}_{\text{MDP}}$ assistance in Doors, Keys & Gems. In Q_{MDP} assistance, we repeatedly take the best action that minimizes the expected goal achievement cost with respect to the posterior over goal specifications. In $\bar{\pi}_{\text{MDP}}$ assistance, we randomly select a single goal specification g from the posterior, and follow the optimal joint policy for that goal.

Table 4.4: Comparison of assistance methods measured in terms accuracy (precision and recall for assistance options), helpfulness (plan length and human cost relative to CLIPS with offline Q_{MDP} assistance), and human similarity (correlation of assistance options with mean human ratings) Metrics are averaged across the dataset per domain, with standard errors reported in brackets.

Inference	Assistance	Accuracy		Helpfulness		Human Sim.
		Assist. Prec.	Assist. Rec.	Rel. Plan Length	Rel. Human Cost	Assist. Cor.
Doors, Keys & Gems						
CLIPS	Q_{MDP} (Offline)	0.97 (0.03)	0.97 (0.03)	1.00 (0.00)	1.00 (0.00)	0.96 (0.01)
Lang. Only IP	Q_{MDP} (Offline)	0.90 (0.06)	0.83 (0.06)	1.26 (0.10)	1.18 (0.07)	0.83 (0.01)
Action Only IP	Q_{MDP} (Offline)	0.43 (0.09)	0.40 (0.09)	1.68 (0.15)	1.46 (0.08)	0.45 (0.01)
CLIPS	Q_{MDP} (Online)	1.00 (0.00)	1.00 (0.00)	0.99 (0.01)	0.99 (0.00)	0.97 (0.00)
Lang. Only IP	Q_{MDP} (Online)	0.90 (0.06)	0.83 (0.06)	1.26 (0.10)	1.18 (0.07)	0.83 (0.01)
Action Only IP	Q_{MDP} (Online)	0.88 (0.06)	0.90 (0.06)	1.15 (0.07)	1.09 (0.03)	0.83 (0.01)
CLIPS	$\bar{\pi}_{MDP}$	0.91 (0.03)	0.91 (0.03)	1.09 (0.04)	1.08 (0.02)	0.96 (0.00)
Lang. Only IP	$\bar{\pi}_{MDP}$	0.79 (0.05)	0.77 (0.05)	1.31 (0.09)	1.19 (0.04)	0.85 (0.01)
Action Only IP	$\bar{\pi}_{MDP}$	0.72 (0.04)	0.41 (0.05)	1.58 (0.12)	1.45 (0.05)	0.58 (0.01)
VirtualHome						
CLIPS	Q_{MDP} (Offline)	0.87 (0.04)	1.00 (0.00)	1.00 (0.00)	1.00 (0.00)	–
Lang. Only IP	Q_{MDP} (Offline)	0.59 (0.05)	0.96 (0.03)	1.33 (0.07)	1.35 (0.08)	–
Action Only IP	Q_{MDP} (Offline)	0.61 (0.07)	0.84 (0.07)	1.30 (0.07)	1.33 (0.07)	–
CLIPS	Q_{MDP} (Online)	0.90 (0.04)	1.00 (0.00)	1.12 (0.15)	1.13 (0.17)	–
Lang. Only IP	Q_{MDP} (Online)	0.59 (0.05)	0.96 (0.03)	1.33 (0.07)	1.35 (0.08)	–
Action Only IP	Q_{MDP} (Online)	0.84 (0.06)	0.92 (0.06)	1.03 (0.04)	1.04 (0.05)	–
CLIPS	$\bar{\pi}_{MDP}$	0.59 (0.06)	0.65 (0.06)	1.22 (0.06)	1.33 (0.07)	–
Lang. Only IP	$\bar{\pi}_{MDP}$	0.45 (0.05)	0.57 (0.06)	1.37 (0.08)	1.48 (0.09)	–
Action Only IP	$\bar{\pi}_{MDP}$	0.46 (0.06)	0.45 (0.05)	1.61 (0.14)	1.77 (0.16)	–

Qualitative Analysis. Figure 4.9 illustrates how (offline) Q_{MDP} assistance differs from $\bar{\pi}_{MDP}$. While $\bar{\pi}_{MDP}$ assistance better reproduces the gradedness seen in human assistance judgments, this comes at the cost of reduced safety and helpfulness. By following a policy π to goal g according to how likely g is, $\bar{\pi}_{MDP}$ fails to aggregate information across multiple uncertain goals when deciding the best action to take. This is most clearly illustrated in Figure 4.9c, where CLIPS assigns higher probability to the blue gem over the green gem. Q_{MDP} assistance safely unlocks *Door 3* instead of *Door 2*, since doing so will ensure that both the blue gem and green gem are reachable. In contrast, $\bar{\pi}_{MDP}$ unlocks *Door 2* on average 66% of the time, which is unsafe. However, as Figure 4.9d shows, $\bar{\pi}_{MDP}$ is slightly better when CLIPS produces the wrong goal inferences (due to the sub-optimality of the human’s actions with respect to the true goal). In this single failure case, Q_{MDP} confidently executes the wrong actions, whereas $\bar{\pi}_{MDP}$ assistance is sometimes successful.

Quantitative Analysis. We present the performance of different assistance methods in Table 4.4. Compared to offline Q_{MDP} assistance, $\bar{\pi}_{MDP}$ assistance was less accurate, and produced longer plans on average. This highlights the importance of minimizing expected cost under uncertainty, rather than just sampling a likely goal to follow. As might be expected, online Q_{MDP} assistance was more accurate, since the posterior over goals was updated as new actions were observed. This significantly improved the performance of action-only inverse planning (note that language-only inverse planning is the same regardless of assistance method, since the human principal communicates no further instructions). However, multimodal goal inference via CLIPS combined with online Q_{MDP} assistance was still the most accurate, since it leveraged linguistic information from the beginning of assistance.

Apart from one failure case in VirtualHome where the assistant and simulated human principal were caught in a livelock, CLIPS with online Q_{MDP} assistance was slightly more helpful on average than offline assistance. Nonetheless, offline assistance was almost as helpful. This suggests that once enough actions are observed to disambiguate the principal’s instruction, CLIPS can still assist effectively even if it has partial to no observability of the principal’s subsequent actions.

Chapter 5

Inferring Shared Goals from Communicating Agents

The human way of communicating—by intentionally informing others of things for cooperative motives—comes so naturally to us that we can hardly conceive of any other.

Michael Tomasello

In Chapter 4, we introduced cooperative language-guided inverse plan search (CLIPS) as a Bayesian architecture that provides language-guided assistance by modeling humans as cooperative communicators. However, the underlying model of communicative cooperation can be used for more than just assistance: Instead of being employed by an *internal* observer (the assistant/listener) to model the person being helped (the principal/speaker), it can be employed by an *external* observer to understand and infer the goals of a *team* of communicating agents.

This chapter illustrates how we can readily adapt CLIPS to perform team-level goal inference from the actions and utterances of agents working on a shared task. Similar to prior work on joint intentionality and group-level theory of mind [141, 143, 163, 212], we model the cooperative team as a single *group agent*, instead of separately modeling each agent as an individual with their own goals and beliefs. As in the assistive version of CLIPS, the speaker may communicate a summary of the team’s joint plan as an instruction to the listener, which is mapped to natural language using a LLM. Both agents may also take actions towards their shared goal, allowing a third-person observer to infer this goal from their actions and words.

To evaluate this approach, we conduct a series of experiments on team-level goal inference in the multi-agent Doors, Keys & Gems environment, comparing the outputs of CLIPS with the goal inferences of third-person human observers when they are shown the same actions and utterances. We also perform experiments in a baseline setting where language is omitted. Unlike the experiments in Chapter 4, we elicit goal inferences at multiple time points in a single trajectory. We find that human inferences are highly correlated with the inferences produced by CLIPS, and that language instructions greatly accelerate convergence to the team’s true goal, with remaining ambiguity resolved by the observed actions. These findings indicate that CLIPS is not just useful for assistance, but also allows for richer models of the communicating social world.

5.1 Modeling Communicating Teams as Group Agents

We previously developed CLIPS as a model of an *individual* cooperative speaker who enacts and articulates a joint plan when interacting with a listener. However, this model does not obviously to a communicating *team*. After all, a complete model of rational communication seems to require more sophistication: Since each agent in the team may not initially know the team’s goal (as is the case for our robot assistant), an observer would have to model not only the team’s shared goal, but also each agent’s *beliefs* about their shared goal, including how those beliefs are formed through goal inference. In addition, agents who know the goal (such as our human principal) may have to be modeled as *pedagogically* selecting utterances in order to best reduce listeners’ uncertainty about the shared goal and plan [154, 213].

While treating each agent as an individual with their own mental states would indeed be the most *realistic* model, we propose that it is often not necessary for understanding the goals of a team. Instead, similar to Shum et al. [163], we opt to model a communicating team as a *group agent* with a shared goal g and a joint plan π . Given this joint plan π , a speaker in the team may utter an instruction u communicating the plan, and each agent $i \in [1, n]$ takes actions $a_{i,t}$ at timestep t according to the plan (a simplified graphical model for the $n = 2$ case is shown in Figure 5.1b). This is essentially the same generative process presented in Chapter 4, but generalized to n agents:

$$\text{Goal Prior:} \quad g \sim P(g) \quad (5.1)$$

$$\text{Joint Planning:} \quad \pi \sim P(\pi|g) \quad (5.2)$$

$$\text{Action Selection:} \quad a_{1:n,t} \sim P(a_{1:n,t}|s_t, \pi) \quad (5.3)$$

$$\text{Utterance Generation:} \quad u_t, c_t, d_t \sim P(u_t, c_t, d_t|s_t, \pi) \quad (5.4)$$

$$\text{State Transition:} \quad s_{t+1} \sim P(s_{t+1}|s_t, a_{1:n,t}) \quad (5.5)$$

A peculiar aspect of this model is that the speaker is assumed to communicate the plan π through an utterance u , despite all agents supposedly having shared mental states: Why communicate the plan, if everybody knows what it is? Nonetheless, there is a good reason to use this model for goal inference: Since it is much simpler than the complete model described earlier, it can serve as a *resource-rational approximation* [128] of the true dynamics, allowing third-person observers to efficiently infer the team’s goal while avoiding the need to represent individual agents that reason recursively about their team members’ mental states.

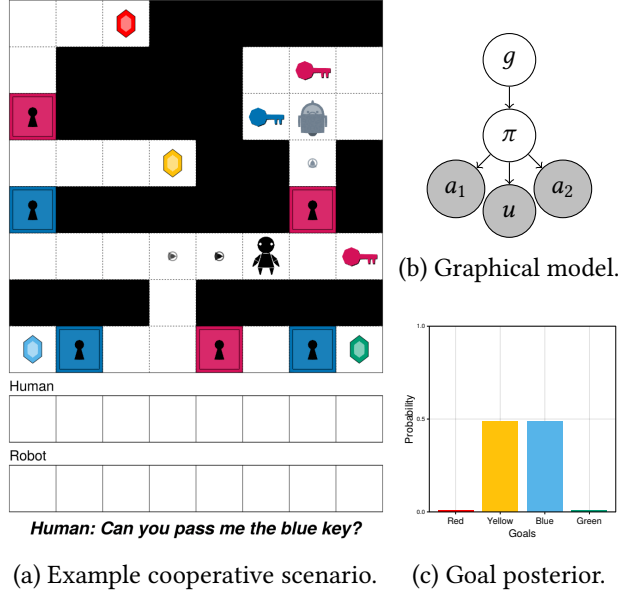


Figure 5.1: Modeling communicating teams: (a) A human-robot team cooperates to achieve a shared goal g (one of the 4 gems), with the human uttering an instruction $u = \text{"Can you pass me the blue key?"}$ to the robot. (b) We model the team as forming a *joint* plan π to achieve their goal g , dictating their actions a_1 (human), a_2 (robot). The human communicates part of this plan as an instruction u . (c) Bayesian inverse planning produces a posterior distribution over goals.

5.2 Team-Level Goal Inference from Actions and Instructions

Having previously described the details of the CLIPS model class in Chapter 4, we now move on how team-level goal inference is achieved. While CLIPS assistive agents update their beliefs by conditioning on only the speaker’s actions, an external observer should condition on the actions of *all* agents in the team $a_{1:n,1:T}$, in addition in to the utterances $u_{1:T}$ and speech decisions $d_{1:T}$:

$$\begin{aligned} P(g, \pi | s_{1:T}, d_{1:T}, u_{1:T}, a_{1:n,1:T}) &\propto P(g, \pi, s_{1:T}, d_{1:T}, u_{1:T}, a_{1:n,1:T}) \\ &= P(g, \pi) \prod_{t=1}^T P(s_t | s_{t-1}, a_{1:n,t-1}) P(u_t, d_t | s_t, \pi) P(a_{1:n,t} | s_t, \pi) \end{aligned} \quad (5.6)$$

We can estimate this posterior distribution using the CLIPS belief update in Algorithm 3. Since we follow Chapter 4 in assuming that planning is deterministic (specifically, by using RTHS with a fixed search budget), the belief update corresponds to an exact Bayesian filtering algorithm which updates unnormalized weights w_t^g for each goal hypothesis g as new evidence arrives:

$$\begin{aligned} w_0^g &\leftarrow P(g) \\ w_t^g &\leftarrow w_{t-1}^g P(u_t, d_t | s_t, \pi_g) P(a_{1:n,t} | s_t, \pi_g) \end{aligned}$$

By normalizing these weights, we get the step- t posterior $P_t^g = w_t^g / \sum_{g'} (w_t^{g'})$ for each goal g .

5.3 Experiments

To evaluate both the accuracy and human-likeness of our approach, we conducted human and computational experiments, comparing the team-level goal inferences of CLIPS against inferences elicited from human participants. As baselines, we evaluated action-only and language-only inverse planning. Experiments were conducted using the multi-agent Doors, Keys & Gems (mDKG) domain, which contained a team of $n = 2$ agents: a human principal and a robot assistant collaborating to retrieve a target gem (Figure 5.1(a)). As in Chapter 4, the robot agent could not pick up gems, reflecting its role as an assistive agent. Action costs were assumed to be equal for both agents, with the wait action carrying 60% the cost of other actions.

5.3.1 Dataset Generation

We constructed 6 instances of the mDKG environment with varying maze designs and item locations. In each of these environment instances, we created 2–4 action sequences from the initial state to a goal gem, generated through a combination of automated planning and manual modification in order to increase the diversity and goal ambiguity associated with each action sequence. For each action sequence, we wrote a natural language instruction that the human might communicate to the robot in a variety of styles (e.g. requests like *"Can you unlock the red and blue door for me?"*, or commands like *"Pass me the blue key."*). For simplicity, instructions were all communicated at the beginning of each action sequence (i.e., u_t was only observed at $t = 1$). In total, we constructed 20 stimuli of action trajectories paired with instruction utterances, reflecting a range of cooperative scenarios.

Bayesian Multi-Agent Goal Inference from Actions and Instructions

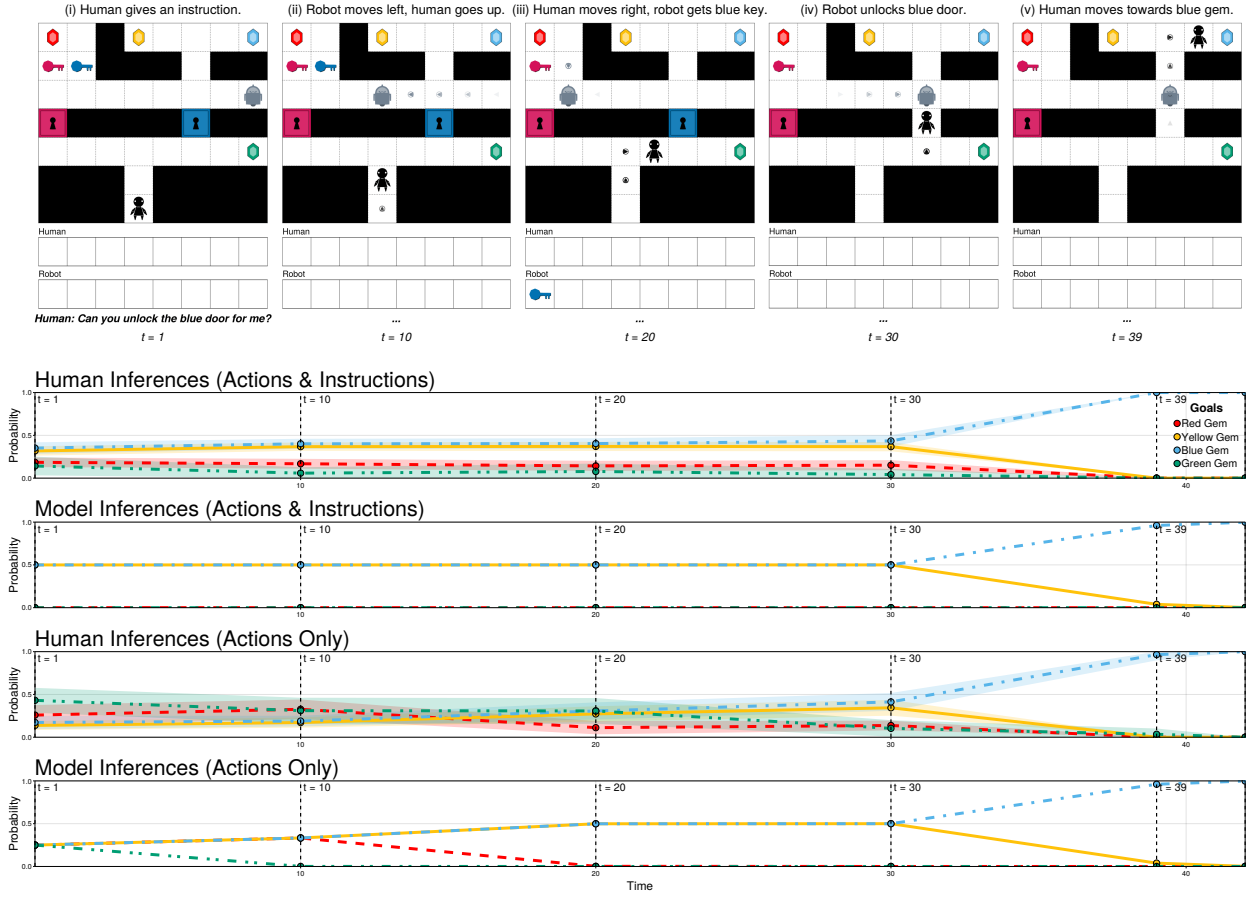


Figure 5.2: Goal inferences over time from the actions and instructions of a human-robot team, where the team’s goal is for the human to pick up one of the four colored gems. (Row 1) Frames from an illustrative state-action trajectory ($s_{1:t}, a_{1:t}$), with the initial utterance $u = \text{"Can you unlock the blue door for me?"}$ shown below the first frame. (Row 2) Average human inferences (w. 95% CI) given both the instruction and actions, elicited at the selected frames. (Row 3) Model inferences via Bayesian inverse planning from instructions and actions. (Row 4) Average human inferences (w. 95% CI) given actions only, without any instructions provided. (Row 5) Model inferences via Bayesian inverse planning from actions only.

5.3.2 Human Experiments

Our human experiment involved two experimental conditions in order to isolate the effect of language information on goal inference: (i) *Actions & Instructions* and (ii) *Actions Only*. In the *Actions & Instructions* condition, participants were shown animated trajectories of the human-robot team as stimuli, along with the instructions given by the human to the robot at the initial state (see Figure 5.2 for selected frames). In the *Actions Only* condition, the participants were shown the same animated stimuli, but without the instruction. Animated trajectories were segmented at certain judgment points, and participants provided their goal inferences at each judgment point by selecting all gems they thought were likely to be the team’s goal. We inserted 4–5 judgment points for each trajectory, depending on the length of the trajectory.

We recruited 120 US participants fluent in English via Prolific, 60 of whom were assigned to each of the experimental conditions. Participants were paid at a rate of 15 USD per hour and receive a bonus for correctly inferring the agents’ goals. Each participant provided goal inferences for 10 out of the 20 stimuli. In total, each stimulus was rated by approximately 30 participants. Before viewing the stimuli, participants went through a tutorial. One participant in the with-instruction condition was excluded from analysis due to outlier responses below the first quartile minus the inter-quartile range.

5.3.3 Model Configuration

We ran team-level goal inference with CLIPS on the same set of stimuli we provided to humans. We used a slightly different version of the utterance model shown in Figure 4.2c, with a plan-dependent probability of making a decision d_t to speak. In particular, we assumed $P(d_t|\pi) = 0.95$ when π contained salient actions to be communicated, and $P(d_t|\pi) = 0.05$ otherwise. This allowed goal inferences to be drawn from decisions *not* to speak (when $d_t = 0$). We fixed the set of 7 few-shot examples $\mathcal{E}$ used by our utterance model, leaving no other free parameters besides the rationality parameter β , which we varied from $1/16$ to 16 in powers of two. To fit the model, we computed Pearson’s correlation coefficient r between model inferences at each judgment point vs. average human inferences at the same judgment point. We found that $\beta = 1.0$ led to high correlation in both experimental conditions, achieving the highest geometric mean of r across conditions. The following results all use $\beta = 1.0$ unless stated otherwise.

5.3.4 Results

Qualitative Analysis Figure 5.2 shows an illustrative example of multi-agent goal inference from actions and instructions over time. In this example, the human principal is able to directly reach the green gem, but the other three gems, along with the robot, are locked behind a red door and a blue door. As such, the human principal requires the robot’s help to unlock one of the doors in order to reach one of those gems. The red gem is closer to the red door, while the yellow and blue gems are closer to the blue door. In the *Actions Only* condition, both humans and action-only inverse planning place a (close-to)-uniform prior over the four gems. But in the *Actions & Instructions* condition, both humans and CLIPS place higher probability on the yellow and blue gems at $t = 1$, while down-weighting the red and green gems. This is because a rational agent would not utter an instruction for the green gem, and would instruct the robot to unlock the red instead of blue door if the goal was the red gem.

As the example progresses through $t = 10$ (robot moves toward the key) and $t = 20$ (robot picks up the blue key), the team’s actions gradually provide information about the goal, such that the action-only model eventually stops considering the green gem and red gem as live possibilities. Humans are slightly more uncertain when shown only actions, down-weighting the red gem, but maintaining the possibility that the green gem might be the goal even at $t = 20$ (perhaps because they consider it possible that each agent is acting independently). In contrast, goal uncertainty is considerably reduced when instructions are provided. Among human participants, lower uncertainty manifested not just as faster convergence to the true goal, but also smaller confidence intervals (i.e. lower population variance). Only at $t = 30$ (robot unlocks the door) do human inferences from actions alone converge towards human inferences with language.

Table 5.1: Goal inference accuracy for humans (averaged across subjects) and models. We report the probability assigned to the true goal $P(g_{\text{true}})$ and Brier score (lower is better) at the initial, median, and final judgment points ($t \in \{\text{first}, \text{median}, \text{last}\}$). Values are averaged over stimuli, and standard deviations are in brackets. The *Instructions Only* row corresponds to the first step of the *Actions & Instructions* condition.

	t = first		t = median		t = last	
	$P(g_{\text{true}})$	Brier Score	$P(g_{\text{true}})$	Brier Score	$P(g_{\text{true}})$	Brier Score
Humans (<i>Actions & Instructions</i>)	0.51 (0.06)	0.10 (0.06)	0.56 (0.21)	0.10 (0.04)	0.94 (0.08)	0.01 (0.01)
Humans (<i>Instructions Only</i>)	0.51 (0.06)	0.10 (0.06)	0.51 (0.06)	0.10 (0.06)	0.51 (0.06)	0.10 (0.06)
Humans (<i>Actions Only</i>)	0.23 (0.05)	0.20 (0.03)	0.44 (0.15)	0.13 (0.05)	0.92 (0.02)	0.00 (0.00)
CLIPS (<i>Actions & Instructions</i>)	0.64 (0.23)	0.09 (0.06)	0.65 (0.23)	0.09 (0.06)	0.99 (0.02)	0.00 (0.00)
Inv. Planning (<i>Instructions Only</i>)	0.64 (0.23)	0.09 (0.06)	0.64 (0.23)	0.09 (0.06)	0.64 (0.23)	0.09 (0.06)
Inv. Planning (<i>Actions Only</i>)	0.25 (0.00)	0.19 (0.00)	0.55 (0.17)	0.11 (0.04)	0.98 (0.04)	0.00 (0.00)

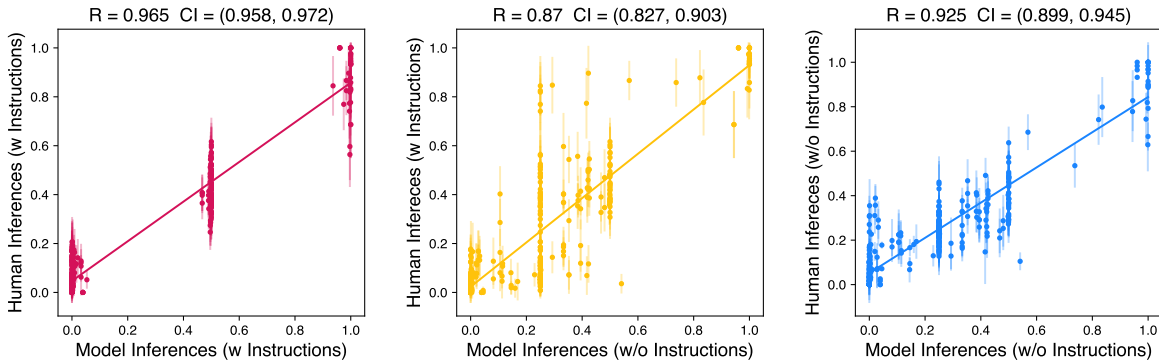


Figure 5.3: Correlation plots between average human inferences and model inferences, in the *Actions & Instructions* condition (*left*) and *Actions Only* condition (*right*). Our model displays a good fit in both conditions. In contrast, action-only inverse planning correlates less well with human inferences in the *Actions & Instructions* condition (*middle*).

Correlational Analysis To quantitatively evaluate the fit between CLIPS and human participants’ inferences, we run a correlational analysis and show the results across the two experimental conditions in Figure 5.3. As can be seen, goal inferences are strongly correlated with human judgments across experimental conditions, with a Pearson’s r of 0.965 (95% CI of [0.958, 0.972]) in the *Actions & Instructions* condition (left plot), and a Pearson’s R of 0.925 (95% CI of [0.899, 0.945]) in the *Actions Only* condition (right plot). The 95% confidence intervals are calculated via bootstrapping with 1000 samples.

Figure 5.3 also shows that the correlation coefficient is higher in the *Actions & Instructions* condition. We suspect that this is because language instructions reduce the uncertainty and variance in human inferences, leading to a better fit. The left-most plot of Figure 5.3 also shows that the probability ratings form small clusters around values of 0, 0.5, and 1, whereas the data points in the *Actions Only* condition have a much wider spread. This indicates that instructions help the observer effectively reduce the set of possible goals to just two or one goals. As a baseline comparison, we plot the correlation between action-only inverse planning and human inferences in the *Actions & Instructions* condition (middle plot). This resulted in a lower correlation coefficient of 0.87, with a 95% CI of [0.827, 0.903], demonstrating the importance of modeling inference from language to explain human goal inferences.

Goal Inference Accuracy Finally, we analyze the performance of both humans and CLIPS in terms of accurately inferring the team’s true goal. At each judgment point (which includes the initial state where no actions are shown), we compute the probability assigned to the true goal $P(g_{\text{true}})$ by both our model and the average human, along with the Brier score $\sum_i (P(g_i) - \mathbf{1}[g_i = g_{\text{true}}])^2$, a measure of well-calibrated inference. We then average these values across the entire dataset at each judgment point. We show these results for the initial, median, and final judgment points in Table 5.1. (In cases where a stimulus has no midpoint, we average the true goal probabilities for the middle two judgment points.)

We find that, on average, CLIPS assigns slightly higher probability to the true goal than humans ($p < 0.001$ in both experimental conditions via a paired t -test), indicating that our model is able to effectively infer the team’s goals for the tested stimuli. Notably, there is a very clear difference between experimental conditions at the initial timestep: When instructions are provided at the start, both humans and the model assign much higher probability to the true goal than when no instructions are observed. This illustrates the importance of language in conveying useful information rapidly, as compared to inferring goals from actions alone.

Since instructions provide a lot of information, we also see that the true goal probability at the median judgment point only increases marginally in the *Actions & Instructions* condition, with the actions observed between the first and median points providing limited extra information. In contrast, in the *Actions Only* condition, the increase in $P(g_{\text{true}})$ is much more pronounced. However, instructions alone are not enough to disambiguate the goal: As inference progresses on to the last judgment point, goal accuracy is significantly higher than at the first judgment point and median judgment point. As a result, CLIPS is more accurate at the final judgment point than inverse planning from instructions alone.

5.4 Discussion and Future Work

In this chapter, we demonstrated how CLIPS can be applied beyond the assistive setting, allowing us to infer the shared goals of a communicating team. We achieved this by using CLIPS as an efficient approximation of team-based communication and cooperation, avoiding the need to recursively reason about each individual team member. In doing so, we also showed how prior work on group-level theory-of-mind [141, 143, 163, 212] can be extended to handle natural linguistic communication, leveraging the flexibility of LLMs as utterance likelihoods. Through our experiments, we found that CLIPS can explain human inferences of team goals in a range of cooperative scenarios, while producing more accurate inferences than unimodal inverse planning.

Still, the full range of cooperative and communicative behavior exhibited by humans goes far beyond what we have modeled here. As work by Stacy et al. [214] and others have shown, we humans *are* able to recursively reason about the individual mental states of two communicators, including the fact that they may not share the same goals, beliefs, or beliefs about their shared goal [142, 173, 214]. We also understand that cooperative speakers do not utter just any aspect of their plans. Instead, they communicate to best aid their listeners in understanding their intentions [154, 213, 215]. How can we model this efficiently without suffering from the curse of recursion? We suggest that future work should investigate *multi-resolution modeling* of multi-agent teams — flexibly representing them as a single group agent or as individual agents where necessary — in order to match both the efficiency and expressivity of human cooperative intelligence.

Part III

Social Normativity

Chapter 6

Modeling Social Normativity via Norm-Augmented Stochastic Games

What is missing... is an adequately specified theory of collective action whereby a group of principals can organize themselves voluntarily to retain the residuals of their own efforts.

Elinor Ostrom

How do people live and work together in large social groups, even when they do not all share the same interests? And how can we achieve similar levels of cooperation in the future machine economy, where large numbers of autonomous agents may act on the behalf of humans and their institutions? In the preceding chapters, we have discussed two core capabilities — *theory of mind* and *communicative cooperation* — which enable cooperation in local, dyadic, or team-based interactions. However, these capabilities are insufficient for cooperation at the societal scale, where agents do not all work towards the same goals, and where the costs of communicating with or inferring the goals of large numbers of agents is prohibitive for any one actor to engage in. Instead, humans solve this problem through *social normativity*: the designation of certain behaviors as acceptable or desirable, and others as unacceptable or undesirable, according to shared standards [216, 217]. By collectively upholding rules for acceptable behavior — that is, *norms* — we can coordinate on mutually-beneficial activities despite our divergent interests, while disincentivizing harmful actions, and penalizing free-riders that profit without contributing their fair share. Furthermore, we can achieve this in a largely *decentralized* manner [218], without the need for a central authority to publicize or enforce norms, because we are able to learn such norms from those immediately around us, and comply with or enforce them ourselves [219, 220]. In doing so, we sustain cooperation without requiring knowledge of or alignment with everyone else’s goals.

To develop the theoretical foundations of social normativity in a manner suitable for machine implementation, this chapter introduces **norm-augmented stochastic games (NSGs)**, an extension of the stochastic game (a.k.a. Markov game) framework for multi-agent sequential decision-making. In NSGs, each agent is equipped with probabilistic beliefs about the shared norms their society might practice, allowing for: (i) agents’ desires to be shaped by the norms they believe in; (ii)

decentralized norm learning via belief updating; (iii) stable compliance and enforcement of norms that is robust to strategic self-interest and the tragedy of the commons. This not only explains key aspects of human social normativity, but also characterizes a design target for normatively-competent autonomous agents. We introduce the concept of NSGs after reviewing background on stochastic games. Next, we introduce equilibrium concepts and convergence results that arise in NSGs, and consider several ways in which a set of norms can achieve strategic stability. We then explain how NSGs generalize and unify prior attempts at formally characterizing social norms. This sets the stage for Chapter 7, which introduces a practical instantiation of Bayesian norm learning by agents in a stochastic game.

6.1 Preliminaries

For us to develop a formal model of social normativity, we first need a way to talk about the social world. We do this by introducing multi-agent stochastic *environments*, the notion of *stochastic games* over these environments, and finally *stochastic Bayesian games*, where each agent has beliefs about how all agents will act. We start with the definition of an environment:

Definition 6.1 (Environment). A *multi-agent stochastic environment*, or simply an *environment*, is a tuple $\mathcal{E} = (I, S, \{A^i\}_{i \in I}, P_s, P_i, P_H)$ where:

- $I = \{1, \dots, M\}$ is a set of M agents.
- S is a set of environment states.
- A^i is a set of actions available to agent $i \in I$.
- $P_s : S \times A^1 \times \dots \times A^M \rightarrow \Delta(S)$ is the (stochastic) environment transition function.
- $P_i \in \Delta(S)$ is a distribution over the initial state s_0 .
- $P_H \in \Delta(\mathbb{N})$ is a distribution over the time horizon H .

We denote the set of joint actions as $A := A^1 \times \dots \times A^M$, writing a joint action as $a_t := (a_t^1, \dots, a_t^M)$, and the joint action for all agents but i as $a_t^{-i} \in A^{-i}$. We also denote:

- $h_t^i = (s_1, a_1^i, s_2, a_2^i, \dots, s_t)$ as the individual history for agent i up to time t .
- $h_t = (s_1, a_1, s_2, a_2, \dots, s_t)$ as the joint history for all agents up to time t .
- H_t^i (respectively, H_t) as the set of all individual (joint) histories up to time t .
- $H^i = \bigcup_{t=1}^{\infty} H_t^i$ (respectively, $H = \bigcup_{t=1}^{\infty} H_t$) as the set of all individual (joint) histories.

The environment begins at an initial state $s_1 \sim P_i(\cdot)$. At each step s_t , every agent i selects an action $a_t^i \in A_i$ at state s_t , and the next state $s_{t+1} \sim P(\cdot | s_t, a_1^t, \dots, a_M^t)$ is sampled according to the transition function. The environment terminates after a (random) time horizon $H \sim P_H(\cdot)$.

An environment merely specifies what happens when agents take actions in the world. The reason that we need social normativity at all, however, is because we *care* about how the world goes, and the direction of our caring can end up being at odds with others. Game theory describes this situation through the metaphor of a *game*. Since our environment is stochastic, we consider *stochastic games* [152], also known as *Markov games* [221].

Definition 6.2 (Stochastic Game). A *stochastic game* $\mathcal{G}$ is a tuple $(\mathcal{E}, \{\prec_i\}_{i \in I})$ where:

- $\mathcal{E} = (I, S, \{A^i\}_{i \in I}, P_s, P_i, P_H)$ is a multi-agent stochastic environment.
- $\preceq^i$ is a partial order on $\Delta(H)$, specifying agent i 's preferences over prospects $\mu_h \in \Delta(H)$.

A prospect $\mu_h \in \Delta(H)$ is a distribution on histories $h \in H$. We say that agent i prefers a prospect μ_a less than μ_b when $\mu_a \preceq^i \mu_b$, that this is *strict* when $\mu_a \prec^i \mu_b$, and they are *indifferent* when $\mu_a \simeq^i \mu_b$. Since $\preceq^i$ is partial, the agent's preferences may be *incomplete* — there exist prospects $\mu_a \parallel^i \mu_b$ where the agent is neither indifferent nor prefers one over the other.

By treating agents' preferences as partial orders, our definition of stochastic games is more general than in standard treatments [152, 221]. We start with this definition because not all of an agent's choices may be comparable, especially on matters of normative importance [222, 223].¹ Furthermore, incomplete preferences do not preclude game theoretic analysis [224]. We can also recover more familiar structure under additional assumptions: Modulo some restrictions², if an agent's preferences $\preceq^i$ satisfy the independence and continuity axioms of utility theory [226] but not completeness, then they can be represented [225, 227] as a *vector-valued utility function* $U^i : H \rightarrow \mathbb{R}^k$, as studied in multi-objective game theory and reinforcement learning [228–231]. If all agents' preferences are also *complete*, we have a utility-based stochastic game, with the typical reward-based formulation as a special case:

Definition 6.3 (Utility-Based Stochastic Game). A *utility-based stochastic game* is a stochastic game $\mathcal{G} = (\mathcal{E}, \{\prec_i\}_{i \in I})$ where each agent's preferences $\prec_i$ are characterized by a utility function $U^i : H \rightarrow \mathbb{R}$ s.t. for all $\mu_a, \mu_b \in \Delta(H)$, we have $\mu_a \preceq^i \mu_b \iff \mathbb{E}_{h \sim \mu_a}[U^i(h)] \leq \mathbb{E}_{h \sim \mu_b}[U^i(h)]$.

Definition 6.4 (Reward-Based Stochastic Game). A *reward-based stochastic game* is a utility-based stochastic game where each agent's utility function U^i is *time-separable*, i.e., there exists a reward function $R^i : S \times A \times S$ s.t. $U^i(h_t) = \sum_{\tau=1}^{t-1} R^i(s_\tau, a_\tau, s_{\tau+1})$.

To study human-like social coordination, we need to model more than just each agents' preferences. We also need to model their *beliefs* about how other agents will behave³. We do this by first formalizing agent behavior as *policies*:

- An history-dependent *policy* $\pi^i : H \rightarrow \Delta(A^i)$ for agent i specifies a distribution over their actions $a^i \in A^i$ given the joint history $h_t \in H$.
- A *joint policy* for all agents $\pi : H \rightarrow \Delta(A)$ specifies the joint distribution over all agents' actions given $h_t \in H$. Joint policies may be *correlated*, i.e. $\pi(a_t; h_t) \neq \prod_{i=1}^M \pi(a_t^i; h_t)$.

We are now ready to introduce *stochastic Bayesian games* [232, 233] — stochastic games where each agent is endowed with *prior beliefs* about the policies of all players.

Definition 6.5 (Stochastic Bayesian Game). A *stochastic Bayesian game* (SBG) is a tuple $\mathcal{BG} = (\mathcal{G}, \Pi, \{B^i\}_{i \in I})$ where:

- $\mathcal{G} = (\mathcal{E}, \{\prec^i\}_{i \in I})$ is a stochastic game.
- $\Pi \subseteq \{H \rightarrow \Delta(A)\}$ is a (countable) set of joint policies.
- $B^i \in \Delta(\Pi)$ is agent i 's (subjective) prior over the joint policy followed by all agents.

Starting from their prior belief B^i , a Bayesian agent will update their beliefs as they observe how other agents' act. Their *posterior belief* $B_t^i(\pi) = B^i(\pi|h_t)$ after observing history h_t is:

$$B_t^i(\pi) = \frac{B^i(\pi) \cdot \prod_{\tau=1}^t \pi(a_\tau; h_\tau)}{\sum_{\pi' \in \Pi} B^i(\pi') \cdot \prod_{\tau=1}^t \pi'(a_\tau; h_\tau)} \quad \text{or, recursively} \quad B_t^i(\pi) = \frac{B_{t-1}^i(\pi) \pi(a_t; h_t)}{\sum_{\pi' \in \Pi} B_{t-1}^i(\pi') \pi'(a_t; h_t)}$$

¹For instance, breaking a promise to a loved one v.s. reneging on an important professional obligation.

²The space of histories H must be discrete, or more generally σ -compact [225].

³Since we assume the environment is fully observable, we do not consider beliefs over the environment here.

Given a belief B , we denote $\pi_B(a_t; h_t) := \sum_{\pi \in \Pi} B(\pi|h_t) \cdot \pi(a_t; h_t)$ as the mixture policy induced by B , and $\pi_B^i(a_t^i; h_t) = \sum_{a_{-t}^{-i}} \pi_B(a_t^i, a_{-t}^{-i}; h_t)$ as the induced individual policy for agent i . We will sometimes abuse notation and write $B(a_t; h_t)$ and $B(a_t^i; h_t)$ in lieu of $\pi_B(a_t; h_t)$ and $\pi_B^i(a_t^i; h_t)$. We also write $B(h|h_t)$ for the posterior predictive distribution over full histories h given h_t . We now introduce two requirements that we shall generally assume of SBGs:

- *Grain-of-Truth*: Each agent i has a non-zero prior belief $B^i(\pi) > 0$ in the *true* joint policy π .
- *Self-Knowledge*: Each agent has a *true* belief about their own policy $\pi_{B^i}^i = \pi^i$.

We note that our focus here is on the *subjective* variant of SBGs [234]. In *objective* SBGs, all agent's prior beliefs B^i equal to each other *and* to the true joint policy π . To distinguish beliefs, each agent instead observes a signal or *type* at the start of the game which informs them of their individual policy π^i , while inducing a posterior over the remaining policies π^{-i} [233]. Since we assume subjective priors and self-knowledge, we do not require initial revelation of types.

6.2 Norm-Augmented Stochastic Games

We have now introduced the machinery required to model beliefs about other agents — a prerequisite for many forms of social coordination. But what is the *content* of these beliefs? While much of this thesis has focused on observers' beliefs about the goals and plans of others (**Part I**), including *shared* goals and plans (**Part II**), we need more to account for large scale cooperation. Though we may not all share the same goals, we understand that society generally operates according to *shared rules*. Rules tell us what behavior we should or should not engage in, allowing us to coordinate much more easily than inferring everyone's plans or goals.

To model this situation, we develop norm-augmented stochastic games — stochastic Bayesian games where agents have beliefs about the shared rules that agents generally follow. This requires us to first define our notion of a rule.

Definition 6.6 (Rule). A *rule* of degree d_v is a function $v : H \times \mathcal{P}_{d_v}(I) \rightarrow \{0, 1\}$, where $v(h_t, J) = 1$ denotes that the agents $J \subseteq I$ (with $|J| = d_v$) have violated the rule at history $h_t \in H_t$, and $v(h_t, J) = 0$ denotes compliance with the rule by all in J .

Given J , the output of v depends only on the joint history h_t^J of those agents, i.e. for any $g_t, h_t \in H$, we have $v(g_t, J) = v(h_t, J)$ when $g_t^J = h_t^J$.

Intuitively, a rule *classifies* whether some set of agents' behavior comply with the rule at each step t . Some rules might be *Markov* or *time-separable*, s.t. $v(h_t, J)$ depends on only the final state transition (s_t, a_t, s_{t+1}) for all $h_t \in H$. In other cases, rules might be *temporally extended* (e.g. the obligation to clear dirty dishes within a certain amount of time, s.t. $v(h_{t+\Delta t}, J) = 1$ only some duration Δt after the obligation was triggered at h_t). If a rule is of degree 1, it applies only to individuals (e.g. an individual obligation to wear a mask). Rules of higher degree can define joint obligations or prohibitions (e.g. against conspiring to commit crime). For subsequent definitions, it shall also be useful to introduce *weighted rule violation counts*:

Definition 6.7 (Weighted Rule Violation Count). Given a rule v of degree d_v and a weighting function $W : H \rightarrow \mathbb{R}_{\geq 0}$, the *weighted rule violation count* $n_v(h_t, J, W) = \sum_{\tau=1}^t W(h_\tau) \cdot v(h_\tau, J)$ is the number of W -weighted violations of v by group J with $|J| = d_v$ in history h_t . The aggregate weighted rule violation count across all agents I is $n_v(h_t, W) = \sum_{J \subseteq I, |J|=d_v} d_v \cdot n_v(h_t, J, W)$.

Definition 6.8 (Belief-Weighted Rule Violation Count). Given a rule v , a belief function $B_v : H \rightarrow [0, 1]$ that maps h_t to the degree of belief in v , and a monotonic transform $f : [0, 1] \rightarrow \mathbb{R}$ (e.g. the identity $f(x) = x$, or a threshold function $f(x) = \mathbf{1}_{x \geq \theta}(x)$), the *belief-weighted rule violation count* is $n_v(h_t, J, f \circ B_v) = \sum_{\tau=1}^t f(B_v(h_\tau)) \cdot v(h_\tau, J)$.

Weighted rule violation counts give us a way to specify beliefs about a norm-compliant society (since such a society would violate norms less, at least on average), and to specify preferences for norm-compliant agents (who would prefer not to violate rules they believe are norms). To make the latter notion more precise, we define *Pareto improvements* with respect to expected belief-weighted rule violation counts:

Definition 6.9. Given a set of rules $\mathcal{N}$, belief functions $B := \{B_v\}_{v \in \mathcal{N}}$, belief transforms $F := \{f_v\}_{v \in \mathcal{N}}$, and agents I , we say that a prospect μ_a is a Pareto improvement upon μ_b in expected belief-weighted rule violation counts iff:

$$\forall v \in \mathcal{N}, J \subseteq I, \quad \mathbb{E}_{h \sim \mu_a} [n_v(h, J, f_v \circ B_v)] \leq \mathbb{E}_{h \sim \mu_b} [n_v(h, J, f_v \circ B_v)]$$

We denote this by $\mu_a \succeq_{(\mathcal{N}, B, F, I)} \mu_b$. When the inequality above is strict for at least one $v \in \mathcal{N}, J \subseteq I$, we call it a strict Pareto improvement, denoted $\mu_a \succ_{(\mathcal{N}, B, F, I)} \mu_b$.

With these ingredients, we can now formally define norm-augmented stochastic games:

Definition 6.10 (Norm-Augmented Stochastic Game). A *norm-augmented stochastic game* (NSG) is a tuple $\mathcal{NG} = (\mathcal{BG}, \mathcal{N})$ where $\mathcal{BG} = (\mathcal{E}, \{\prec^i\}_{i \in I}, \Pi, \{B^i\}_{i \in I})$ is a stochastic Bayesian game and $\mathcal{N} \subseteq \{H \times \mathcal{P} \rightarrow \{0, 1\}\}$ is a countable set of rules, with the following additional structure:

1. *Belief in Shared Norms:* Each agent's prior B^i about the joint policy π factorizes into a prior $B^i(N)$ over shared norms $N \in \mathcal{N}$ and a conditional belief over policies $B^i(\pi|N)$:

$$B^i(\pi) = \sum_{N \in \mathcal{N}} B^i(N) B^i(\pi|N)$$

2. *Belief in Relative Norm Compliance:* Let $B^i(\pi|h_t, N)$ be i 's posterior belief after observing h_t , conditioned on rules N . This induces a predictive distribution $B^i(h|h_t, N)$ over full histories h after h_t where all agents follow the N -conditioned policy induced by $B^i(\pi|h_t, N)$. Let $W^i : H \rightarrow \mathbb{R}_{\geq 0}$ be a weighting function for each agent i . For all agents i , rulesets $N \in \mathcal{N}$ and histories $h_t \in H$, we require:

$$\mathbb{E}_{h \sim B^i(h|h_t, N)} [\sum_{v \in N} n_v(h, W^i)] \leq \mathbb{E}_{h \sim B^i(h|h_t, N=\emptyset)} [\sum_{v \in N} n_v(h, W^i)]$$

That is, each agent i believes that the expected number of W^i -weighted violations of N when society follows N is no greater than the expected number of W^i -weighted violations of N when society follows an empty ruleset $\emptyset$.

3. *Conditional Preference for Norm Compliance:* Let $B_v^i(h_t) = B^i(v \in N|h_t)$ be agent i 's posterior belief in v being a norm after observing h_t , and $B^i = \{B_v^i\}_{v \in \mathcal{N}}$. There exists a (possibly empty) set of *norm internalizing agents* $NI \in I$ whose preferences $\preceq^i$ satisfy the following w.r.t. to a base preference order $\preceq_0^i$, their norm beliefs B^i , and belief transforms $F^i := \{f_v^i\}_{v \in \mathcal{N}}$:

$$\begin{aligned} \mu_a \succeq^i \mu_b & \text{ if } \mu_a \succeq_0^i \mu_b \quad \text{and} \quad \mu_a \succeq_{(\mathcal{N}, B^i, F^i, I)} \mu_b \\ \forall \mu_a, \mu_b \in \Delta(H) \quad \mu_a \succ^i \mu_b & \text{ if } \mu_a \succ_0^i \mu_b \quad \text{and} \quad \mu_a \succeq_{(\mathcal{N}, B^i, F^i, I)} \mu_b \\ \mu_a \succ^i \mu_b & \text{ if } \mu_a \succeq_0^i \mu_b \quad \text{and} \quad \mu_a \succ_{(\mathcal{N}, B^i, F^i, I)} \mu_b \end{aligned}$$

That is, a norm internalizing agent i prefers Pareto improvements in belief-weighted norm violation counts, as long as this does not contradict their base preferences $\preceq_0^i$.

Why define NSGs in terms of conditions 1–3? As argued by Gintis [15] and Vanderschraaf [235], allowing for *correlations* in agents’ policies better captures the shared nature of social normativity. *Belief in Shared Norms* (condition 1) introduces these correlations by having the joint policy depend on a single (unobserved) set of shared rules, similar to how local cooperation can be achieved by assuming shared intentionality [12, 26, 143, 212]. *Belief in Relative Norm Compliance* (condition 2) just imposes a weak consistency condition on agent’s beliefs, requiring that policies conditioned on N actually lead to less violations of ruleset N .

Finally, *Conditional Preference for Norm Compliance* (condition 3) allows us to model *intrinsic* preferences for norm compliance that humans frequently exhibit [15, 236], while accounting for how such preferences are often conditional on believing that others’ follow the same norms [16]. Furthermore, intrinsic preferences for norm compliance can enable cooperation where correlated beliefs are not enough [15, 16] — especially useful when we are engineering cooperative machines.

We note that conditions 2 and 3 above are very general: We do not require a specific level of norm compliance in agent’s beliefs about how society behaves, nor do we specify how norm internalizing agents should trade-off violations of one norm versus other norms, or against their base preferences. For the purposes of practical implementation, however, we may wish to impose stricter requirements. For example, we might require beliefs in ϵ -norm compliance:

Definition 6.11 (ϵ -Norm Compliance). In an NSG, we say that agent i believes that the joint policy π exhibits ϵ -norm compliance if for all rulesets $N \in \mathcal{N}$ and histories $h_t \in H$, we have:

$$\mathbb{E}_{h \sim B^i(h|h_t, N)} \left[\frac{1}{|h|} \sum_{v \in N} n_v(h, W_1) \right] \leq \epsilon \quad \text{where} \quad \forall h \in H, W_1(h) = 1$$

where $|h|$ is the number of time steps that history h spans. In other words, conditional on N , the agent believes that the expected norm violation frequency is less than ϵ . This satisfies *Belief in Relative Norm Compliance* when the time horizon $|h| = H$ is known, and the default norm violation frequency $\mathbb{E}_{h \sim B^i(h|h_t, N=\emptyset)} [1/H \sum_{v \in N} n_v(h, W_1)]$ is greater than or equal to ϵ .

In our original introduction of NSGs [18], we required that all agents’ beliefs satisfy a version of ϵ -norm compliance. However, belief in ϵ -compliance does not account for the intuition that agents are less likely to comply with norms when the opportunity costs are high. For instance, one might not expect another agent to pick up their litter if it is blown across the street, and one would not expect *any* agent to do so in the middle of a hurricane. Unless ϵ is set very high, situations like these preclude a fixed ϵ across all histories. A more reasonable model of norm compliance should take into account the costs of complying. For this, it is useful to consider a refinement of *Conditional Preference* in the case of utility-based stochastic games.

Definition 6.12 (Norm-Augmented Utility Functions). Let $U_0^i : H \rightarrow \mathbb{R}$ be a base utility function for a norm internalizing agent $i \in \text{NI}$. Their *norm-augmented utility function* is:

$$U_N^i(h) = U_0^i(h) - \sum_{v \in N} \sum_{J \in \mathcal{J}_{i,v}} n_v(h, J, B_v^i, f_v^i)$$

where $\mathcal{J}_{i,v} = \{J | i \in J \subseteq I, |J| = d_v\}$ is the set of v -applicable groups that agent i is part of, $B_v^i(h_t) = B^i(v \in N | h_t)$ is agent i ’s posterior belief in v being a norm, and f_v^i a monotonic transform.

A norm-augmented utility function subtracts the sum of the belief-weighted violation counts from some base utility function. By using a different transform (e.g. $f_v(x) = c_v x$) for each rule v , we can also specify how violations of one norm are traded off against base utility, or

against violations of other norms. It is easy to see that norm-augmented utility functions satisfy *Conditional Preference* by letting $U_{\mathcal{N}}^i$ and U_0^i characterize $\preceq^i$ and $\preceq_0^i$ respectively. Of course, this is only one convenient way of satisfying *Conditional Preference*. To recover the ability to represent incompleteness, one might instead impose partial comparability [237] of violation counts.

If each agent i assumes that all other agents $j \neq i$ are norm internalizers with norm-augmented utility functions, and also knows their base utility functions U_0^j , beliefs B^j , and transforms f_v^j , then a reasonable way to concretely model the joint policy π is *(Boltzmann) rational norm compliance*.

Definition 6.13 ((Boltzmann) Rationality in SBGs). We say a history-dependent policy $\pi_\beta^i(a_t^i; h_t)$ is *Boltzmann rational* with respect to a utility function $U^i(h)$ and belief B^i when:

$$\pi_\beta^i(a_t^i; h_t) \propto \exp(\beta \mathbb{E}_{h \sim B^i(h|h_t, a_t^i)}[U^i(h)])$$

where $\beta > 0$ is a rationality parameter (higher β is more optimal), and B^i exhibits self-knowledge (i.e. $\pi_{B^i}^i = \pi_\beta^i$), s.t. the predictive distribution $B^i(h|h_t, a_t^i)$ over full histories is given by:

$$B^i(h|h_t, a_t^i) = P_H(|h|) \sum_{a_t^{-i} \in A^{-i}} B^i(a_t^{-i}|h_t) \cdot P_s(s_{t+1}|s_t, a_t^i, a_t^{-i}) \cdot \prod_{\tau=t+1}^{|h|} \pi_\beta^i(a_\tau^i; h_\tau) \cdot B^i(a_\tau^{-i}|h_\tau) \cdot P_s(s_{\tau+1}|s_\tau, a_\tau)$$

Taking $\beta \rightarrow \infty$, we recover *standard rationality*, i.e., subjective utility maximization.

Definition 6.14 ((Boltzmann) Rational Norm Compliance). In an NSG, we say that an agent i believes that the joint policy π exhibits *(Boltzmann) rational norm compliance* if their conditional belief over policies $B^i(\pi|N)$ induces a mixture policy of the form:

$$\pi_{B^i}(a_t; h_t, N) := \prod_{j=1}^M \pi_{B^i}^j(a_t^j; h_t, N)$$

where each individual policy $\pi_{B^i}^j(a_t^j; h_t, N)$ is (Boltzmann) rational with respect to $B^j(\cdot|N)$ and the N -conditioned norm-augmented utility function $U_N^j(h)$:

$$U_N^j(h) = U_0^j(h) - \sum_{v \in N} \sum_{j \in \mathcal{J}_{f,v}} n_v(h, j, B_v^j, f_v^j) \quad \text{where} \quad \forall h_t, B_v^j(h_t) := 1$$

If each $B^j(\cdot|N)$ for all agents $j \neq i$ also induces a (Boltzmann) rational mixture policy s.t.:

$$\forall i, j \in I, \quad \pi_{B^j}(a_t; h_t, N) = \pi_{B^i}(a_t; h_t, N)$$

then we say there is *mutual belief in (Boltzmann) rational norm compliance*. Furthermore, this joint policy is a fixed point corresponding to a logit-quantal response equilibrium [238, 239] when $\beta < \infty$, converging to a Nash equilibrium as $\beta \rightarrow \infty$.

We can weaken the above definition, allowing each agent i to have a (potentially false) belief that agent j has base utility U_0^{ij} and belief B^{ij} . In such a situation, agent i could still believe in mutual belief in (Boltzmann) rational norm compliance, even if this is not actually true. More generally, agent i might have priors about the beliefs and preferences of other agents [240, 241].

Since there may be degenerate cases where — in wanting to comply with some norms N , society ends up complying with them less in equilibrium than if they had never wanted to comply with N at all⁴ — even mutual belief in (Boltzmann) rational norm compliance does not guarantee *Belief in Relative Norm Compliance*. However, we conjecture that the former will imply the latter under fairly weak additional assumptions, which we leave for future research.

⁴For instance, a norm requiring every agent to give way to other agents when queuing up.

6.3 Normative Equilibria and Norm Convergence

With our definitions in hand, we are now able to discuss convergence and equilibria in NSGs. Setting aside for now whether a set of norms are rational to comply with, we consider an ideal norm learning scenario, where a set of learning agents $L \subseteq I$ interact with expert agents $E = I \setminus L$ who follow a policy consistent with some true set of norms N^* . We show that the posterior belief $B^i(N^*|h_t)$ converges to 1 as $t \rightarrow \infty$ under a few assumptions:

Proposition 6.1 (Convergence of Norm Learning from Experts). Consider a NSG with learning agents $L \subseteq I$ and expert agents $E = I \setminus L$. Let the true norms N^* and joint expert policy π^E be determined by sampling $(N^*, \pi^E) \sim P_N(N)P_E(\pi^E|N^*)$ according to some natural distribution. Each expert is informed of their part of π^E , which they follow. Suppose:

1. *Grain of Truth*: Each agent $i \in L$ assigns non-zero joint prior probability to the true norms and expert policy, i.e., $B^i(N^*)B^i(\pi^E|N^*) > 0$.
2. *Self-Knowledge*: Each agent has a true belief about their own policy, i.e., $\pi_{B^i}^i = \pi^i$.
3. *Identifiability*. For the true joint policy π^* and any alternative joint policy π' induced by norms $N' \neq N^*$, there exists a non-empty set of history-action pairs $\mathcal{D}_\epsilon(N^*, N) = \{(h, a) : D_{KL}(\pi^*(\cdot|h) || \pi'(\cdot|h)) \geq \epsilon\}$ that discriminate between the two policies.
4. *Informativeness*. For the true joint policy π^* and any alternative joint policy π' induced by norms $N' \neq N^*$, the probability of visiting discriminating history-action pairs in $\mathcal{D}_\epsilon(N^*, N')$ infinitely often is 1. Formally, let $\mathcal{V}_t = \{(h_\tau, a_\tau) : \tau \leq t, (h_\tau, a_\tau) \in \mathcal{D}_\epsilon(N^*, N')\}$ be the set of discriminating pairs visited up to time t . Then $\mathbb{P}_{\pi^*}(|\mathcal{V}_t| \rightarrow \infty \text{ as } t \rightarrow \infty) = 1$.

Then, for each learning agent $i \in L$, the posterior belief in the true norms converges to certainty almost surely when h_t is generated by following the true joint policy π^* :

$$\lim_{t \rightarrow \infty} B^i(N^*|h_t) = 1 \quad \text{a.s.}$$

Proof Sketch. The result follows from the consistency of Bayesian inference under the conditions above. Since the beliefs of learning agents' are not misspecified, the true norms N^* have non-zero prior probability. And since norms are identifiable and informative, the number of discriminating history-action pairs keeps growing. When comparing any alternative policy π to the true policy π^* , the total log-likelihood ratio is the sum of log-likelihood ratios for each discriminating $(h_\tau, a_\tau) \in \mathcal{V}_t$. The expected value for each of these ratios under π^* is the KL-divergence $\mathbb{E}_{\pi^*}[\frac{\pi^*(a_\tau; h_\tau)}{\pi(a_\tau; h_\tau)}]$, which is larger than 1 by the identifiability condition. As such, the expected value of the total likelihood ratio is unbounded as $t \rightarrow \infty$, implying convergence to N^* . $\square$

Corollary 6.1 (Convergence of Norm Learning under Policy Revelation). Let the true norms N^* and joint policy π^* be determined by sampling $(N^*, \pi^*) \sim P_N(N)P_\pi(\pi^*|N^*)$ according to some natural distribution. Each agent $i \in I$ is informed of their part of the joint policy π^{i*} , which they follow. Suppose the conditions in Proposition 6.1 hold. Then, for all agents $i \in I$, the posterior belief in the true norms converges to certainty almost surely:

$$\lim_{t \rightarrow \infty} B^i(N^*|h_t) = 1 \quad \text{a.s.}$$

Proof Sketch. Nothing in the proof for Proposition 6.1 precludes experts from also being learners. $\square$

While Corollary 6.1 may seem like a strong result, the requirement that each agent be informed of their part of the joint policy (i.e., their type) limits its usefulness as an account of norm learning. Since norms are highly informative of the revealed policies (and vice versa), the benefit of being able to converge to the true norms N^* after already knowing π^i is limited. At best, it is useful in cases where one has to learn norms that do not apply to oneself (e.g. role-based norms).

An open question for our theory is whether convergence of norm learning and following still applies when all agents have distinct subjective priors, and no policy revelation occurs at the start of the game. In this case, agents will not have access to a correlated signal at the beginning the game, and the behavior they observe from each other may not reflect any “true” norms N^* . Instead, agent behavior only becomes correlated through interaction with the environment and each other. Related Bayesian learning results can be found in Kalai and Lehrer [234, 242], but these results focus on beliefs about future histories, not beliefs about latent information like norms. Nonetheless, we hypothesize that norm learning agents will often display *self-confirming* behavior [243], eventually converging to some set of norms N even though there were no “true” norms initially. Indeed, we will present experiments demonstrating this phenomenon in Chapter 7.

The results above assume that some set of agents are informed of the norm-guided policy they should play, and simply follow it rather than considering whether it is in their interest to do so. This loses the most interesting aspect of norms — that they are shared rules that help us coordinate and adjudicate our often conflicting interests. To study this aspect of social normativity, we need to examine whether certain sets of norms are strategically stable. We call such a stable normative arrangement a *normative equilibrium*. We define normative equilibria with respect to existing equilibrium notions that are applicable to stochastic Bayesian games more broadly, considering both of the rationality concepts introduced in Definition 6.13.

Definition 6.15 (Subjective Equilibrium). In a utility-based SBG, a collection of beliefs $\{B^i\}_{i \in I}$ and the induced policies $\{\pi_{B^i}\}_{i \in I}$ form a *subjective Nash equilibrium* [234] w.r.t. the true joint policy π^* if for each agent $i \in I$, and at each history $h_t \in H$, we have:

- *Subjective Optimization*: $\pi_{B^i}^i$ is optimal with respect to i ’s beliefs about other agents’ policies:

$$\forall \tilde{\pi}^i \neq \pi_{B^i}^i \quad \mathbb{E}_{a_t^i \sim \pi_{B^i}^i(a_t^i; h_t)} [\mathbb{E}_{h \sim B^i(h|h_t, a_t^i)} [U^i(h)]] \geq \mathbb{E}_{a_t^i \sim \tilde{\pi}^i(a_t^i; h_t)} [\mathbb{E}_{h \sim B^i(h|h_t, a_t^i)} [U^i(h)]]$$

- *Uncontradicted Beliefs*: $B^i(h|h_t) = \pi^*(h|h_t)$ when h_t has positive probability $\pi^*(h_t) > 0$.

Furthermore, suppose agents’ beliefs $\{B^i\}_{i \in I}$ are *correlated*,⁵ i.e. there exists a signal $m \sim P(m)$ s.t. $P(B^1, \dots, B^M) = P(m) \prod_{i=1}^M P(B^i|m)$, then they form a *subjective correlated equilibrium*.

Definition 6.16 (Subjective Boltzmann Equilibrium). In a utility-based SBG, a collection of beliefs $\{B^i\}_{i \in I}$ and the induced policies $\{\pi^i\}_{i \in I}$ form a *subjective Boltzmann equilibrium* (or *subjective quantal response equilibrium* [238]) with rationality parameter $\beta > 0$ if we replace *Subjective Optimization* in Definition 6.15 with *Boltzmann Rationality* (Definition 6.13):

$$\pi^i(a_t^i; h_t) \propto \exp(\beta \mathbb{E}_{h \sim B^i(h|h_t, a_t^i)} [U^i(h)])$$

Similarly, we define a *subjective Boltzmann correlated equilibrium* as a subjective Boltzmann equilibrium with correlated beliefs.

We remark that it is possible to extend subjective equilibria to games with incomplete preferences by replacing utility maximization with Pareto improvements (similar to [224]).

⁵Which is distinct from saying that they *believe* their beliefs or policies to be correlated.

In NSGs, any equilibrium set of beliefs $\{B^i\}_{i \in I}$ assigns (posterior) probabilities to each set of possible norms $N \in \mathcal{N}$. We characterize normative equilibria as those equilibria where each agent's beliefs have converged on some set of norms N .

Definition 6.17 (ϵ -Normative Equilibrium). In a NSG, a collection of beliefs $\{B^i\}_{i \in I}$ and the induced policies $\{\pi^i\}_{i \in I}$ form an ϵ -subjective normative equilibrium if:

1. $\{B^i\}_{i \in I}$ and $\{\pi^i\}_{i \in I}$ form a subjective (Boltzmann) equilibrium.
2. For each agent $i \in I$, there exists a *non-empty* set of rules $N^i \subseteq \mathcal{N}$ such that $B^i(N^i) \geq 1 - \epsilon$.

That is, in an ϵ -subjective normative equilibrium, each agent has converged to assigning high probability (at least $1 - \epsilon$) to some set of subjective norms, which may differ across agents.⁶

In the case where a single set of norms $N^* \subseteq \mathcal{N}$ is assigned a high probability $B^i(N^*) \geq 1 - \epsilon$ by *all* agents $i \in I$, we call this an ϵ -objective normative equilibrium. We also say that N^* is in an ϵ -normative equilibrium.

Why characterize normative equilibria in terms of convergence in *beliefs*, rather than the *policies* which are actually followed? Recall that in our definition of NSGs, a link between beliefs and behavior is established by *Belief in Relative Norm Compliance*, which can be supplemented by conditions such as *Belief in ϵ -Norm Compliance* or *Belief in Boltzmann Rational Norm Compliance*. Furthermore, note that in subjective equilibrium, each agents' beliefs must correspond to reality on all positive-probability histories by the *Uncontradicted Beliefs* criterion. As such, if all agents believe in some set of norms N with sufficiently high probability at equilibrium, then the corresponding policy will satisfy a norm compliance condition with high probability.

Unfortunately, Definition 6.17 only characterizes what it means for norms to be strategically stable, and does not provide insight about what *kinds* of norms N are likely to stable. Below, we suggest a few general ways that stability might be achieved:

- *Coordination Norms*: Suppose agents are playing a coordination game, i.e. games where two or more sets of histories $H^a, H^b, \dots$ have positive utility for all agents, and where any unilateral deviation from those histories results in a decrease in utility for one or more players. A coordination norm is a rule v that maps any history in one of those sets (say H^a) to zero, and other histories to one. If all agents have a strong belief in v (i.e. they expect a low norm violation count w.r.t. v), they will prefer to comply with v rather than deviate from it, even without norm-augmented utilities.
- *Strong Internalization*: Suppose all agents have strongly internalized a norm v , s.t. the cost of violating v in their norm-augmented utility function U^i is larger than the maximum value of their base utility $\max_h U_0^i(h)$. If each agent complying with v does not preclude others from complying v (e.g. v only prohibits anti-social actions within the full control of each agent), then all agents will prefer to comply with v rather than deviate from it.
- *Punishment Norms*: Suppose each agent i believes w.h.p. that society follows a norm set $\{v_1, v_2\}$, where v_2 is punishment norm requiring some other agent j to punish i if v_1 is violated by i , decreasing their utility U^i by a sufficiently large amount. Then all agents will prefer to comply with v_1 rather than deviate from it. Furthermore, $\{v_1, v_2\}$ will be in equilibrium if, when some agent i does violate v_1 , at least one agent j prefers to comply with v_2 (e.g. due to internalization, or the benefits of signaling to all that v_1 is enforced).

⁶This may arise due to non-identifiability, or imperfect monitoring in the partially observable extension of NSGs.

Finally, we make a brief remark on convergence. As in the case of norm learning without policy revelation, it is an open question when a group of agents will converge to a normative equilibrium. Kalai and Lehrer [234] give results for convergence to subjective equilibria under the assumption that all agents are perfect Bayesian learners. We expect that these results also generalize to subjective Boltzmann equilibria. However, convergence to a subjective equilibrium does not guarantee convergence in belief to a (non-empty) set of norms N — agents might instead believe that *no* norms are followed with high probability, or fail to converge due to non-identifiability. Nonetheless, we conjecture that convergence can be assured under conditions similar to those used in Proposition 6.1, perhaps by constructing a correlation device out of agents’ interaction histories in a manner similar to [234]. We leave the pursuit of this question to future work.

6.4 Discussion and Related Work

Norm-augmented stochastic games constitute a general framework for modeling and engineering many aspects of human social normativity in temporally-extended settings. By characterizing norms and normativity in the language of stochastic games with uncertain beliefs, NSGs make precise not just how practical deliberation might be constrained by norms, but also account for the epistemology of norm learning. In developing this account, we have drawn upon a rich literature across philosophy [16, 217], economics [15, 218, 244], game theory [152, 234, 238], AI [232, 233], and cognitive science [26, 29]. Below, we summarize how NSGs relate to and build upon prior characterizations of social normativity, while noting the limitations that remain in our framework.

Norms as Rules with Shared Expectations. Our approach to modeling social normativity builds upon Bicchieri’s [16] definition of social norms as situational rules that agents conditionally prefer to follow when they believe others follow the same rules and expect compliance to them. In NSGs, we capture situational rules as functions that classify behavioral histories, rather than just individual or joint strategies in normal form games. Furthermore, we give a Bayesian characterization of agents’ beliefs about what rules society follows, explaining how agents might form these beliefs in the first place. We also partly capture Bicchieri’s notion of conditional preference, allowing for intrinsic preferences for norm compliance that are belief-dependent (in addition to the instrumental preferences for norm compliance that arise naturally in a game theoretic setting).

One aspect of Bicchieri’s account not addressed by NSGs is the stipulation that conditional preferences for norm compliance should depend not just on *empirical expectations* that others’ will follow a rule, but also belief in *normative expectations* — a second-order belief that others expect one to comply with the rule. Bicchieri takes dependence on normative expectations to be the key distinguishing factor between norms that are properly *social* versus what she calls *descriptive norms*. NSGs elide this distinction, and hence technically only model what Bicchieri would call descriptive norms.

A straightforward way to address this limitation would be to enrich the *Conditional Preference* condition of NSGs, allowing for preferences to depend on second-order beliefs, and in particular, beliefs that other agents’ believe that society will follow some rule. However, this leads to a less tractable model for dubious benefit, since it is not clear that first and second-order beliefs and what rules are followed come apart very often. More importantly, second-order *empirical* expectations are still not quite *normative* expectations, given their reversed directionality of fit [245].

A better approach might be to treat normative expectations as more desire-like, implying willingness to enforce or express approval for some rule. Past work on social norm learning has implicitly taken this approach, allowing for inferences about a norm to be drawn from enforcement actions [29]. To account for this in NSGs, we might modify *Belief in Relative Norm Compliance* to incorporate *Belief in Relative Norm Endorsement*, stipulating some connection between a norm set N and observable signals of approval, enforcement, or normative judgment. In this way, when agents update their beliefs about N , they would do it on the basis of not just observed compliance, but also evidence that compliance is normatively expected.

Norms as Correlating Devices. Gintis [15] has argued that social norms function as choreographers or correlating devices that tell individuals which strategies to play, forming a correlated strategy. Similarly, Vanderschraaf [235] has argued that conventions should be understood as correlated equilibria. Unlike Nash equilibria, which require agents’ policies to be statistically independent, correlation allows for a much richer set of equilibrium strategies (and hence stable norms) to be achieved. However, prior work making the connection between norms and correlated equilibria has often directly identified a norm or convention with the full correlated strategy (i.e., the joint policy), typically in the context of simple normal form games. This stands in contrast to our intuitive understanding of norms as *abstract rules* that constrain certain aspects of our behavior, rather than detailing every action we should take in every situation.

To bridge this gap, NSGs treat norms as high-level rules that are associated with *sets* of joint policies. In particular, a rule set N is associated with a set joint policies that (approximately) comply with those rules. This structure features in each agent’s beliefs about societal behavior in a way that maintains correlation, per *Belief in Shared Norms*. Furthermore, NSGs provide a way to explain how agents’ converge upon correlated strategies in equilibrium. Since agents believe that everyone’s actions are correlated, acting in accordance with those beliefs (e.g. by complying with norms) will cause their *actual* policies to be correlated, helping agents achieve coordination.

Norms as Logical or Deontic Constraints. In normative multi-agent systems research [246, 247], norms are often formalized as obligations, prohibitions, or permissions expressed in various logical systems [248–250], especially deontic logic [226]. This approach provides a formal semantics for normative reasoning that machines can implement. However, it is not always clear how to integrate these approaches with decision-theoretic frameworks like game theory or reinforcement learning. Furthermore, while logic provides a flexible and structured medium for representing the *content* of norms, it does not account for their *social grounding*, nor how logical or deontic statements might be learned from observations.

While NSGs abstract away from the details of how norms are represented, our treatment of norms as rules that classify agent histories is highly compatible with a logical representation. Indeed, in Chapter 7, logical formulas serve as the implementation language for the rules of the NSG that we study. Similar to previous studies on Bayesian norm learning [17, 251, 252] and rule induction [253, 254], this integration enables us to analyze how rational agents might learn and respond to structured norms with well-defined compositional semantics.

Chapter 7

Learning Social Norms via Bayesian Inverse Planning

見賢思齊焉，見不賢而內自省也。

On seeing a person with virtue, consider emulating them. On seeing a person without virtue, reflect on whether you share their defects.

Confucius

As autonomous agents become increasingly embedded in human society, a key technical and political challenge will be to ensure that their deployment leads to mutual benefit and flourishing, while avoiding the risks of conflict, collusion, and miscoordination [255]. To address this challenge, shared normative standards — such as social norms — serve as promising targets for aligning autonomous systems with societal interests even while they primarily serve the goals of their users [249, 256–258]. How then can we build agents that learn and comply with norms?

In the previous chapter, we introduced norm-augmented stochastic games (NSGs) as model of human-like social normativity, including the acquisition of social norms. In NSGs, each agent assumes that all other agents may be complying with a shared set of norms even while pursuing their own interests, allowing them to infer the existence of a norm from apparent violations of self-interest, and to aggregate such observations across a population of agents, updating their beliefs about which norms best explain collective behavior. Having inferred these norms, an agent can then comply with them, either because of an intrinsic motivation [259], or because it is strategically valuable [260]. As such, shared normativity is often *self-sustaining*: Once enough agents infer a shared set of norms, they are likely to conform to them, thereby creating public knowledge that the norms are practiced, and enabling new entrants to rapidly learn the norms.

So far, however, our treatment of norms and norm learning has been highly abstract. While NSGs provide a general theoretical framework, instantiating this framework comes with a number of practical challenges: (i) How should norms be represented, so as to capture the rich space of constraints and standards that mediate social interaction? (ii) How can agents reliably comply with norms while still pursuing their own interests, especially in the presence of complex, history-dependent norms? (iii) Most importantly, given its apparent intractability, how can the Bayesian approach to norm learning assumed by NSGs be practically implemented or approximated?

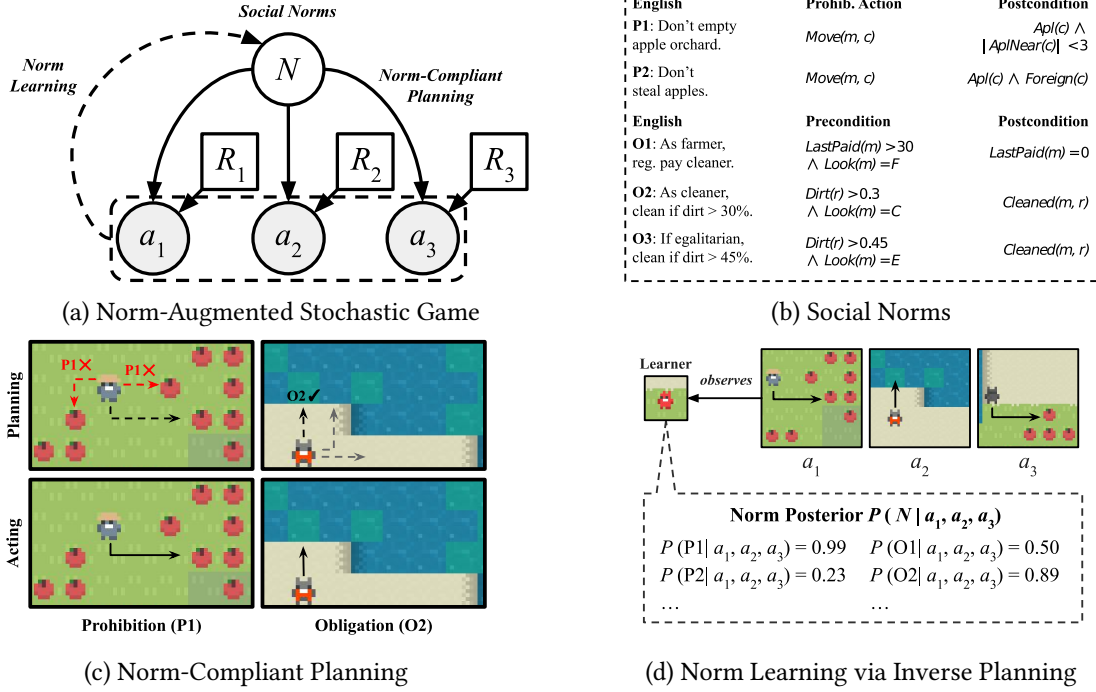


Figure 7.1: Overview of our architecture, summarized by (a) the (simplified) graphical model for a norm-augmented stochastic game. Agents learn to comply with rule-based social norms (b), which are factored into prohibitions (e.g. P1, P2) and obligations (e.g. O1, O2, O3). Agents comply with norms while pursuing their interests via model-based planning (c), which switches between a reward-oriented mode that obeys prohibitions while collecting rewards (left), and an obligation-oriented mode that plans to satisfy a postcondition (right). Agents learn these norms via (d) Bayesian inverse planning, updating the probabilities of each potential norm based on observations of others’ actions.

This chapter takes up these challenges directly, introducing a practical architecture for approximately Bayesian norm learning in a norm-augmented stochastic game (Figure 7.1a). We introduce a logical rule-based representation of social norms (Figure 7.1b), which take the form of obligations or prohibitions, and show how agents can comply with these norms through model-based planning (Figure 7.1c). Finally, we show how agents can perform approximately Bayesian *rule induction* [253, 261] via bounded inverse planning in the vein of SIPS (Chapter 1), inferring norms from observations of violation and compliance with potential obligations and prohibitions (Figure 7.1d).

To evaluate our approach, we adapt and combine several existing games from the Melting Pot suite of environments [262, 263], including elements of Commons Harvest [264, 265], Clean Up [266], and Territory [262]. The resulting environment $\mathcal{E}$ is shown in Figure 7.1, serving as the basis for a reward-based NSG $\mathcal{NG} = (\mathcal{E}, \{R^i\}_{i \in I}, \Pi, \{B^i\}_{i \in I}, \mathcal{N})$. Agents can collect and eat apples to gain reward but otherwise incur a cost for actions (see Appendix for details). Apples regrow at a density-dependent rate (nearby apples lead to faster regrowth), but the rate decreases as the river gets polluted over time. To prevent pollution, agents have to use a clean-up action on the river. In addition, several regions are visually marked as each agent’s territory (but do not otherwise have effects), and agents have one of three distinct appearances that demarcate *roles*. By combining these elements, we allowed for a range of social norms $\mathcal{N}$, including resource management norms, property norms, role-based division of labor, and compensation for pro-social labor.

In contrast to norm learning frameworks based on (model-free) reinforcement learning (RL) [267–269], our approach foregrounds the *epistemic* and *cognitive* aspects of social norms, framing norm learning as an (approximately) rational Bayesian process of learning structured rules [17, 253, 254, 270, 271], rather than a reactive process of adapting one’s habitual actions [272, 273]. As such, our account captures the speed and flexibility of norm learning and coordination in human children and adults [274–276]. Through our experiments, we demonstrate that approximately Bayesian norm learning is indeed feasible in long-horizon multi-agent contexts while requiring orders of magnitude less experience than model-free norm learning from sanctions [267, 272]. We also find that shared normativity enables both the maintenance and convergence of normative systems: Mutual belief in the norms is maintained despite the “deaths” of experienced agents and the “births” of normative novices, while groups of agents can bootstrap themselves towards a stable set of norms by learning from each others’ actions and experimentally complying with tentative norms.

Collectively, these findings highlight the importance of integrating game-theoretic accounts of norms and conventions [15, 277, 278] with the conceptual richness and flexibility of human learning [253, 279, 280]. By uniting the structured representations of normative concepts common in normative systems research [246, 247, 250] with a Bayesian decision-theoretic approach to multi-agent coordination [234, 244], our architecture captures the dynamism of human normative cognition that is omitted from model-free RL approaches, while paving the way for normatively-competent autonomous systems that combine model-free and model-based learning [281, 282].

7.1 Representing Social Norms

NSGs are a general setting for multi-agent norm learning, but they leave unspecified the set $\mathcal{N}$ of possible norms. Drawing upon deontic and temporal logic [17, 248, 283], we consider learning both (maintenance-based) *prohibitions* $\mathcal{N}_P$, which forbid the effects of certain actions at all times, and (achievement-based) *obligations* $\mathcal{N}_O$, which require pursuing *temporally extended* goals, giving our full set of norms $\mathcal{N} = \mathcal{N}_P \cup \mathcal{N}_O$. Several examples are shown in Figure 7.1b. Note that the distinction between prohibitions and obligations is not fundamental: Prohibitions can be represented as obligations via negating the effects and vice versa. Nonetheless, we found it more natural to represent maintenance norms in prohibitive form (e.g. “Don’t empty the apple orchard.”) and achievement norms in obligative form (e.g. “Pay the cleaner in the next 30 steps.”).

Following prior work on norm and rule learning [253, 271], we express norms as rules in a first-order language $\mathcal{L}$, which includes predicates and functions defined over objects in a state s . Given a first-order expression ϕ , we denote its valuation in s as $s[\phi]$. Using this notation, we represent a prohibition norm $\nu_P \in \mathcal{N}_P$ as a tuple $\langle \text{Prohib}, \text{Post} \rangle$ where *Prohib* is a function of objects in s that returns a set of potentially prohibited actions, and the postcondition *Post* holds in a state s whenever the prohibited effect occurs. In other words, if an action $a^i \in s[\text{Prohib}]$ leads to state s' such that $s'[\text{Post}]$ is true, this means that the prohibition has been violated, and hence that $\nu_P(h^i, s, a^i, s') = 1$.¹ As an example, an agent m in our environment is prohibited from moving to a cell c (*Prohib* = *Move*(m, c)) if that cell contains apples and is in foreign territory (*Post* = *Apl*(c) $\wedge$ *Foreign*(c)) as depicted in Figure 7.1b.

¹We only consider rules ν with degree $d_\nu = 1$, so we omit the dependence on J introduced in Definition 6.6.

Next, we define obligations $v_O \in \mathcal{N}_O$ as tuples $\langle \text{Pre}, \text{Post}, \tau \rangle$, where Pre is a precondition formula that holds true when the obligation is triggered, and Post is a postcondition formula that holds true once the obligation has been discharged, with τ as the duration during which the obligation must be satisfied. Given a history $h^i = (s_{t-k}, a_{t-k}^i, \dots, s_t, a_t^i)$ up to time step t , this means that $v_O(h) = 1$ (the obligation is violated) if there exists some $k \leq \tau$ such that $s_{t-k}[\text{Pre}]$ is true but $s_{t-k+j}[\text{Post}]$ is false for all $k < j \leq \tau$. As another example, when agents are in the cleaner role ($\text{Look}(m) = C$) and find the river more than 30% polluted ($\text{Dirt}(r) > 0.3$) they are obliged to have cleaned the river ($\text{Cleaned}(m, r)$) within the next 20 steps ($\tau = 20$). Therefore, $(\text{Pre} = (\text{Dirt}(r) > 0.3 \wedge \text{Look}(m) = C), \text{Post} = \text{Cleaned}(m, r), \tau = 20)$.

7.2 Norm-Compliant Planning

Having introduced how norms classify behavior, we now describe how agents can plan to comply with a set of norms $N = N_P \cup N_O$ where $N_P \subseteq \mathcal{N}_P$ and $N_O \subseteq \mathcal{N}_O$. We assume that all of our agents are norm internalizers, as defined in Definition 6.10. As such, their preferences dictate maximization of their norm-augmented utility function (Definition 6.12). Since we consider a reward-based NSG, the norm-augmented utility function decomposes into a sum of rewards minus belief-weighted violation counts:

$$\begin{aligned} U_{\mathcal{N}}^i(h_t) &= \sum_{\tau=1}^{t-1} R^i(s_\tau, a_\tau^i, s_{\tau+1}) - \sum_{v \in \mathcal{N}} n_v(h, \{i\}, B_v^i, f_v^i) \\ &= \sum_{\tau=1}^{t-1} R^i(s_\tau, a_\tau^i, s_{\tau+1}) + \sum_{\tau=1}^t \sum_{v \in \mathcal{N}} f_v^i(B^i(v \in N|h_\tau)) \cdot v(h_\tau) \end{aligned} \quad (7.1)$$

where $B^i(v \in N|h_\tau)$ is agent i 's posterior belief in v after history h_τ , and $f_v^i : [0, 1] \rightarrow \mathbb{R}$ is a monotonic transform that accounts for belief thresholding or cost multipliers. When there is no belief thresholding, we let $f_v^i(x) = C_v^i x$, so that there is a fixed cost C_v^i for any norm violation. We first assume that the agent is certain that a set of norms N holds (so $B^i(v \in N|h_\tau) = 1$ for all $v \in N$, and zero otherwise), then discuss how agents can plan under uncertainty about which norms are followed by society.

Even when an agent is certain about N , complying with all of them can be challenging due to temporally extended obligations. As such, we design our agents to use two different planning modes, namely *reward-oriented planning* and *obligation-oriented planning*. Each of these corresponds to a different sub-problem that is a *proxy* of the true game. Agents switch from the former mode to the latter mode whenever an obligation precondition is triggered.

In reward-oriented planning, agents maximize Equation 7.1 except with the cost of obligation norms ignored. Since prohibition norms are not temporally extended, this gives us a proxy reward $R_{N_P}^i(s, a^i, s')$ that only depends on the current state transition. We maximize this reward through model-based planning, assuming that each agent i has a reasonably accurate model $P_s^i(s'|s, a^i)$ of the environment and other agents. In particular, we use real-time dynamic programming (RTDP) [53] to compute estimates of the state and action value functions:

$$Q_{N_P}^i(s, a^i) = \sum_{s' \in \mathcal{S}} P_s^i(s'|s, a^i) [R_{N_P}^i(s, a^i, s') + V_{N_P}^i(s')] \quad (7.2)$$

$$V_{N_P}^i(s) = \max_{a^i \in \mathcal{A}^i} Q_{N_P}^i(s, a^i) \quad (7.3)$$

While reward-oriented planning accounts for prohibitions, it ignores the potential future costs that might arise due to obligations becoming active. Instead, our agents reactively switch to obligation-oriented planning once the precondition Pre of some obligation $v_O = \langle \text{Pre}, \text{Post}, \tau \rangle$ is met and then plan to achieve Post as a goal. This creates a (stochastic) shortest-path problem, where any state s with $s[\text{Post}] = 1$ is treated as a *terminal* state, with a modified reward function that accounts only for action costs $C^i(a^i)$, prohibition costs $C_{v_p}^i$ and delivers a reward of 1 when Post is satisfied:

$$R_i^{v_O \cup N_p}(s, a^i, s') = C^i(a^i) + \sum_{v_p \in N_p} C_{v_p}^i \cdot v_p(s, a^i, s') + s'[\text{Post}] \quad (7.4)$$

We similarly maximize this reward via model-based planning, giving us value estimates $Q_i^{v_O \cup N_p}$ and $V_i^{v_O \cup N_p}$ for each obligation v_O that becomes active. Agents thus act by selecting actions that maximize either $Q_i^{N_p}$ or $Q_i^{v_O \cup N_p}$ depending on whether an obligation v_O is active, performing several iterations of RTDP before acting. Obligations are satisfied in the order that their preconditions are triggered, and each agent maintains a queue of such obligations to satisfy.

Thus far, we have described how planning occurs when agents are *certain* about a set of norms N . In general, however, agents only have a *belief* $B_t^i(N)$ over norms. To handle this uncertainty, we consider two simple approaches, *thresholding* and *sampling*, which avoid the need to sum over all possible norms $v \in N$. When thresholding, agents comply with a norm v whenever its probability is higher than a threshold $B_t^i(v \in N) \geq \theta$, corresponding to setting $f_v^i(x) = C_v^i \cdot \mathbf{1}_{x \geq \theta}(x)$. When sampling, agents sample of set of norms $N' \sim B_t^i(N)$ to comply with for a certain number of steps K , before resampling. This is also known as probability matching [284] or Thompson sampling [285]. Each approach has properties that may be useful. Thresholding is low variance and may be appropriate when norm violation is sufficiently costly. In contrast, sampling can promote exploration, especially under high uncertainty about which norms are true [286].

7.3 Bayesian Norm Learning

In Bayesian norm learning, an agent i computes a posterior distribution $B^i(N|h_t)$ over the norms followed by society given the history h_t of interactions so far. As discussed in Chapter 6, this posterior is defined with respect to a conditional belief over joint policies $B^i(\pi|N)$, which induces a mixture policy $\pi_{B^i}(h_t; N) = \sum_{\pi} B(\pi|N) \cdot \pi(h_t)$:

$$B^i(N|h_t) \propto B^i(N) \sum_{\pi} B(\pi|N) \cdot \pi(h_t) \equiv B^i(N) \cdot \pi_{B^i}(h_t; N) \quad (7.5)$$

What should the shape of $\pi_{B^i}(h_t; N)$ be, so as to render norm learning tractable? One concrete model we considered was *Boltzmann rational norm compliance* (Definition 6.14). Under this model, agent i assumes that all agents are Boltzmann-rational w.r.t. their N -augmented expected utility, resulting in a joint policy $\pi_{B^i}(h_t; N)$ that puts more probability on actions that increase expected utility while avoiding N -violations, but always assigns non-zero probability to any action a_t .²

$$\pi_{B^i}(a_t; h_t, N) \propto \prod_{j=1}^M \exp(\beta \mathbb{E}_{h \sim B^j(h|h_t, a_t^j)} [U_N^j(h)]) \quad (7.6)$$

²As long as no actions lead to negative infinite utility.

Still, as we have seen in earlier chapters (Chapter 1), Boltzmann rational policies can be intractable to compute, due to the intractability of computing the expected utility in Equation 7.6. Furthermore, if agent i is not Boltzmann rational, but assumes Boltzmann rational norm compliance, this violates the *Self-Knowledge* condition (Proposition 6.1) that allows agent i to avoid updating their beliefs based on actions they take themselves. As such, we make two modifications: First, we assume that agent i has self-knowledge.³ Second, we assume only *bounded* Boltzmann rationality when modeling the other agents, using the same approximate Q -values computed by norm compliant planning via RTDP (Section 7.2).

Even under these modeling assumptions, inferring the exact posterior over N — the *rule induction* problem [253, 261] — is still highly intractable, since it requires enumerating over all subsets $N \subseteq \mathcal{N}$. Instead, we compute a *mean-field approximation* [287] of the posterior, assuming that it factorizes into approximate local posteriors $\tilde{B}^i(v \in H|h_t, a_t^{-i})$ for each rule $v \in \mathcal{N}$:

$$B_i(N|h_t, a_t^{-i}) \approx \tilde{B}^i(N|h_t, a_t^{-i}) \propto \prod_{v \in \mathcal{N}} \tilde{B}^i(v \in N|h_t, a_t^{-i}) \quad (7.7)$$

This approximation allows us to update beliefs about each norm v *independently*:

$$\tilde{B}^i(v \in N|h_t, a_t^{-i}) \propto \tilde{B}^i(v \in N|h_t) \cdot \tilde{\pi}_{B^i}^{-i}(a_t^{-i}; h_t, v) \quad (7.8)$$

where $\tilde{\pi}_{B^i}^{-i}(a_t^{-i}; h_t, v)$ approximates the Boltzmann rational policy for agents $-i$ under the assumption that v is the *only* norm they comply with. As noted above, we can compute this using the Q -values derived by bounded norm compliant planning:

$$\tilde{\pi}_{B^i}^{-i}(a_t^{-i}; h_t, v) \propto \prod_{j \neq i} \exp(Q_v^j(s, a^j)) \quad (7.9)$$

Altogether, this approach to Bayesian norm learning can be viewed as a form of *Bayesian inverse planning*, where agent i inverts an approximate model of norm-compliant planning to infer the norms that all other agents comply with.

7.4 Experiments

Having developed our model of norm-guided learning and planning, we investigate whether it reproduces the aspects of human normativity that our theory suggests. To do, we conducted experiment studying four key phenomena: (1) *Passive Norm Learning*: We explore the speed and effectiveness of learning norms via (approximately) Bayesian updating from passive observations [17], assuming that norms are shared by other agents. (2) *Norm-Enabled Social Outcomes*: We investigate whether our model of norm learning can foster collectively beneficial coordination [15, 278], given the right set of norms [218, 288, 289]. (3) *Intergenerational Norm Transmission*: We assess if norms can be sustained across generations of agents, and under what circumstances this occurs. (4) *Norm Emergence and Convergence*: We investigate whether shared normativity enables norms to organically emerge and stabilize through Bayesian learning without explicit sanctions.

In all of our experiments, we used a sample size of 13 runs for each condition, determined based on an effect size of 0.8 with $\alpha = 0.05$ (one-sample t -test assuming a normal distribution).

³Only in the sense that agent i knows what action they will take at the moment they take it, not necessarily in advance. Hence, there is nothing for agent i to learn from taking that action. This can be seen as an alternate justification for applying the **do**-operator in the belief update used by CLIPS (Chapter 4).

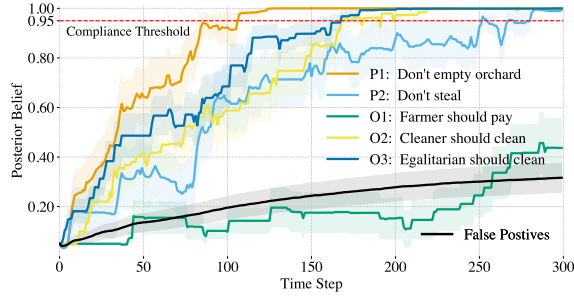


Figure 7.2: Passive Norm Learning. Average posterior belief of the learner in each of the five norms practiced by the experienced agents, N_{active} . Most norms were acquired within ≤ 300 steps.

$ N_{\text{active}} $	$t/H = 25\%$	50%	75%	100%
	Pre/Rec	Pre/Rec	Pre/Rec	Pre/Rec
0	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00
1	0.15/0.09	0.08/0.40	0.06/0.54	0.06/0.57
2	0.19/0.27	0.14/0.59	0.13/0.76	0.12/0.82
3	0.24/0.37	0.21/0.63	0.18/0.77	0.18/ 0.83
4	0.35/0.36	0.28/0.67	0.24/0.77	0.22/ 0.83
5	0.47/0.25	0.43/0.57	0.34/0.79	0.32/ 0.83

Table 7.1: Average Precision (Pre) and Recall (Rec) for the learner successfully acquiring ($P(v) \geq 0.95$) the active norms N_{active} after a certain % of the episode, where H denotes episode length.

7.4.1 Passive Norm Learning

In our norm learning experiments, we investigated whether a single learning agent l could acquire the same norms believed in by a set of $M_{\text{Exp}} = 3$ experienced agents, each playing one of three roles $z \in Z := \{C, F, E\}$ (read as “cleaner”, “farmer” and “egalitarian”). The learning agent also had an egalitarian role ($z_{\text{Lea}} = E$), and each episode lasted $H = 300$ steps. All experienced agents assigned a probability of 1 to the norms $N_{\text{active}} = \{P1, P2, O1, O2, O3\}$ shown in Figure 7.1b. The full space $\mathcal{N}$ of 68 possible norms was generated by enumerating over a variety of predicates and numeric thresholds (see Appendix). The learning agent handled uncertainty over norms via thresholding, and had a high norm violation cost, motivating it to comply with learned norms.

Figure 7.2 demonstrates the versatility and efficiency of our learning model across a diverse set of norms. The majority of norms practiced by the experienced agents were rapidly and effectively internalized within 300 steps, in contrast to the *millions* of steps required by model-free norm learning approaches [272]. This was despite some norms being complied with only occasionally (e.g. the cleaner’s obligation to clean the dirty river), providing a limited learning signal for observers. Interestingly, Obligation 1 (concerning the farmer’s payment) was *not* learned very well, with an average confidence of 0.4 even after 300 steps. This was likely due to the fact that the obligation was hard to satisfy, requiring the farmer to run after its assigned cleaner to pay them apples. As a result, the learning agent often inferred that the norm was *non-existent* rather than merely hard to satisfy. To address this, more sophisticated norm learners should distinguish unintentional failure to comply with a norm from the absence of a norm. After all, many human obligations can be hard to fulfill, but we still take them to be in force despite their difficulty.

Table 7.1 shows our model’s learning curve over time for varying numbers of actively practiced norms, as reflected in the precision and recall metrics. While we observed a higher recall with more time steps, precision was low in most cases, indicating that the learner was “over-learning” norms. From qualitative inspections of each run, we found that this was due to several reasons: First, the learning agent sometimes captured the underlying dynamics of the environment. Even though norms were not explicitly followed, agents sometimes inferred them to be true if certain states of an environment were always avoided. For instance, agents learned to not be allowed to move when there was $> 55\%$ of dirt in the river which was a state that was *implicitly* avoided due to the river being cleaned on a regular basis. Such observations shed light on the phenomenon of overgeneralized norm learning in real-world contexts [254]: Similar to unintentionally failed

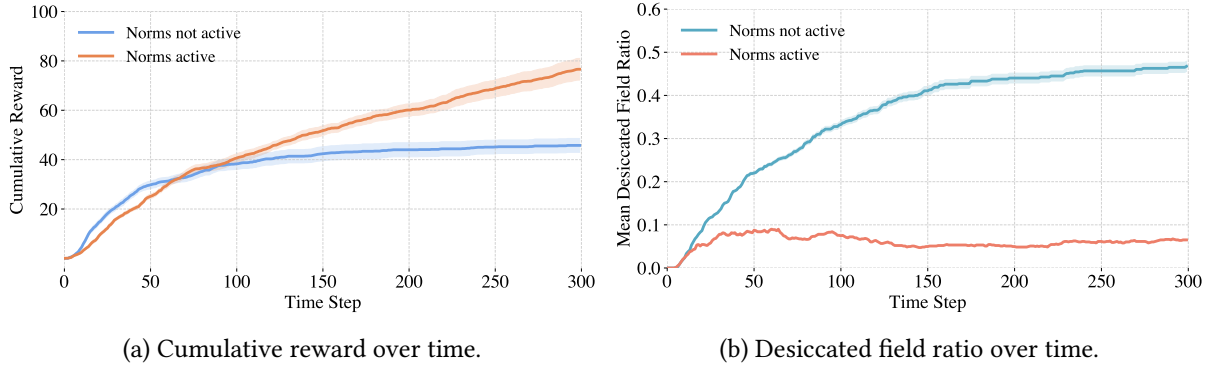


Figure 7.3: Social outcomes in the presence of a norm learning agent. We measure whether beneficial outcomes are achieved when one agent learns from three experienced agents practicing a set of norms. Either all norms in $\{P1, P2, O1, O2, O3\}$ are practiced (*Norms active*) or none of them are (*Norms not active*).

obligations, there exist “accidentally fulfilled prohibitions”, and sophisticated learners should learn to recognize and account for both.

Second, we noticed that agents often learned *logically overlapping* norms, reflecting the fact that some rules in $\mathcal{N}$ logically entailed other rules (e.g. if $\text{Dirt}(r) > 0.4$ is true, then $\text{Dirt}(r) > 0.3$ is also true). This behavior was likely due to the mean-field approximation of the posterior in Equation 7.7, which prevented *explaining away* among overlapping norms [290]. Future work should investigate how Bayesian norm learning can be made efficient without the mean-field approximation, perhaps via divide-and-conquer strategies [291] that first infer local posteriors over individual norms, then combine them into joint posteriors over sets of norms.

7.4.2 Norm-Enabled Social Outcomes

Norms are instrumental in steering pro-social behavior and cultivating cooperation within societies [289, 292–294], and past research has illustrated the efficacy of norm compliance in addressing social challenges and enabling favorable outcomes [218, 265, 266, 295, 296]. We investigated whether it was possible to achieve such outcomes even with one agent as norm learner, measuring social welfare in terms of the collective cumulative reward obtained by all agents [267] (hereafter *cumulative reward*) (Figure 7.3a) and sustainability of the environment [265] (Figure 7.3b), as measured by the ratio of desiccated apple fields in our environment relative to the total number of fields. While these were only two measures of desirable social outcomes (others might include equality, efficiency, etc. [265]), they nonetheless offered a useful measure of the benefit provided by norm-guided coordination. We conducted this analysis with the independent variable being the set N of norms, where $N \in \{\emptyset, \{P1, P2, O1, O2, O3\}\}$. All other experimental conditions were consistent with those described earlier: one learning agent, and three experienced ones.

Perhaps unsurprisingly, the presence of norms all five norms led to a clear increase in social welfare (Figure 7.3). Specifically, agents tended to achieve greater collective rewards with these norms in our environment (Figure 7.3a). Unlike the scenario with no active norms, cumulative reward exhibited a consistent uptrend with time. Further evidence was provided by our sustainability metric: The percentage of desiccated fields (where apples do not regrow) increased rapidly in the absence of resource management norms, but remained at a low value when norms were present (Figure 7.3b).

7.4.3 Intergenerational Norm Transmission

When agents have limited lifespans, or only join a community for fixed amounts of time, the stability of that community’s norms requires the successful transmission of norms across generations [297–300]. Since our account of norm learning depends on there being enough agents who *practice* a norm for common knowledge of the norm to be maintained, we predicted that norms would be sustained as long there were enough experienced agents to learn from over a reasonable period of time [301–304]. For shorter lifespans, we predicted that social norms would be unstable, drifting from those originally practiced.

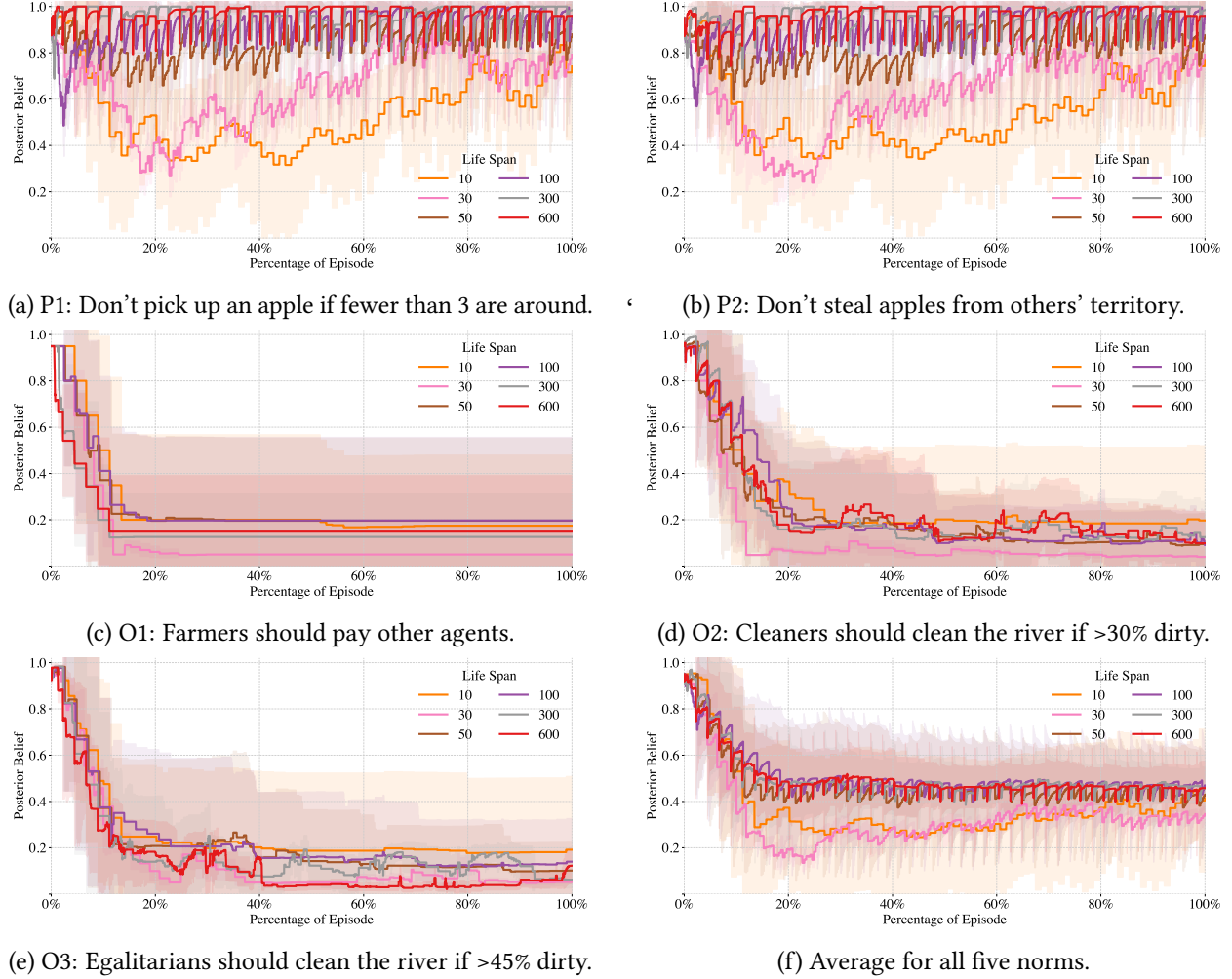


Figure 7.4: Inter-generational transmission or decay of norms. Mean norm belief averaged across all six agents across generations. (a)-(e) shows the mean belief in each of the norms described in Figure 7.1b. (f) is the average belief across the norms in (a)-(e).

To test these hypotheses, we ran an experiment with $M = 6$ learning agents, while assigning each of the three role-associated appearances $z \in Z$ to two out of the six agents (effectively forming a parent-child pair for each role). Agents used sampling to select which norms to comply with. Each agent i had a maximum life span of L , and an initial age of $\frac{1}{M}L$. Each run was simulated for $1.5ML$ steps (i.e. three generational cycles). Upon an agent i 's demise, it was replaced by an

equivalent “child” agent with the same role-associated appearance z_i , but with a low uniform prior over norms $P_i(v) = 1 - \theta$ for all $v \in \mathcal{N}$. Initially, 3 experienced agents were present to seed the norms. As in earlier experiments, all agents were intrinsically motivated to comply with the learned norms.

The results of this experiment are shown in Figure 7.4, revealing interesting patterns of norm transmission contingent upon the life spans of agents and the types of norms. Starting with the prohibition against eating too many apples (Figure 7.4a), we observed striking rapidity in both its acquisition and transmission. Indeed, a life span as brief as 50 steps was sufficient for the effective transmission of these prohibition norms, aligning with the findings of our passive learning experiments (Figure 7.2). The faster learning speed can also be attributed to a larger number of agents to learn from (five instead of three), all adhering to the same prohibition.

Obligation norms presented a more complex scenario (see e.g. Figure 7.4d). Due to the role-conditioned nature of the obligations we considered, learning opportunities were considerably more restricted compared to prohibitions. When learning the norms that applied to a particular role, an agent could only observe the two agents that were in that role, not all 6 agents. If they were the “child” agent in that role, they had only one agent to learn from (the “parent” in the same role). This resulted in a delayed learning curve for obligation norms, with agents requiring longer lifespans to learn and transmit them. Indeed, our results indicated that even an extended life span of 600 steps did not ensure that obligations were reliably transmitted.

Given the shorter learning time for obligations (300 steps) that we observed in Section 7.4.1, this inability to transmit obligations was unexpected. What we found instead was collective forgetting of all initially active obligations. Beyond the fact that obligations were less frequently observed, a possible factor was sampling: Since agents resampled the norms they would comply with every $K = 10$ steps, this added stochasticity that interfered with the long-horizon goals involved in obligation-oriented planning: If an obligation was not sampled at any step while fulfilling its goal, an agent would give up on the goal and go back to maximizing reward. This in turn created less of a learning signal for other agents.

Notwithstanding these unanticipated factors, our overall hypothesis about norm transmission was validated: Under *some* conditions, social norms are successfully maintained (Figure 7.4f), and this is more likely to occur with more opportunities to learn norms from agents who practice them. While more theoretical work is required, this suggests that in some regimes, norms are not only in a normative equilibrium but *evolutionarily stable* [305]. In contrast, when observations are scarce, norms might be abandoned [306], even by agents who used to comply with them.

7.4.4 Norm Emergence and Convergence

The emergence of norms is a complex phenomenon that has been studied both empirically [267, 277, 307, 308] and theoretically [300, 309, 310]. While prior work has studied norm emergence through adaption of policies in response to punishment [307], or learning to punish in accordance with public signals [267], our approach builds upon models of ad-hoc cooperation through joint intentionality [143], bootstrapping a shared set of norms in the same way that a team of cooperating agents can bootstrap a common goal without explicit communication [141]. Unlike the aforementioned mechanisms, this model of norm emergence does not require norm enforcement, as long as agents are sufficiently motivated (intrinsically or extrinsically) to comply with the norms that emerge.

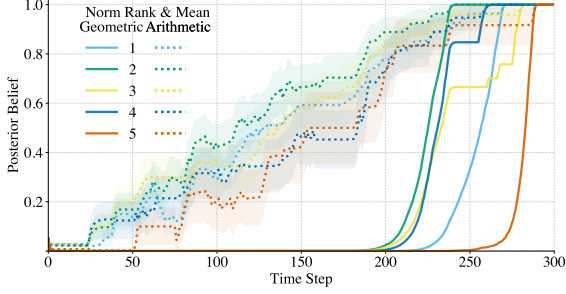


Figure 7.5: Norm emergence and convergence. Geometric and arithmetic mean of the posterior beliefs of all agents for the top 5 norms with final group belief $P_{\text{all}}^{t=300}(\nu) \geq \theta = 0.95$.

Norm	$\hat{p}_{\text{emerge}}$
1. Don't move if river is >55% dirty.	61%
2. Don't move if river is >60% dirty.	61%
3. Egalitarians should clean the river if it is >30% dirty.	61%
4. Don't move if looking East.	46%
5. Don't pick up an apple if no apples are around it.	38%
6. Don't move onto others' property.	31%
7. Don't steal apples from others' property.	31%
8. Don't pick up an apple if fewer than 2 are around it.	31%

Figure 7.6: Emergent norms (top 8) sorted by frequency of emergence $\hat{p}_{\text{emerge}}$ across 13 runs. A norm ν is emergent if it is in the top 5 norms for that run, or if the final group belief $P_{\text{all}}^{t=300}(\nu) \geq 0.95$.

To test whether such bootstrapping in fact occurs, we simulated a population of $M = 3$ learning agents with $z \in \{C, E, F\}$ (every role represented once). As in our norm transmission experiments, agents select norms to comply with via sampling, effectively performing a form of *norm exploration* when uncertain about which norms hold.

Figure 7.5 summarizes the results of these experiments and demonstrates the emergence of shared norms among our agents. The gradual increase in the arithmetic mean over multiple runs signifies an emergent belief in the existence of certain norms, even though such beliefs may not initially be shared by all agents. This eventually results in a sharp rise in the geometric mean, demonstrating convergence and unanimous agreement among agents on specific norms. Through repeated interaction and learning, our agents coalesce around a set of norms they collectively perceive as true, illustrating how pure observational learning combined with experimental norm compliance can eventually lead to a stable set of norms.

Examining the emergent norms more closely (Table 7.6), we found that many of the learned norms were non-functional, or “silly” norms [311]. This was likely due to intrinsic norm violation costs, which help to stabilize inefficient or even harmful norms [216]. As such, we expect that emergent norms are more likely to be beneficial when agents are more strategic about which norms they comply with. Nonetheless, recent work has shown how non-functional norms can play an epistemic role in stabilizing a normative system [311], and our model provides one mechanism for how such auxiliary norms might arise.

7.5 Discussion

How should we build autonomous systems that learn to comply with the norms of their social environments? In this chapter, we prototyped one such approach by instantiating the framework of *norm-augmented stochastic games* (NSGs) in a complex multi-agent setting. Tackling the challenges of norm specification, norm-compliant planning, and Bayesian norm learning, we designed agents that each pursue their own objectives while complying with the norms that they infer from collective behavior. In doing so, we show how agents can *rapidly acquire structured norms* by observing experienced agents, *sustain mutual belief* in these norms in a changing population, and *converge upon shared norms* even when none are initially present. These features not only capture important facets of human normativity, but may also prove crucial for future autonomous agents.

Nonetheless, many conceptual and technical questions about norm learning remain unexplored by our experiments, or are not explicitly accounted for by our modeling framework. We review these questions in the following discussion, and consider how our framework could be extended to address them.

The Role of Sanctions in Learning and Sustaining Norms Perhaps the most important difference between our experiments and previous work on decentralized norm learning [267, 277, 307] is the absence of *sanctions* — acts of punishment or approval — in motivating and sustaining norm compliance. Rather, our experiments focused on the *epistemic* aspect of norm learning, i.e., learning the *content* of normative rules, and ensuring they remain commonly known, leaving open the *motivational* aspect of norm learning. In part, this is because motivational learning is not as important for machines: We can often design them to *want* to comply with norms.

That said, nothing about the NSG formalism precludes the study of sanctions, which are important when we cannot control the motivations of other agents. Sanctioning could be introduced as a primitive action (e.g. the “zapping beams” supported by Melting Pot [262, 263]), a learned policy (e.g. taking others’ apples in retaliation for theft), or a (meta)-norm that obligates punishment of agents who do not comply with norms. Sophisticated agents might then learn or plan to use sanctions against norm violators. Would-be violators — e.g. agents with no intrinsic violation costs — might learn to expect punishment for norm violation, thereby choosing to comply. In fact, by separating the epistemic and motivational aspects of norm learning, NSGs may deliver a clearer account of the role of sanctions: Sanctions provide both *evidence* that a norm exists, and *motivation* to comply with it. By adjusting their strengths, we can investigate their contributions to norm learning and stability. We could also disambiguate intrinsic motivation accounts of punishment [312] from more strategic accounts, where agents punish to signal support for existing norms [219, 313, 314], influencing the normative beliefs of other agents in ways that they deem beneficial.

Model-Free vs. Model-Based Norm Learning Our paper takes a model-based approach to norm-oriented learning and planning, achieving orders of magnitude more sample efficiency than primarily model-free approaches [267, 272]. However, this comes at the cost of greater runtime complexity. To address these challenges, model-free and model-based approaches can be fruitfully combined. For example Bayesian learning can be accelerated via bottom-up proposals [315] and amortized inference [316, 317].

The question of model-free vs. model-based learning also relates to how norms are *represented*: Some norms — those that we tend to call *practices* or *rituals* [318] — are perhaps best represented as *habits* [319] or *skills*, which can be learned model-free [272]. Other norms are more naturally thought of as *abstract rules*, classifying behavior by its permissibility [226], while enabling public interpretability and generalization to new scenarios [293]. Our framework emphasizes the latter because we believe it is important for the conceptual richness of normative cognition: If norms lacked a symbolic, language-like structure, we would find it hard to compose, generalize, communicate, and argue about them [320].

While symbolic rules have become less popular in machine learning, recent advances in large language models (LLMs) suggest that they can be made considerably more scalable. As methods like Constitutional AI [321] suggest, a norm or principle ν could be represented in natural language itself, using an LLM to *classify* whether behavior complies with the norm [256]. Alternatively, LLMs might *translate* natural language, such as legal documents, into formal specifications [250,

322], enabling grounded and reliable reasoning in a particular domain [129, 174, 323] in the vein of CLIPS (Chapter 4). LLMs could thus serve as priors over norms $B^i(N)$ for a norm learner, or as approximate posteriors $B^i(N|u)$ over norms N given a piece of normative language u .

Symbolic norms do not eliminate the value of model-free learning, however: In general, learning how to *generate* norm-compliant behavior is harder than learning to *discriminate* it [324]. Oftentimes, it may be more efficient to learn habitual practices that comply with a norm, rather than planning them. Relatedly, the *motivation* to comply with norms might be learned via model-free mechanisms [281], even if norms are learned in a model-based manner.

Normative Reasoning and Norm Adaptation In our experiments, agents passively learn existing norms, then comply with them once learned. Yet humans are significantly more flexible. We do not just learn norms, but also decide when it makes sense to follow them [325, 326], and adjudicate when they are fairly applied [220]. This is because we can *reason* about the function of norms [327] and how they affect each of us [292], adapting them to novel circumstances [310].

Yet given the complexity of reasoning about ideal norms, it is costly to do this all the time. Instead, if we are to build normatively competent agents, we should perhaps take a leaf from recent accounts of how humans manage this complexity via resource-rational contractualist reasoning [282]: defaulting to learned rules in tried-and-true situations, adjusting them in new scenarios via processes of universalization [325, 326] or virtual bargaining [328, 329], and forging new norms via agreement when it is feasible and worthwhile. By shifting fluidly between these levels of normative cognition, we might eventually build systems that not only *comply* with the norms of our society, but help us deliberate upon and *improve* them [257].

7.6 Appendix

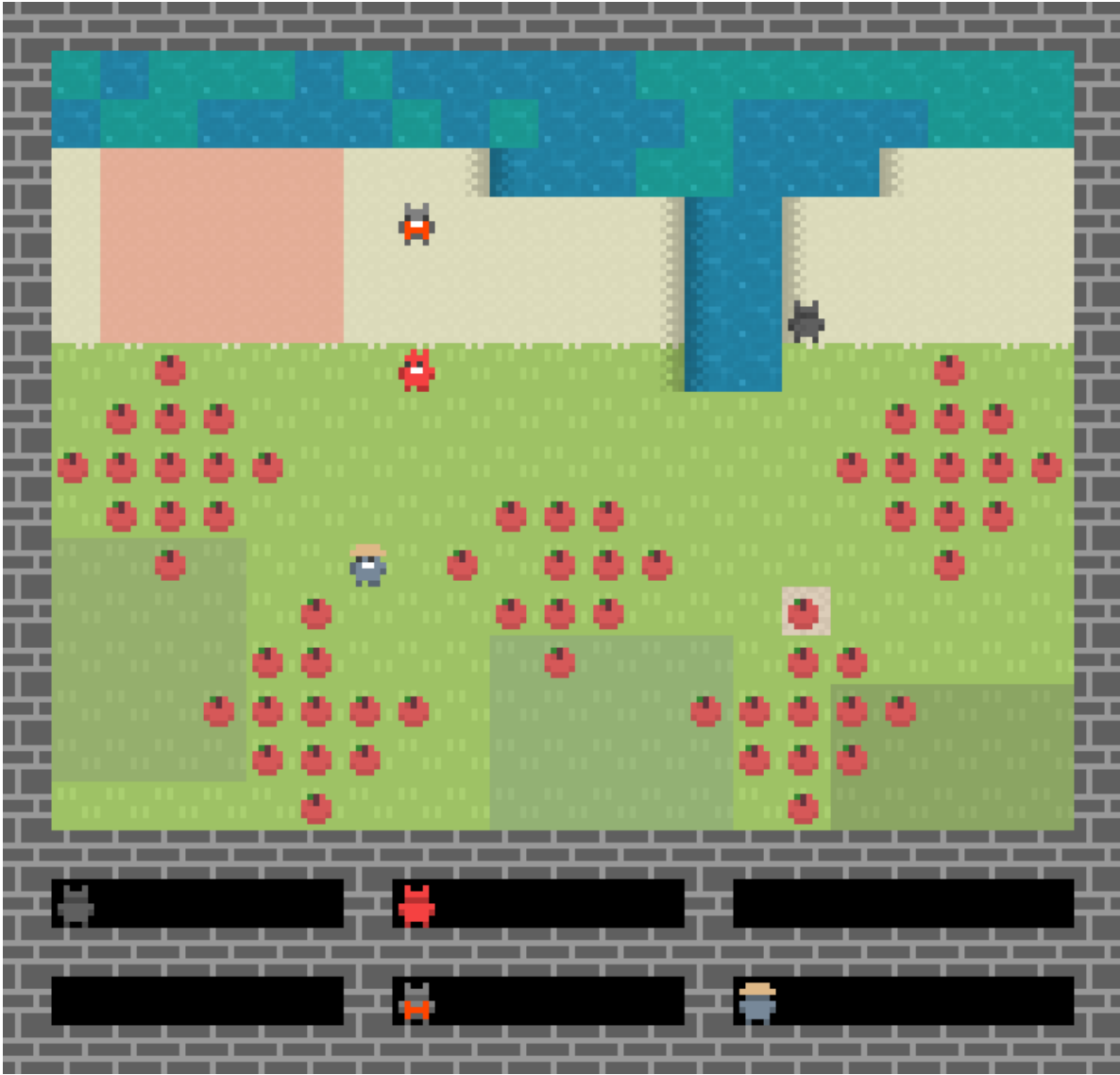


Figure 7.7: A multi-agent environment for norm learning. The three experienced agents are depicted in grey each representing a specific role based on their appearance. The learning agent wears no hat or vest, mimicking the “egalitarian” role based on its appearance. Each agent’s territory is visible as an agent-colored overlay. The river is located on the top of the frame with brighter parts representing the dirt. Apples are located in orchards that might, in turn, be in part on an agent’s territory. If a field is desiccated, it is unlikely to regrow apples, as signified by the sand-like color underneath.

Environment Parameters. The individual reward for eating an apple for every agent is 1 while the action cost is 0.01. Each prohibition norm violation results in a violation cost of 1 that is applied to the value function as described in Equation 7.3 while, as mentioned above, the reward for the fulfillment of an obligation is 1, as specified in Equation 7.4. Our specific environment’s prohibition and obligation norms are specified in Figure 7.1b.

Prohibitions (31)
If a cell holds an apple $\rightarrow$ Don't move there.
If a cell is not your property $\rightarrow$ Don't move there.
If a cell holds an apple and it's not your property $\rightarrow$ Don't move there.
If the river dirt fraction $> f \rightarrow$ Don't move at all. where $f \in \{0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60\}$
If you're looking towards direction $d \rightarrow$ Don't move at all. where $d \in \{\text{North, East, South, West}\}$
If a cell holds an apple and the number of apples around $< n \rightarrow$ Don't move there. where $n \in \{1, 2, 3, 4, 5, 6, 7, 8\}$
If a cell holds an apple and it's not your property and the number of apples around $< n \rightarrow$ Don't move there. where $n \in \{1, 2, 3, 4, 5, 6, 7, 8\}$
Obligations (37)
If the river dirt fraction $> f$ and you look like $z \rightarrow$ Clean the river. where $f \in \{0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60\}$, $z \in \{\text{Cleaner, Farmer, Egalitarian}\}$
If the number of steps you haven't paid anyone $> s$ and you look like $z \rightarrow$ Pay someone. where $s \in \{10, 15, 20, 25, 30\}$, $z \in \{\text{Cleaner, Farmer, Egalitarian}\}$
If another agent violated a norm $\rightarrow$ Sanction them.

Table 7.2: List of potential norms $\mathcal{N}$ considered by norm-learning agents.

Norm-Compliant Planning Parameters. We use real-time dynamic programming (RTDP) [53] for the norm-compliant planning algorithm with a rollout depth of 20 every 2 steps and a discount factor of $\gamma = 0.9$. Agents comply with norms at a belief threshold $\theta = 0.95$ (in Sections 7.4.1, 7.4.2), or when using Thompson sampling, with a sampling frequency of either $K = 10$ (in Section 7.4.3) or $K = 15$ (in Section 7.4.4). We used higher K in the latter case to allow for more time for bootstrap learning and hence emergence to occur.

Norm Learning Parameters. Every potential norm out of the 68 generated ones (see Tables 7.2) was given an initial marginal prior probability of $P(v) = 0.05$. We employed a Boltzmann temperature of 1. The space of potential norms $\mathcal{N}$ was enumerated by considering pre-conditions or post-conditions with up to three parameters, then eliminating infeasible or tautological norms. Norms are feasible when their pre/post-conditions are achievable.

Part IV

Probabilistic Programming Infrastructure

Chapter 8

Programmable Sequential Monte Carlo

We are convinced nevertheless that Monte Carlo methods will one day reach an impressive maturity.

J.M. Hammersley and D. C. Handscomb

Throughout most of this thesis, we have relied heavily on probabilistic programming infrastructure in order to achieve efficient Bayesian inference over complex probabilistic models. In particular, both SIPS (Chapter 1) and CLIPS (Chapter 4) make use of sequential Monte Carlo (SMC) sampling [120, 122, 330, 331], also known as particle filtering, along with its deterministic limit (i.e. exact Bayesian filtering) to perform rapid inference over the goals and plans of other agents. Achieving this efficiency required an iterative process of model-inference co-design, with changes to the generative model necessitating changes to the inference algorithm to preserve accuracy and speed. Without the right tooling, however, this would have resulted in significant overheads in research and engineering time: Each modification to the generative model or inference algorithm changes the probabilistic quantities that an SMC method has to compute, which — if not automated — has to be carefully re-derived and re-implemented.

As a solution to this challenge, this chapter introduces a library for **programmable sequential Monte Carlo** inference called *GenParticleFilters.jl* [20], which builds on the Gen.jl probabilistic programming framework [4]. This library provides a wide variety of building blocks that developers can compose into full SMC samplers, including subroutines for particle initialization, updating, resampling, rejuvenation, and resizing. This makes *GenParticleFilters.jl* one of the most feature-complete probabilistic programming libraries for SMC to date, as exemplified by its support for Sequential Monte Carlo with Probabilistic Program Proposals (SMCP³), an algorithm class that subsumes most existing frameworks for SMC [122]. By automating the low-level math required to ensure the soundness of SMC, *GenParticleFilters.jl* enables rapid prototyping of SMC algorithms that are tailored to specific model classes, enabling applications including SIPS, active learning of Gaussian process kernel structure [332] and online synthesis of probabilistic programs [333].

In the following sections, we review SMC as an algorithm class and the correctness properties it maintains. We then introduce the building blocks provided by *GenParticleFilters.jl*, specifying their semantics in terms of how they transform weighted particle collections while preserving the correctness of SMC. Along the way, we describe use cases for each building block we provide.

8.1 Background

In sequential Monte Carlo, we are interested in approximating a sequence of target probability distributions $(P_t)_{t \in [1, T]}$, each defined over a space X_t . Typically, we can only evaluate each P_t up to a constant. That is, we have a sequence of *unnormalized* target distributions $(\tilde{P}_t)_{t \in [1, T]}$ for which we can evaluate $\tilde{P}_t(x_t)$ given some $x_t \in X_t$. Given the normalizing constant $Z_t = \int_X \tilde{P}(dx)$, we could compute the normalized target as $P_t = \tilde{P}_t/Z_t$. However, Z_t is generally unknown and intractable to compute exactly. In posterior inference, for example, the unnormalized target $\tilde{P}_t(x_t)$ corresponds to the joint probability $p(x_t, y_{1:t})$ of latents x_t and observations $y_{1:t}$ under some generative model $p(x_t, y_{1:t}) = p(x_t)p(y_{1:t}|x_t)$, and we are interested in approximating the posterior $P_t(x_t) = p(x_t|y_{1:t})$ for all $t \in [1, T]$. This requires approximating $Z_t = p(y_{1:t})$.

To solve such problems, SMC evolves a collection of weighted samples $\{(x_t^i, w_t^i)\}_{i=1}^{N_t}$, also known as *particles*, through a sequence of stochastic transformations, such that the particle collection at step t is an approximation of the target P_t . Each of these particle collections is required to maintain a property called *proper weighting*:

Definition 8.1 (Proper Weighting). A particle (x, w) is *properly weighted* with respect to an unnormalized target $\tilde{P}$ if $\mathbb{E}[wf(x)] = \mathbb{E}_{x \sim P}[Z \cdot f(x)]$ for any integrable function $f(x)$, where $Z = \int_X \tilde{P}(dx)$ is the normalizing constant. Similarly, a particle collection $\{(x^i, w^i)\}_{i=1}^N$ is properly weighted w.r.t. $\tilde{P}$ if $\mathbb{E}[\frac{1}{N} \sum_{i=1}^N w_i f(x_i)] = \mathbb{E}_{x \sim P}[Z \cdot f(x)]$.

Here, (x, w) are random variables distributed according to some stochastic process (e.g. the process of running SMC). One useful feature of proper weighting is that we can unbiasedly estimate the normalizing constant Z by computing the average weight $\frac{1}{N} \sum_{i=1}^N w^i$ (i.e. by setting $f(x) = 1$):

$$\mathbb{E}[\frac{1}{N} \sum_{i=1}^N w^i] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[w^i] = \mathbb{E}_{x \sim P}[Z \cdot 1] = Z \quad (8.1)$$

Since we have the ability to estimate Z , we can use this to compute a *self-normalized* estimate of the expectation of an integrable test function $f(x)$:

Definition 8.2 (Self-Normalized Estimator). Given a particle collection $\{(x^i, w^i)\}_{i=1}^N$ properly weighted for $\tilde{P}$, the *self-normalized estimator* for $f(x)$ is $\hat{f}_{\text{SNE}}^N := \sum_{i=1}^N w^i f(x^i) / \sum_{i=1}^N w^i$.

For example, when x is discrete, we can estimate the probability $P(x = v)$ under the target by choosing $f(x) = \mathbb{1}_v(x)$, computing the self-normalized estimate $\sum_{x^i=v} w^i / \sum_i w^i$. As such, by generating properly weighted samples, SMC allows to estimate both posterior expectations and posterior probabilities.

Although the self-normalized estimator divides out an unbiased estimate of Z , the estimator itself is biased since division does not preserve unbiasedness. However, most SMC algorithms also produce *consistent* estimators:

Definition 8.3 (Consistency). Given a scheme for generating particle collections $\{(x^i, w^i)\}_{i=1}^N$ parametric in N , the self-normalized estimator $\hat{f}_{\text{SNE}}^N$ is (strongly) *consistent* if

$$\Pr \left(\lim_{N \rightarrow \infty} \hat{f}_{\text{SNE}}^N = \mathbb{E}_{x \sim P(x)}[f(x)] \right) = 1.$$

To generate particle collections that are properly weighted with respect to the unnormalized targets $(\tilde{P}_t)_{t \in [1, T]}$, SMC first *initializes* a set of particles targeting $\tilde{P}_1$ (Section 8.2). SMC then applies a sequence of particle *updates* which transform and reweight particles targeting $\tilde{P}_{t-1}$ into particles targeting $\tilde{P}_t$ (Section 8.3). Between updates, an SMC algorithm may also *rejuvenate* particles, diversifying their values and weights while keeping the target fixed at $\tilde{P}_t$ (Section 8.4). To focus computational effort on particles with higher weights, SMC may also *resample* existing particles (Section 8.5). Finally, to control the total amount of computational effort spent on inference, SMC may *resize* the number of particles (Section 8.6).

8.2 Particle Initialization

To initialize a particle filter, SMC uses *importance sampling*. In its most standard form, importance sampling requires a proposal distribution Q defined over the sample space X that one can sample from, evaluate exactly at any $x \in X$, and which has full support¹ with respect to the (unnormalized) target $\tilde{P}$. With this proposal, we can generate a properly weighted sample by sampling $x \sim Q(\cdot)$, and computing the *importance weight* $w = \tilde{P}(x)/Q(x)$, since:

$$\mathbb{E}_{x \sim Q}[wf(x)] = \mathbb{E}_{x \sim Q}\left[\frac{\tilde{P}(x)}{Q(x)}f(x)\right] = \mathbb{E}_{x \sim \tilde{P}}[Zf(x)]$$

However, there are more general particle initialization strategies supported by *GenParticleFilters.jl*. As explained in Lew et al. [136], the proposal $Q(x)$ need not have a tractable density, and only needs to return an unbiased estimate of $1/Q(x)$ along with a sample x . A proposal also need not sample each initial x in the particle filter independently. Instead, it is possible to use a *joint* proposal distribution over all $x^{1:N}$ samples, as in *multiple importance sampling* [334].

Definition 8.4 (Particle Initialization). Given an initial target $\tilde{P}_1$ defined over X_1 and sample size N , a *particle initialization* strategy generates a particle collection $\{(x^i, w^i)\}_{i=1}^N$ that is properly weighted for $\tilde{P}_1$, using a (joint) proposal distribution Q over X_1^N .

When using this more general strategy, proper weighting can be achieved as follows:

Proposition 8.1. Let $\bar{Q}(x)$ be the marginal probability of generating x by sampling $x^{1:N} \sim Q$ from the joint proposal, sampling an index $i \sim \text{Uniform}(1, N)$, and setting $x = x^i$. Suppose $\bar{Q}$ has full support w.r.t. to target $\tilde{P}$. Then the following particle collection is properly weighted w.r.t. $\tilde{P}$:

$$\{(x^i, w^i)\}_{i=1}^N \quad \text{where} \quad x^{1:N} \sim Q, \quad w^i = \frac{\tilde{P}(x^i)}{\bar{Q}(x^i)}$$

Proof. Let $\bar{Q}^N$ be the exchangeable distribution defined by sampling $x^{1:N} \sim Q$ and applying a random permutation of the indices. Expectations of sums are invariant under (random) permutations. As such, we have:

$$\begin{aligned} \mathbb{E}_{x^{1:N} \sim Q}\left[\frac{1}{N} \sum_{i=1}^N w^i f(x^i)\right] &= \mathbb{E}_{x^{1:N} \sim \bar{Q}^N}\left[\frac{1}{N} \sum_{i=1}^N w^i f(x^i)\right] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{x^i \sim \bar{Q}}[w^i f(x^i)] \\ &= \mathbb{E}_{x \sim \bar{Q}}\left[\frac{\tilde{P}(x)}{\bar{Q}(x)}f(x)\right] = \mathbb{E}_{x \sim \tilde{P}}[Zf(x)]. \end{aligned}$$

□

¹A proposal Q has full support with respect to a target $\tilde{P}$ if $Q(x) > 0$ wherever $\tilde{P}(x) > 0$.

The standard joint proposal is the one that factorizes independently $Q(x^{1:N}) = \prod_{i=1}^N Q(x^i)$. However, this can lead to unwanted variance and unnecessary repetitions, especially in the case where the sample space X_1 is discrete, or when important regions of sample space are low in probability. To allow users more control over the variance of their SMC algorithms, *GenParticleFilters.jl* also provides support for *stratified initialization*, which proposes $x^{1:N}$ equally distributed² across a set of M strata $\{S_i\}_{i=1}^M$ that partition the initial target's sample space X_1 (i.e. $\cup_{i=1}^M S_i = X_1$ and $S_i \cap S_j = \emptyset$ for all i, j). The corresponding marginal proposal $\bar{Q}$ is equivalent to uniformly sampling the stratum index $m \sim \text{Uniform}(1, M)$, then sampling from a proposal $x \sim Q(\cdot|S_m)$ conditioned on the selected stratum S_m . The resulting marginal probability is $\bar{Q}(x) = \frac{1}{M}Q(x|S_m)$, which we use as the denominator of the importance weight w . Both independent and stratified particle initialization are supported by the method `pf_initialize`:

```
pf_initialize(
    model::GenerativeFunction, model_args::Tuple, observations::ChoiceMap,
    [proposal::GenerativeFunction, proposal_args::Tuple], n_particles::Int
)
```

Initialize a particle filter, generating traces (i.e. particles) from the model constrained to the provided observations. A custom proposal can optionally be used to propose choices for the initial set of traces. Returns a `ParticleFilterState`.

```
pf_initialize(
    model::GenerativeFunction, model_args::Tuple, observations::ChoiceMap, strata,
    [proposal::GenerativeFunction, proposal_args::Tuple], n_particles::Int
)
```

Perform *stratified* initialization of a particle filter, given a set of strata specified as an iterator over choicemaps. Each generated trace is constrained to both the provided observations and the choicemap for the stratum it belongs to. A custom proposal can optionally be used to propose unconstrained choices. For N particles and M strata, each stratum is first allocated $B = \lfloor N / M \rfloor$ particles. The remaining R particles are distributed at random among the strata.

Figure 8.1: Documentation for `pf_initialize`.

Stratified initialization is used in SIPS (Chapter 1) to ensure that an equal number of particles is allocated to each possible goal. With `pf_initialize`, this can be done by providing a list of choicemaps (dictionaries mapping random variables in a generative function to values), each constraining the goal variable to a different value. The remaining latent variables are proposed conditioned on the values in the choicemap for a stratum, exploiting the fact that generative functions in *Gen* implement a family of choicemap-dependent proposals [335]. In the case where there are no other latent variables in the generative model besides those that are stratified over, we end up with a particle collection that *exactly* represents the target posterior of interest.

²Stratification with unequal weighting of strata is also possible, but not currently supported.

8.3 Particle Updating

Once we have a particle collection approximating a target $\tilde{P}_{t-1}$ in our sequence $(\tilde{P}_t)_{t \in [1, T]}$, we can transform this into a collection targeting $\tilde{P}_t$ by *updating* each particle (x, w) separately. This operation is also known as *mutation*, *propagation*, or *extension* (in the case where the sample space grows $X_{t-1} \subset X_t$).

Definition 8.5 (Particle Updating). Given a particle (x, w) properly weighted w.r.t. $\tilde{P}_{t-1}$, a *particle update* from $\tilde{P}_{t-1}$ to $\tilde{P}_t$ returns a new particle $(\tilde{x}, w \cdot v)$ properly weighted w.r.t. $\tilde{P}_t$, where v is known as the *incremental (importance) weight*.

There are a wide variety of particle updates that maintain proper weighting. For example, when $\tilde{P}_{t-1}$ and $\tilde{P}_t$ have the same sample space X (as in annealed importance sampling [336]), one can simply keep the particle's value fixed at $\tilde{x} = x$, computing the incremental weight as $v = \tilde{P}_t(x) / \tilde{P}_{t-1}(x)$ to reflect the change in target. In a state space model $p(z_{1:T}, y_{1:T}) = \prod_t P(z_t | z_{t-1}) P(y_t | z_t)$, where one wishes to infer a filtering posterior $p(z_{1:t} | y_{1:t})$ for each t , a common choice is the *bootstrap update*: Given $x := z_{1:t-1}$, we sample $z_t \sim P(z_t | z_{t-1})$ from the prior dynamics to get the new particle value $\tilde{x} := z_{1:t}$, and set the incremental weight as $v = P(y_t | z_t)$. As it turns out, these are just special cases of a general class called *SMCP³ updates* [122]:

Definition 8.6 (SMCP³ update). A *SMCP³ update* from $\tilde{P}_{t-1}$ to $\tilde{P}_t$ is defined by auxiliary spaces U_K and U_L , a forward probability kernel $Q_K : X_{t-1} \rightarrow U_K$, a backward probability kernel $Q_L : X_t \rightarrow U_L$, and a (forward) deterministic transform $f_K : X_{t-1} \times U_K \rightarrow X_t \times U_L$. To apply the update to a particle (x, w) that is properly weighted for $\tilde{P}_{t-1}$, we:

1. Sample auxiliary randomness $u_K \sim Q_K(\cdot; x)$ from the forward kernel;
2. Transform the previous sample and forward auxiliary randomness (x, u_K) into an updated sample and backward auxiliary randomness $(\tilde{x}, u_L) = f_K(x, u_K)$;
3. Compute the incremental weight as

$$v := \frac{\tilde{P}_t(\tilde{x})}{\tilde{P}_{t-1}(x)} \frac{Q_L(u_L; \tilde{x})}{Q_K(u_K; x)} \left| \frac{\partial f_K^{\text{cont.}}}{\partial(x, u_K)} \right|,$$

where $\left| \frac{\partial f_K^{\text{cont.}}}{\partial(x, u_K)} \right|$ is the determinant of the Jacobian matrix of the continuous part of f_K ³;

4. Return the updated particle $(\tilde{x}, w \cdot v)$.

For the update to be valid, the following requirements must be satisfied:

- *Invertibility*: There exists a *backward* deterministic transform $f_L : X_t \times U_L \rightarrow X_{t-1} \times U_K$ s.t. for any (x, u_K) where $\tilde{P}_{t-1}(x) Q_K(u_K; x) > 0$, $f_L(f_K(x, u_K)) = (x, u_K)$.
- *Full Support*: For any $(\tilde{x}, u_L)$ where $\tilde{P}_t(\tilde{x}) Q_L(u_L; \tilde{x}) > 0$, there must exist $(x, u_K) = f_L(\tilde{x}, u_L)$ where $\tilde{P}_{t-1}(x) Q_K(u_K; x) > 0$.

Theorem 8.1. Given an SMCP³ update $(U_K, U_L, Q_K, Q_L, f_K)$ from $\tilde{P}_{t-1}$ to $\tilde{P}_t$ and a particle (x, w) properly weighted for $\tilde{P}_{t-1}$, the particle $(\tilde{x}, w \cdot v)$ that results from applying the update is properly weighted for $\tilde{P}_t$.

³More generally, this term is a Radon-Nikodym derivative, see [194].

```

translator = UpdatingTraceTranslator(
    p_new_args::Tuple = (),
    p_argdiffs::Tuple = (),
    new_observations::ChoiceMap = EmptyChoiceMap(),
    q_forward::GenerativeFunction,
    q_forward_args::Tuple = (),
    q_backward::GenerativeFunction,
    q_backward_args::Tuple = (),
    transform::Union{TraceTransformDSLProgram,Nothing} = nothing
)

```

Constructor for a updating trace translator, which updates the trace of model given a forward proposal `q_forward`, backward proposal `q_backward`, new arguments for that model `p_new_args`, and new observations. Optionally, a trace transform can be provided, specifying how the input model trace and forward proposal trace get mapped to the output model trace and backward proposal trace.

```
(output_trace, log_weight) = translator(input_trace; kwargs...)
```

Run an `UpdatingTraceTranslator` on an input trace, returning the output trace and the (incremental) importance weight.

Figure 8.2: Documentation for `UpdatingTraceTranslator`.

We give here only a proof sketch for Theorem 8.1 in the case where all spaces X_{t-1}, X_t, U_K, U_L are discrete (s.t. $|\partial f_K^{\text{cont.}} / \partial(x, u_K)| = 1$), leaving the full proof to Lew et al. [122]. Let $f(\tilde{x})$ be a function over X_t . We want to show that $\mathbb{E}[(w \cdot v)f(\tilde{x})] = \mathbb{E}_{\tilde{x} \sim P_t}[Z_t f(\tilde{x})]$. Noting that $\mathbb{E}_{u_K \sim Q_K(\cdot; x)}[vf(\tilde{x})]$ is also a function of x , we apply the fact that (x, w) is properly weighted for $\tilde{P}_{t-1}$:

$$\begin{aligned}
\mathbb{E}_{(x, w, u_K)}[(w \cdot v)f(\tilde{x})] &= \mathbb{E}_{(x, w)}[w \cdot \mathbb{E}_{u_K \sim Q_K(\cdot; x)}[vf(\tilde{x})]] \\
&= \mathbb{E}_{x \sim P_{t-1}}[Z_{t-1} \mathbb{E}_{u_K \sim Q_K(\cdot; x)}[vf(\tilde{x})]] = Z_{t-1} \mathbb{E}_{x \sim P_{t-1}}[vf(\tilde{x})]
\end{aligned}$$

Expanding the-right hand side, we have:

$$\begin{aligned}
Z_{t-1} \mathbb{E}_{x \sim P_{t-1}}[vf(\tilde{x})] &= Z_{t-1} \mathbb{E}_{x \sim P_{t-1}} \left[\frac{\tilde{P}_t(\tilde{x})}{\tilde{P}_{t-1}(x)} \frac{Q_L(u_L; \tilde{x})}{Q_K(u_K; x)} f(\tilde{x}) \right] \\
&= Z_{t-1} \sum_{x \in X_{t-1}} \sum_{u_K \in U_K} P_{t-1}(x) Q_K(u_K; x) \frac{\tilde{P}_t(\tilde{x})}{\tilde{P}_{t-1}(x)} \frac{Q_L(u_L; \tilde{x})}{Q_K(u_K; x)} f(\tilde{x}) \\
&= \sum_{x \in X_{t-1}} \frac{Z_{t-1} P_{t-1}(x)}{\tilde{P}_{t-1}(x)} \sum_{u_K \in U_K} \tilde{P}_t(\tilde{x}) Q_L(u_L; \tilde{x}) f(\tilde{x}) \quad \text{where } (\tilde{x}, u_L) = f_K(x, u_K) \\
&= \sum_{\tilde{x} \in X_t} \sum_{u_L \in U_L} \tilde{P}_t(\tilde{x}) Q_L(u_L; \tilde{x}) f(\tilde{x}) \quad (\text{by full support + invertibility}) \\
&= \sum_{\tilde{x} \in X_t} \tilde{P}_t(\tilde{x}) f(\tilde{x}) = \mathbb{E}_{\tilde{x} \sim P_t}[Z_t f(\tilde{x})]
\end{aligned}$$

We have as desired that $\mathbb{E}[(w \cdot v)f(\tilde{x})] = \mathbb{E}_{\tilde{x} \sim P_t}[Z_t f(\tilde{x})]$.

```
pf_update!(state::ParticleFilterState, new_args::Tuple,
           argdiffs::Tuple, observations::ChoiceMap,
           fwd_proposal::GenerativeFunction, fwd_args::Tuple,
           bwd_proposal::GenerativeFunction, bwd_args::Tuple,
           [transform::TraceTransformDSLProgram])
```

Perform a particle filter update, with a custom forward and backward kernel. New latent choices are sampled from `fwd_proposal`, and any discarded choices are evaluated under `bwd_proposal`. For each particle:

- `fwd_proposal` is evaluated with arguments $(t_old, fwd_args \dots)$, where `t_old` is the old model trace, and produces its own trace `t_fwd`.
- The old model trace is replaced by a new model trace `t_new` by merging the choices in `t_old` and `t_fwd`, sampling any remaining choices from the model's default proposal, and discarding `disc_choices`, the choices in `t_old` inconsistent with those in `t_fwd`.
- The probability of `disc_choices` is assessed under `bwd_proposal` with the arguments (t_new, bwd_args) , giving a backward weight. This weight is used as a correction within the importance weight update.

The choicemap of `t_new` and `disc_choices` satisfy the following conditions:

1. `observations` is a subset of `get_choices(t_new)`;
2. `get_choices(t_old) \ disc_choices` is a subset of `get_choices(t_new)`;
3. `get_choices(t_fwd)` is a subset of `get_choices(t_new)`.
4. `disc_choices` is within the support of `bwd_proposal`

If a trace transform is also provided, then more general forward and backward kernels can be used: `t_new` (the model's new trace) and `t_bwd` (the trace for the backward kernel) are constructed as a function of `t_old`, `t_fwd`, and any new observations.

Figure 8.3: Documentation for the most general form of `pf_update!`.

GenParticleFilters.jl enables users to specify SMCP³ updates via constructs called *updating trace translators* (Figure 8.2), an extension of the notion of trace translators introduced by Cusumano-Towner et al. [331]. When an updating trace translator is called on a trace x from a generative function, it returns the new trace $\tilde{x}$ and the relative weight v . Users can also pass all the components for an SMCP³ update directly to `pf_update!` (Figure 8.3). Internally, this constructs an updating trace translator, and applies the update to an entire particle collection $\{(x^i, w^i)\}_{i=1}^N$.

SMCP³ updates are highly flexible, allowing the use of auxiliary randomness and deterministic transforms when designing updates. For example, instead of the bootstrap update in state-space models, users can implement *nested importance sampling* [337] as an SMCP³ update. This involves generating M extensions $\{z_{1:t}^i\}_{i=1}^M$ of the previous sample $x = z_{1:t-1}$ (forming the auxiliary randomness u_K sampled by Q_K), selecting one extension $z_{1:t}^i$ to retain according to its normalized importance weights, and setting the new sample to that extension $\tilde{x} = z_{1:t}^i$ (corresponding to the first component of the transform f_K). By increasing the number of nested importance samples M , this update gets arbitrarily close to sampling from the local posterior over z_t . Simpler updates are also possible. If we do not use auxiliary randomness (as in [120]), we can set $U_K = X_t$, $U_L = X_{t-1}$, and use the identity transform $(\tilde{x}, u_L) = f_K(x, u_K) := (u_K, x)$. We refer readers to Lew et al. [122] for more design guidance on SMCP³, including guidance on the optimal backward kernel Q_L .

```
pf_rejuvenate!(state::ParticleFilterState, kern, kern_args=(), n_iters=1;
               method=:move, kwargs...)
```

Rejuvenates particles by repeated application of a kernel `kern`. `kern` should be a callable which takes a trace as its first argument, and returns a tuple with a trace as the first return value. `method` specifies the rejuvenation method: `:move` for MCMC moves without reweighting, and `:reweight` for rejuvenation with reweighting.

```
pf_move_accept!(state::ParticleFilterState, kern, kern_args=(), n_iters=1; kwargs...)
```

Rejuvenates particles by repeated application of a MCMC kernel `kern`. `kern` should be a callable which takes a trace as its first argument, and returns a tuple `(trace, accept)`, where `trace` is the (potentially) new trace, and `accept` is true if the MCMC move was accepted. The kernel is repeatedly applied to each trace for `n_iters`.

```
pf_move_reweight!(state::ParticleFilterState, kern, [kern_args=(), n_iters=1])
```

Rejuvenates particles by repeated application of a reweighting kernel `kern`. `kern` should be a callable which takes a trace as its first argument, and returns a tuple `(trace, rel_weight)`, where `trace` is the new trace, and `rel_weight` is the relative log-importance weight. The kernel is repeatedly applied to each trace for `n_iters`, and the weights accumulated.

Figure 8.4: Documentation for `pf_rejuvenate!` and its variants.

8.4 Particle Rejuvenation

In particle updating, we were interested in moving a particle collection from one target to the next. However, there are many cases where we may want to modify a particle collection while keeping the target *fixed*. For example, after resampling (Section 8.5), the diversity of particle values x is generally reduced, and we may want to increase their diversity. When approximating a filtering posterior $p(z_{1:t}|y_{1:t})$, we may also want to update older values z_τ where $\tau < t$, correcting for the fact that they were sampled by a process that only saw earlier observations $y_{1:\tau}$ and not subsequent ones $y_{\tau+1:t}$. Both of these can be achieved via *particle rejuvenation* moves:

Definition 8.7 (Particle Rejuvenation). Given a particle (x, w) properly weighted w.r.t. $\tilde{P}_t$, *particle rejuvenation* returns a new particle $(\tilde{x}, w \cdot v)$ properly weighted w.r.t. to the same target $\tilde{P}_t$.

Particle rejuvenation is a special case of particle updating (when $\tilde{P}_{t-1} = \tilde{P}_t$), but certain kinds of rejuvenation moves can be implemented more efficiently than the general SMCP³ update in Section 8.3. In particular, when we choose both the forward and backward proposals Q_K, Q_L to be a Markov chain Monte Carlo (MCMC) kernel $K(\cdot; x)$ that is stationary with respect to the (normalized) target P_t , the resulting incremental weight is always $v = 1$, and does not have to be explicitly computed. This follows directly from stationarity:

$$v = \frac{\tilde{P}_t(\tilde{x}) K(x; \tilde{x})}{\tilde{P}_t(x) K(x; \tilde{x})} = \frac{P_t(\tilde{x}) K(x; \tilde{x})}{P_t(x) K(x; \tilde{x})} = 1$$

To exploit the fact that MCMC rejuvenation does not require weight computation, *GenParticleFilters.jl* supports multiple kinds of rejuvenation moves (Figure 8.4). In *move-accept* rejuvenation (`pf_move_accept!`), users provide an MCMC kernel (e.g. a Metropolis-Hasting kernel) that returns a new trace given an old one. This design allows users to draw on the library of MCMC kernels provided by Gen.jl [4, 335], and (in some cases) avoid the overhead of weight computation. In *move-reweight* rejuvenation [338] (`pf_move_reweight!`), users instead provide a reweighting kernel that returns a new trace and a relative weight v . Reweighting kernels have the same I/O specification as trace translators, but can be implemented more efficiently in special cases.

8.5 Particle Resampling

A phenomenon that often occurs in SMC is *weight degeneracy*, where one or a few particles end up having much higher weights than the others after multiple rounds of updating. This is not necessarily undesirable, since the target P_t could in fact place a lot of posterior mass on specific values $x \in X_t$, and we may want the empirical distribution represented by our particle collection to have low divergence w.r.t. P_t (as in [339]).

However, weight degeneracy can hurt the quality of the SMC approximation for *future* targets $(P_{t+1}, P_{t+2}, \dots)$, because if a high probability sample x is represented only once, then it can only have one descendant $\tilde{x}$ after a particle update. Instead, we would like a way for high-weight particles to have *multiple* descendants in subsequent particle collections, allowing the high-probability regions of future targets to be better explored. This can be achieved through *particle resampling*:

Definition 8.8 (Particle Resampling). Given a particle collection $\{(x^i, w^i)\}_{i=1}^N$ properly weighted w.r.t. $\tilde{P}_t$, *particle resampling* returns a new collection of N particles $\{(\tilde{x}^j, \tilde{w}^j)\}_{j=1}^N$ which is properly weighted w.r.t. to the same target $\tilde{P}_t$, where each new particle $(\tilde{x}^j, \tilde{w}^j)$ is descended from a parent particle with index a_j s.t. $\tilde{x}^j = x^{a_j}$.

The simplest form of resampling is *multinomial resampling*, which selects each particle (x, w) for inclusion in the new collection with probability equal to its normalized weight $\bar{w} = w / \sum_i w^i$, and sets the new weight $\tilde{w} = \sum_i w^i / N$ for all resulting particles. On average, the resulting collection will contain multiple copies of particles that were highly weighted, allowing SMC to focus computational effort on tracking their descendants.

When N is small, however, multinomial sampling can be high in variance, since there is always a chance of discarding even the most highly weighted particles. Instead we can use resampling schemes which allocate copies of a particle more deterministically, reducing variance [340]:

- In *residual resampling*, a particle with a normalized weight of $\bar{w}$ is ensured at least $\lfloor N\bar{w} \rfloor$ copies in the new collection.
- In *stratified resampling*, the interval $[0, 1]$ is partitioned into N strata, and a uniform sample u_i is drawn in each sub-interval. If u_i lands in $[W^{j-1}, W^j)$ where $W^j = \sum_{k \leq j} \bar{w}^k$, sample x^j is included in the new collection.
- In *block resampling* [341, 342], the particle collection is partitioned into a set of blocks, and resampling is performed only within each block (via one of the strategies above).

Block resampling is useful when we want to ensure there are always particles associated with certain subsets $S \subseteq X$ of the sample space. For example, SIPS (Chapter 1) uses block resampling on a per-goal basis, ensuring that there are always particles tracking each possible goal.

```
pf_resample!(state::ParticleFilterState, method=:multinomial; kwargs...)
```

Resamples particles in the filter, stochastically pruning low-weight particles. The resampling method can optionally be specified: `:multinomial` (default), `:residual`, or `:stratified`. To resample within a block of particles with indices `idxs`, call this method on `state[idxs]`.

Keyword Arguments

- `priority_fn = nothing`: An optional function that maps (log) particle weights to custom log priority scores (e.g. $w \rightarrow w/2$ for less aggressive pruning).

```
pf_multinomial_resample!(state::ParticleFilterState; kwargs...)
```

Performs multinomial resampling (i.e. simple random resampling) of the particles in the filter. Each trace (i.e. particle) is resampled with probability proportional to its weight.

```
pf_residual_resample!(state::ParticleFilterState; kwargs...)
```

Performs residual resampling of the particles in the filter, which reduces variance relative to multinomial sampling. For each particle with normalized weight w_i , $\lfloor Nw_i \rfloor$ copies are resampled, where N is the total number of particles. The remainder are sampled with probability proportional to $Nw_i - \lfloor Nw_i \rfloor$ for each particle i .

```
pf_stratified_resample!(state::ParticleFilterState; kwargs...)
```

Performs stratified resampling of the particles in the filter, which reduces variance relative to multinomial sampling. First, uniform random samples $u_1, \dots, u_n$ are drawn within the strata $[0, 1/n), \dots, [n-1/n, 1)$, where n is the number of particles. Then, given the cumulative normalized weights $W_k = \sum_{j=1}^k w_j$, sample the k th particle for each u_i where $W_{k-1} \leq u_i < W_k$.

Figure 8.5: Documentation for `pf_resample!` and its variants.

All of the resampling schemes described above are supported by *GenParticleFilters.jl* (Figure 8.5). In addition, users can specify *priority score* via the `priority_fn` keyword argument to control the probability that each particle is resampled [343]. A priority function $\alpha : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ maps the weight w of a particle to a priority score $\alpha(w)$, which is used in place of w by each resampling scheme. For example, in multinomial resampling, particles are resampled with probability $\alpha(w)/\sum_i \alpha(w^i)$ instead. By choosing a priority function such as $\alpha(w) = w^{1/2}$, resampling can be made to prune low-weight particles less aggressively.

Given the many resampling schemes covered here, how is proper weighting ensured? We can derive the proper weights by viewing resampling as a process that generates a random number of copies n^i for each sample x^i s.t. $\sum_i n^i = N$. Proper weighting is then ensured as follows:

Proposition 8.2. Let $R(n^{1:N}; \{(x^i, w^i)\}_{i=1}^N)$ be the joint distribution over the number of copies n^i of each x^i induced by resampling a collection $\{(x^i, w^i)\}_{i=1}^N$ properly weighted for $\tilde{P}$, where $\sum n^i = N$. Let $\bar{n}^i$ be the *expected* number of copies $\mathbb{E}[n^i | \{(x^i, w^i)\}_{i=1}^N]$ under R . For every resampled particle $\tilde{x}^j = x^{a_j}$, set its weight as $\tilde{w}^j := w^{a_j} / \bar{n}^{a_j}$. If $\bar{n}^i > 0$ for each x^i with $w^i > 0$, the resulting particle collection $\{(\tilde{x}^j, \tilde{w}^j)\}_{j=1}^N$ is also properly weighted for $\tilde{P}$.

Proof. We want to show that $\mathbb{E}[\frac{1}{N} \sum_{j=1}^N \tilde{w}^j f(\tilde{x}^j)] = \mathbb{E}_{x \sim P}[Zf(x)]$. We have:

$$\begin{aligned} \mathbb{E}\left[\frac{1}{N} \sum_{j=1}^N \tilde{w}^j f(\tilde{x}^j)\right] &= \mathbb{E}\left[\frac{1}{N} \sum_{j=1}^N \frac{w^{a_j}}{\bar{n}^{a_j}} f(\tilde{x}^{a_j})\right] = \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N n^i \frac{w^i}{\bar{n}^i} f(x^i)\right] \\ &= \mathbb{E}_{\{(x^i, w^i)\}_{i=1}^N} \left[\frac{1}{N} \sum_{i=1}^N \mathbb{E}\left[\frac{n^i}{\bar{n}^i} \mid \{(x^i, w^i)\}_{i=1}^N\right] w^i f(x^i) \right] \\ &= \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^N w^i f(x^i)\right] = \mathbb{E}_{x \sim P}[Zf(x)], \end{aligned}$$

where the last line follows by proper weighting of the original collection $\{(x^i, w^i)\}_{i=1}^N$. $\square$

Using Proposition 8.2, we can easily derive the proper weights for the resampling schemes discussed here. It is easy to see that multinomial, residual, and stratified sampling without priority scores has an expected copy number $\bar{n} = N \frac{w}{\sum_i w^i}$ for any particle (x, w) , s.t. the new proper weight after resampling is the average weight $\tilde{w} = \sum_i w^i / N$. When a priority function α is used, we have $\bar{n} = N \frac{\alpha(w)}{\sum_i \alpha(w^i)}$, so the new weight is $\tilde{w} = \frac{\sum_i \alpha(w^i)}{N} \frac{w}{\alpha(w)}$. If we perform block resampling and use (non-priority-weighted) resampling within one block A but not another block B (e.g. keeping one copy each of the top- k weighted particles, and resampling the rest), then the $|A|$ resampled particles have a weight of $\tilde{w} = \sum_{i \in A} w^i / |A|$, while the particles in B keep their original weights. *GenParticleFilters.jl* implements these weights for each supported resampling scheme.

8.6 Particle Filter Resizing

While particle resampling allows us to focus computational effort on specific particles, it keeps the total sample budget fixed at the original number N . However, we may want to adaptively change the number of particles if circumstances call for it. For example, when tracking an object through occlusion with a particle filter, we may want to *increase* the total number of particles to reflect our increased uncertainty about the object's position. Particle increases may also be warranted if some inference quality diagnostic falls below a threshold [131, 344]. But if we are highly confident about a few hypotheses, we may want to *decrease* the number of particles to improve the runtime and efficiency of SMC. In open-ended SIPS (Chapter 3), we both increase and decrease the number of particles, adding a fixed number of new hypotheses at each step, before resampling back down to a smaller number [10]. We call this general class of operations *particle filter resizing*:

Definition 8.9 (Particle Filter Resizing). Given a particle collection $\{(x^i, w^i)\}_{i=1}^N$ properly weighted w.r.t. $\tilde{P}_t$, *particle filter resizing* returns a new collection of $M \neq N$ particles $\{(\tilde{x}^j, \tilde{w}^j)\}_{j=1}^M$ which is properly weighted w.r.t. to the same target $\tilde{P}_t$, where each new particle $(\tilde{x}^j, \tilde{w}^j)$ is either descended from a parent particle with index a_j s.t. $\tilde{x}^j = x^{a_j}$, or freshly sampled via an initialization scheme.

In *GenParticleFilters.jl*, we support a wide array of resizing operations, including *replicating* particles, *dereplicating* (the descendants of) replicated particles, *resampling* with a variable sample size, *coalescing* identical particles, and *introducing* new ones. These are summarized in Figure 8.6.


```
pf_resize!(state::ParticleFilterState, n_particles::Int,  
           method=:multinomial; kwargs...)
```

Resizes a particle filter by resampling existing particles until a total of `n_particles` have been sampled. The resampling method can optionally be specified: `:multinomial` (default), `:residual` or `:optimal`.

```
pf_replicate!(state::ParticleFilterState, n_replicates::Int;  
              layout=:contiguous)
```

Expands a particle filter by replicating each particle `n_replicates` times, in either a `:contiguous` or `:interleaved` layout.

```
pf_dereplicate!(state::ParticleFilterState, n_replicates::Int;  
                layout=:contiguous, method=:keepfirst)
```

Shrinks a particle filter by retaining one out of every `n_replicates` particles. The total number of particles must be a multiple of `n_replicates`. Retained particles can be selected by different methods. If `method` is `:keepfirst`, only the first particle of each set of replicates is retained, exactly reversing the effect of `pf_replicate!`. If `method` is `:sample`, the retained particle is sampled according to its normalized weight among the replicates.

```
pf_coalesce!(state::ParticleFilterState; by=get_choices)
```

Coalesces traces that are equivalent according to `by` (defaulting to `get_choices`). Each set of equivalent traces is replaced by a single trace with weight equal to the sum of the original weights (multiplied by the ratio of the number of new to old traces).

```
pf_introduce!(state::ParticleFilterState, [model::GenerativeFunction, model_args,  
      observations::ChoiceMap, [proposal::GenerativeFunction, proposal_args],  
      n_particles::Int)
```

Introduce `n_particles` new traces, constrained to the provided observations. If `model` and `model_args` are omitted, then this function will use the same model and arguments as the first trace in the particle filter. A custom proposal can be used to propose choices.

Figure 8.6: Documentation for `pf_resize!` and other resizing operations.

Besides the multinomial and residual resampling schemes described earlier, changing the particle count allows us to make use of schemes like the optimal resampling strategy of Fearnhead and Clifford [345]. In optimal resampling, the new number of particles M is less than the original N , and particles are chosen to optimally reduce the variance of the between the new (smaller) collection and the original particle approximation. More generally, once the particle count can be adapted, a much wider design space is open, since resizing is natural approach for allocating more computation to specific particles descendants, or for exploring particular regions of sample space.

To achieve proper weighting for this diverse array of resizing operations⁴, we can generalize Proposition 8.2 to the case where the number of particles can change from M to N .

Proposition 8.3. Let $R(n^{1:N}; \{(x^i, w^i)\}_{i=1}^N)$ be the joint distribution over the number of copies n^i of each x^i induced by resampling a collection $\{(x^i, w^i)\}_{i=1}^N$ properly weighted for $\tilde{P}$, where the new total count is $\sum n^i = M$. Let $\bar{n}^i$ be the *expected* number of copies $\mathbb{E}[n^i | \{(x^i, w^i)\}_{i=1}^N]$ under R . For every resampled particle $\tilde{x}^j = x^{a_j}$, set its weight as $\tilde{w}^j := \frac{w^{a_j}}{\bar{n}^{a_j}} \frac{M}{N}$. If $\bar{n}^i > 0$ for each x^i with $w^i > 0$, the resulting particle collection $\{(\tilde{x}^j, \tilde{w}^j)\}_{j=1}^M$ is also properly weighted for $\tilde{P}$.

Proof. Similar to Proposition 8.2, we want to show $\mathbb{E}[\frac{1}{M} \sum_{j=1}^M \tilde{w}^j f(\tilde{x}^j)] = \mathbb{E}_{x \sim P}[Zf(x)]$. We have:

$$\begin{aligned} \mathbb{E} \left[\frac{1}{M} \sum_{j=1}^M \tilde{w}^j f(\tilde{x}^j) \right] &= \mathbb{E} \left[\frac{1}{M} \sum_{j=1}^M \frac{w^{a_j}}{\bar{n}^{a_j}} \frac{M}{N} f(\tilde{x}^{a_j}) \right] = \mathbb{E} \left[\frac{1}{M} \sum_{i=1}^N n^i \frac{w^i}{\bar{n}^i} \frac{M}{N} f(x^i) \right] \\ &= \mathbb{E}_{\{(x^i, w^i)\}_{i=1}^N} \left[\frac{1}{N} \sum_{i=1}^N \mathbb{E} \left[\frac{n^i}{\bar{n}^i} \frac{M}{N} \mid \{(x^i, w^i)\}_{i=1}^N \right] w^i f(x^i) \right] \\ &= \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N w^i f(x^i) \right] = \mathbb{E}_{x \sim P}[Zf(x)], \end{aligned}$$

□

Compared to resampling, the only difference in the proper weight is that there is an additional factor of M/N . As a result, the proper weight for multinomial and/or residual resizing continues to be the average weight of the original particles. When dereplicating a particle collection by sampling within each block of L replicates, the proper weight for a block's retained particle is just the average weight within that block. When coalescing two particles into one, we have to multiply the weight of the resulting particle by M/N after summing the original weights. All of these results can be derived from Proposition 8.3, and are implemented in *GenParticleFilters.jl*.

8.7 Discussion and Future Work

This chapter surveyed the wide range of SMC building blocks provided by *GenParticleFilters.jl*. Along the way, we provided mathematical details for how each operation preserves correctness in the form of proper weighting, all of which is automated and implemented our library. However, there are still many SMC operations and variants that we do not fully support, and which deserve future research and implementation:

(Re)sampling Without Replacement. In particle initialization, resampling, and resizing, our library supports correlated sampling strategies such as stratification to reduce variance. Another common strategy that uses correlation is *(re)sampling without replacement*, which is especially useful for inference over discrete sample spaces (as in stochastic beam search [346, 347]). To support this feature, *GenParticleFilters.jl* could be extended to allow users to specify joint proposals in particle initialization, or distributions over sets of retained particles in resampling/resizing.

⁴In the case where fresh particles are not introduced, since particle introduction can be treated like initialization.

Recombination Moves. SMC algorithms can be viewed as a Bayesian instantiation of the more general class of genetic algorithms [348, 349]. Hence, particle updating and resampling are also known as *mutation* and *selection*. Another operation that features in genetic algorithms is *recombination* (also known as *cross-over*), where components of multiple candidates are recombined to form new ones [350]. Recombination has been used in evolutionary MCMC algorithms [351, 352] and some SMC algorithms [353]. However, it has not been shown how to make recombination moves in SMC sound (i.e. maintain proper weighting). Given its similarity to correlated initialization, properly weighted recombination could potentially be derived using some of the machinery introduced in this chapter, and implemented via an extension of trace translators [331].

Parallel SMC. By and large, SMC is an embarrassingly parallel algorithm, and even resampling and resizing have parallel-friendly variants [341, 342, 354]. SMC also has *asynchronous* variants, which allow particles to wait for the completion of shared sub-operations (e.g. batched language model calls, as in [194]) before continuing independently. While our library allows for parallelism via manual division of particles into separate blocks, this limits the scope of initialization and resampling strategies that produce correlated samples (via stratification, etc.). As such, creating an interface that automates away low-level parallelization concerns remains an open design problem.

Conditional SMC. As a Monte Carlo algorithm, SMC can itself be viewed as generative process that one might perform inference over. In *conditional SMC* (*cSMC*), we perform *meta-inference* [135] over an SMC algorithm: Given a sample x from the SMC-generated approximation of the final target $\tilde{P}_T$, we infer the process that led to x being sampled. cSMC is useful for embedding SMC as a subroutine within larger probabilistic programs [136] or inference algorithms [355], as it allows us to unbiasedly estimate the marginal probability that SMC generates a specific value x . Recent work by Lew et al. [136] has shown how a SMC-cSMC algorithm pair can be composed from building blocks that implement both the forward and conditional variants of each SMC operation. Future work could integrate this compositional construction with *GenParticleFilters.jl*, extending compositional cSMC to support the SMC building blocks described in this chapter.

Tree-Structured SMC. In this chapter, we have presented SMC as a process that approximates a *sequence* of target distributions $(\tilde{P}_T)_{t \in [1, T]}$. However, an even more general formulation of SMC operates over *trees* of target distributions: Given a target $\tilde{P}_i$ with parents $\{\tilde{P}_j\}_{j \in \text{Pa}(i)}$, a particle approximation for $\tilde{P}_i$ is generated by conditioning on samples from each of the parents. A prominent instantiation of tree-structured SMC is divide-and-conquer SMC [291], which breaks down the inference problem of sampling from some target $\tilde{P}$ into a tree of inference sub-problems. To support tree-structured SMC, *GenParticleFilters.jl* could be extended with variants of `pf_update!` that take in *multiple* particle collections, instead of just evolving a single collection.

Chapter 9

Probabilistic Programming with Large Language Models

[A]nyone speaking a language possesses, implicitly, an enormous knowledge of the statistics of the language.

Claude Shannon

In this thesis, we have developed an approach to cooperative intelligence rooted in rational, model-based AI engineering — specifically through probabilistic programming and inverse planning. This stands in contrast to the data-driven scaling paradigm that dominates current AI research. A reasonable skeptic might thus question whether our approach can truly scale to the complex, open-ended domain of human cooperation, especially given the knowledge engineering challenges faced by earlier model-based approaches to AI. After all, how could one possibly specify probabilistic models encompassing the vast space of human goals, or the ways these goals might be expressed through natural language?

We have given parts of our answer in earlier chapters, demonstrating how inverse planning can be scaled towards open-ended goal inference (Chapter 3) and how large language models (LLMs) can be embedded within probabilistic models of cooperative agency (Chapter 4). This chapter completes our response, explaining the broader role that LLMs can play in scaling a rational probabilistic approach to AI engineering.

Specifically, we introduce infrastructure for **probabilistic programming with LLMs**. By embedding language models as *generative functions* in a general-purpose probabilistic programming language (PPL), we can specify generative models of the (social) world that coherently compose LLMs with other symbolic, probabilistic, and neural components. This allows us to address key knowledge engineering challenges that might otherwise be inhibiting: LLMs can serve as *likelihood functions* over natural language, bridging the gap between human communication and formal representations. They can also function as *priors* over aspects of the world — such as human goals — capturing statistical knowledge that would be impractical to specify manually. Additionally, LLMs can act as *proposal distributions* for inference over hypotheses represented in a formal language. In the sections to follow, we explain how LLMs can be integrated in a probabilistic programming language, then demonstrate how LLMs can be used in the aforementioned roles.

<pre>GPT3GenerativeFunction(model::String = "text-davinci-002" temperature::Float64 = 1.0 max_tokens::Int = 1024 stop = nothing)</pre>	<pre>gpt3 = GPT3GenerativeFunction(stop="\n") @gen function qa_model() q ~ gpt3("Question:") a ~ gpt3(q * "\n" * "Answer:") return (q, a) end</pre>	<pre>observations = choicemap((:a, " Mount Olympus")) traces, weights, _ = importance_sampling(qa_model, (), observations, 100)</pre>
--	---	---

(a) LLM gen. function constructor. (b) LLM probabilistic program $P(q, a)$. (c) Importance sampling for $P(q|a)$.

Figure 9.1: Overview of the *GenGPT3.jl* library for probabilistic programming with LLMs [21]. *GenGPT3.jl* wraps LLMs accessible via the OpenAI LLM Completions API in Gen’s [4] generative function interface. Users can (a) construct an LLM generative function, specifying their choice of model and stop token. In (b), a probabilistic program `qa_model` makes two calls to an LLM, first to sample a question q , then an answer a to that question. In (c), we write an inference program that takes an observed answer $a = \text{"Mount Olympus"}$, and infers q via importance sampling over `qa_model`.

9.1 Language Models as Generative Functions

LLMs are neural networks — typically following the transformer architecture [356] — that parameterize probability distributions over language data. Language data is treated as a sequence of textual sub-units called *tokens* (e.g. word, characters, or byte-pair encodings [357, 358]) from some vocabulary $\mathcal{V}$. We focus here on *autoregressive LLMs*, which parameterize a *next-token distribution* $P_{\text{LLM}}(w_n | w_{1:n-1})$ over w_n given the previous tokens $w_{1:n-1}$. Most transformer-based LLMs also have a maximum token count n_{max} , beyond which the next-token distribution is undefined.

For our use cases, we want to model distributions over *strings* of tokens, which can have variable length. We also want these string to be *completions* of some initial prefix string $s_{\text{start}} = w_{1:m}$, often called a *prompt*. To do so, we can designate a particular token $w_{\text{stop}} \in \mathcal{V}$ as a *stop token*. The distribution $P_{\text{LLM}}(s; s_{\text{start}})$ over variable-length completions $s = w_{m+1:n}$ is then characterized by sampling tokens from the LLM until w_{stop} or the maximum token limit n_{max} is reached:

$$P_{\text{LLM}}(s; s_{\text{start}}) = \begin{cases} \prod_{i=m+1}^n P_{\text{LLM}}(w_i | w_{1:i-1}) & \text{if } w_n = w_{\text{stop}} \text{ or } n = n_{\text{max}} \\ 0 & \text{otherwise} \end{cases} \quad (9.1)$$

By viewing LLMs as distributions over strings, we can embed them as *generative functions* [4, 335] in a probabilistic programming language. As briefly covered in Section 8.1, generative functions are an abstract data type for probabilistic computations that support the following minimal interface:

- *simulate*: *sampling* of an execution trace t given arguments x ;
- *generate*: *constrained sampling* with random choices in t fixed to particular values;
- *assess*: probability *evaluation* of a full execution trace t .

In our case, the argument x of an LLM generative function is the prompt s_{start} , and an execution trace is just the generated string s under $P_{\text{LLM}}(s; s_{\text{start}})$. As such, we can easily implement *simulate* if we have sampling access to an LLM. For *generate*, we assume the only random choice that can be constrained is the full string s . We can then implement both *generate* and *assess* if we have access to the token probabilities¹ of each w_i in s , allowing us to evaluate Equation 9.1.

¹Many but not all web-based LLM APIs provided access to this information. Note that this does not require access to the *full* next-token distribution at each step, only the probability of the tokens actually constrained.

The generative function interface (GFI) is compositional, so once we have implemented it for an LLM, we can make calls to an LLM generative function inside of compositional language for writing generative functions — that is, a probabilistic programming language. Figure 9.1 shows an example of this integration as implemented in *GenGPT3.jl* [21], a library that wraps the OpenAI LLM Completions API within the GFI. Using *GenGPT3.jl*, users have access to the GPT-3 family of LLMs [170] as generative functions (Figure 9.1a). Users can then write LLM probabilistic programs like `qa_model` (Figure 9.1b), using GPT-3 as a prior over possible questions under the prompt “Question: ”, concatenating the generated question q with more text, then calling GPT-3 again to sample an answer a to that question. Since `qa_model` is also a generative function, we can use it with *Gen.jl*’s inference library [4], performing importance sampling to infer the question q from an observed answer a (Figure 9.1c).

9.1.1 Batched LLM Generative Functions for Efficiency

While the interface shown in Figure 9.1 is expressive enough to allow users to write a wide array of LLM probabilistic programs, it is not the most efficient. Since LLMs can be parallelized on a GPU, sampling or scoring a batch of N completions in parallel can often be N times faster than doing so in serial. This difference is exacerbated when calling a LLM via a web API, since each API call incurs substantial overhead. For instance, each execution of `qa_model` (Figure 9.1b) involves two LLM API calls, taking about 2 seconds. As a result, importance sampling with 100 samples (Figure 9.1c) takes around 200 seconds.

To (partially) address this challenge, *GenGPT3.jl* provides support for both *batched* and *mixture* LLM generative functions (Figure 9.2a, L1–2). Instead of taking a single prompt s_{start} , a batched LLM generative function (`MultiGPT3GF`) takes in a batch of N prompts $s_{\text{start}}^{1:N}$, and returns N completions $s^{1:N}$. A mixture LLM generative function (`GPT3Mixture`) takes N prompts $s_{\text{start}}^{1:N}$ and a vector θ of mixture probabilities, returning a single completion s sampled from the mixture distribution $\sum_{i=1}^N \theta_i P_{\text{LLM}}(s; s_{\text{start}}^i)$. Both of these generative functions make batched LLM API calls in their implementations, substantially reducing API overhead.

With these abstractions, we can rewrite `qa_model` in batched form $P(q^{1:N}, a)$, as shown in Figure 9.2: Instead of sampling q once, we sample N questions $q^{1:N}$ in a batch, and pass them all as prompts to a mixture LLM, from which sample one answer a . Sampling just one trace from `batched_qa_model` with a constrained is equivalent to running the importance sampling algorithm in Figure 9.1c with N samples. Due to batching, however, it has a much lower runtime.

Model transformations like the one described here are known as *pseudo-marginalization* [135, 359] (since the variable q is approximately marginalized out). We make similar use of `GPT3Mixture` in our implementation of CLIPS (Chapter 1), allowing us to (exactly) marginalize over a space of possible latent prompts. Still, the need to manually rewrite a model presents a barrier to the use of LLMs in probabilistic programming. Ideally, batching would be an optimization automatically performed by an inference algorithm, not implemented by the user. We revisit this issue in the Discussion, surveying recent approaches to automatically leveraging batched computation.

```
batched_gpt3 = MultiGPT3GF(stop="\n")
gpt3_mixture = GPT3Mixture(stop="\n")
@gen function batched_qa_model(N::Int)
  prompts = fill("Question: ", N)
  qs ~ batched_gpt3(prompts)
  prompts = qs .* "\n" .* "Answer:"
  a ~ gpt3_mixture(prompts, fill(1/N, N))
  return (qs, a)
end
```

(a) Batched LLM probabilistic program.

```
obs = choicemap((:a, " Mount Olympus"))
trace, weight =
  generate(batched_qa_model, (100,), obs)
```

(b) Batched inference over $P(q, a)$.

Figure 9.2: Probabilistic programming with batched LLM generative functions.

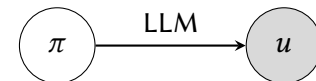
9.2 Roles for Language Models in Probabilistic Programs

Having shown how LLMs can be embedded in probabilistic programs, we now illustrate several different roles they can play in probabilistic modeling and inference. We consider three primary roles where we see LLMs being especially useful for a rational probabilistic approach to AI engineering: (i) likelihood functions over language, (ii) informative priors and (iii) data-driven proposal distributions.

9.2.1 Likelihood Functions

```
@gen function utterance_model()
  intention ~ labeled_uniform(["(buy ice-cream 1)", "(buy pizza 2)"])
  prompt =
    """Intention: (buy lollipop 3)
    Utterance: Could I have three lollipops please?
    Intention: $(intention)
    Utterance: """
  utterance ~ language_model(prompt)
  return utterance
end
```

(a) Probabilistic program for $P(\pi, u) = P(\pi)P(u|\pi)$.



(b) Graphical model of $P(\pi, u)$.

```
obs = choicemap({:utterance, "Deux pizzas."})
_, logprobs, _ = enumerative_inference(utterance_model, (), obs)
```

(c) Inference program for $P(\pi|u)$.

Figure 9.3: LLM utterance likelihood $P(u|\pi)$ in a model $P(\pi, u)$ of intentional utterances. (a, b) The generative model samples an intention π , then samples an utterance u conditioned on π and few-shot examples. (c) In inference, we condition on an observed u , using the LLM as a likelihood function.

As we have demonstrated extensively in CLIPS (Chapter 4), LLMs can be used as likelihood functions over human utterances, allowing us to specify models of cooperative communication that operate the full vocabulary of natural language. This greatly expands the reach of rational models of communication such as rational speech act theory [138, 140, 173, 213], which have historically been limited to small vocabularies or synthetic languages. Figure 9.3 shows a simple example of how LLMs can be used in this way: We prompt an LLM with (i) an example of how to translate formal “intentions” into natural language; (ii) an intention π sampled from a prior $P(\pi)$. This parameterizes a distribution over utterances $P(u|\pi)$ (Figure 9.3a). We then condition on an observed utterance u , allowing us to infer a distribution over intentions π (Figure 9.3c).

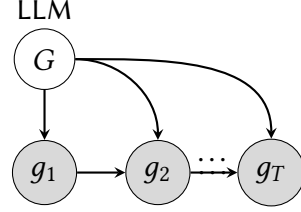
While the example here is simple, this basic approach can be applied to much richer models of utterance generation — in CLIPS, for instance, a multi-step *plan* π is communicated as one or more utterances over time [12]. Crucially, utterance generation is not deferred entirely to the LLM. Instead, we can write probabilistic programs that embed *structured domain knowledge* about how language is likely to be generated (e.g. according to cooperative principles [137, 138]), and how language is *jointly* distributed with other data we might observe (e.g. actions taken by a cooperative agent). For everything else that we do not care to (or know how to) explicitly model — which exact sentence structure to use, politeness markers, informal speech, or even dialect — we can use LLMs to model as a kind of “linguistic noise”.

The utility of this approach is not limited to cooperative AI applications. In probabilistic data cleaning [360] and citation matching [361, 362], for example, language models can be used to model typographical errors and formatting differences. In any setting where we want to model the relationship between latent structured representations and “noisy” language observations, we can make productive use of LLM likelihoods.

9.2.2 Informative Priors

```
@gen function agent_model(T::Int)
  prompt = """
    Goal: (and (sliced lettuce) (sliced cucumber) (chopped onion))
    Goal: (and (mashed avocado) (diced tomato))
    Goal: """
  goal ~ language_model(prompt)
  subgoals_remaining = parse_to_subgoals(goal)
  subgoals_achieved = String[]
  for t in 1:T
    subgoal = {(:subgoal, t)} ~ labeled_uniform(subgoals_remaining)
    push!(subgoals_achieved, subgoal)
    pop!(subgoals_remaining, subgoal)
  end
  return subgoals_achieved
end
```

(a) Probabilistic program for $P(G, g_{1:T}) = P(G) \prod_{t=1}^T P(g_t | g_{1:t-1}, G)$.



(b) Graphical model of $P(G, g_{1:T})$.

```
obs = choicemap(((:subgoal, 1), "(chopped olive)"),
               ((:subgoal, 2), "(sliced tomato)"))
n_samples = 100
traces, logprobs, _ = importance_sampling(
  agent_model, (2,), obs, n_samples
)
```

(c) Inference program for $P(G | g_{1:T})$.

Figure 9.4: LLM goal prior $P(G)$ in a sequential model $P(G, g_{1:T})$ of a subgoal-achieving agent. (a, b) The generative model samples a goal G from an example-conditioned LLM prior, parses G into subgoals, and achieves the subgoals in random order. (c) We condition on two observed subgoals to infer $P(G | g_1, g_2)$.

Human reasoning often incorporates learned statistical knowledge about the world: when inferring intentions or goals, we leverage not just our common-sense theory of (boundedly) rational agency (Chapter 2), but also prior knowledge about typical behaviors or preferences [10, 103]; when trying to figure what item on a faded shopping receipt cost a hundred dollars, we draw upon the likely distribution of items and prices that might be found in a particular store [193].

Since LLMs are trained on vast corpora of text and code on the Internet, they can serve as proxies for this cultural knowledge. Correspondingly, many researchers have investigated the usefulness of LLMs in specifying priors for perception and planning tasks [363], causal inference [364, 365], and numerical or logical concepts [366]. Indeed, the first paper to introduce probabilistic programming with LLM sub-components uses LLMs as context-specific priors [193].

Figure 9.4 shows an example of how LLMs can be used as goal priors in the context of open-ended goal inference (cf. Chapter 3). We model an agent that achieves a sequence of subgoals over T steps (Figure 9.4a). We first prompt an LLM with example goals in formal PDDL syntax [46, 47], sampling a new goal G in the same format. We then parse the goal into subgoals, which the agent achieves in random order at each step $t \in [1, T]$. Inference over this model can be performed via importance sampling (with the prior as the proposal), conditioned on the agent first achieving the subgoal (chopped olive), followed by (sliced tomato) (Figure 9.4c). One plausible inference here might be that the goal G is Greek salad — represented as the set of subgoals (and (chopped olive) (sliced tomato) (chopped cucumber) (sprinkled feta)) — since that would be consistent with the observed subgoals, and be reasonably likely under the LLM goal prior. In contrast, an observationally consistent but implausible inference might be (and (chopped olive) (sliced tomato) (boiled feta)), since people almost never boil feta.

In the example above, we represent goals G in a formal language. In some exploratory research, we have also used LLMs as priors over goals represented as templated English recipes, finding that LLM-generated goals are close to human-generated goals in their plausibility and feasibility [367]. It remains an open question how best to represent the structured hypotheses that LLMs are used as priors over. We discuss several approaches below.

Free-Form Natural Language. Latent hypotheses could be represented in free-form natural language, as in [192, 366, 368, 369]. This requires other parts of the model to be able to interpret natural language, e.g. via LLM likelihoods [192], or language-to-code translation [366, 368].

Structured Natural Language. Priors could instead be defined over structured natural language [367], enabling easier parsing and control over the representational scope of the hypotheses, while avoiding the restrictiveness that may come with generating hypotheses in code [366].

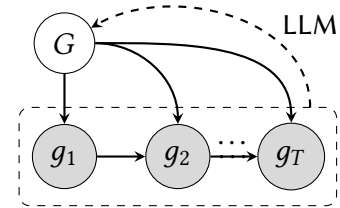
Formal Language. For the easiest integration with systems that operate over formal machine language (planners, program interpreters, etc.), hypotheses could be directly represented in structured code. To encourage more diverse generations, natural language documentation could be generated before the code itself, with the documentation serving as *auxiliary randomness* that aids the hypothesis generation process. However, ensuring that the generated code is syntactically and semantically sound may require recent advances in constrained LLM decoding [195, 196].

Program-Structured Priors. Instead of having LLMs generate structured hypotheses wholesale, they could be used within a probabilistic program prior to generate specific aspects of the overall hypothesis (as in [193]). This requires making stronger conditional independence assumptions about different aspects of a hypothesis, but provides more control over the prior distribution.

9.2.3 Proposal Distributions

```
@gen function goal_proposal(obs_subgoals)
  subgoals_str = subgoal_choicemap_to_str(obs_subgoals)
  prompt = """
  Below are some subgoals and the goals they might be part of:
  Subgoals: (chopped onion) (sliced lettuce)
  Goal: (and (sliced lettuce) (sliced cucumber) (chopped onion))
  Subgoals: (diced tomato)
  Goal: (and (mashed avocado) (diced tomato))
  Subgoals: (sliced beets) (dressed beets vinegar) (sliced apple)
  Goal: (and (sliced beets) (sliced apple) (dressed beets vinegar))
  Subgoals: $(subgoals_str)
  Goal: """
  goal ~ language_model(prompt)
  return goal
end
```

(a) Probabilistic program with LLM proposal $Q(G; g_{1:T})$.



(b) LLM as proposal $Q(G; g_{1:T})$.

```
obs = choicemap(((:subgoal, 1), "(chopped olive)"),
               ((:subgoal, 2), "(sliced tomato)"))
traces, logprobs, _ = importance_sampling(
  agent_model, (2,), obs, 20,
  goal_proposal, (obs,)
)
```

(c) Inference program for $P(G|g_{1:T})$.

Figure 9.5: LLM proposal over goals $Q(G; g_{1:T})$ given observed subgoals $g_{1:T}$. (a) We prompt an LLM with the observed subgoals and few-shot examples of how to generate a complete goal from subgoals. (b, c) We use this as a custom proposal for importance sampling over the agent model in Figure 9.4.

Since LLMs can be used as context-conditioned priors over structured hypotheses, they can also serve as data-driven proposals — or approximate posteriors — over such hypotheses. In Figure 9.5, we show an example of such a proposal, which we use for inference over the agent model $P(G, g_{1:T})$ in Figure 9.4. We prompt an LLM to guess the full goal G from the observed subgoals $g_{1:T}$ (Figure 9.5a). The LLM thus functions as a *subgoal-conditioned goal proposal* $Q(G; g_{1:T})$ (Figure 9.5b), similar to the n -gram-based proposals in open-ended SIPS (Chapter 3). We use this proposal in importance sampling (Figure 9.5c), allowing us to reasonably approximate the posterior $P(G|g_{1:T})$ with less samples than importance sampling with the prior as the proposal (Figure 9.4c).

If LLMs can directly be used as approximate posteriors, why bother integrating them in a probabilistic programming framework at all? Some recent research has taken this approach, directly using LLM to approximate posterior predictive distributions [370, 371]. However, as we have seen with CLIPS (Chapter 4) and other studies evaluating LLMs on probabilistic mental state inference [38, 372], LLMs are neither accurate nor well-calibrated for the posterior inference tasks required for human-like cooperative intelligence. Below, we consider some ways that LLMs can be more fruitfully utilized as proposals.

Exploiting LLM-Friendly Sub-Problems. LLMs are more likely to be useful as proposals when they are prompted in a way that is close to the data they are trained on. In Figure 9.5, we do this by effectively having the LLM solve an auto-completion task, generating a full goal given the constituent subgoals achieved so far. This is easier than the full goal inference task considered in (open-ended) SIPS and CLIPS, which requires inferring goals from actions or percepts. This suggests a distinct role for LLM proposals that can be exploited by importance or rejection sampling schemes: By prompting LLMs to solve a convenient sub-problem of the full inference problem we care about (e.g. inferring goals from subgoals, not actions), then evaluating the proposed samples with respect to our full generative model (e.g. a model of boundedly rational planning), we can ensure the coherence of the resulting inferences

Avoiding LLM Mode Collapse. Even with this qualified usage, LLMs may not always be ideal proposals. In exploratory research [367], we found that larger fine-tuned LLMs often had spurious biases towards specific hypotheses, potentially as a result of the mode collapse encouraged by reinforcement learning from human feedback [208, 373]. Such proposals are overly *narrow*, whereas good proposals need to be *broad* in order to avoid unbounded variance in posterior estimates [374]. As a result, it might be preferable to use LLM base models with small parameter counts as proposals, since they do not exhibit mode collapse. It may even be desirable to *distill* the statistical knowledge of LLMs into much smaller models (e.g. n -gram models) by using them to generate synthetic data with the appropriate distributional properties [132]. This would also be much less costly than sampling repeatedly from LLMs with hundreds of billions of parameters.

Semantic Parsing Proposals. One use case where LLMs are likely to avoid mode collapse is in *semantic parsing via constrained generation* [195–198, 375]. In semantic parsing, we are interested in parsing a string or utterance u into a formal expression $c \in \mathcal{L}$, where $\mathcal{L}$ is a formal language with syntactic and semantic constraints. To handle ambiguous utterances u , we may also want the parsed expression c to be likely under some probability distribution. In CLIPS [12] and extensions [376], this distribution is the product $P(c|g, \pi)P(u|c)$ of how likely the command c is under a goal and plan $P(c|g, \pi)$, and the utterance likelihood $P(u|c)$. Since LLMs are frequently trained on large amounts of both text and code — and since formal language constraints make for a narrower target distribution — LLMs are natural as proposals $Q(c; u)$ for this setting. By embedding such proposals in recently-developed SMC methods for constrained generation [195, 196], the gap between target and proposal can be further reduced. We expect that using LLMs as semantic parsing proposals will be key to scaling CLIPS-like pragmatic instruction following to real applications, where the space of possible commands $c \in \mathcal{L}$ will be too large for enumerative inference.

9.3 Discussion and Related Work

The idea of probabilistic programming with LLMs was first introduced by Lew et al. [193], who used masked language models as conditional priors over the locations and identities of everyday objects. Since then, many researchers have taken up and explored the idea in various directions:

- Dohan et al. [377] proposed *language model cascades* — probabilistic programs where most sub-routines are language models — as a framework for LLM composition.
- Wong et al. [174] developed *rational meaning construction* as a computational account of the relationship between human language and thought, arguing that LLMs can serve as proxies for the human ability to flexibly translate natural language into a probabilistic language-of-thought, formalized as probabilistic programs.
- Contemporaneously, we developed the work presented in this chapter [21] and Chapters 4–5 [12, 14], focusing on the role that LLMs can play as *sub-components* of probabilistic programs, rather than as translation functions from language to probabilistic code.

More recent work has begun to unite some of these threads. Recognizing the promise of sequential Monte Carlo inference for constrained LLM decoding, Lew et al. [194] developed *sequential Monte Carlo steering* of LLMs, while simultaneously introducing the *LLaMPPL* library for specifying LLM constrained generation tasks as probabilistic programs. Unlike the LLM probabilistic programs in this chapter, *LLaMPPL* programs allow users to mix modeling and inference code, directly specifying a Feynman-Kac model [378] over LLM token sequences. *LLaMPPL* also addresses the batching limitations described in Section 9.1.1, using an asynchronous programming model where multiple SMC particles queue up for batched computation of LLM next-token probabilities, then run independently between LLM calls.

Since LLM generation under formal constraints is a posterior inference task, SMC steering is well-suited for the semantic parsing use case described in Section 9.2.3, and also for the use of LLMs as flexible translators in rational meaning construction [38, 174, 372, 379]. Developing this use case further, Loula et al. [195] have shown how SMC can efficiently incorporate both syntactic and semantic constraints in LLM generation. They also show that better approximation of the constrained LLM posterior correlates significantly with other measures of performance, validating the importance of a Bayesian approach. Even more recently, Lipkin et al. [196] has shown how to further accelerate constrained generation via adaptive rejection sampling, while Grand et al. [380] demonstrate that LLMs can write *LLaMPPL* programs to steer the generations of (smaller) LLMs.

While progress in this area has been rapid, considerable work remains to be done to integrate LLMs into a fully-fledged probabilistic programming stack. The approach taken by *GenGPT3.jl* [21] is highly expressive, but suffers from batching limitations and the inefficiencies of working with API-based LLMs. In contrast, *LLaMPPL* [194] and its associated libraries [195] are not general-purpose PPLs, limiting the ease of integration with architectures like CLIPS. In general, the efficient integration of local or remote LLMs into a general purpose PPL remains an open design challenge — one that will require addressing how PPLs can automate batched computation and asynchronous calls to remote or GPU-bound processes. These challenges will have to be addressed to exploit the full potential of probabilistic programming with LLMs.

Conclusion

Conclusion

This power is due to co-operation itself. When the labourer cooperates systematically with others, [they] strip off the fetters of [their] individuality, and develop the capabilities of [their] species.

Karl Marx

What are the computational and algorithmic foundations of how humans cooperate? And how, by studying them, can we engineer cooperatively intelligent machines? Throughout this thesis, we have seen how taking a rational approach to reverse engineering human cooperation has led us to new insights about building more cooperative machines — and how the resulting algorithmic and conceptual insights have deepened our understanding of human cooperation itself. By taking seriously the tractability limits of human theory-of-mind, we were able to develop SIPS not only as an account of human action understanding, but also as a state-of-the-art algorithm for online Bayesian goal inference. By reflecting on the role LLMs could play in a cooperative model of human communication, we not only hit upon a promising new route to reliable AI copilots, but also expanded the reach of traditional rational speech act models to handle rich linguistic utterances. And by taking a Bayesian, game-theoretic approach to analyzing social normativity, we laid the groundwork for not just normatively competent autonomous systems, but also a richer understanding of how humans might acquire social norms with such remarkable efficiency.

Taken together, these advances suggest a different route to advancing human-AI cooperation than is currently popular today: By encoding our best computational theories of how minds work when they cooperate, we have built systems that cooperate more reliably, more efficiently, and more intelligibly than approaches that treat cooperation as just another behavioral pattern to be learned. The success of these systems — performing accurate goal inference with essentially no training data requirements, or following ambiguous instructions with greater reliability than state-of-the-art LLMs — suggests that rational engineering may offer a viable alternative to the scaling paradigm that dominates contemporary AI.

But what of the vast landscape of human cooperative capacities that remains unexplored? While SIPS has shown how we might scalably infer human goals and plans, how does this translate to modeling and inferring beliefs, or the rich tapestry of other mental attitudes that populate our intuitive psychology — hopes, fears, regrets, resentments, loves, and dreams? While CLIPS has made headway into pragmatic instruction following, what will it take to go from there to the full spectrum of cooperative communication — from elaborated explanations to spirited argumentation, from casual coordination to high-stakes negotiation? And while our work on norm-augmented stochastic games has demonstrated the feasibility of Bayesian norm learning, how might we

extend this to model the complex institutional structures that shape human societies, or to capture the intricate processes of normative reasoning through which humans justify and critique the norms they practice?

Undoubtedly, there remains a tremendous amount of work to be done before we can come close to approximating human-like cooperative intelligence. Nevertheless, there appear to be at least some domains where the rational engineering of cooperative systems stands poised to make immediate impact. For example, many of the “agentic” AI applications of interest today appear amenable to our approach. Web browsing agents, for instance, operate in environments with well-defined action spaces and clear goal structures, making them ideal candidates for planning and inverse planning. Video game NPCs navigate virtual worlds whose physics and affordances are perfectly known. In these domains and others where the world’s structure admits tractable modeling, the rational engineering approach offers significant advantages: greater reliability stemming from coherent planning under uncertainty, enhanced interpretability through explicit modeling of mental states, and improved data and computational efficiency by leveraging scientifically grounded theories about how minds work. By continuing to advance not just the research, but also the platform infrastructure for model-based planning, probabilistic programming, and inverse planning, we believe it will be possible to deliver compelling alternatives to current “agentic AI” in applications like those mentioned here.

As for the future that might be further afield — one where there are perhaps general-purpose autonomous agents running amok in our social world — we refrain from saying too much out of uncertainty. But it is our hope that by continuing to do this work — and getting it right — we can stand to benefit from a safer, more cooperative, and ultimately more fulfilling world: One where AI isn’t the pathway to our demise, but instead to a world where we can live and flourish together, despite our many differences.

References

- [1] A. Dafoe, Y. Bachrach, G. Hadfield, E. Horvitz, K. Larson, and T. Graepel. “Cooperative AI: Machines must learn to find common ground”. In: *Nature* 593.7857 (2021), pp. 33–36.
- [2] K. M. Collins, I. Sucholutsky, U. Bhatt, K. Chandra, L. Wong, M. Lee, C. E. Zhang, T. Zhi-Xuan, M. Ho, V. Mansinghka, et al. “Building machines that learn and think with people”. In: *Nature human behaviour* 8.10 (2024), pp. 1851–1863.
- [3] N. Goodman, V. Mansinghka, D. M. Roy, K. Bonawitz, and J. B. Tenenbaum. “Church: a language for generative models”. In: *arXiv preprint arXiv:1206.3255* (2012).
- [4] M. F. Cusumano-Towner, F. A. Saad, A. K. Lew, and V. K. Mansinghka. “Gen: A general-purpose probabilistic programming system with programmable inference”. In: *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 2019, pp. 221–236.
- [5] T. Zhi-Xuan, J. Mann, T. Silver, J. Tenenbaum, and V. Mansinghka. “Online Bayesian Goal Inference for Boundedly Rational Planning Agents”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [6] C. L. Baker, R. Saxe, and J. B. Tenenbaum. “Action understanding as inverse planning”. In: *Cognition* 113.3 (2009), pp. 329–349.
- [7] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey. “Maximum entropy inverse reinforcement learning”. In: *Proceedings of the 23rd National Conference on Artificial Intelligence*. Vol. 3. 2008, pp. 1433–1438.
- [8] M. Ramírez and H. Geffner. “Probabilistic plan recognition using off-the-shelf classical planners”. In: *Twenty-Fourth AAAI Conference on Artificial Intelligence*. 2010.
- [9] M. Chiari, A. E. Gerevini, F. Percassi, L. Putelli, I. Serina, and M. Olivato. “Goal recognition as a deep learning task: The GRNet approach”. In: *Proceedings of the International Conference on Automated Planning and Scheduling*. Vol. 33. 2023, pp. 560–568.
- [10] T. Zhi-Xuan, G. Kang, V. Mansinghka, and J. Tenenbaum. “Infinite Ends from Finite Samples: Open-Ended Goal Inference as Top-Down Bayesian Filtering of Bottom-Up Proposals”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 46. 2024.
- [11] A. Alanqary, G. Z. Lin, J. Le, T. Zhi-Xuan, V. K. Mansinghka, and J. B. Tenenbaum. “Modeling the Mistakes of Boundedly Rational Agents Within a Bayesian Theory of Mind”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 43. 43. 2021.

- [12] T. Zhi-Xuan, L. Ying, V. Mansinghka, and J. B. Tenenbaum. “Pragmatic Instruction Following and Goal Assistance via Cooperative Language-Guided Inverse Planning”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024.
- [13] D. Hadfield-Menell, S. J. Russell, P. Abbeel, and A. Dragan. “Cooperative inverse reinforcement learning”. In: *Advances in neural information processing systems*. 2016, pp. 3909–3917.
- [14] L. Ying, T. Zhi-Xuan, V. Mansinghka, and J. B. Tenenbaum. “Inferring the goals of communicating agents from actions and instructions”. In: *Proceedings of the AAAI Symposium Series*. Vol. 2. 1. 2023, pp. 26–33.
- [15] H. Gintis. “Social norms as choreography”. In: *Politics, Philosophy & Economics* 9.3 (2010), pp. 251–264.
- [16] C. Bicchieri. *The grammar of society: The nature and dynamics of social norms*. Cambridge University Press, 2005.
- [17] S. Cranefield, F. Meneguzzi, N. Oren, and B. T. R. Savarimuthu. “A Bayesian approach to norm identification”. In: *ECAI 2016*. Ios Press, 2016, pp. 622–629.
- [18] N. Oldenburg and T. Zhi-Xuan. “Learning and Sustaining Shared Normative Systems via Bayesian Rule Induction in Markov Games”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 2024.
- [19] M. Cusumano-Towner, A. K. Lew, and V. K. Mansinghka. “Automating Involutive MCMC using Probabilistic and Differentiable Programming”. In: *arXiv preprint arXiv:2007.09871* (2020).
- [20] T. Zhi-Xuan. *GenParticleFilters.jl*. 2020.
- [21] T. Zhi-Xuan. *GenGPT3.jl: GPT-3 as a generative function in Gen.jl*. Nov. 2022.
- [22] G. Gergely, Z. Nádasdy, G. Csibra, and S. Bíró. “Taking the intentional stance at 12 months of age”. In: *Cognition* 56.2 (1995), pp. 165–193.
- [23] S. Liu and E. S. Spelke. “Six-month-old infants expect agents to minimize the cost of their actions”. In: *Cognition* 160 (2017), pp. 35–42.
- [24] F. Warneken and M. Tomasello. “Altruistic helping in human infants and young chimpanzees”. In: *science* 311.5765 (2006), pp. 1301–1303.
- [25] M. Tomasello. *Origins of Human Communication*. MIT Press, 2010.
- [26] M. Tomasello and M. Carpenter. “Shared intentionality”. In: *Developmental science* 10.1 (2007), pp. 121–125.
- [27] M. E. Bratman. *Shared Agency: A Planning Theory of Acting Together*. en. Google-Books-ID: jcs8BAAAQBAJ. Oxford University Press, Dec. 2013. ISBN: 978-0-19-989794-0.
- [28] H. Rakoczy and M. F. Schmidt. “The early ontogeny of social norms”. In: *Child Development Perspectives* 7.1 (2013), pp. 17–21.
- [29] Z.-X. Tan and D. C. Ong. “Bayesian Inference of Social Norms as Shared Constraints on Behavior”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 41. 41. July 2019, pp. 2919–2925.

- [30] J. Knobe. “Theory of mind and moral cognition: Exploring the connections”. In: *Trends in cognitive sciences* 9.8 (2005), pp. 357–359.
- [31] M. Kleiman-Weiner, T. Gerstenberg, S. Levine, and J. B. Tenenbaum. “Inference of Intention and Permissibility in Moral Decision Making.” In: *CogSci*. Citeseer. 2015.
- [32] J. F. Fisac, M. A. Gates, J. B. Hamrick, C. Liu, D. Hadfield-Menell, M. Palaniappan, D. Malik, S. S. Sastry, T. L. Griffiths, and A. D. Dragan. “Pragmatic-Pedagogic Value Alignment”. In: *arXiv preprint arXiv:1707.06354* (2017).
- [33] C. L. Baker, J. Jara-Ettinger, R. Saxe, and J. B. Tenenbaum. “Rational quantitative attribution of beliefs, desires and percepts in human mentalizing”. In: *Nature Human Behaviour* 1.4 (2017), pp. 1–10.
- [34] D. C. Dennett. *The intentional stance*. MIT press, 1989.
- [35] C. Baker, R. Saxe, and J. Tenenbaum. “Bayesian Theory of Mind: Modeling Joint Belief-Desire Attribution”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*, 33 (33). 2011.
- [36] J. Jara-Ettinger, L. Schulz, and J. Tenenbaum. “The Naive Utility Calculus as a unified, quantitative framework for action understanding”. In: *PsyArXiv* (2019).
- [37] T. Zhi-Xuan, N. Gothoskar, F. Pollok, D. Gutfreund, J. B. Tenenbaum, and V. K. Mansinghka. “Solving the Baby Intuitions Benchmark with a Hierarchically Bayesian Theory of Mind”. In: *RSS 2022 Workshop on Social Intelligence in Humans and Robots*. 2022.
- [38] L. Ying, T. Zhi-Xuan, L. Wong, V. Mansinghka, and J. B. Tenenbaum. “Understanding Epistemic Language with a Language-augmented Bayesian Theory of Mind”. In: *Transactions of the Association for Computational Linguistics* (2025).
- [39] T. Shu, A. Bhandwaldar, C. Gan, K. Smith, S. Liu, D. Gutfreund, E. Spelke, J. Tenenbaum, and T. Ullman. “AGENT: A benchmark for core psychological reasoning”. In: *International conference on machine learning*. PMLR. 2021, pp. 9614–9625.
- [40] S. D. Houlihan, M. Kleiman-Weiner, L. B. Hewitt, J. B. Tenenbaum, and R. Saxe. “Emotion prediction as computation over a generative theory of mind”. In: *Philosophical Transactions of the Royal Society A* 381.2251 (2023), p. 20220047.
- [41] C. H. Papadimitriou and J. N. Tsitsiklis. “The complexity of Markov decision processes”. In: *Mathematics of operations research* 12.3 (1987), pp. 441–450.
- [42] C. Backstrom and P. Jonsson. “All PSPACE-complete planning problems are equal but some are more equal than others”. In: *Proceedings of the International Symposium on Combinatorial Search*. Vol. 2. 1. 2011, pp. 10–17.
- [43] I. Van Rooij. “The tractable cognition thesis”. In: *Cognitive science* 32.6 (2008), pp. 939–984.
- [44] M. Blokpoel, J. Kwisthout, T. P. Van der Weide, and I. van Rooij. “How action understanding can be rational, Bayesian and tractable”. In: (2010).
- [45] M. F. Cusumano-Towner and V. K. Mansinghka. “Using probabilistic programs as proposals”. In: *arXiv preprint arXiv:1801.03612* (2018).
- [46] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins. *PDDL - The Planning Domain Definition Language*. 1998.

- [47] T. Zhi-Xuan. “PDDL. jl: An Extensible Interpreter and Compiler Interface for Fast and Flexible AI Planning”. PhD thesis. Massachusetts Institute of Technology, 2022.
- [48] D. Ramachandran and E. Amir. “Bayesian inverse reinforcement learning”. In: *Proceedings of the 20th International Joint Conference on Artificial Intelligence*. 2007, pp. 2586–2591.
- [49] G. R. Ghosal, M. Zurek, D. S. Brown, and A. D. Dragan. “The effect of modeling human rationality level on learning rewards from multiple feedback types”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 5. 2023, pp. 5983–5992.
- [50] M. Fox and D. Long. “PDDL2. 1: An extension to PDDL for expressing temporal planning domains”. In: *Journal of artificial intelligence research* 20 (2003), pp. 61–124.
- [51] H. L. Younes and M. L. Littman. “PPDDL1. 0: An extension to PDDL for expressing planning domains with probabilistic effects”. In: *Techn. Rep. CMU-CS-04-162* 2 (2004), p. 99.
- [52] D. P. Bertsekas and J. N. Tsitsiklis. “An analysis of stochastic shortest path problems”. In: *Mathematics of Operations Research* 16.3 (1991), pp. 580–595.
- [53] A. G. Barto, S. J. Bradtke, and S. P. Singh. “Learning to Act using Real-Time Dynamic Programming”. In: *Artificial Intelligence* 72.1-2 (1995), pp. 81–138.
- [54] B. Bonet and H. Geffner. “Labeled RTDP: Improving the Convergence of Real-Time Dynamic Programming.” In: *ICAPS*. Vol. 3. 2003, pp. 12–21.
- [55] R. E. Korf. “Real-Time Heuristic Search”. In: *Artificial intelligence* 42.2-3 (1990), pp. 189–211.
- [56] S. Koenig and M. Likhachev. “Real-Time Adaptive A*”. In: *Proceedings of the fifth international joint conference on Autonomous agents and multiagent systems*. 2006, pp. 281–288.
- [57] S. Koenig and X. Sun. “Comparing real-time and incremental heuristic search for real-time situated agents”. In: *Autonomous Agents and Multi-Agent Systems* 18 (2009), pp. 313–341.
- [58] C. Hernández, X. Sun, S. Koenig, and P. Meseguer. “Tree Adaptive A*”. In: *The 10th International Conference on Autonomous Agents and Multiagent Systems*. 2011, pp. 123–130.
- [59] T. G. Dietterich. “The MAXQ Method for Hierarchical Reinforcement Learning.” In: *ICML*. Vol. 98. Citeseer. 1998, pp. 118–126.
- [60] A. Y. Ng and S. J. Russell. “Algorithms for Inverse Reinforcement Learning”. In: *Proceedings of the Seventeenth International Conference on Machine Learning*. 2000, pp. 663–670.
- [61] P. Abbeel and A. Y. Ng. “Apprenticeship learning via inverse reinforcement learning”. In: *Proceedings of the twenty-first international conference on Machine learning*. 2004, p. 1.
- [62] D. S. Brown and S. Niekum. “Deep Bayesian Reward Learning from Preferences”. In: *arXiv preprint arXiv:1912.04472* (2019).
- [63] B. Michini and J. P. How. “Improving the efficiency of Bayesian inverse reinforcement learning”. In: *2012 IEEE International Conference on Robotics and Automation*. IEEE. 2012, pp. 3651–3656.
- [64] N. D. Goodman, C. L. Baker, E. B. Bonawitz, V. K. Mansinghka, A. Gopnik, H. Wellman, L. Schulz, and J. B. Tenenbaum. “Intuitive theories of mind: A rational approach to false belief”. In: *Proceedings of the twenty-eighth annual conference of the cognitive science society*. Vol. 6. Cognitive Science Society Vancouver. 2006.

- [65] C. L. Baker, J. B. Tenenbaum, and R. R. Saxe. “Goal inference as inverse planning”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 29. 2007.
- [66] J. Jara-Ettinger, H. Gweon, L. E. Schulz, and J. B. Tenenbaum. “The naive utility calculus: Computational principles underlying commonsense psychology”. In: *Trends in cognitive sciences* 20.8 (2016), pp. 589–604.
- [67] M. F. Cusumano-Towner, A. Radul, D. Wingate, and V. K. Mansinghka. “Probabilistic programs for inferring the goals of autonomous agents”. In: *arXiv preprint arXiv:1704.04977* (2017).
- [68] I. R. Seaman, J.-W. van de Meent, and D. Wingate. “Nested Reasoning About Autonomous Agents Using Probabilistic Programs”. In: *arXiv* (2018), arXiv–1812.
- [69] M. Bratman. *Intention, plans, and practical reason*. Vol. 10. Harvard University Press Cambridge, MA, 1987.
- [70] M. Ramírez and H. Geffner. “Plan recognition as planning”. In: *Twenty-First International Joint Conference on Artificial Intelligence*. 2009.
- [71] S. Sohrabi, A. V. Riabov, and O. Udrea. “Plan Recognition as Planning Revisited.” In: *IJCAI*. 2016, pp. 3258–3264.
- [72] D. Höller, G. Behnke, P. Bercher, and S. Biundo. “Plan and goal recognition as HTN planning”. In: *2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE. 2018, pp. 466–473.
- [73] G. A. Kaminka, M. Vered, and N. Agmon. “Plan recognition in continuous domains”. In: *Thirty-Second AAAI Conference on Artificial Intelligence*. 2018.
- [74] M. Vered, R. F. Pereira, M. C. Magnaguagno, F. Meneguzzi, and G. A. Kaminka. “Online goal recognition as reasoning over landmarks”. In: *Workshops at the Thirty-Second AAAI Conference on Artificial Intelligence*. 2018.
- [75] P. Thaker, J. B. Tenenbaum, and S. J. Gershman. “Online learning of symbolic concepts”. In: *Journal of Mathematical Psychology* 77 (2017), pp. 10–20.
- [76] R. Self, M. Harlan, and R. Kamalapurkar. “Online inverse reinforcement learning for nonlinear systems”. In: *2019 IEEE Conference on Control Technology and Applications (CCTA)*. IEEE. 2019, pp. 296–301.
- [77] N. Rhinehart and K. Kitani. “First-person activity forecasting from video with online inverse reinforcement learning”. In: *IEEE transactions on pattern analysis and machine intelligence* (2018).
- [78] O. Evans and N. D. Goodman. “Learning the preferences of bounded agents”. In: *NIPS Workshop on Bounded Optimality*. Vol. 6. 2015.
- [79] O. Evans, A. Stuhlmüller, and N. Goodman. “Learning the preferences of ignorant, inconsistent agents”. In: *Thirtieth AAAI Conference on Artificial Intelligence*. 2016.
- [80] R. Shah, N. Gundotra, P. Abbeel, and A. D. Dragan. “On the feasibility of learning, rather than assuming, human biases for reward inference”. In: *arXiv preprint arXiv:1906.09624* (2019).

- [81] S. Armstrong and S. Mindermann. “Occam’s razor is insufficient to infer the preferences of irrational agents”. In: *Advances in Neural Information Processing Systems*. 2018, pp. 5598–5609.
- [82] T. L. Griffiths, F. Lieder, and N. D. Goodman. “Rational use of cognitive resources: Levels of analysis between the computational and the algorithmic”. In: *Topics in cognitive science* 7.2 (2015), pp. 217–229.
- [83] M. K. Ho, D. Abel, J. D. Cohen, M. L. Littman, and T. L. Griffiths. “The Efficiency of Human Cognition Reflects Planned Information Processing”. In: *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*. 2020.
- [84] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. “Strips planning in infinite domains”. In: *arXiv preprint arXiv:1701.00287* (2017).
- [85] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. “PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning”. In: *Proceedings of the International Conference on Automated Planning and Scheduling*. Vol. 30. 2020, pp. 440–448.
- [86] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. “Mastering the game of Go with deep neural networks and tree search”. In: *nature* 529.7587 (2016), p. 484.
- [87] H. Wimmer and J. Perner. “Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children’s understanding of deception”. In: *Cognition* 13.1 (1983), pp. 103–128.
- [88] J. Phillips, A. Morris, and F. Cushman. “How we know what not to think”. In: *Trends in cognitive sciences* 23.12 (2019), pp. 1026–1040.
- [89] F. Callaway, F. Lieder, P. Das, S. Gul, P. M. Krueger, and T. Griffiths. “A resource-rational analysis of human planning.” In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. 2018.
- [90] C. A. Schult and H. M. Wellman. “Explaining human movements and actions: Children’s understanding of the limits of psychological explanation”. In: *Cognition* 62.3 (1997), pp. 291–324.
- [91] H. M. Wellman and K. H. Lagattuta. “Theory of mind for learning and teaching: The nature and role of explanation”. In: *Cognitive development* 19.4 (2004), pp. 479–497.
- [92] L. Young, F. Cushman, M. Hauser, and R. Saxe. “The neural basis of the interaction between theory of mind and moral judgment”. In: *Proceedings of the National Academy of Sciences* 104.20 (2007), pp. 8235–8240.
- [93] F. Cushman. “Crime and punishment: Distinguishing the roles of causal and intentional analyses in moral judgment”. In: *Cognition* 108.2 (2008), pp. 353–380.
- [94] S. Liu, T. D. Ullman, J. B. Tenenbaum, and E. S. Spelke. “Ten-month-old infants infer the value of goals from the costs of actions”. In: *Science* 358.6366 (2017), pp. 1038–1041.
- [95] M. Kryven, T. Ullman, W. Cowan, and J. Tenenbaum. “Outcome or Strategy? A Bayesian Model of Intelligence Attribution.” In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. 2016.

- [96] T. Otter, J. Johnson, J. Rieskamp, G. M. Allenby, J. D. Brazell, A. Diederich, J. W. Hutchinson, S. MacEachern, S. Ruan, and J. Townsend. "Sequential sampling models of choice: Some recent advances". In: *Marketing letters* 19.3 (2008), pp. 255–267.
- [97] A. Bobu, D. R. Scobee, J. F. Fisac, S. S. Sastry, and A. D. Dragan. "Less is more: Rethinking probabilistic models of human behavior". In: *Proceedings of the 2020 ACM/IEEE International Conference on Human-Robot Interaction*. 2020, pp. 429–437.
- [98] M. E. Bratman, D. J. Israel, and M. E. Pollack. "Plans and resource-bounded practical reasoning". In: *Computational intelligence* 4.3 (1988), pp. 349–355.
- [99] H. Geffner. "Heuristics, planning and cognition". In: *Heuristics, Probability and Causality. A Tribute to Judea Pearl*. College Publications (2010).
- [100] J. Lee, J. Fong, B. C. Kok, H. Soh, Y. Xie, F. Zhou, H. Soh, F. Gu, W. Sng, T. Taunyazov, et al. "Getting to Know One Another: Calibrating Intent, Capabilities and Trust for Human-Robot Collaboration". In: *Proceedings of Robotics: Science and Systems*. Vol. 9. SAGE Publications Sage UK: London, England. 2020, pp. 4235–4245.
- [101] S. Reddy, A. D. Dragan, and S. Levine. "Where do you think you're going? inferring beliefs about dynamics from behavior". In: *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. 2018, pp. 1461–1472.
- [102] J. Hoffmann. "FF: The Fast-Forward planning system". In: *AI magazine* 22.3 (2001), pp. 57–57.
- [103] G. B. Moskowitz and I. Olcaysoy Okten. "Spontaneous goal inference (SGI)". In: *Social and Personality Psychology Compass* 10.1 (2016), pp. 64–80.
- [104] F. Heider and M. Simmel. "An experimental study of apparent behavior". In: *The American Journal of Psychology* 57.2 (1944), pp. 243–259.
- [105] A. C. Graesser, M. Singer, and T. Trabasso. "Constructing inferences during narrative text comprehension." In: *Psychological review* 101.3 (1994), p. 371.
- [106] R. R. Hassin, H. Aarts, and M. J. Ferguson. "Automatic goal inferences". In: *Journal of Experimental Social Psychology* 41.2 (2005), pp. 129–140.
- [107] F. Van Overwalle, M. Van Duynslaeger, D. Coomans, and B. Timmermans. "Spontaneous goal inferences are often inferred faster than spontaneous trait inferences". In: *Journal of Experimental Social Psychology* 48.1 (2012), pp. 13–18.
- [108] G. Gergely and G. Csibra. "Teleological reasoning in infancy: The naive theory of rational action". In: *Trends in cognitive sciences* 7.7 (2003), pp. 287–292.
- [109] J. Jara-Ettinger, H. Gweon, J. B. Tenenbaum, and L. E. Schulz. "Children's understanding of the costs and rewards underlying rational action". In: *Cognition* 140 (2015), pp. 14–23.
- [110] T. Ullman, C. Baker, O. Macindoe, O. Evans, N. Goodman, and J. Tenenbaum. "Help or hinder: Bayesian models of social goal inference". In: *Advances in Neural Information Processing Systems* 22 (2009).
- [111] M. Kleiman-Weiner, M. K. Ho, J. L. Austerweil, M. L. Littman, and J. B. Tenenbaum. "Coordinate to cooperate or compete: abstract goals and joint intentions in social interaction". In: *COGSCI*. 2016.

- [112] M. Vered, G. A. Kaminka, and S. Biham. “Online goal recognition through mirroring: Humans and agents”. In: *Annual Conference on Advances in Cognitive Systems 2016*. Cognitive Systems Foundation. 2016.
- [113] J. Kwisthout and I. Van Rooij. “Bridging the gap between theory and practice of approximate Bayesian inference”. In: *Cognitive Systems Research* 24 (2013), pp. 2–8.
- [114] M. Blokpoel, J. Kwisthout, T. P. van der Weide, T. Wareham, and I. van Rooij. “A computational-level explanation of the speed of goal inference”. In: *Journal of Mathematical Psychology* 57.3-4 (2013), pp. 117–133.
- [115] T. L. Griffiths and J. B. Tenenbaum. “Optimal predictions in everyday cognition”. In: *Psychological science* 17.9 (2006), pp. 767–773.
- [116] L. Schulz. “Finding new facts; thinking new thoughts”. In: *Advances in child development and behavior* 43 (2012), pp. 269–294.
- [117] D. Guthrie and M. Hepple. “Storing the web in memory: Space efficient language models with constant time retrieval”. In: *Proceedings of the 2010 conference on Empirical Methods in Natural Language Processing*. 2010, pp. 262–272.
- [118] N. Daw and A. Courville. “The pigeon as particle filter”. In: *Advances in Neural Information Processing Systems* 20 (2008), pp. 369–376.
- [119] E. Vul, G. Alvarez, J. Tenenbaum, and M. Black. “Explaining human multiple object tracking as resource-constrained approximate inference in a dynamic probabilistic model”. In: *Advances in Neural Information Processing Systems* 22 (2009).
- [120] P. Del Moral, A. Doucet, and A. Jasra. “Sequential Monte Carlo samplers”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 68.3 (2006), pp. 411–436.
- [121] N. Chopin. “A sequential particle filter method for static models”. In: *Biometrika* 89.3 (2002), pp. 539–552.
- [122] A. K. Lew, G. Matheos, T. Zhi-Xuan, M. Ghavamizadeh, N. Gothoskar, S. Russell, and V. K. Mansinghka. “SMCP3: Sequential Monte Carlo with Probabilistic Program Proposals”. In: *AISTATS*. 2023.
- [123] A. Morris, J. Phillips, K. Huang, and F. Cushman. “Generating options and choosing between them depend on distinct forms of value representation”. In: *Psychological science* 32.11 (2021), pp. 1731–1746.
- [124] B. Beronov, C. Weilbach, F. Wood, and T. Campbell. “Sequential Core-Set Monte Carlo”. In: *Uncertainty in Artificial Intelligence*. PMLR. 2021, pp. 2165–2175.
- [125] K. Chandra, T. Chen, T.-M. Li, J. Ragan-Kelley, and J. B. Tenenbaum. “Inferring the Future by Imagining the Past”. In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.
- [126] R. Speer. *wordfreq: a Python library for looking up the frequencies of words in many languages*. 2016.
- [127] A. Beale. *12Dicts Package*. 2016.

- [128] F. Lieder and T. L. Griffiths. “Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources”. In: *Behavioral and Brain Sciences* 43 (2020), e1.
- [129] T. Zhi-Xuan, L. Ying, V. Mansinghka, and J. B. Tenenbaum. “Pragmatic Instruction Following and Goal Assistance via Cooperative Language Guided Inverse Plan Search”. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multi-Agent Systems*. 2024.
- [130] P. Del Moral, A. Doucet, and A. Jasra. “On adaptive resampling strategies for sequential Monte Carlo methods”. In: *Bernoulli* 18.1 (2012), pp. 252–278.
- [131] V. Elvira, J. Míguez, and P. M. Djurić. “Adapting the number of particles in sequential Monte Carlo methods through an online scheme for convergence assessment”. In: *IEEE Transactions on Signal Processing* 65.7 (2016), pp. 1781–1794.
- [132] P. West, C. Bhagavatula, J. Hessel, J. Hwang, L. Jiang, R. Le Bras, X. Lu, S. Welleck, and Y. Choi. “Symbolic Knowledge Distillation: from General Language Models to Commonsense Models”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Seattle, United States: Association for Computational Linguistics, 2022, pp. 4602–4625.
- [133] A. L. Blum and M. L. Furst. “Fast planning through planning graph analysis”. In: *Artificial Intelligence* 90.1-2 (1997), pp. 281–300.
- [134] C. Hernández and P. Meseguer. “Improving LRTA*(k)”. In: *Proceedings of the 20th International Joint Conference on Artificial Intelligence*. 2007, pp. 2312–2317.
- [135] A. K. Lew, M. Cusumano-Towner, and V. K. Mansinghka. “Recursive Monte Carlo and variational inference with auxiliary variables”. In: *Uncertainty in Artificial Intelligence*. PMLR. 2022, pp. 1096–1106.
- [136] A. K. Lew, M. Ghavamizadeh, M. C. Rinard, and V. K. Mansinghka. “Probabilistic programming with stochastic probabilities”. In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023), pp. 1708–1732.
- [137] H. P. Grice. “Logic and Conversation”. In: *Speech Acts*. Brill, 1975, pp. 41–58.
- [138] N. D. Goodman and M. C. Frank. “Pragmatic language interpretation as probabilistic inference”. In: *Trends in cognitive sciences* 20.11 (2016), pp. 818–829.
- [139] D. Fried, J. Andreas, and D. Klein. “Unified Pragmatic Models for Generating and Following Instructions”. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Ed. by M. Walker, H. Ji, and A. Stent. New Orleans, Louisiana: Association for Computational Linguistics, June 2018, pp. 1951–1963. DOI: [10.18653/v1/N18-1177](https://doi.org/10.18653/v1/N18-1177).
- [140] T. R. Sumers, M. K. Ho, T. L. Griffiths, and R. D. Hawkins. “Reconciling truthfulness and relevance as epistemic and decision-theoretic utility.” In: *Psychological Review* (2023).
- [141] N. Tang, S. Stacy, M. Zhao, G. Marquez, and T. Gao. “Bootstrapping an Imagined We for Cooperation.” In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. 2020.

- [142] S. Stacy, C. Li, M. Zhao, Y. Yun, Q. Zhao, M. Kleiman-Weiner, and T. Gao. “Modeling communication to coordinate perspectives in cooperation”. In: *arXiv preprint arXiv:2106.02164* (2021).
- [143] S. A. Wu, R. E. Wang, J. A. Evans, J. B. Tenenbaum, D. C. Parkes, and M. Kleiman-Weiner. “Too many cooks: Bayesian inference for coordinating multi-agent collaboration”. In: *Topics in Cognitive Science* 13.2 (2021), pp. 414–432.
- [144] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba. “Virtualhome: Simulating household activities via programs”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, pp. 8494–8502.
- [145] X. Puig, T. Shu, S. Li, Z. Wang, Y.-H. Liao, J. B. Tenenbaum, S. Fidler, and A. Torralba. “Watch-and-help: A challenge for social perception and human-ai collaboration”. In: *arXiv preprint arXiv:2010.09890* (2020).
- [146] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. “Do as I can, not as I say: Grounding language in robotic affordances”. In: *arXiv preprint arXiv:2204.01691* (2022).
- [147] Y. Wan, J. Mao, and J. Tenenbaum. “HandMeThat: Human-Robot Communication in Physical and Social Environments”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 12014–12026.
- [148] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. “Progprompt: Generating situated robot task plans using large language models”. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2023, pp. 11523–11530.
- [149] L. M. Schulze Buschoff, E. Akata, M. Bethge, and E. Schulz. “Visual cognition in multimodal large language models”. In: *Nature Machine Intelligence* (2025), pp. 1–11.
- [150] C. Jin, Y. Wu, J. Cao, J. Xiang, Y.-L. Kuo, Z. Hu, T. Ullman, A. Torralba, J. B. Tenenbaum, and T. Shu. “MMToM-QA: Multimodal Theory of Mind Question Answering”. In: *arXiv preprint arXiv:2401.08743* (2024).
- [151] S. Saisubramanian, K. H. Wray, L. Pineda, and S. Zilberstein. “Planning in stochastic environments with goal uncertainty”. In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2019, pp. 1649–1654.
- [152] L. S. Shapley. “Stochastic games”. In: *Proceedings of the national academy of sciences* 39.10 (1953), pp. 1095–1100.
- [153] B. Bonet and H. Geffner. “Solving POMDPs: RTDP-Bel vs. Point-based Algorithms.” In: *IJCAI*. Pasadena CA. 2009, pp. 1641–1646.
- [154] S. Milli and A. D. Dragan. “Literal or pedagogic human? analyzing human model misspecification in objective learning”. In: *Uncertainty in artificial intelligence*. PMLR. 2020, pp. 925–934.
- [155] D. K. Lewis. *Convention: A Philosophical Study*. Harvard University Press, 1969. ISBN: 9780674170254.

- [156] P. Doshi and P. J. Gmytrasiewicz. “Monte Carlo sampling methods for approximating interactive POMDPs”. In: *Journal of Artificial Intelligence Research* 34 (2009), pp. 297–337.
- [157] T.-H. Nguyen, D. Hsu, W.-S. Lee, T.-Y. Leong, L. Kaelbling, T. Lozano-Perez, and A. Grant. “Capir: Collaborative action planning with intention recognition”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*. Vol. 7. 1. 2011, pp. 61–66.
- [158] O. Macindoe, L. P. Kaelbling, and T. Lozano-Pérez. “POMCoP: Belief Space Planning for Sidekicks in Cooperative Games.” In: *AIIDE*. 2012.
- [159] P. R. Cohen. “On Knowing What to Say: Planning Speech Acts”. PhD thesis. University of Toronto, 1978.
- [160] P. R. Cohen and C. R. Perrault. “Elements of a plan-based theory of speech acts”. In: *Cognitive science* 3.3 (1979), pp. 177–212.
- [161] C. R. Perrault and J. F. Allen. “A plan-based analysis of indirect speech acts”. In: *Computational Linguistics* 6.3-4 (1980), pp. 167–182.
- [162] C. Boutilier. “Planning, learning and coordination in multiagent decision processes”. In: *TARK*. Vol. 96. Citeseer. 1996, pp. 195–210.
- [163] M. Shum, M. Kleiman-Weiner, M. L. Littman, and J. B. Tenenbaum. “Theory of minds: Understanding behavior in groups through inverse planning”. In: *Proceedings of the AAAI conference on Artificial Intelligence*. Vol. 33. 01. 2019, pp. 6163–6170.
- [164] A. Egan. “Some counterexamples to causal decision theory”. In: *The Philosophical Review* 116.1 (2007), pp. 93–114.
- [165] D. A. Herrmann. *Naturalizing Decision Theory*. University of California, Irvine, 2023.
- [166] J. Pearl. “The do-calculus revisited”. In: *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*. 2012, pp. 3–11.
- [167] A. Shah, S. Li, and J. Shah. “Planning with uncertain specifications (puns)”. In: *IEEE Robotics and Automation Letters* 5.2 (2020), pp. 3414–3421.
- [168] M. L. Littman, A. R. Cassandra, and L. P. Kaelbling. “Learning policies for partially observable environments: Scaling up”. In: *Machine Learning Proceedings 1995*. Elsevier, 1995, pp. 362–370.
- [169] M. Hauskrecht. “Value-function approximations for partially observable Markov decision processes”. In: *Journal of artificial intelligence research* 13 (2000), pp. 33–94.
- [170] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [171] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Al-tenschmidt, S. Altman, S. Anadkat, et al. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [172] J. L. Austin. *How to do things with words*. Vol. 88. Oxford University Press, 1975.

- [173] P. Gmytrasiewicz. “How to do things with words: A Bayesian approach”. In: *Journal of Artificial Intelligence Research* 68 (2020), pp. 753–776.
- [174] L. Wong, G. Grand, A. K. Lew, N. D. Goodman, V. K. Mansinghka, J. Andreas, and J. B. Tenenbaum. “From word models to world models: Translating from natural language to the probabilistic language of thought”. In: *arXiv preprint arXiv:2306.12672* (2023).
- [175] N. D. Goodman and D. Lassiter. “Probabilistic semantics and pragmatics: Uncertainty in language and thought”. In: *The Handbook of Contemporary Semantic Theory* (2015), pp. 655–686.
- [176] J. Jara-Ettinger. “Theory of mind as inverse reinforcement learning”. In: *Current Opinion in Behavioral Sciences* 29 (2019), pp. 105–110.
- [177] E. C. Williams, N. Gopalan, M. Rhee, and S. Tellex. “Learning to parse natural language to grounded reward functions with weak supervision”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2018, pp. 4430–4436.
- [178] H.-Y. Tung, A. W. Harley, L.-K. Huang, and K. Fragkiadaki. “Reward learning from narrated demonstrations”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, pp. 7004–7013.
- [179] J. Fu, A. Korattikara, S. Levine, and S. Guadarrama. “From language to goals: Inverse reinforcement learning for vision-based instruction following”. In: *arXiv preprint arXiv:1902.07742* (2019).
- [180] H. J. Jeon, S. Milli, and A. Dragan. “Reward-rational (implicit) choice: A unifying formalism for reward learning”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 4415–4426.
- [181] C. F. Camerer, T.-H. Ho, and J.-K. Chong. “A cognitive hierarchy model of games”. In: *The Quarterly Journal of Economics* 119.3 (2004), pp. 861–898.
- [182] N. Tang, S. Gong, M. Zhao, C. Gu, J. Zhou, M. Shen, and T. Gao. “Exploring an Imagined “We” in human collective hunting: Joint commitment within shared intentionality”. In: *Proceedings of the annual meeting of the cognitive science society*. Vol. 44. 44. 2022.
- [183] S. Russell. “Human-Compatible Artificial Intelligence”. In: *Human-like machine intelligence* (2021), pp. 3–23.
- [184] C. Laidlaw, E. Bronstein, T. Guo, D. Feng, L. Berglund, J. Svegliato, S. Russell, and A. Dragan. “AssistanceZero: Scalably Solving Assistance Games”. In: *arXiv preprint arXiv:2504.07091* (2025).
- [185] S. Tellex, T. Kollar, S. Dickerson, M. Walter, A. Banerjee, S. Teller, and N. Roy. “Understanding natural language commands for robotic navigation and mobile manipulation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 25. 1. 2011, pp. 1507–1514.
- [186] S. Squire, S. Tellex, D. Arumugam, and L. Yang. “Grounding English commands to reward functions”. In: *Robotics: Science and Systems*. 2015.
- [187] D. K. Misra, J. Sung, K. Lee, and A. Saxena. “Tell me Dave: Context-sensitive grounding of natural language to manipulation instructions”. In: *The International Journal of Robotics Research* 35.1-3 (2016), pp. 281–300.

- [188] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh. “Reward Design with Language Models”. In: *The Eleventh International Conference on Learning Representations*. 2023.
- [189] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, et al. “Language to Rewards for Robotic Skill Synthesis”. In: *arXiv preprint arXiv:2306.08647* (2023).
- [190] J. X. Liu, Z. Yang, B. Schornstein, S. Liang, I. Idrees, S. Tellex, and A. Shah. “Lang2LTL: Translating Natural Language Commands to Temporal Specification with Large Language Models”. In: *Workshop on Language and Robotics at CoRL 2022*. 2022.
- [191] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, and A. B. Murthy. “LLMs Cant Plan, But Can Help Planning in LLM-Modulo Frameworks”. In: *Forty-First International Conference on Machine Learning*. 2024.
- [192] H. Kim, M. Sclar, T. Zhi-Xuan, L. Ying, S. Levine, Y. Liu, J. B. Tenenbaum, and Y. Choi. “Hypothesis-Driven Theory-of-Mind Reasoning for Large Language Models”. In: *arXiv preprint arXiv:2502.11881* (2025).
- [193] A. K. Lew, M. H. Tessler, V. K. Mansinghka, and J. B. Tenenbaum. “Leveraging Unstructured Statistical Knowledge in a Probabilistic Language of Thought”. In: *Proceedings of the Annual Conference of the Cognitive Science Society*. 2020.
- [194] A. K. Lew, T. Zhi-Xuan, G. Grand, and V. K. Mansinghka. “Sequential Monte Carlo Steering of Large Language Models using Probabilistic Programs”. In: *arXiv preprint arXiv:2306.03081* (2023).
- [195] J. Loula et al. “Syntactic and Semantic Control of Large Language Models via Sequential Monte Carlo”. In: *The Thirteenth International Conference on Learning Representations*. 2025.
- [196] B. Lipkin, B. LeBrun, J. H. Vigly, J. Loula, D. R. MacIver, L. Du, J. Eisner, R. Cotterell, V. Mansinghka, T. J. O’Donnell, et al. “Fast Controlled Generation from Language Models with Adaptive Weighted Rejection Sampling”. In: *arXiv preprint arXiv:2504.05410* (2025).
- [197] B. T. Willard and R. Louf. “Efficient Guided Generation for LLMs”. In: *arXiv preprint arXiv:2307.09702* (2023).
- [198] R. Shin, C. H. Lin, S. Thomson, C. Chen, S. Roy, E. A. Platanios, A. Pauls, D. Klein, J. Eisner, and B. Van Durme. “Constrained language models yield few-shot semantic parsers”. In: *arXiv preprint arXiv:2104.08768* (2021).
- [199] Z. Sunberg and M. Kochenderfer. “Online algorithms for POMDPs with continuous state, action, and observation spaces”. In: *Proceedings of the International Conference on Automated Planning and Scheduling*. Vol. 28. 2018, pp. 259–263.
- [200] M. Held and R. M. Karp. “The traveling-salesman problem and minimum spanning trees: Part II”. In: *Mathematical programming* 1.1 (1971), pp. 6–25.
- [201] D. Aversa, S. Sardina, and S. Vassos. “Pruning and preprocessing methods for inventory-aware pathfinding”. In: *2016 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE. 2016, pp. 1–8.
- [202] C. Hernández, T. Uras, S. Koenig, J. A. Baier, X. Sun, and P. Meseguer. “Reusing cost-minimal paths for goal-directed navigation in partially known terrains”. In: *Autonomous Agents and Multi-Agent Systems* 29 (2015), pp. 850–895.

- [203] J. Choi and K.-E. Kim. “Hierarchical bayesian inverse reinforcement learning”. In: *IEEE transactions on cybernetics* 45.4 (2014), pp. 793–805.
- [204] K. Murphy and S. Russell. “Rao-Blackwellised particle filtering for dynamic Bayesian networks”. In: *Sequential Monte Carlo methods in practice*. Springer, 2001, pp. 499–515.
- [205] R. Giora. “On the graded salience hypothesis”. In: *Intercultural Pragmatics* 1.1 (2004), pp. 113–123.
- [206] I. Kecskes et al. “The role of salience in processing pragmatic units”. In: *Acta Linguistica Hungarica (Since 2017 Acta Linguistica Academica)* 51.3-4 (2004), pp. 309–324.
- [207] A. K. Hopkins, A. Renda, and M. Carbin. “Can LLMs Generate Random Numbers? Evaluating LLM Sampling in Controlled Domains”. In: *ICML 2023 Workshop: Sampling and Optimization in Discrete Space*. 2023.
- [208] R. Kirk, I. Mediratta, C. Nalmpantis, J. Luketina, E. Hambro, E. Grefenstette, and R. Raileanu. “Understanding the Effects of RLHF on LLM Generalisation and Diversity”. In: *The Twelfth International Conference on Learning Representations*. 2024.
- [209] F. Yates. “Systematic sampling”. In: *Philosophical Transactions of the Royal Society of London. Series A, Mathematical and Physical Sciences* 241.834 (1948), pp. 345–377.
- [210] M. Sclar, S. Kumar, P. West, A. Suhr, Y. Choi, and Y. Tsvetkov. “Minding Language Models’ (Lack of) Theory of Mind: A Plug-and-Play Multi-Character Belief Tracker”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by A. Rogers, J. Boyd-Graber, and N. Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 13960–13980. doi: [10.18653/v1/2023.acl-long.780](https://doi.org/10.18653/v1/2023.acl-long.780).
- [211] W. R. Thompson. “On the theory of apportionment”. In: *American Journal of Mathematics* 57.2 (1935), pp. 450–456.
- [212] S. Stacy. “The Imagined We: Shared Bayesian Theory of Mind for Modeling Communication”. PhD thesis. Los Angeles, 2022.
- [213] P. Shafto, N. D. Goodman, and T. L. Griffiths. “A rational account of pedagogical reasoning: Teaching by, and learning from, examples”. In: *Cognitive psychology* 71 (2014), pp. 55–89.
- [214] S. Stacy, S. Gong, A. Parab, M. Zhao, K. Jiang, and T. Gao. “A Bayesian theory of mind approach to modeling cooperation and communication”. In: *Wiley Interdisciplinary Reviews: Computational Statistics* 16.1 (2024), e1631.
- [215] L. Ying, K. Jha, S. Aarya, J. B. Tenenbaum, A. Torralba, and T. Shu. “GOMA: Proactive embodied cooperative communication via goal-oriented mental alignment”. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2024, pp. 7099–7106.
- [216] C. Bicchieri, R. Muldoon, and A. Sontuoso. “Social Norms”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta. Winter 2018. Metaphysics Research Lab, Stanford University, 2018.
- [217] G. Brennan, L. Eriksson, R. E. Goodin, and N. Southwood. *Explaining norms*. OUP UK, 2013.
- [218] E. Ostrom. *Governing the commons: The evolution of institutions for collective action*. Cambridge university press, 1990.

- [219] G. K. Hadfield and B. R. Weingast. “What Is Law? A Coordination Model of the Characteristics of Legal Order”. In: *Journal of Legal Analysis* 4.2 (Dec. 2012), pp. 471–514. ISSN: 2161-7201. DOI: [10.1093/jla/las008](https://doi.org/10.1093/jla/las008).
- [220] G. K. Hadfield. *Rules for a flat world: Why humans invented law and how to reinvent it for a complex global economy*. Oxford University Press, 2017.
- [221] M. L. Littman. “Markov games as a framework for multi-agent reinforcement learning”. In: *Machine Learning Proceedings 1994*. Ed. by W. W. Cohen and H. Hirsh. San Francisco (CA): Morgan Kaufmann, Jan. 1994, pp. 157–163. ISBN: 978-1-55860-335-6. DOI: [10.1016/B978-1-55860-335-6.50027-1](https://doi.org/10.1016/B978-1-55860-335-6.50027-1).
- [222] E. Anderson. *Value in Ethics and Economics*. Harvard University Press, 1995.
- [223] R. Chang. *Incommensurability, incomparability, and practical reason*. Harvard University Press, 1997.
- [224] S. Bade. “Nash equilibrium in games with incomplete preferences”. In: *Economic Theory* 26 (2005), pp. 309–332.
- [225] Ö. Evren. “On the existence of expected multi-utility representations”. In: *Economic Theory* 35.3 (2008), pp. 575–592.
- [226] G. H. Von Wright. “On the logic of norms and actions”. In: *New studies in deontic logic: Norms, actions, and the foundations of ethics*. Springer, 1981, pp. 3–35.
- [227] J. Dubra, F. Maccheroni, and E. A. Ok. “Expected utility theory without the completeness axiom”. In: *Journal of Economic Theory* 115.1 (2004), pp. 118–133.
- [228] D. Blackwell. “An analog of the minimax theorem for vector payoffs”. In: *Pacific Journal of Mathematics* 6.1 (1956), pp. 1–8.
- [229] L. S. Shapley and F. D. Rigby. “Equilibrium points in games with vector payoffs”. In: *Naval Research Logistics Quarterly* 6.1 (1959), pp. 57–61.
- [230] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, et al. “A practical guide to multi-objective reinforcement learning and planning”. In: *Autonomous Agents and Multi-Agent Systems* 36.1 (2022), p. 26.
- [231] P. Vamplew, B. J. Smith, J. Kallstrom, G. Ramos, R. Radulescu, D. M. Roijers, C. F. Hayes, F. Heintz, P. Mannion, P. J. Libin, et al. “Scalar reward is not enough: a response to Silver, Singh, Precup and Sutton (2021)”. In: *Autonomous Agents And Multi-Agent Systems* 36.2 (2022).
- [232] S. V. Albrecht and S. Ramamoorthy. “A game-theoretic model and best-response learning method for ad hoc coordination in multiagent systems”. In: *Proceedings of the 2013 International Conference on Autonomous Agents and Multi-Agent Systems*. 2013, pp. 1155–1156.
- [233] S. V. Albrecht, J. W. Crandall, and S. Ramamoorthy. “Belief and truth in hypothesised behaviours”. In: *Artificial Intelligence* 235 (2016), pp. 63–94.
- [234] E. Kalai and E. Lehrer. “Subjective games and equilibria”. In: *Games and Economic Behavior* 8.1 (1995), pp. 123–163.

- [235] P. Vanderschraaf. “Convention as correlated equilibrium”. In: *Erkenntnis* 42.1 (1995), pp. 65–87.
- [236] H. Gintis. “The hitchhiker’s guide to altruism: Gene-culture coevolution, and the internalization of norms”. In: *Journal of theoretical biology* 220.4 (2003), pp. 407–418.
- [237] A. Sen. “Interpersonal aggregation and partial comparability”. In: *Econometrica: Journal of the Econometric Society* (1970), pp. 393–409.
- [238] R. D. McKelvey and T. R. Palfrey. “Quantal response equilibria for extensive form games”. In: *Experimental economics* 1 (1998), pp. 9–41.
- [239] J. Černý, B. An, and A. N. Zhang. “Quantal Correlated Equilibrium in Normal Form Games”. In: *Proceedings of the 23rd ACM Conference on Economics and Computation*. 2022, pp. 210–239.
- [240] P. J. Gmytrasiewicz and P. Doshi. “A Framework for Sequential Planning in Multi-Agent Settings”. en. In: *Journal of Artificial Intelligence Research* 24 (July 2005), pp. 49–79. ISSN: 1076-9757. DOI: [10.1613/jair.1579](https://doi.org/10.1613/jair.1579).
- [241] D. Antos and A. Pfeffer. “Representing Bayesian games without a common prior.” In: *AAMAS*. 2010, pp. 1457–1458.
- [242] E. Kalai and E. Lehrer. “Rational learning leads to Nash equilibrium”. In: *Econometrica: Journal of the Econometric Society* (1993), pp. 1019–1045.
- [243] D. Fudenberg and D. K. Levine. “Self-confirming equilibrium”. In: *Econometrica: Journal of the Econometric Society* (1993), pp. 523–545.
- [244] R. J. Aumann. “Correlated Equilibrium as an Expression of Bayesian Rationality”. In: *Econometrica* 55.1 (1987). Publisher: [Wiley, Econometric Society], pp. 1–18. ISSN: 0012-9682. DOI: [10.2307/1911154](https://doi.org/10.2307/1911154).
- [245] K. Paprzycka. “Normative expectations, intentions, and beliefs”. In: *Southern Journal of Philosophy* 37.4 (1999), pp. 629–652.
- [246] G. Boella and L. van der Torre. “An architecture of a normative system: counts-as conditionals, obligations and permissions”. In: *Proceedings of the fifth international joint conference on Autonomous agents and multiagent systems*. AAMAS ’06. New York, NY, USA: Association for Computing Machinery, May 2006, pp. 229–231. ISBN: 978-1-59593-303-4. DOI: [10.1145/1160633.1160671](https://doi.org/10.1145/1160633.1160671).
- [247] G. Boella, L. van der Torre, and H. Verhagen. “Introduction to normative multiagent systems”. en. In: *Computational & Mathematical Organization Theory* 12.2 (Oct. 2006), pp. 71–79. ISSN: 1572-9346. DOI: [10.1007/s10588-006-9537-7](https://doi.org/10.1007/s10588-006-9537-7).
- [248] D. Kasenberg and M. Scheutz. “Inverse norm conflict resolution”. In: *Proceedings of the First AAAI/ACM Conference on Artificial Intelligence, Ethics, and Society*. 2018.
- [249] T. Arnold and D. Kasenberg. “Value Alignment or Misalignment: What Will Keep Systems Accountable?” In: *AAAI Workshop on AI, Ethics, and Society*. 2017.
- [250] J. J. Camilleri. “Contracts and Computation”. en. Doctoral. Gothenburg: University of Gothenburg, 2017.

- [251] S. Cranefield and A. Dhiman. “Identifying Norms from Observation Using MCMC Sampling”. In: *IJCAI*. 2021, pp. 118–124. URL: <https://doi.org/10.24963/ijcai.2021/17>.
- [252] Z. X. Tan, J. Brawer, and B. Scassellati. “That’s Mine! Learning Ownership Relations and Norms for Robots”. In: *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*. (Accepted) 2019.
- [253] N. D. Goodman, J. B. Tenenbaum, J. Feldman, and T. L. Griffiths. “A rational analysis of rule-based concept learning”. In: *Cognitive science* 32.1 (2008), pp. 108–154.
- [254] S. Nichols, S. Kumar, T. Lopez, A. Ayars, and H.-Y. Chan. “Rational learners and moral rules”. In: *Mind & Language* 31.5 (2016), pp. 530–554.
- [255] J. Song, H. Ren, D. Sadigh, and S. Ermon. “Multi-agent generative adversarial imitation learning”. In: (2018).
- [256] J. J. Nay. *Law Informs Code: A Legal Informatics Approach to Aligning Artificial Intelligence with Humans*. arXiv:2209.13020 [cs]. Nov. 2022. DOI: [10.48550/arXiv.2209.13020](https://doi.org/10.48550/arXiv.2209.13020).
- [257] T. Zhi-Xuan. “What Should AI Owe To Us? Accountable and Aligned AI Systems via Contractualist AI Alignment”. en. In: (2022).
- [258] T. Zhi-Xuan, M. Carroll, M. Franklin, and H. Ashton. “Beyond Preferences in AI Alignment”. In: *Philosophical Studies* (2024), pp. 1–51.
- [259] L. Li, B. Britvan, and M. Tomasello. “Young children conform more to norms than to preferences”. In: *Plos one* 16.5 (2021), e0251228.
- [260] A. Morris and F. Cushman. “A common framework for theories of norm compliance”. In: *Social Philosophy and Policy* 35.1 (2018), pp. 101–127.
- [261] S. Muggleton. “Bayesian Inductive Logic Programming”. en. In: *Machine Learning Proceedings 1994*. Ed. by W. W. Cohen and H. Hirsh. San Francisco (CA): Morgan Kaufmann, Jan. 1994, pp. 371–379. ISBN: 978-1-55860-335-6. DOI: [10.1016/B978-1-55860-335-6.50052-0](https://doi.org/10.1016/B978-1-55860-335-6.50052-0).
- [262] J. Z. Leibo, E. Duéñez-Guzmán, A. S. Vezhnevets, J. P. Agapiou, P. Sunehag, R. Koster, J. Matyas, C. Beattie, I. Mordatch, and T. Graepel. *Scalable Evaluation of Multi-Agent Reinforcement Learning with Melting Pot*. arXiv:2107.06857 [cs]. July 2021.
- [263] J. P. Agapiou et al. *Melting Pot 2.0*. arXiv:2211.13746 [cs] version: 3. Jan. 2023.
- [264] M. A. Janssen, R. Holahan, A. Lee, and E. Ostrom. “Lab Experiments for the Study of Social-Ecological Systems”. In: *Science* 328.5978 (Apr. 2010). Publisher: American Association for the Advancement of Science, pp. 613–617. DOI: [10.1126/science.1183532](https://doi.org/10.1126/science.1183532).
- [265] J. Pérolat, J. Z. Leibo, V. Zambaldi, C. Beattie, K. Tuyls, and T. Graepel. “A multi-agent reinforcement learning model of common-pool resource appropriation”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017.
- [266] E. Hughes et al. “Inequity aversion improves cooperation in intertemporal social dilemmas”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018.

- [267] E. Vinitzky, R. Köster, J. P. Agapiou, E. A. Duéñez-Guzmán, A. S. Vezhnevets, and J. Z. Leibo. “A learning agent that acquires social norms from public sanctions in decentralized multi-agent settings”. en. In: *Collective Intelligence 2.2* (Apr. 2023). Publisher: SAGE Publications, p. 26339137231162025. ISSN: 2633-9137. DOI: [10.1177/26339137231162025](https://doi.org/10.1177/26339137231162025).
- [268] A. Lerer and A. Peysakhovich. *Learning Existing Social Conventions via Observationally Augmented Self-Play*. arXiv:1806.10071 [cs]. Mar. 2019.
- [269] N. Jaques, A. Lazaridou, E. Hughes, C. Gulcehre, P. Ortega, D. Strouse, J. Z. Leibo, and N. De Freitas. “Social influence as intrinsic motivation for multi-agent deep reinforcement learning”. In: *International conference on machine learning*. PMLR. 2019, pp. 3040–3049.
- [270] B. T. R. Savarimuthu, S. Cranefield, M. A. Purvis, and M. K. Purvis. “Identifying prohibition norms in agent societies”. en. In: *Artificial Intelligence and Law 21.1* (Mar. 2013), pp. 1–46. ISSN: 1572-8382. DOI: [10.1007/s10506-012-9126-7](https://doi.org/10.1007/s10506-012-9126-7).
- [271] T. Zhi-Xuan, J. Brawer, and B. Scassellati. *That’s Mine! Learning Ownership Relations and Norms for Robots*. en. arXiv:1812.02576 [cs]. Jan. 2019.
- [272] R. Köster, K. R. McKee, R. Everett, L. Weidinger, W. S. Isaac, E. Hughes, E. A. Duéñez-Guzmán, T. Graepel, M. Botvinick, and J. Z. Leibo. *Model-free conventions in multi-agent reinforcement learning with heterogeneous preferences*. arXiv:2010.09054 [cs]. Dec. 2020.
- [273] A. Ayars. “Can model-free reinforcement learning explain deontological moral judgments?” In: *Cognition* 150 (2016), pp. 232–242.
- [274] H. Rakoczy, F. Warneken, and M. Tomasello. “The sources of normativity: Young children’s awareness of the normative structure of games.” In: *Developmental Psychology* 44.3 (2008), pp. 875–881.
- [275] H. Rakoczy, N. Brosche, F. Warneken, and M. Tomasello. “Young children’s understanding of the context-relativity of normative rules in conventional games”. en. In: *British Journal of Developmental Psychology* 27.2 (2009), pp. 445–456. ISSN: 2044-835X. DOI: [10.1348/026151008X337752](https://doi.org/10.1348/026151008X337752).
- [276] M. F. Schmidt, H. Rakoczy, and M. Tomasello. “Young children attribute normativity to novel actions without pedagogy or normative language”. en. In: *Developmental Science* 14.3 (2011). _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-7687.2010.01000.x>, pp. 530–539. ISSN: 1467-7687. DOI: [10.1111/j.1467-7687.2010.01000.x](https://doi.org/10.1111/j.1467-7687.2010.01000.x).
- [277] R. Axelrod and W. D. Hamilton. “The evolution of cooperation”. In: *science* 211.4489 (1981), pp. 1390–1396.
- [278] K. Binmore and L. Samuelson. “An economist’s perspective on the evolution of norms”. In: *Journal of Institutional and Theoretical Economics (JITE)/Zeitschrift für die gesamte Staatswissenschaft* (1994), pp. 45–63.
- [279] N. D. Goodman, J. B. Tenenbaum, and T. Gerstenberg. *Concepts in a probabilistic language of thought*. Tech. rep. Center for Brains, Minds and Machines (CBMM), 2014.
- [280] S. T. Piantadosi and R. A. Jacobs. “Four problems solved by the probabilistic language of thought”. In: *Current Directions in Psychological Science* 25.1 (2016), pp. 54–59.

- [281] F. Cushman. “Action, outcome, and value: A dual-system framework for morality”. In: *Personality and social psychology review* 17.3 (2013), pp. 273–292.
- [282] S. Levine, N. Chater, J. Tenenbaum, and F. Cushman. *Resource-rational contractualism: A triple theory of moral cognition*. en-us. May 2023. DOI: [10.31234/osf.io/p48t7](https://doi.org/10.31234/osf.io/p48t7).
- [283] C. E. Alchourrón. “Logic of Norms and Logic of Normative Propositions”. In: *Logique et Analyse* 12.47 (1969). Publisher: Peeters Publishers, pp. 242–268. ISSN: 0024-5836.
- [284] W. Gaissmaier and L. J. Schooler. “The smart potential behind probability matching”. In: *Cognition* 109.3 (2008), pp. 416–422.
- [285] D. J. Russo, B. V. Roy, A. Kazerouni, I. Osband, and Z. Wen. “A Tutorial on Thompson Sampling”. English. In: *Foundations and Trends® in Machine Learning* 11.1 (July 2018). Publisher: Now Publishers, Inc., pp. 1–96. ISSN: 1935-8237, 1935-8245. DOI: [10.1561/22000000070](https://doi.org/10.1561/22000000070).
- [286] B. Eysenbach and S. Levine. *If MaxEnt RL is the Answer, What is the Question?* en. Oct. 2019.
- [287] T. Tanaka. “A Theory of Mean Field Approximation”. In: *Advances in Neural Information Processing Systems*. Vol. 11. MIT Press, 1998.
- [288] C. Bicchieri and Y. Fukui. “The great illusion: Ignorance, informational cascades, and the persistence of unpopular norms”. In: *Experience, Reality, and Scientific Explanation: Essays in Honor of Merrilee and Wesley Salmon*. Springer, 1999, pp. 89–121.
- [289] E. Fehr and I. Schurtenberger. “Normative foundations of human cooperation”. en. In: *Nature Human Behaviour* 2.7 (July 2018), pp. 458–468. ISSN: 2397-3374. DOI: [10.1038/s41562-018-0385-5](https://doi.org/10.1038/s41562-018-0385-5).
- [290] M. P. Wellman and M. Henrion. “Explaining’explaining away””. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 15.3 (1993), pp. 287–292.
- [291] F. Lindsten, A. M. Johansen, C. A. Naesseth, B. Kirkpatrick, T. B. Schön, J. A. Aston, and A. Bouchard-Côté. “Divide-and-conquer with sequential Monte Carlo”. In: *Journal of Computational and Graphical Statistics* 26.2 (2017), pp. 445–458.
- [292] T. M. Scanlon. *What We Owe to Each Other*. en. Google-Books-ID: 9OPsDwAAQBAJ. Harvard University Press, Nov. 2000. ISBN: 978-0-674-00423-8.
- [293] G. K. Hadfield and B. R. Weingast. “Microfoundations of the Rule of Law”. In: *Annual Review of Political Science* 17.1 (2014). _eprint: <https://doi.org/10.1146/annurev-polisci-100711-135226>, pp. 21–42. DOI: [10.1146/annurev-polisci-100711-135226](https://doi.org/10.1146/annurev-polisci-100711-135226).
- [294] F. D’Agostino, G. Gaus, and J. Thrasher. “Contemporary Approaches to the Social Contract”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta. Winter 2021. Metaphysics Research Lab, Stanford University, 2021.
- [295] K. M. Lhaksmana, Y. Murakami, and T. Ishida. “Role-Based Modeling for Designing Agent Behavior in Self-Organizing Multi-Agent Systems”. In: *International Journal of Software Engineering and Knowledge Engineering* 28.01 (Jan. 2018). Publisher: World Scientific Publishing Co., pp. 79–96. ISSN: 0218-1940. DOI: [10.1142/S0218194018500043](https://doi.org/10.1142/S0218194018500043).

- [296] P. J. K. Christoffersen, A. A. Haupt, and D. Hadfield-Menell. *Get It in Writing: Formal Contracts Mitigate Social Dilemmas in Multi-Agent RL*. arXiv:2208.10469 [cs, econ]. Aug. 2022. DOI: [10.48550/arXiv.2208.10469](https://doi.org/10.48550/arXiv.2208.10469).
- [297] J. Aldous and R. Hill. “Social Cohesion, Lineage Type, and Intergenerational Transmission”. In: *Social Forces* 43.4 (May 1965), pp. 471–482. ISSN: 0037-7732. DOI: [10.2307/2574453](https://doi.org/10.2307/2574453).
- [298] R. González et al. “The Role of Family in the Intergenerational Transmission of Collective Action”. en. In: *Social Psychological and Personality Science* 12.6 (Aug. 2021). Publisher: SAGE Publications Inc, pp. 856–867. ISSN: 1948-5506. DOI: [10.1177/1948550620949378](https://doi.org/10.1177/1948550620949378).
- [299] K.-P. Tam. “Understanding Intergenerational Cultural Transmission Through the Role of Perceived Norms”. en. In: *Journal of Cross-Cultural Psychology* 46.10 (Nov. 2015). Publisher: SAGE Publications Inc, pp. 1260–1266. ISSN: 0022-0221. DOI: [10.1177/0022022115600074](https://doi.org/10.1177/0022022115600074).
- [300] R. X. Hawkins, N. D. Goodman, and R. L. Goldstone. “The Emergence of Social Norms and Conventions”. en. In: *Trends in Cognitive Sciences* 23.2 (Feb. 2019), pp. 158–169. ISSN: 13646613. DOI: [10.1016/j.tics.2018.11.003](https://doi.org/10.1016/j.tics.2018.11.003).
- [301] G. Trommsdorff. “Parent–Child Relations Over the Lifespan: A Cross-Cultural Perspective”. In: *Parenting Beliefs, Behaviors, and Parent-Child Relations*. Num Pages: 42. Psychology Press, 2005. ISBN: 978-0-203-94290-1.
- [302] Bowles. *Group Competition, Reproductive Leveling, and the Evolution of Human Altruism / Science*. 2006.
- [303] R. A. Watson and E. Szathmáry. “How Can Evolution Learn?” English. In: *Trends in Ecology & Evolution* 31.2 (Feb. 2016). Publisher: Elsevier, pp. 147–157. ISSN: 0169-5347. DOI: [10.1016/j.tree.2015.11.009](https://doi.org/10.1016/j.tree.2015.11.009).
- [304] V. Bengtson. *Global Aging and Challenges to Families*. en. Routledge, Jan. 2018. ISBN: 978-1-351-32814-2.
- [305] J. M. Smith. *Evolution and the Theory of Games*. Cambridge University Press, 1982.
- [306] D. Panke and U. Petersohn. “Why international norms disappear sometimes”. In: *European journal of international relations* 18.4 (2012), pp. 719–742.
- [307] B. T. R. Savarimuthu, S. Cranefield, M. K. Purvis, and M. A. Purvis. “Norm emergence in agent societies formed by dynamically changing networks”. en. In: *Web Intelligence and Agent Systems: An International Journal* 7.3 (Jan. 2009). Publisher: IOS Press, pp. 223–232. ISSN: 1570-1263. DOI: [10.3233/WIA-2009-0164](https://doi.org/10.3233/WIA-2009-0164).
- [308] R. Shibusawa and T. Sugawara. “Norm Emergence via Influential Weight Propagation in Complex Networks”. In: *2014 European Network Intelligence Conference* (Sept. 2014). Conference Name: 2014 European Network Intelligence Conference (ENIC) ISBN: 9781479969142 Place: Wroclaw, Poland Publisher: IEEE, pp. 30–37. DOI: [10.1109/ENIC.2014.28](https://doi.org/10.1109/ENIC.2014.28).
- [309] E. Ullmann-Margalit. *The emergence of norms*. OUP Oxford, 2015.
- [310] A. Morris-Martin, M. De Vos, and J. Padget. “Norm emergence in multiagent systems: a viewpoint paper”. en. In: *Autonomous Agents and Multi-Agent Systems* 33.6 (Nov. 2019), pp. 706–749. ISSN: 1387-2532, 1573-7454. DOI: [10.1007/s10458-019-09422-0](https://doi.org/10.1007/s10458-019-09422-0).

- [311] D. Hadfield-Menell, M. Andrus, and G. K. Hadfield. “Legible Normativity for AI Alignment: The Value of Silly Rules”. In: arXiv:1811.01267 [cs]. Nov. 2018.
- [312] J. M. Darley, K. M. Carlsmith, and P. H. Robinson. “Incapacitation and just deserts as motives for punishment”. In: *Law and human behavior* 24 (2000), pp. 659–683.
- [313] F. Cushman, A. Sarin, and M. Ho. “Punishment as communication”. In: *The Oxford handbook of moral psychology* (2019), pp. 197–209.
- [314] S. Radkani, J. Tenenbaum, and R. Saxe. “Modeling punishment as a rational communicative social action”. In: *Proceedings of the annual meeting of the cognitive science society*. Vol. 44. 44. 2022.
- [315] S.-C. Zhu, R. Zhang, and Z. Tu. “Integrating bottom-up/top-down for object recognition by data driven Markov chain Monte Carlo”. In: *Proceedings IEEE Conference on Computer Vision and Pattern Recognition. CVPR 2000 (Cat. No. PR00662)*. Vol. 1. IEEE. 2000, pp. 738–745.
- [316] T. Wang, Y. Wu, D. Moore, and S. J. Russell. “Meta-learning MCMC proposals”. In: *Advances in neural information processing systems* 31 (2018).
- [317] I. Dasgupta, E. Schulz, J. B. Tenenbaum, and S. J. Gershman. “A theory of learning to infer.” In: *Psychological review* 127.3 (2020), p. 412.
- [318] K. Hagen. “The propriety of Confucius: A sense-of-ritual”. In: *Asian Philosophy* 20.1 (2010), pp. 1–25.
- [319] W. M. Pauli, J. Cockburn, E. R. Pool, O. D. Perez, and J. P. O’Doherty. “Computational approaches to habits in a model-free world”. In: *Current Opinion in Behavioral Sciences* 20 (2018), pp. 104–109.
- [320] J. Mikhail. “Universal moral grammar: Theory, evidence and the future”. In: *Trends in cognitive sciences* 11.4 (2007), pp. 143–152.
- [321] Y. Bai, S. Kadavath, S. Kundu, A. Aske, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. “Constitutional AI: Harmlessness from AI feedback”. In: *arXiv preprint arXiv:2212.08073* (2022).
- [322] J. X. Liu, Z. Yang, I. Idrees, S. Liang, B. Schornstein, S. Tellex, and A. Shah. “Lang2LTL: Translating Natural Language Commands to Temporal Robot Task Specification”. In: *arXiv preprint arXiv:2302.11649* (2023).
- [323] T. X. Olausson, A. Gu, B. Lipkin, C. E. Zhang, A. Solar-Lezama, J. B. Tenenbaum, and R. Levy. “LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers”. In: *arXiv preprint arXiv:2310.15164* (2023).
- [324] Y. N. Wu, R. Gao, T. Han, and S.-C. Zhu. “A tale of three probabilistic families: Discriminative, descriptive, and generative models”. In: *Quarterly of Applied Mathematics* 77.2 (2019), pp. 423–465.
- [325] S. Levine, M. Kleiman-Weiner, L. Schulz, J. Tenenbaum, and F. Cushman. “The logic of universalization guides moral judgment”. In: *Proceedings of the National Academy of Sciences* 117.42 (2020), pp. 26158–26169.

- [326] J. Kwon, T. Zhi-Xuan, J. Tenenbaum, and S. Levine. “When it is not out of line to get out of line: The role of universalization and outcome-based reasoning in rule-breaking judgments”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 45. 45. July 2023.
- [327] L. Kohlberg and R. H. Hersh. “Moral development: A review of the theory”. In: *Theory into practice* 16.2 (1977), pp. 53–59.
- [328] S. Levine, M. Kleiman-Weiner, N. Chater, F. Cushman, and J. B. Tenenbaum. “When rules are over-ruled: Virtual bargaining as a contractualist method of moral judgment”. In: *Cognition* 250 (2024), p. 105790.
- [329] D. Trujillo, M. Zhang, T. Zhi-Xuan, J. B. Tenenbaum, and S. Levine. “Resource-Rational Virtual Bargaining for Moral Judgment: Toward a Probabilistic Cognitive Model”. In: *Topics in Cognitive Science* (2024).
- [330] A. Doucet, N. De Freitas, and N. Gordon. “An introduction to sequential Monte Carlo methods”. In: *Sequential Monte Carlo methods in practice* (2001), pp. 3–14.
- [331] M. Cusumano-Towner, B. Bichsel, T. Gehr, M. Vechev, and V. K. Mansinghka. “Incremental inference for probabilistic programs”. In: *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2018, pp. 571–585.
- [332] G. Z. Lin. “Bayesian Active Structure Learning for Gaussian Process Probabilistic Programs”. MA thesis. Massachusetts Institute of Technology, 2022.
- [333] T. Mills, J. Tenenbaum, and S. Cheyette. “Human spatiotemporal pattern learning as probabilistic program synthesis”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [334] V. Elvira, L. Martino, D. Luengo, and M. F. Bugallo. “Generalized Multiple Importance Sampling”. In: *Statistical Science* 34.1 (2019), pp. 129–155.
- [335] M. F. Cusumano-Towner. “Gen: a high-level programming platform for probabilistic inference”. PhD thesis. Massachusetts Institute of Technology, 2020.
- [336] R. M. Neal. “Annealed importance sampling”. In: *Statistics and computing* 11 (2001), pp. 125–139.
- [337] C. Naesseth, F. Lindsten, and T. Schon. “Nested sequential monte carlo methods”. In: *International Conference on Machine Learning*. PMLR. 2015, pp. 1292–1301.
- [338] R. A. G. Marques and G. Storvik. “Particle move-reweighting strategies for online inference”. In: *Preprint series. Statistical Research Report <http://urn.nb.no/URN:NBN:no-23420>* (2013).
- [339] A. Saeedi, T. D. Kulkarni, V. K. Mansinghka, and S. J. Gershman. “Variational particle approximations”. In: *Journal of Machine Learning Research* 18.69 (2017), pp. 1–29.
- [340] R. Douc and O. Cappé. “Comparison of resampling schemes for particle filtering”. In: *ISPA 2005. Proceedings of the 4th International Symposium on Image and Signal Processing and Analysis, 2005*. IEEE. 2005, pp. 64–69.
- [341] C. Vergé, C. Dubarry, P. Del Moral, and E. Moulines. “On parallel implementation of sequential Monte Carlo methods: the island particle model”. In: *Statistics and Computing* 25.2 (2015), pp. 243–260.

- [342] N. Whitely, A. Lee, and K. Heine. “On the role of interaction in sequential Monte Carlo algorithms”. In: *Bernoulli* (2016), pp. 494–529.
- [343] R. Chen. “Sequential Monte Carlo methods and their applications”. In: *IMS Lecture Notes Series, Markov Chain Monte Carlo* 7 (2005), pp. 147–182.
- [344] D. Fox. “KLD-sampling: Adaptive particle filters”. In: *Advances in neural information processing systems* 14 (2001).
- [345] P. Fearnhead and P. Clifford. “On-line inference for hidden Markov models via particle filters”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 65.4 (2003), pp. 887–899.
- [346] W. Kool, H. Van Hoof, and M. Welling. “Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement”. In: *International Conference on Machine Learning*. PMLR. 2019, pp. 3499–3508.
- [347] C. I. Meister, A. Amini, T. Vieira, and R. Cotterell. “Conditional Poisson stochastic beams”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. 2021, pp. 664–681.
- [348] T. Higuchi. “Monte Carlo filter using the genetic algorithm operators”. In: *Journal of Statistical Computation and Simulation* 59.1 (1997), pp. 1–23.
- [349] J. W. Suchow, D. D. Bourgin, and T. L. Griffiths. “Evolution in mind: Evolutionary dynamics, cognitive processes, and bayesian inference”. In: *Trends in cognitive sciences* 21.7 (2017), pp. 522–530.
- [350] T. Baeck, D. Fogel, and Z. Michalewicz. *Evolutionary Computation 1: Basic Algorithms and Operators*. Basic algorithms and operators. Taylor & Francis, 2000. ISBN: 9780750306645.
- [351] M. Strens. “Evolutionary MCMC sampling and optimization in discrete spaces”. In: *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*. 2003, pp. 736–743.
- [352] M. M. Drugan and D. Thierens. “Recombination operators and selection strategies for evolutionary Markov Chain Monte Carlo algorithms”. In: *Evolutionary intelligence* 3 (2010), pp. 79–101.
- [353] H. Wu and D. Wang. “Worst-Case Analysis is Maximum-A-Posteriori Estimation”. In: *arXiv preprint arXiv:2310.09774* (2023).
- [354] L. M. Murray, A. Lee, and P. E. Jacob. “Parallel resampling in the particle filter”. In: *Journal of Computational and Graphical Statistics* 25.3 (2016), pp. 789–805.
- [355] R. Frigola, F. Lindsten, T. B. Schön, and C. E. Rasmussen. “Bayesian inference and learning in Gaussian process state-space models with particle MCMC”. In: *Advances in neural information processing systems* 26 (2013).
- [356] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

- [357] R. Sennrich, B. Haddow, and A. Birch. “Neural Machine Translation of Rare Words with Sub-word Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2016, pp. 1715–1725.
- [358] V. Zouhar, C. Meister, J. Gastaldi, L. Du, T. Vieira, M. Sachan, and R. Cotterell. “A Formal Perspective on Byte-Pair Encoding”. In: *Findings of the Association for Computational Linguistics: ACL 2023*. 2023, pp. 598–614.
- [359] C. Andrieu and G. O. Roberts. “The Pseudo-Marginal Approach for Efficient Monte Carlo Computations”. In: *The Annals of Statistics* (2009), pp. 697–725.
- [360] A. Lew, M. Agrawal, D. Sontag, and V. Mansinghka. “PClean: Bayesian data cleaning at scale with domain-specific probabilistic programming”. In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2021, pp. 1927–1935.
- [361] H. Pasula, B. Marthi, B. Milch, S. J. Russell, and I. Shpitser. “Identity uncertainty and citation matching”. In: *Advances in Neural Information Processing Systems* 15 (2002).
- [362] B. Marthi, B. Milch, and S. Russell. “First-order probabilistic models for information extraction”. In: *IJCAI 2003 Workshop on Learning Statistical Models from Relational Data*. 2003, p. 2.
- [363] B. Z. Li, W. Chen, P. Sharma, and J. Andreas. “LaMPP: Language Models as Probabilistic Priors for Perception and Action”. In: *arXiv e-prints* (2023), arXiv–2302.
- [364] K. Choi, C. Cundy, S. Srivastava, and S. Ermon. “LMPriors: Pre-Trained Language Models as Task-Specific Priors”. In: *NeurIPS 2022 Foundation Models for Decision Making Workshop*.
- [365] E. Kiciman, R. Ness, A. Sharma, and C. Tan. “Causal reasoning and large language models: Opening a new frontier for causality”. In: *Transactions on Machine Learning Research* (2023).
- [366] K. Ellis. “Human-like few-shot learning via bayesian reasoning over natural language”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 13149–13178.
- [367] T. Zhi-Xuan, P. S. Lunis, N. Fernandez Echeverri, V. Mansinghka, and J. Tenenbaum. “Language Models as Informative Goal Priors in a Bayesian Theory of Mind”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 45. 45. 2023.
- [368] G. Grand, V. Pepe, J. Andreas, and J. Tenenbaum. “Loose LIPS Sink Ships: Asking Questions in Battleship with Language-Informed Program Sampling”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 46. 2023.
- [369] R. Ma, J. Qu, A. Bobu, and D. Hadfield-Menell. “Goal Inference from Open-Ended Dialog”. In: *arXiv preprint arXiv:2410.13957* (2024).
- [370] J. Requeima, J. Bronskill, D. Choi, R. Turner, and D. K. Duvenaud. “LLM Processes: Numerical predictive distributions conditioned on natural language”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 109609–109671.
- [371] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson. “Large language models are zero-shot time series forecasters”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 19622–19635.

- [372] L. Ying, K. M. Collins, M. Wei, C. E. Zhang, T. Zhi-Xuan, A. Weller, J. B. Tenenbaum, and L. Wong. “The Neuro-Symbolic Inverse Planning Engine (NIPE): Modeling Probabilistic Social Inferences from Linguistic Inputs”. In: *ICML 2023 Workshop on Theory of Mind in Communicating Agents*. 2023.
- [373] L. O’Mahony, L. Grinsztajn, H. Schoelkopf, and S. Biderman. “Attributing mode collapse in the fine-tuning of large language models”. In: *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*. 2024.
- [374] A. Owen and Y. Zhou. “Safe and effective importance sampling”. In: *Journal of the American Statistical Association* 95.449 (2000), pp. 135–143.
- [375] G. Poesia, A. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani. “Synchromesh: Reliable Code Generation from Pre-trained Language Models”. In: *International Conference on Learning Representations*. 2022.
- [376] L. Ying, J. X. Liu, S. Aarya, Y. Fang, S. Tellex, J. B. Tenenbaum, and T. Shu. “SIFTOM: Robust Spoken Instruction Following through Theory of Mind”. In: *arXiv preprint arXiv:2409.10849* (2024).
- [377] D. Dohan, W. Xu, A. Lewkowycz, J. Austin, D. Bieber, R. G. Lopes, Y. Wu, H. Michalewski, R. A. Saurous, J. Sohl-Dickstein, et al. “Language model cascades”. In: *arXiv preprint arXiv:2207.10342* (2022).
- [378] P. Moral. *Feynman-Kac formulae: genealogical and interacting particle systems with applications*. Springer, 2004.
- [379] C. Zhang, L. Wong, G. Grand, and J. Tenenbaum. “Grounded physical language understanding with probabilistic programs and simulated worlds”. In: *Proceedings of the Annual Meeting of the Cognitive Science Society*. Vol. 45. 45. 2023.
- [380] G. Grand, J. B. Tenenbaum, V. K. Mansinghka, A. K. Lew, and J. Andreas. “Self-Steering Language Models”. In: *arXiv preprint arXiv:2504.07081* (2025).